APP\core__init__.py

0 lines • 0 bytes

APP\core\app__init__.py

0 lines • 0 bytes

APP\core\app\file_processor.py

429 lines • 15,130 bytes

```
import os
import logging
import time
import concurrent.futures
import threading
from datetime import datetime
from pathlib import Path
from typing import List, Dict, Any, Callable, Optional

from core.app.performance import resolve_max_workers
from core.infra import is_file_writable
from core.infra.utils_logging import safe_file_label, scrub_path


logger = logging.getLogger(__name__)


def _parse_date_candidates(
    raw_date: Any,
    candidate_years: Optional[List[int]] = None,
) -> List[datetime]:
    if raw_date is None:
        return []

    date_text = str(raw_date).strip()
    if not date_text or date_text == "01.01.1900":
        return []

    for fmt in ("%d.%m.%Y", "%d.%m.%y"):
        try:
            return [datetime.strptime(date_text, fmt)]
        except ValueError:
            continue

    try:
        day_month = datetime.strptime(date_text, "%d.%m")
    except ValueError:
        return []

    years = candidate_years or [datetime.now().year]
    parsed: List[datetime] = []
    seen = set()
    for year in years:
        if year in seen:
            continue
        seen.add(year)
        try:
            parsed.append(day_month.replace(year=year))
        except ValueError:
            continue
    return parsed


def _has_explicit_year(raw_date: Any) -> bool:
    if raw_date is None:
        return False

    date_text = str(raw_date).strip()
    if not date_text:
        return False

    for fmt in ("%d.%m.%Y", "%d.%m.%y"):
        try:
            datetime.strptime(date_text, fmt)
            return True
        except ValueError:
            continue
    return False


def date_matches_target(raw_date: Any, target_date: datetime) -> bool:
    if raw_date is None:
        return False

    date_text = str(raw_date).strip()
    if not date_text or date_text == "01.01.1900":
        return False

    if date_text == target_date.strftime("%d.%m.%Y"):
        return True

    if not _has_explicit_year(date_text):
        return False
```

```
    parsed = _parse_date_candidates(date_text, [target_date.year])
    return any(dt.date() == target_date.date() for dt in parsed)


def _resolve_row_date_for_filters(
    raw_date: Any,
    target_date: Optional[datetime],
    start_date: Optional[datetime],
    end_date: Optional[datetime],
    today: datetime,
    fallback_year: Optional[int] = None,
) -> tuple[Optional[datetime], Optional[str]]:
    date_text = str(raw_date).strip() if raw_date is not None else ""
    if not date_text or date_text == "01.01.1900":
        return None, "invalid_date"

    if not _has_explicit_year(date_text) and not fallback_year:
        return None, "missing_year"

    if target_date is not None:
        years = [target_date.year]
    elif start_date is not None and end_date is not None:
        years = list(range(start_date.year, end_date.year + 1))
    else:
        if fallback_year and 1900 <= fallback_year <= 2100:
            years = [fallback_year]
        else:
            years = [today.year]

    candidates = _parse_date_candidates(raw_date, years)
    if not candidates:
        return None, "invalid_date"

    if target_date is not None:
        matched = [dt for dt in candidates if dt.date() == target_date.date()]
    elif start_date is not None and end_date is not None:
        matched = [
            dt
            for dt in candidates
            if start_date.date() <= dt.date() <= end_date.date()
        ]
    else:
        matched = candidates

    if not matched:
        return None, "out_of_scope"

    non_future = [dt for dt in matched if dt.date() <= today.date()]
    if not non_future:
        return None, "future_date"

    return min(non_future), None


def check_master_lock(config_controller) -> bool:
    fallback = Path(config_controller.config_path).parent / "Master_Report.xlsx"
    paths_cfg = config_controller.config.setdefault("paths", {})
    master_cfg = paths_cfg.get("master_file")
    if master_cfg:
        master_path = config_controller.resolve_path(str(master_cfg))
    else:
        master_path = fallback
        paths_cfg["master_file"] = str(fallback)
        save_config = getattr(config_controller, "save_config", None)
        if callable(save_config):
            save_config()
    try:
        master_path.parent.mkdir(parents=True, exist_ok=True)
    except OSError as e:
        try:
            fallback.parent.mkdir(parents=True, exist_ok=True)
            master_path = fallback
            if Path(paths_cfg.get("master_file", "")) != fallback:
                paths_cfg["master_file"] = str(fallback)
                save_config = getattr(config_controller, "save_config", None)
                if callable(save_config):
                    save_config()
            logging.warning(
                "Master path unavailable (%s). Using fallback: %s",
                e,
                scrub_path(str(fallback)),
            )
        except OSError:
```

```
            return False
    return is_file_writable(str(master_path))


def process_file_task(
    persistence,
    ingestion,
    validator,
    emp_name,
    meta,
    session_id,
    target_date_str,
    start_date_str=None,
    end_date_str=None,
    processed_file_ids: Optional[List[int]] = None,
    processed_file_ids_lock: Optional[Any] = None,
    cancel_event: Optional[threading.Event] = None,
    today: Optional[datetime] = None,
) -> int:
    if cancel_event is not None and cancel_event.is_set():
        return 0

    path_str = meta["path"] if isinstance(meta, dict) else meta
    if not os.path.exists(path_str):
        raise FileNotFoundError(
            f"Не найден файл отчета для сотрудника {emp_name}: {path_str}"
        )

    logging.info("Processing file %s", safe_file_label(path_str))
    file_ids = persistence.save_file_metadata(session_id, [meta])
    file_id = file_ids[0]
    try:
        # Build cache key
        file_stat = os.stat(path_str)
        file_mtime = file_stat.st_mtime
        file_size = file_stat.st_size
        date_key = target_date_str or "all"

        # Try row cache first
        cached_rows = persistence.get_row_cache(path_str, file_mtime, file_size, date_key)
        if cached_rows is not None:
            _t0 = time.monotonic()
            rows = cached_rows
            _elapsed = time.monotonic() - _t0
            logging.info(f"[PERF] Cache hit {safe_file_label(path_str)}: {_elapsed:.3f}s ({len(rows)} rows)")
        else:
            _t0 = time.monotonic()
            rows = ingestion.read_file(
                Path(path_str),
                strict_io=True,
                target_date_str=target_date_str,
                start_date_str=start_date_str,
                end_date_str=end_date_str,
            )
            _elapsed = time.monotonic() - _t0
            logging.info(f"[PERF] Read {safe_file_label(path_str)}: {_elapsed:.1f}s ({len(rows)} rows)")
            # Save to cache (async, non-blocking)
            persistence.save_row_cache(path_str, file_mtime, file_size, date_key, rows)
        valid_rows = []
        template_errors = []
        target_date = (
            datetime.strptime(target_date_str, "%d.%m.%Y") if target_date_str else None
        )
        start_date = (
            datetime.strptime(start_date_str, "%d.%m.%Y") if start_date_str else None
        )
        end_date = datetime.strptime(end_date_str, "%d.%m.%Y") if end_date_str else None
        today = today or datetime.now()  # Use shared timestamp from caller for consistency

        for row in rows:
            row_date = row.get("date")
            if not row_date:
                template_errors.append(
                    {
                        "file_path": path_str,
                        "sheet_name": row.get("_sheet_name"),
                        "row_index": row.get("_row_index"),
                        "error_type": "missing_date",
                        "details": "missing date skipped",
                        "raw_data": row,
                    }
                )
                continue
```

```
            row_year_hint = row.get("_sheet_year_hint")
            resolved_date, skip_reason = _resolve_row_date_for_filters(
                row_date,
                target_date,
                start_date,
                end_date,
                today,
                fallback_year=row_year_hint,
            )

            if skip_reason in {"invalid_date", "future_date", "missing_year"}:
                details = (
                    f"future date skipped: {row_date}"
                    if skip_reason == "future_date"
                    else (
                        f"missing year in sheet date skipped: {row_date}"
                        if skip_reason == "missing_year"
                        else f"invalid date skipped: {row_date}"
                    )
                )
                template_errors.append(
                    {
                        "file_path": path_str,
                        "sheet_name": row.get("_sheet_name"),
                        "row_index": row.get("_row_index"),
                        "error_type": skip_reason,
                        "details": details,
                        "raw_data": row,
                    }
                )
                continue

            if skip_reason == "out_of_scope" or resolved_date is None:
                continue

            row["date"] = resolved_date.strftime("%d.%m.%Y")
            if validator.validate_row(row):
                row["employee_name"] = emp_name
                valid_rows.append(row)

        has_template_warnings = False
        for row in valid_rows:
            warning_kind = row.get("_template_warning_kind")
            if row.get("is_template_warning") or warning_kind:
                if row.get("is_template_warning"):
                    has_template_warnings = True
                template_errors.append(

                    {
                        "file_path": path_str,
                        "sheet_name": row.get("_sheet_name"),
                        "row_index": row.get("_row_index"),
                        "error_type": warning_kind or "template_warning",
                        "details": row.get(
                            "_template_warning_reason",
                            row.get("_source_info", "Template warning"),
                        ),
                        "raw_data": row,
                    },
                )

        if template_errors:
            persistence.save_template_errors_batch(session_id, template_errors)

        if valid_rows:
            persistence.save_staging_data(file_id, valid_rows)
        persistence.update_file_status(
            file_id,
            "processed_with_warnings" if has_template_warnings else "processed",
        )
        if processed_file_ids is not None:
            if processed_file_ids_lock is not None:
                with processed_file_ids_lock:
                    processed_file_ids.append(file_id)
            else:
                processed_file_ids.append(file_id)
        return len(valid_rows)
    except Exception as e:
        persistence.update_file_status(file_id, "error", str(e))
        raise


def run_engine(
```

```
    persistence,
    ingestion,
    validator,
    employees_map,
    session_id,
    target_date_str,
    on_progress,
    start_date_str=None,
    end_date_str=None,
    on_file_error: Optional[Callable[[List[Dict[str, str]]], None]] = None,
    cancel_event: Optional[threading.Event] = None,
) -> bool:
    total_files = sum(len(paths) for paths in employees_map.values())
    if total_files == 0:
        return False

    processed_files = 0
    total_rows_processed = 0
    file_errors: List[Dict[str, str]] = []

    processed_file_ids = []
    processed_file_ids_lock = threading.Lock()

    # Compute current date once before spawning threads so that all
    # workers use the same reference — avoids inconsistent "future date"
    # rejections when the job straddles midnight.
    today = datetime.now()

    with concurrent.futures.ThreadPoolExecutor(
        max_workers=resolve_max_workers(getattr(ingestion, "config", None))
    ) as executor:
        future_to_file = {}
        for emp_name, metas in employees_map.items():
            for meta in metas:
                if cancel_event is not None and cancel_event.is_set():
                    break
                future = executor.submit(
                    process_file_task,
                    persistence,
                    ingestion,
                    validator,
                    emp_name,
                    meta,
                    session_id,
                    target_date_str,
                    start_date_str,
                    end_date_str,
                    processed_file_ids,
                    processed_file_ids_lock,
                    cancel_event,
                    today,
                )
                future_to_file[future] = meta
            if cancel_event is not None and cancel_event.is_set():
                break

        for future in concurrent.futures.as_completed(future_to_file):
            try:
                total_rows_processed += future.result()
            except Exception as e:
                meta = future_to_file.get(future)
                file_path = meta.get("path") if isinstance(meta, dict) else str(meta)
                file_errors.append({"path": str(file_path), "error": str(e)})
                logging.error("Failed to process file %s: %s", safe_file_label(file_path), e)

            processed_files += 1
            on_progress((processed_files / total_files) * 50)

    # Wait for pending DB writes to drain before returning so the
    # caller can safely check cancel or read consolidated data.
    try:
        persistence.wait_for_writes()
    except Exception as e:
        for file_id in processed_file_ids:
            try:
                persistence.update_file_status(file_id, "error", str(e))
            except Exception as update_err:
                logging.error(f"Failed to update file status for {file_id}: {update_err}")
        raise

    logging.info(
        f"Ingestion complete. Total valid rows processed: {total_rows_processed}"
    )
```

```
    if file_errors and callable(on_file_error):
        try:
            on_file_error(file_errors)
        except Exception as callback_error:
            logging.error(f"Failed to report file errors: {callback_error}")
    return True


def get_master_path(config_controller) -> Path:
    master_str = config_controller.config.get("paths", {}).get(
        "master_file", "Master_Report.xlsx"
    )
    return config_controller.resolve_path(master_str)
```

APP\core\app\message_formatting.py

131 lines • 4,459 bytes

```
from __future__ import annotations

import ntpath
import os
from datetime import datetime
from typing import Dict, List, Optional


def _basename(path: str) -> str:
    return os.path.basename(path) or ntpath.basename(path)


def format_file_error_note(file_errors: List[Dict[str, str]]) -> Optional[str]:
    if not file_errors:
        return None

    def format_item(err: Dict[str, str]) -> str:
        name = _basename(err.get("path", ""))
        error = str(err.get("error") or "").strip()
        if not name:
            return ""
        if not error:
            return name
        if len(error) > 80:
            error = error[:77].rstrip() + "..."
        return f"{name} ({error})"

    sample = ", ".join(
        format_item(err)
        for err in file_errors[:3]
        if err.get("path")
    )
    suffix = "" if len(file_errors) <= 3 else " ..."
    if sample:
        end = "." if not suffix else ""
        return f"Не удалось обработать {len(file_errors)} файл(ов): {sample}{suffix}{end}"
    return f"Не удалось обработать {len(file_errors)} файл(ов)."


def build_empty_data_message(
    target_date_str: Optional[str],
    start_date_str: Optional[str],
    end_date_str: Optional[str],
    range_note: Optional[str],
    file_error_note: Optional[str],
) -> str:
    if start_date_str and end_date_str:
        msg = f"Нет данных за период {start_date_str} — {end_date_str}"
    elif target_date_str:
        msg = "Нет данных за дату"
    else:
        msg = "Нет данных"

    notes = [note for note in (range_note, file_error_note) if note]
    if notes:
        return f"{msg}. {' '.join(notes)}"
    return msg


def build_success_message(
    session_type: str,
    start_date_str: Optional[str],
    end_date_str: Optional[str],
    range_note: Optional[str],
    file_error_note: Optional[str],
) -> str:
    if start_date_str and end_date_str:
        msg = f"Готово! ({start_date_str} — {end_date_str})"
    elif session_type == "all_dates":
        msg = "Готово!"
    else:
        msg = "Успешно!"

    notes = [note for note in (range_note, file_error_note) if note]
    if notes:
        return f"{msg} {' '.join(notes)}"
    return msg


def normalize_consolidation_dates(
    target_date_str: Optional[str],
    start_date_str: Optional[str],
    end_date_str: Optional[str],
```

```
) -> tuple[Optional[str], Optional[str], Optional[str], Optional[str], Optional[str]]:
    today = datetime.now().date()
    range_note: Optional[str] = None
    effective_target_date = target_date_str
    effective_start_date = start_date_str
    effective_end_date = end_date_str

    if target_date_str:
        td = datetime.strptime(target_date_str, "%d.%m.%Y").date()
        if td > today:
            msg = (
                f"Дата {target_date_str} в будущем. "
                "Будущие даты исключаются из сборки."
            )
            return None, None, None, msg, None

    if start_date_str and end_date_str:
        sd = datetime.strptime(start_date_str, "%d.%m.%Y").date()
        ed = datetime.strptime(end_date_str, "%d.%m.%Y").date()
        if sd > today:
            msg = (
                f"Период {start_date_str} — {end_date_str} полностью в будущем. "
                "Будущие даты исключаются из сборки."
            )
            return None, None, None, msg, None
        if ed > today:
            effective_end_date = today.strftime("%d.%m.%Y")
            range_note = (
                f"Даты после {effective_end_date} исключены "
                "(будущие даты не учитываются)."
            )

    return effective_target_date, effective_start_date, effective_end_date, None, range_note


def build_issue_preview(issues: List[str], limit: int = 3) -> str:
    unique_issues: list[str] = []
    seen: set[str] = set()
    for issue in issues:
        if issue in seen:
            continue
        seen.add(issue)
        unique_issues.append(issue)

    preview = "; ".join(unique_issues[:limit])
    if len(unique_issues) > limit:
        preview += f"; и еще {len(unique_issues) - limit}"
    return preview
```

APP\core\app\partial.py

72 lines • 2,623 bytes

```
"""Mark interrupted output files as ``.partial`` so users can clean up.

When a long-running report is cancelled mid-write, the destination
workbook may exist on disk in an incomplete state. Removing it would
silently destroy work the user might want to inspect; leaving it
with the real filename risks the user opening a corrupted file by
accident later.

The convention: rename ``foo.xlsx`` → ``foo.partial.xlsx`` on cancel.
The user can then either delete the partial file or rename it back
once they understand what was written. This is the same pattern
used by web browsers for in-progress downloads.
"""
from __future__ import annotations

import logging
from pathlib import Path
from typing import List, Optional

_PARTIAL_SUFFIX = ".partial"


def mark_partial(path: Optional[Path]) -> List[Path]:
    """Rename ``path`` (or its contents) to a ``.partial`` variant.

    ``None`` and missing paths are no-ops. When ``path`` is a directory,
    every ``*.xlsx`` file inside is renamed. Returns the list of
    resulting partial paths (empty when nothing was renamed).
    """
    if path is None:
        return []
    src = Path(path)
    if not src.exists():
        return []
    candidates: List[Path] = []
    if src.is_dir():
        candidates.extend(sorted(src.glob("*.xlsx")))
    else:
        candidates.append(src)

    renamed: List[Path] = []
    for entry in candidates:
        new_name = entry.with_name(entry.stem + _PARTIAL_SUFFIX + entry.suffix)
        # Avoid clobbering an existing partial from a previous attempt.
        counter = 1
        target = new_name
        while target.exists():
            target = entry.with_name(
                f"{entry.stem}{_PARTIAL_SUFFIX}.{counter}{entry.suffix}"
            )
            counter += 1
        try:
            entry.rename(target)
        except OSError as exc:
            logging.debug("Could not mark %s as partial: %s", entry, exc)
            continue
        renamed.append(target)
    return renamed


def format_partial_note(partials: List[Path]) -> str:
    """Return a human-readable note listing the partial files."""
    if not partials:
        return ""
    if len(partials) == 1:
        return f" Неполный файл: {partials[0].name} (переименован в .partial)."
    joined = ", ".join(p.name for p in partials[:3])
    suffix = "" if len(partials) <= 3 else f" и ещё {len(partials) - 3}"
    return f" Неполные файлы ({len(partials)}): {joined}{suffix}."


__all__ = ["mark_partial", "format_partial_note"]
```

APP\core\app\performance.py

22 lines • 658 bytes

```
import os
from typing import Any, Mapping


def default_max_workers() -> int:
    return min(32, (os.cpu_count() or 4) * 2)


def resolve_max_workers(config: Mapping[str, Any] | None) -> int:
    if not isinstance(config, Mapping):
        return default_max_workers()
    performance = config.get("performance")
    if not isinstance(performance, Mapping):
        return default_max_workers()
    raw_value = performance.get("max_workers")
    if raw_value in (None, ""):
        return default_max_workers()
    try:
        value = int(raw_value)
    except (TypeError, ValueError):
        return default_max_workers()
    return max(1, min(64, value))
```

APP\core\app_controller.py

1142 lines • 45,914 bytes

```
import concurrent.futures
import logging
import os
import threading
import uuid
from datetime import datetime
from collections.abc import Sequence
from pathlib import Path
from typing import List, Dict, Any, Optional, Callable

from core.infra import DiscoveryService, is_file_writable
from core.infra.utils_logging import scrub_path
from core.app.file_processor import (
    check_master_lock,
    run_engine,
    date_matches_target,
)
from core.app.performance import default_max_workers, resolve_max_workers
from core.app.message_formatting import (
    build_empty_data_message as build_controller_empty_data_message,
    build_issue_preview,
    build_success_message as build_controller_success_message,
    format_file_error_note as format_controller_file_error_note,
    normalize_consolidation_dates as normalize_controller_consolidation_dates,
)
from core.infra.reconciliation import build_combined_reconciliation_report
from core.infra.rsch import build_rsch_report
from core.services.rsch_service import RschService
from core.services.reconciliation_service import ReconciliationService
from core.app.partial import format_partial_note, mark_partial
from core.infra.utils import cleanup_local_copy_cache

_ADAPTIVE_WORKERS = default_max_workers()


class ConsolidationRunError(RuntimeError):
    def __init__(self, message: str, code: str = "failed") -> None:
        super().__init__(message)
        self.code = code


def _is_cancelled(cancel_event: Optional[threading.Event]) -> bool:
    """Return True when the user requested cancellation.

    Helper for the threaded run_* entry points. The cancel_event is set
    by the GUI when the user clicks the Cancel button on the
    OperationStatusBar; the helper exists so call sites stay short and
    the check stays cheap (no allocation per iteration).
    """
    return cancel_event is not None and cancel_event.is_set()


def _cancel_message(
    on_complete: Callable[[bool, str, Optional[Path]], None],
    *output_paths: Optional[Path],
) -> None:
    """Wrap the standard cancel completion in a partial-files note.

    All six ``run_*`` methods funnel their cancel branch through this
    helper so the user gets a consistent message and the on-disk
    partial files are renamed (not deleted) before the GUI shows the
    toast. Network cache files are scrubbed in the same call.

    Pass each candidate output path (file or directory) that the
    operation may have started writing; the helper marks every
    existing ``*.xlsx`` under those paths as ``.partial`` and appends
    a short note to the user-facing message.
    """
    cleanup_local_copy_cache(max_age_days=0, min_delete_age_seconds=0)
    partials: List[Path] = []
    for path in output_paths:
        partials.extend(mark_partial(path))
    note = format_partial_note(partials)
    message = "Операция отменена пользователем." + note
    try:
        # Tag the message so the GUI completion handler can detect
        # cancellation by type (isinstance) instead of Russian substring
        # matching — see core.ui.gui_commands._base.CancelledByUser.
        # Lazy import keeps the controller decoupled from tkinter at
        # module load (headless paths do not import the UI layer).
        from core.ui.gui_commands._base import CancelledByUser
        tagged = CancelledByUser(message)
    except Exception:
```

```
        tagged = message
    on_complete(False, tagged, None)


class AppController:
    """Main orchestrator — coordinates file discovery, ingestion, and export."""

    def __init__(
        self,
        config_controller,
        ingestion,
        persistence,
        validator,
        logic_service,
        writer,
        alias_repo=None,
    ):
        self.config_controller = config_controller
        self.ingestion = ingestion
        self.persistence = persistence
        self.validator = validator
        self.logic_service = logic_service
        self.writer = writer
        self.alias_repo = alias_repo

        self.discovery = DiscoveryService(config_controller, ingestion, persistence=persistence)
        self._max_workers = resolve_max_workers(
            getattr(config_controller, "config", None)
        )
        self._executor = concurrent.futures.ThreadPoolExecutor(
            max_workers=self._max_workers,
            thread_name_prefix="hermes-worker",
        )
        self._lock = threading.Lock()
        self._cached_employees = {}
        self._active_threads: List[threading.Thread] = []
        self._rsch = RschService(
            start_thread_fn=self._start_thread,
            get_drive_letters_fn=self._get_network_drive_letters,
        )
        self._reconciliation = ReconciliationService(
            start_thread_fn=self._start_thread,
            get_drive_letters_fn=self._get_network_drive_letters,
            alias_repo=self.alias_repo,
        )

    @property
    def rename_detected(self):
        return self.discovery.rename_detected

    def _get_employees_map(
        self, *, allow_network_scan: bool = True
    ) -> Dict[str, List[Dict[str, Any]]]:
        """Discover and return employee-to-file mapping."""
        return self.discovery.get_employees_map(allow_network_scan=allow_network_scan)

    def _resolve_master_path(self) -> Path:
        """Resolve the output master report path."""
        fallback = Path(self.config_controller.config_path).parent / "Master_Report.xlsx"
        paths_cfg = self.config_controller.config.setdefault("paths", {})
        master_cfg = paths_cfg.get("master_file")
        if master_cfg:
            master_path = self.config_controller.resolve_path(str(master_cfg))
        else:
            master_path = fallback
            paths_cfg["master_file"] = str(fallback)
            save_config = getattr(self.config_controller, "save_config", None)
            if callable(save_config):
                save_config()
        try:
            master_path.parent.mkdir(parents=True, exist_ok=True)
            return master_path
        except OSError as e:
            logging.warning(
                "Master path unavailable (%s). Using fallback: %s",
                e,
                scrub_path(str(fallback)),
            )
            if Path(paths_cfg.get("master_file", "")) != fallback:
                paths_cfg["master_file"] = str(fallback)
                save_config = getattr(self.config_controller, "save_config", None)
                if callable(save_config):
                    save_config()
```

```
        return fallback

    def _get_curator_config(self, curator: Optional[str]) -> Optional[Dict[str, Any]]:
        if not curator:
            return None
        curators = self.config_controller.config.get("curators", {})
        if not isinstance(curators, dict):
            return None
        cfg = curators.get(curator)
        if not isinstance(cfg, dict) or not cfg.get("enabled", True):
            return None
        return cfg

    def _resolve_report_path(self, curator_config: Optional[Dict[str, Any]]) -> Path:
        if curator_config and curator_config.get("output_path"):
            return self.config_controller.resolve_path(str(curator_config["output_path"]))
        return self._resolve_master_path()

    def _is_report_writable(self, curator_config: Optional[Dict[str, Any]]) -> bool:
        if not curator_config:
            return check_master_lock(self.config_controller)
        path = self._resolve_report_path(curator_config)
        try:
            path.parent.mkdir(parents=True, exist_ok=True)
        except OSError:
            return False
        return is_file_writable(str(path))

    def _get_enabled_curator_employee_keys(self) -> set[str]:
        curators = self.config_controller.config.get("curators", {})
        if not isinstance(curators, dict):
            return set()

        keys = set()
        for cfg in curators.values():
            if not isinstance(cfg, dict) or not cfg.get("enabled", True):
                continue
            key = str(cfg.get("employees_key") or "").strip().lower()
            if key:
                keys.add(key)
        return keys

    @staticmethod
    def _employee_name_matches_any_key(name: str, keys: set[str]) -> bool:
        lowered_name = str(name).lower()
        return any(key in lowered_name for key in keys)

    @staticmethod
    def _filter_employees_for_curator(
        employees_map: Dict[str, List[Dict[str, Any]]],
        curator_config: Optional[Dict[str, Any]],
        general_excluded_keys: Optional[set[str]] = None,
    ) -> Dict[str, List[Dict[str, Any]]]:
        if not curator_config:
            excluded_keys = general_excluded_keys or set()
            if not excluded_keys:
                return employees_map
            return {
                name: metas
                for name, metas in employees_map.items()
                if not AppController._employee_name_matches_any_key(name, excluded_keys)
            }

        key = str(curator_config.get("employees_key") or "").strip().lower()
        if not key:
            return employees_map

        included_keys = {key}

        return {
            name: metas
            for name, metas in employees_map.items()
            if AppController._employee_name_matches_any_key(name, included_keys)
        }

    def _start_thread(self, target: Callable) -> None:
        """Start a task thread and track it for graceful shutdown."""
        t = threading.Thread(target=target, daemon=True)
        with self._lock:
            self._active_threads = [th for th in self._active_threads if th.is_alive()]
            self._active_threads.append(t)
        t.start()
```

```
    @staticmethod
    def _normalize_selected_datetime(selected_date) -> datetime:
        if isinstance(selected_date, datetime):
            return selected_date
        return datetime.combine(selected_date, datetime.min.time())

    def _get_network_drive_letters(self) -> tuple[str, ...]:
        cfg = getattr(self.config_controller, "config", {}) or {}
        network_cfg = cfg.get("network") if isinstance(cfg, dict) else {}
        raw_letters = (
            network_cfg.get("drive_letters", ("Q", "Z"))
            if isinstance(network_cfg, dict)
            else ("Q", "Z")
        )
        if isinstance(raw_letters, str):
            raw_letters = [raw_letters]
        letters = tuple(str(letter).strip() for letter in raw_letters if str(letter).strip())
        return letters or ("Q", "Z")

    @staticmethod
    def _has_target_date(
        available_dates: List[Any], target_date_obj: datetime, today: datetime
    ) -> bool:
        if target_date_obj.date() > today.date():
            return False
        for available_date in available_dates or []:
            if date_matches_target(available_date, target_date_obj):
                return True
        return False

    def _check_employee_refresh_status(
        self,
        name: str,
        employees_map: Dict[str, List[Dict[str, Any]]],
        target_date_obj: datetime,
        today: datetime,
    ) -> tuple[str, str, str, str]:
        metas = employees_map.get(name, [])
        paths = [m["path"] if isinstance(m, dict) else m for m in metas]
        paths = [p for p in paths if os.path.exists(p)]

        if not paths:
            return (name, " НЕТ ФАЙЛА", "---", "missing")

        path_str = (
            os.path.basename(paths[0]) if len(paths) == 1 else f"[{len(paths)} файлов]"
        )
        status, tag = " НЕТ ДАТЫ", "partial"

        try:
            forced_refresh_tokens = set()
            for p in paths:
                p_path = Path(p)
                stat = os.stat(p_path)
                stat_mtime = stat.st_mtime
                stat_size = stat.st_size
                cache = self.persistence.get_file_cache(str(p_path))

                cache_hit = (
                    cache
                    and cache["mtime"] == stat_mtime
                    and cache["size"] == stat_size
                )

                if cache_hit:
                    available = cache.get("dates", [])
                    if self._has_target_date(available, target_date_obj, today):
                        status, tag = " ГОТОВО", "ready"
                        break

                    force_token = (
                        str(p_path),
                        target_date_obj.strftime("%d.%m.%Y"),
                        stat_mtime,
                        stat_size,
                    )
                    if force_token not in forced_refresh_tokens:
                        forced_refresh_tokens.add(force_token)
                        available = self.ingestion.get_available_dates(
                            p_path,
                            remote_mtime=stat_mtime,
                            remote_size=stat_size,
                            strict_io=True,
```

```
                    )
                    self.persistence.save_file_cache(
                        str(p_path),
                        stat_mtime,
                        stat_size,
                        available,
                    )
            else:
                available = self.ingestion.get_available_dates(
                    p_path,
                    remote_mtime=stat_mtime,
                    remote_size=stat_size,
                    strict_io=True,
                )
                self.persistence.save_file_cache(
                    str(p_path),
                    stat_mtime,
                    stat_size,
                    available,
                )

            if self._has_target_date(available, target_date_obj, today):
                status, tag = " ГОТОВО", "ready"
                break
    except Exception as e:
        logging.exception("Error %s: %s", name, e)
        status, tag = "[ERR] ОШИБКА", "missing"

    return (name, status, path_str, tag)

def _run_refresh_status(
    self,
    selected_date,
    on_progress: Callable[[float], None],
    on_complete: Callable[[List[tuple], Dict, bool], None],
    on_error: Optional[Callable[[str], None]] = None,
) -> None:
    try:
        import time as _time
        _t0 = _time.monotonic()
        logging.info("[NET] Загружаю статусы файлов...")
        employees_map = self._get_employees_map(allow_network_scan=False)
        unique_names = sorted(employees_map.keys())
        if not unique_names:
            on_complete([], {}, False)
            return

        target_date_obj = self._normalize_selected_datetime(selected_date)
        today = datetime.now()
        total_count = len(unique_names)
        local_data: List[tuple] = []

        executor = self._executor
        futures = {
            executor.submit(
                self._check_employee_refresh_status,
                name,
                employees_map,
                target_date_obj,
                today,
            ): name
            for name in unique_names
        }
        try:
            for i, future in enumerate(
                concurrent.futures.as_completed(futures, timeout=30)
            ):
                local_data.append(future.result())
                on_progress(((i + 1) / total_count) * 100)
        except concurrent.futures.TimeoutError:
            # A single hung stat lookup (network drive stall, etc.)
            # used to block the whole refresh indefinitely because
            # as_completed had no per-future deadline. Cancel the
            # stragglers, surface which employees we skipped, and
            # keep going with whatever we collected so the user
            # sees a partial result instead of a frozen UI.
            timed_out = [n for fut, n in futures.items() if not fut.done()]
            logging.warning(
                "[NET] Refresh status timed out after 30s; "
                "%d/%d employees not finished: %s",
                len(timed_out),
                total_count,
                ", ".join(timed_out[:10])
```

```
                + ("..." if len(timed_out) > 10 else ""),
            )
            for fut in futures:
                if not fut.done():
                    fut.cancel()
            on_progress((len(local_data) / total_count) * 100)

        local_data.sort(key=lambda x: x[0])
        with self._lock:
            self._cached_employees = employees_map
        _elapsed = _time.monotonic() - _t0
        logging.info(f"[PERF] Refresh status: {_elapsed:.1f}s for {total_count} employees")
        on_complete(local_data, employees_map, self.rename_detected)
    except Exception as e:
        logging.exception("Refresh error: %s", e)
        on_complete([], {}, False)
        if callable(on_error):
            on_error(f"Ошибка обновления статусов: {e}")

@staticmethod
def _build_session_meta(
    session_type: str,
    target_date_str: Optional[str],
    start_date_str: Optional[str],
    end_date_str: Optional[str],
) -> Dict[str, Any]:
    meta: Dict[str, Any] = {"type": session_type}
    if target_date_str:
        meta["date"] = target_date_str
    if start_date_str and end_date_str:
        meta["start_date"] = start_date_str
        meta["end_date"] = end_date_str
    return meta

def _resolve_employees_for_run(self, use_fresh_employees: bool) -> Dict[str, List[Dict[str, Any]]]:
    if use_fresh_employees:
        employees_map = self._get_employees_map()
        with self._lock:
            self._cached_employees = employees_map
        return employees_map

    with self._lock:
        employees_map = dict(self._cached_employees) if self._cached_employees else None
    if employees_map:
        return employees_map

    employees_map = self._get_employees_map()
    with self._lock:
        if not self._cached_employees:
            self._cached_employees = employees_map
    return employees_map

@staticmethod
def _format_file_error_note(file_errors: List[Dict[str, str]]) -> Optional[str]:
    return format_controller_file_error_note(file_errors)

@staticmethod
def _build_empty_data_message(
    target_date_str: Optional[str],
    start_date_str: Optional[str],
    end_date_str: Optional[str],
    range_note: Optional[str],
    file_error_note: Optional[str],
) -> str:
    return build_controller_empty_data_message(
        target_date_str,
        start_date_str,
        end_date_str,
        range_note,
        file_error_note,
    )

def _build_no_files_message(self) -> str:
    raw_count = getattr(self.discovery, "last_discovered_count", 0)
    discovered_count = raw_count if isinstance(raw_count, int) else 0
    if discovered_count:
        return (
            f"Найдено {discovered_count} файл(ов), но 0 привязано к сотрудникам. "
            "Проверьте employees.yaml или привязки файлов."
        )
    return "Нет файлов"

@staticmethod
```

```
    def _build_success_message(
        session_type: str,
        start_date_str: Optional[str],
        end_date_str: Optional[str],
        range_note: Optional[str],
        file_error_note: Optional[str],
    ) -> str:
        return build_controller_success_message(
            session_type,
            start_date_str,
            end_date_str,
            range_note,
            file_error_note,
        )

    @staticmethod
    def _normalize_consolidation_dates(
        target_date_str: Optional[str],
        start_date_str: Optional[str],
        end_date_str: Optional[str],
    ) -> tuple[Optional[str], Optional[str], Optional[str], Optional[str], Optional[str]]:
        return normalize_controller_consolidation_dates(
            target_date_str,
            start_date_str,
            end_date_str,
        )

    def _execute_consolidation(
        self,
        on_progress: Callable[[float], None],
        on_complete: Callable[[bool, str, Optional[Path]], None],
        target_date_str: Optional[str],
        start_date_str: Optional[str],
        end_date_str: Optional[str],
        session_type: str,
        use_fresh_employees: bool,
        curator: Optional[str] = None,
        on_failure_code: Optional[Callable[[str], None]] = None,
        cancel_event: Optional[threading.Event] = None,
    ) -> None:
        import time as _time
        _t0 = _time.monotonic()
        session_id = str(uuid.uuid4())

        def _is_cancelled() -> bool:
            return cancel_event is not None and cancel_event.is_set()

        try:
            curator_config = self._get_curator_config(curator)
            effective_target_date, effective_start_date, effective_end_date, date_error, range_note = (
                self._normalize_consolidation_dates(
                    target_date_str,
                    start_date_str,
                    end_date_str,
                )
            )
            if date_error:
                if callable(on_failure_code):
                    on_failure_code("invalid_date")
                on_complete(False, date_error, None)
                return

            meta = self._build_session_meta(
                session_type,
                effective_target_date,
                effective_start_date,
                effective_end_date,
            )
            self.persistence.save_session(session_id, meta)

            employees_map = self._resolve_employees_for_run(use_fresh_employees)
            employees_map = self._filter_employees_for_curator(
                employees_map,
                curator_config,
                self._get_enabled_curator_employee_keys(),
            )
            if not employees_map:
                self.persistence.update_session_status(session_id, "failed")
                if callable(on_failure_code):
                    on_failure_code("no_files")
                on_complete(False, self._build_no_files_message(), None)
                return
```

```
file_errors: List[Dict[str, str]] = []

def collect_file_errors(errors: List[Dict[str, str]]) -> None:
    file_errors.extend(errors)

if not run_engine(
    self.persistence,
    self.ingestion,
    self.validator,
    employees_map,
    session_id,
    effective_target_date,
    on_progress,
    start_date_str=effective_start_date,
    end_date_str=effective_end_date,
    on_file_error=collect_file_errors,
    cancel_event=cancel_event,
):
    self.persistence.update_session_status(session_id, "failed")
    if callable(on_failure_code):
        on_failure_code("no_files")
    on_complete(False, self._build_no_files_message(), None)
    return

if _is_cancelled():
    self.persistence.update_session_status(session_id, "cancelled")
    _cancel_message(on_complete)
    return

consolidated_data = self.logic_service.consolidate(
    session_id,
    curator=curator,
    curator_config=curator_config,
)
# After the SQL aggregation step, push the bar to ~70% so the
# user can see we are in the final stretch (write_report
# is the last heavy step before completion).
on_progress(70)

if _is_cancelled():
    self.persistence.update_session_status(session_id, "cancelled")
    _cancel_message(on_complete)
    return
errors = self.persistence.get_template_errors(session_id)
file_error_note = self._format_file_error_note(file_errors)

if not consolidated_data:
    self.persistence.update_session_status(session_id, "failed")
    if callable(on_failure_code):
        on_failure_code("empty_data")
    on_complete(
        False,
        self._build_empty_data_message(
            effective_target_date,
            effective_start_date,
            effective_end_date,
            range_note,
            file_error_note,
        ),
        None,
    )
    return

master_path = self._resolve_report_path(curator_config)
write_kwargs: Dict[str, Any] = {}
if errors:
    write_kwargs["template_errors"] = errors
if curator_config:
    write_kwargs["curator"] = curator
    write_kwargs["curator_config"] = curator_config

if not self.writer.write_report(consolidated_data, master_path, **write_kwargs):
    self.persistence.update_session_status(session_id, "failed")
    if callable(on_failure_code):
        on_failure_code("write_error")
    on_complete(False, "Ошибка записи основного отчёта", None)
    return

logging.info("[MAIN] Основной отчёт собран: %s", scrub_path(str(master_path)))
_elapsed = _time.monotonic() - _t0
logging.info(f"[PERF] Consolidation ({session_type}): {_elapsed:.1f}s")
on_progress(95)
# `on_progress(100)` is intentionally left to the *completion
```

```
            # handler* — that is when the OperationState's end() resets
            # the bar to 0. Otherwise the bar reaches 100% for a single
            # frame and then immediately resets, which makes it look
            # like nothing happened.
            has_actionable_warnings = any(
                error.get("error_type") not in {"soft_structure"} for error in errors
            )
            has_runtime_warnings = bool(range_note or file_error_note)
            self.persistence.update_session_status(
                session_id,
                "completed_with_warnings"
                if has_actionable_warnings or has_runtime_warnings
                else "completed",
            )

            on_complete(
                True,
                self._build_success_message(
                    session_type,
                    effective_start_date,
                    effective_end_date,
                    range_note,
                    file_error_note,
                ),
                master_path,
            )
        except Exception as e:
            logging.exception("Error: %s", e)
            self.persistence.update_session_status(session_id, "failed")
            if callable(on_failure_code):
                on_failure_code("exception")
            on_complete(False, str(e), None)

    def refresh_status(
        self,
        selected_date,
        on_progress: Callable[[float], None],
        on_complete: Callable[[List[tuple], Dict, bool], None],
        curator=None,
        on_error: Optional[Callable[[str], None]] = None,
    ):
        """Refresh employee file status for the selected date."""
        if not check_master_lock(self.config_controller):
            on_complete([], {}, False)
            msg = "Мастер-файл занят в Excel. Закройте его и повторите."
            logging.error("Master file is locked. Please close it in Excel.")
            if callable(on_error):
                on_error(msg)
            return

        def task():
            self._run_refresh_status(
                selected_date,
                on_progress,
                on_complete,
                on_error=on_error,
            )

        self._start_thread(task)

    # ── Unified consolidation core (Item 6) ──────────────────────

    def _run_consolidation_core(
        self,
        on_progress: Callable[[float], None],
        on_complete: Callable[[bool, str, Optional[Path]], None],
        target_date_str: Optional[str] = None,
        start_date_str: Optional[str] = None,
        end_date_str: Optional[str] = None,
        session_type: str = "daily",
        use_fresh_employees: bool = False,
        curator: Optional[str] = None,
        cancel_event: Optional[threading.Event] = None,
    ):
        """Core consolidation logic shared by all consolidation entry points.

        Builds one Master report for all employees.

        Args:
            on_progress: Progress callback (0-100).
            on_complete: Completion callback (ok, message, path).
            target_date_str: Single date filter (dd.mm.YYYY) or None.
            start_date_str: Range start (dd.mm.YYYY) or None.
```

```
            end_date_str: Range end (dd.mm.YYYY) or None.
            session_type: 'daily', 'all_dates', or 'monthly'.
            use_fresh_employees: If True, re-discover employees from disk.
            cancel_event: optional threading.Event; if set, the task stops
                at the next checkpoint and reports cancelled to the caller.
        """
        curator_config = self._get_curator_config(curator)
        if not self._is_report_writable(curator_config):
            on_complete(
                False, "Файл отчёта занят в Excel! Закройте его и повторите.", None
            )
            return

        def task():
            self._execute_consolidation(
                on_progress,
                on_complete,
                target_date_str,
                start_date_str,
                end_date_str,
                session_type,
                use_fresh_employees,
                curator,
                cancel_event=cancel_event,
            )

        self._start_thread(task)

    def run_consolidation(
        self,
        selected_date,
        on_progress: Callable[[float], None],
        on_complete: Callable[[bool, str, Optional[Path]], None],
        curator: Optional[str] = None,
        cancel_event: Optional[threading.Event] = None,
    ):
        """Run consolidation for a single selected date."""
        self._run_consolidation_core(
            on_progress,
            on_complete,
            target_date_str=selected_date.strftime("%d.%m.%Y"),
            session_type="daily",
            curator=curator,
            cancel_event=cancel_event,
        )

    def run_consolidation_all_dates(
        self,
        on_progress: Callable[[float], None],
        on_complete: Callable[[bool, str, Optional[Path]], None],
        curator: Optional[str] = None,
        cancel_event: Optional[threading.Event] = None,
    ):
        """Run consolidation for all available dates."""
        self._run_consolidation_core(
            on_progress,
            on_complete,
            session_type="all_dates",
            use_fresh_employees=True,
            curator=curator,
            cancel_event=cancel_event,
        )

    def run_consolidation_month(
        self,
        start_date,
        end_date,
        on_progress: Callable[[float], None],
        on_complete: Callable[[bool, str, Optional[Path]], None],
        curator: Optional[str] = None,
        cancel_event: Optional[threading.Event] = None,
    ):
        """Run consolidation for a date range (monthly)."""
        self._run_consolidation_core(
            on_progress,
            on_complete,
            start_date_str=start_date.strftime("%d.%m.%Y"),
            end_date_str=end_date.strftime("%d.%m.%Y"),
            session_type="monthly",
            curator=curator,
            cancel_event=cancel_event,
        )
```

```
    def _run_consolidation_blocking(
        self,
        *,
        target_date_str: Optional[str],
        start_date_str: Optional[str],
        end_date_str: Optional[str],
        session_type: str,
        curator: Optional[str],
    ) -> Path:
        curator_config = self._get_curator_config(curator)
        if not self._is_report_writable(curator_config):
            label = curator or "общий"
            raise RuntimeError(f"Файл отчета {label} занят в Excel! Закройте его и повторите.")

        result: dict[str, Any] = {}

        def on_complete(ok: bool, msg: str, path: Optional[Path]) -> None:
            result["ok"] = ok
            result["msg"] = msg
            result["path"] = path

        def on_failure_code(code: str) -> None:
            result["code"] = code

        self._execute_consolidation(
            lambda _progress: None,
            on_complete,
            target_date_str,
            start_date_str,
            end_date_str,
            session_type,
            False,
            curator,
            on_failure_code=on_failure_code,
        )

        if not result.get("ok"):
            raise ConsolidationRunError(
                str(result.get("msg") or "Не удалось собрать отчет"),
                str(result.get("code") or "failed"),
            )
        path = result.get("path")
        if path is None:
            raise RuntimeError("Отчет собран, но путь к файлу не получен")
        return Path(path)

    def _write_empty_auto_reconciliation_report(self, curator: str, target_date) -> Path:
        curator_config = self._get_curator_config(curator)
        if not curator_config:
            raise RuntimeError(f"Не настроен отчет куратора {curator}")
        if not self._is_report_writable(curator_config):
            raise RuntimeError(f"Файл отчета {curator} занят в Excel! Закройте его и повторите.")

        output_path = self._resolve_report_path(curator_config)
        write_empty_report = getattr(self.writer, "write_empty_daily_report", None)
        if not callable(write_empty_report):
            raise RuntimeError("Экспортер не поддерживает пустой отчет для автосверки")

        if not write_empty_report(
            output_path,
            target_date,
            curator=curator,
            curator_config=curator_config,
        ):
            raise RuntimeError(f"Ошибка записи пустого отчета куратора {curator}")
        return output_path

    def _run_auto_master_report(
        self,
        *,
        label: str,
        curator: Optional[str],
        date_suffix: str,
        target_date,
    ) -> Path:
        try:
            return self._run_consolidation_blocking(
                target_date_str=date_suffix,
                start_date_str=None,
                end_date_str=None,
                session_type="daily",
                curator=curator,
            )
```

```
        except ConsolidationRunError as exc:
            if exc.code == "empty_data":
                if curator:
                    return self._write_empty_auto_reconciliation_report(curator, target_date)
                raise RuntimeError(f"{label}: Нет данных за {date_suffix}") from exc
            raise RuntimeError(f"{label}: {exc}") from exc
        except RuntimeError as exc:
            raise RuntimeError(f"{label}: {exc}") from exc

    def run_auto_reconciliation_day(
        self,
        source_paths: Path | Sequence[Path],
        output_path: Path,
        selected_date,
        on_complete: Callable[[bool, str, Optional[Path]], None],
        cancel_event: Optional[threading.Event] = None,
    ) -> None:
        """Rebuild daily reports, rebuild RSC, then write one 3-sheet reconciliation."""

        def task() -> None:
            try:
                target_date = selected_date.date() if hasattr(selected_date, "date") else selected_date
                date_suffix = target_date.strftime("%d.%m.%Y")
                output_path_resolved = Path(output_path)
                rsch_output = output_path_resolved.parent / f"РСЧ_отчет_{date_suffix}.xlsx"

                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, output_path_resolved, rsch_output)
                    return

                master_paths = {
                    "Общий": self._run_auto_master_report(
                        label="Общий",
                        curator=None,
                        date_suffix=date_suffix,
                        target_date=target_date,
                    ),
                    "Борис": self._run_auto_master_report(
                        label="Борис",
                        curator="boris",
                        date_suffix=date_suffix,
                        target_date=target_date,
                    ),
                    "Иван": self._run_auto_master_report(
                        label="Иван",
                        curator="ivan",
                        date_suffix=date_suffix,
                        target_date=target_date,
                    ),
                }

                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, output_path_resolved, rsch_output)
                    return

                if isinstance(source_paths, (str, Path)):
                    normalized_sources = [Path(source_paths)]
                else:
                    normalized_sources = [Path(p) for p in source_paths]

                issues: List[str] = []
                rsch_rows = build_rsch_report(
                    normalized_sources,
                    rsch_output,
                    target_date=target_date,
                    issues_sink=issues,
                    drive_letters=self._get_network_drive_letters(),
                )
                if rsch_rows == 0:
                    on_complete(
                        True,
                        f"Нет данных РСЧ за {date_suffix}. Автосверка пропущена.",
                        None,
                    )
                    return

                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, output_path_resolved, rsch_output)
                    return

                try:
                    counts = build_combined_reconciliation_report(
                        master_paths,
```

```
                    rsch_output,
                    output_path_resolved,
                    target_date=target_date,
                    drive_letters=self._get_network_drive_letters(),
                    alias_repo=self.alias_repo,
                )
            except ValueError as e:
                on_complete(
                    True,
                    f"Сверка за {date_suffix}: {e}",
                    None,
                )
                return
            issue_note = (
                f" Есть проблемы с листами РСЧ: {build_issue_preview(issues)}"
                if issues
                else ""
            )
            msg = (
                "Автосверка собрана: "
                f"Общий {counts.get('Общий', 0)}, "
                f"Борис {counts.get('Борис', 0)}, "
                f"Иван {counts.get('Иван', 0)} строк. "
                f"РСЧ: {rsch_rows} строк."
                f"{issue_note}"
            )
            on_complete(True, msg, output_path_resolved)
        except Exception as e:
            logging.exception("Auto reconciliation error: %s", e)
            on_complete(False, f"Ошибка автоматической сверки: {e}", None)

    self._start_thread(task)

def run_reconciliation_report(
    self,
    master_paths: Dict[str, Path],
    rsch_path: Path,
    output_path: Path,
    selected_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
    """Build a reconciliation report from already selected report and RSC files."""
    self._reconciliation.run_reconciliation_report(
        master_paths, rsch_path, output_path, selected_date, on_complete, cancel_event
    )

def run_rsch_report(
    self,
    source_paths: Path | Sequence[Path],
    output_path: Path,
    selected_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
    """Build an RSC control report from selected workbooks."""
    self._rsch.run_rsch_report(
        source_paths, output_path, selected_date, on_complete, cancel_event
    )

def run_rsch_report_period(
    self,
    source_paths: Path | Sequence[Path],
    output_path: Path,
    start_date,
    end_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
    """Build one RSC control workbook with one sheet per row date in range."""
    self._rsch.run_rsch_report_period(
        source_paths, output_path, start_date, end_date, on_complete, cancel_event
    )

def run_rsch_report_period_batch(
    self,
    source_paths: Path | Sequence[Path],
    output_dir: Path,
    start_date,
    end_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
```

```
        """Build separate RSC workbooks for each date in the selected range."""
        self._rsch.run_rsch_report_period_batch(
            source_paths, output_dir, start_date, end_date, on_complete, cancel_event
        )

    def run_rsch_report_batch(
        self,
        source_paths: Sequence[Path],
        output_paths: Sequence[Path],
        selected_date,
        on_complete: Callable[[bool, str, Optional[Path]], None],
        cancel_event: Optional[threading.Event] = None,
    ) -> None:
        """Build separate RSC reports per source workbook."""
        self._rsch.run_rsch_report_batch(
            source_paths, output_paths, selected_date, on_complete, cancel_event
        )

    def shutdown(self):
        """Gracefully shut down: wait for threads and close DB."""
        logging.info("[APP] Завершение: ожидаю завершения записей в БД...")
        self.persistence.wait_for_writes()

        # Shutdown thread pool executor
        self._executor.shutdown(wait=True)
        logging.info("[APP] Завершение: пул потоков остановлен.")

        # Wait for active task threads to finish (with timeout)
        with self._lock:
            threads = list(self._active_threads)
        for t in threads:
            t.join(timeout=10)

        self.persistence.close()
        logging.info("[APP] Завершение: БД закрыта.")

    # ── Customer aliases (reconciliation mapping) ──────────

    def get_customer_aliases(self):
        """Return all customer alias entries for UI display."""
        if self.alias_repo is None:
            return []
        return self.alias_repo.get_all_aliases()

    def add_customer_alias(self, master_customer: str, rsch_customer: str, notes: str = "") -> int:
        """Add a new customer alias mapping. Returns new alias id."""
        if self.alias_repo is None:
            raise RuntimeError("Алиасы не настроены")
        return self.alias_repo.add_alias(master_customer, rsch_customer, notes)

    def update_customer_alias(
        self, alias_id: int, master_customer: str, rsch_customer: str, notes: str = ""
    ) -> None:
        """Update an existing customer alias."""
        if self.alias_repo is None:
            raise RuntimeError("Алиасы не настроены")
        self.alias_repo.update_alias(alias_id, master_customer, rsch_customer, notes)

    def delete_customer_alias(self, alias_id: int) -> None:
        """Delete a customer alias by id."""
        if self.alias_repo is None:
            raise RuntimeError("Алиасы не настроены")
        self.alias_repo.delete_alias(alias_id)
```

APP\core\bootstrap.py

477 lines • 18,719 bytes

```
"""Bootstrap — Composition Root для Dependency Injection."""

import logging
import importlib.util
import re
import sqlite3
from pathlib import Path, PureWindowsPath
from typing import Tuple, Optional, Dict, Any, List

from pydantic import BaseModel

from core.infra import ConfigController, MasterWriter
from core.infra.utils import path_exists_with_timeout
from core.infra.ingestion_engine import IngestionEngine
from core.infra.session_repo import SessionRepository as SQLiteManager
from core.domain import LogicService, Validator
from core.app_controller import AppController


# ── Item 8: Pydantic config validation ──────────────────────


class PathsConfig(BaseModel):
    """Validates the 'paths' section of config."""

    master_file: str = "Master_Report.xlsx"
    source_root: str = "REPORTS"
    source_roots: Optional[List[str]] = None


class DatabaseConfig(BaseModel):
    """Validates the 'database' section."""

    path: str = "cache.db"
    schema_path: str = "db_schema.sql"
    timeout: float = 30.0
    allow_local_fallback: bool = False


class LockingConfig(BaseModel):
    """Validates the 'locking' section."""

    retry_count: int = 3
    retry_backoff: float = 1.0


class NetworkConfig(BaseModel):
    """Validates the 'network' section."""

    drive_letters: List[str] = ["Q", "Z"]


class AppConfig(BaseModel):
    """Top-level config model with safe defaults."""

    paths: PathsConfig = PathsConfig()
    database: DatabaseConfig = DatabaseConfig()
    locking: LockingConfig = LockingConfig()
    network: NetworkConfig = NetworkConfig()
    sh1: Dict[str, Any] = {}
    logic: Dict[str, Any] = {}
    curators: Dict[str, Any] = {}
    validation: Dict[str, Any] = {}

    model_config = {"extra": "allow"}


# ── Item 10: Path traversal protection ──────────────────────


def _validate_path_safety(
    resolved: Path, app_root: Path, allow_external: bool = False
) -> Path:
    """Ensure resolved path does not escape the app_root via traversal.

    Returns the resolved path, or raises ValueError if traversal detected.
    """
    try:
        resolved_abs = resolved.resolve()
        app_root_abs = app_root.resolve()
        if resolved_abs.is_relative_to(app_root_abs):
            return resolved_abs
        if allow_external:
```

```
            return resolved_abs
        raise ValueError(
            f"Path traversal detected: '{resolved}' escapes app root '{app_root}'"
        )
    except OSError as os_err:
        # On OSError (e.g. network path unavailable), fall back to strict string check
        resolved_str = str(resolved).lower().rstrip("\\/")
        app_root_str = str(app_root).lower().rstrip("\\/")
        if not resolved_str.startswith(app_root_str):
            raise ValueError(
                f"Path traversal detected: '{resolved}' escapes app root '{app_root}'"
            ) from os_err
        return resolved


def _is_effectively_absolute(path_str: str) -> bool:
    p = Path(path_str)
    if p.is_absolute():
        return True
    if path_str.startswith("\\\\"):
        return True
    return bool(re.match(r"^[A-Za-z]:[\\/]", path_str))


def _can_open_sqlite_path(db_path: Path, timeout: float) -> tuple[bool, str | None]:
    try:
        db_path.parent.mkdir(parents=True, exist_ok=True)
    except OSError as e:
        return False, f"parent dir error: {e}"

    conn = None
    try:
        conn = sqlite3.connect(db_path, timeout=timeout)
        return True, None
    except sqlite3.OperationalError as e:
        return False, str(e)
    finally:
        if conn is not None:
            conn.close()


def _is_stale_app_local_path(
    path_str: str, app_root: Path, default_filename: str
) -> bool:
    """Detect app-local paths persisted from another installation directory."""
    if not _is_effectively_absolute(path_str):
        return False

    for parsed_path in (Path(path_str), PureWindowsPath(path_str)):
        if parsed_path.name.lower() != default_filename.lower():
            continue
        if parsed_path.parent.name.lower() == app_root.name.lower():
            return True
    return False


def _is_current_app_local_default_path(
    path_str: str, app_root: Path, default_filename: str
) -> bool:
    if not _is_effectively_absolute(path_str):
        return False
    try:
        return Path(path_str).resolve() == (app_root / default_filename).resolve()
    except OSError:
        return Path(path_str).name.lower() == default_filename.lower()


def _check_db_integrity_on_startup(persistence) -> None:
    """Run PRAGMA integrity_check and warn on corruption.

    Only called during GUI startup.  If the database file was left in an
    inconsistent state (e.g. after a hard shutdown during a write), the
    check will surface a visible warning so the user can restore from
    backup or regenerate the index.

    The warning message is also stashed on ``Bootstrap._integrity_warning``
    so that the GUI layer (main.py) can surface it via a messagebox after
    the Tk root has been created. Headless callers already get the log
    line.
    """
    try:
        ok, msg = persistence.connection_manager.check_integrity()
        if not ok:
```

```
            warning = (
                "PRAGMA integrity_check failed: "
                f"{msg}. "
                "Data may be corrupted — consider restoring from backup."
            )
            logging.error(warning)
            Bootstrap._integrity_warning = warning
    except Exception:
        pass  # never block startup on integrity check failures


class Bootstrap:
    """Статический класс для инициализации всех зависимостей приложения."""

    # Surfaced by _check_db_integrity_on_startup when PRAGMA
    # integrity_check fails. main.py reads this after creating the Tk
    # root and shows a messagebox. Headless callers already get the
    # log line.
    _integrity_warning: Optional[str] = None

    @staticmethod
    def _has_required_dependency() -> bool:
        return importlib.util.find_spec("python_calamine") is not None

    @staticmethod
    def _coerce_raw_config(raw_cfg: Any) -> Dict[str, Any]:
        if not isinstance(raw_cfg, dict):
            logging.warning("Config is not a dict — using safe defaults.")
            return {}
        return dict(raw_cfg)

    @staticmethod
    def _validate_section(
        raw_cfg: Dict[str, Any], section_name: str, model_cls: type[BaseModel]
    ) -> Dict[str, Any]:
        section_raw = raw_cfg.get(section_name, {})
        if section_raw is None:
            section_raw = {}
        if not isinstance(section_raw, dict):
            logging.warning(
                "Config validation warning: %s is not a dict. Using safe defaults.",
                section_name,
            )
            return model_cls().model_dump(exclude_none=True)

        validated = model_cls().model_dump(exclude_none=True)
        extras = {
            key: value
            for key, value in section_raw.items()
            if key not in model_cls.model_fields
        }

        for key in model_cls.model_fields:
            if key not in section_raw:
                continue
            try:
                partial = model_cls.model_validate({key: section_raw[key]})
                partial_dump = partial.model_dump(exclude_none=True)
                if key in partial_dump:
                    validated[key] = partial_dump[key]
            except Exception as e:
                logging.warning(
                    "Config validation warning for %s.%s: %s. Using default value.",
                    section_name,
                    key,
                    e,
                )

        validated.update(extras)
        return validated

    @staticmethod
    def _validate_and_normalize_config(
        cfg_ctl: ConfigController,
    ) -> tuple[Dict[str, Any], Dict[str, Any]]:
        raw_cfg = cfg_ctl.config
        raw_cfg_dict = raw_cfg if isinstance(raw_cfg, dict) else {}
        cfg = Bootstrap._coerce_raw_config(raw_cfg)

        cfg["paths"] = Bootstrap._validate_section(raw_cfg_dict, "paths", PathsConfig)
        cfg["database"] = Bootstrap._validate_section(
            raw_cfg_dict, "database", DatabaseConfig
        )
```

```
        cfg["locking"] = Bootstrap._validate_section(
            raw_cfg_dict, "locking", LockingConfig
        )
        cfg["network"] = Bootstrap._validate_section(raw_cfg_dict, "network", NetworkConfig)

        if not isinstance(cfg.get("sh1"), dict):
            cfg["sh1"] = {}
        if not isinstance(cfg.get("logic"), dict):
            cfg["logic"] = {}
        if not isinstance(cfg.get("curators"), dict):
            cfg["curators"] = {}
        if not isinstance(cfg.get("validation"), dict):
            cfg["validation"] = {}

        cfg.setdefault("paths", {})
        cfg.setdefault("sh1", {})
        cfg.setdefault("logic", {})
        cfg.setdefault("curators", {})

        cfg_ctl.config = cfg
        return cfg, raw_cfg_dict

    @staticmethod
    def _resolve_rel_factory(app_root: Path):
        def resolve_rel(p: Optional[str], default_filename: Optional[str] = None) -> str:
            if not p:
                if default_filename:
                    return str(app_root / default_filename)
                return ""
            path = Path(p)
            allow_external = _is_effectively_absolute(p)
            resolved = path if allow_external else app_root / path
            resolved = _validate_path_safety(resolved, app_root, allow_external)
            return str(resolved)

        return resolve_rel

    @staticmethod
    def _resolve_database_paths(
        cfg: Dict[str, Any], app_root: Path, resolve_rel
    ) -> tuple[str, str, float, bool]:
        persist_recovered_paths = False
        db_cfg = cfg.setdefault("database", {})
        raw_db_path = db_cfg.get("path", "cache.db")
        if raw_db_path and _is_stale_app_local_path(str(raw_db_path), app_root, "cache.db"):
            raw_db_path = "cache.db"
            persist_recovered_paths = True
        db_path = db_cfg["path"] = resolve_rel(raw_db_path, "cache.db")

        raw_schema_path = db_cfg.get("schema_path", "db_schema.sql")
        if raw_schema_path and _is_stale_app_local_path(
            str(raw_schema_path), app_root, "db_schema.sql"
        ):
            raw_schema_path = "db_schema.sql"
            persist_recovered_paths = True
        schema_path = db_cfg["schema_path"] = resolve_rel(
            raw_schema_path, "db_schema.sql"
        )
        db_timeout = float(db_cfg.get("timeout", 30.0))

        db_ok, db_error = _can_open_sqlite_path(Path(db_path), db_timeout)
        if not db_ok:
            if not bool(db_cfg.get("allow_local_fallback", False)):
                raise RuntimeError(
                    f"Configured DB path is not writable: {db_path} ({db_error}). "
                    "Refusing local fallback in production. Set "
                    "database.allow_local_fallback: true for dev/test fallback."
                )
            logging.warning(
                "Configured DB path is not writable (%s). Falling back to local app DB.",
                db_error,
            )
            fallback_db = str(app_root / "cache.db")
            if db_path != fallback_db:
                persist_recovered_paths = True
            db_path = db_cfg["path"] = fallback_db

        schema_candidate = Path(schema_path)
        if not schema_candidate.exists():
            fallback_schema = app_root / "db_schema.sql"
            if fallback_schema.exists():
                logging.warning(
                    "Configured schema path missing: %s. Using fallback: %s",
```

```
                schema_candidate,
                fallback_schema,
            )
            fallback_schema_str = str(fallback_schema)
            if schema_path != fallback_schema_str:
                persist_recovered_paths = True
            schema_path = db_cfg["schema_path"] = fallback_schema_str

    return db_path, schema_path, db_timeout, persist_recovered_paths

@staticmethod
def _save_portable_runtime_config(cfg_ctl: ConfigController, app_root: Path) -> None:
    """Persist app-local defaults as relative values, while keeping runtime cfg absolute."""
    save_config = getattr(cfg_ctl, "save_config", None)
    if not callable(save_config):
        return

    cfg = cfg_ctl.config
    if not isinstance(cfg, dict):
        save_config()
        return

    db_cfg = cfg.get("database")
    if not isinstance(db_cfg, dict):
        save_config()
        return

    original_path = db_cfg.get("path")
    original_schema = db_cfg.get("schema_path")
    try:
        if original_path and (
            _is_stale_app_local_path(str(original_path), app_root, "cache.db")
            or _is_current_app_local_default_path(str(original_path), app_root, "cache.db")
        ):
            db_cfg["path"] = "cache.db"
        if original_schema and (
            _is_stale_app_local_path(str(original_schema), app_root, "db_schema.sql")
            or _is_current_app_local_default_path(
                str(original_schema), app_root, "db_schema.sql"
            )
        ):
            db_cfg["schema_path"] = "db_schema.sql"
        save_config()
    finally:
        db_cfg["path"] = original_path
        db_cfg["schema_path"] = original_schema

@staticmethod
def _resolve_path_config(
    cfg: Dict[str, Any], raw_cfg: Dict[str, Any], app_root: Path, resolve_rel
) -> None:
    p_cfg = cfg.get("paths", {})
    raw_paths = raw_cfg.get("paths", {}) if isinstance(raw_cfg, dict) else {}
    if not isinstance(raw_paths, dict):
        raw_paths = {}

    master_base = app_root.parent if (app_root / "main.py").exists() else app_root
    # Don't override user's explicit master_file setting
    if "master_file" not in raw_paths or not raw_paths.get("master_file"):
        p_cfg["master_file"] = str(master_base / "Master_Report.xlsx")
    p_cfg["source_root"] = resolve_rel(p_cfg.get("source_root", "REPORTS"), "REPORTS")
    if "source_roots" in p_cfg and isinstance(p_cfg["source_roots"], list):
        p_cfg["source_roots"] = [resolve_rel(r) for r in p_cfg["source_roots"]]

    sr = p_cfg.get("source_root")
    raw_sr = str(raw_paths.get("source_root", "")).replace("\\", "/").lower().strip()
    source_root_is_default = "source_root" not in raw_paths or raw_sr in {
        "",
        "reports",
        "./reports",
    }

    if source_root_is_default and sr and not path_exists_with_timeout(sr):
        # Don't silently invent a Q:/ fallback — break on non-Windows /
        # non-mapped-Q systems. Downstream DiscoveryService already
        # auto-detects configured network drive_letters (Q/Z) when
        # source_roots is empty, so leaving it empty is safe.
        logging.warning(
            "source_root по умолчанию (%s) недоступен и source_roots не "
            "заданы в config.yaml. Приложение продолжит работу, но "
            "реальные отчёты могут быть не найдены. Настройте "
            "paths.source_roots в config.yaml.",
            sr,
```

```
        )

    @staticmethod
    def _build_runtime_components(
        cfg_ctl: ConfigController,
        cfg: Dict[str, Any],
        db_path: str,
        schema_path: str,
        config_path: str,
    ):
        drive_letters = tuple(cfg.get("network", {}).get("drive_letters", ["Q", "Z"]))
        ingestion = IngestionEngine(cfg.get("sh1", {}).get("markers", {}), drive_letters)
        ingestion.config = cfg
        persistence = SQLiteManager(db_path, schema_path, cfg)
        _check_db_integrity_on_startup(persistence)
        from core.infra.customer_alias_repo import CustomerAliasRepository
        alias_repo = CustomerAliasRepository(persistence.connection_manager)
        validator = Validator(cfg)
        logic = LogicService(persistence, cfg)
        writer = MasterWriter(cfg, config_path)
        app_ctl = AppController(cfg_ctl, ingestion, persistence, validator, logic, writer, alias_repo=alias_repo)
        return cfg_ctl, app_ctl, persistence

    @staticmethod
    def setup(
        app_root: Path, config_path: str
    ) -> Tuple[
        Optional[ConfigController], Optional[AppController], Optional[SQLiteManager]
    ]:
        """Инициализировать все зависимости и вернуть контроллеры.

        Returns:
            (config_controller, app_controller, persistence).
        Raises:
            RuntimeError: если критическая зависимость (python-calamine)
                не найдена. Вызывающий код должен перехватить и показать
                пользователю дружелюбное сообщение.
        """
        if not Bootstrap._has_required_dependency():
            raise RuntimeError(
                "Не найдена обязательная библиотека 'python-calamine'. "
                "Установите её: pip install python-calamine==0.6.2"
            )

        cfg_ctl = ConfigController(config_path)
        cfg, raw_cfg = Bootstrap._validate_and_normalize_config(cfg_ctl)
        resolve_rel = Bootstrap._resolve_rel_factory(app_root)
        db_path, schema_path, _db_timeout, persist_recovered_paths = (
            Bootstrap._resolve_database_paths(cfg, app_root, resolve_rel)
        )

        if persist_recovered_paths:
            Bootstrap._save_portable_runtime_config(cfg_ctl, app_root)

        Bootstrap._resolve_path_config(cfg, raw_cfg, app_root, resolve_rel)
        return Bootstrap._build_runtime_components(
            cfg_ctl,
            cfg,
            db_path,
            schema_path,
            config_path,
        )
```

APP\core\domain__init__.py

17 lines • 564 bytes

```
from core.domain.interfaces import (
    IDataSource as IDataSource,
    IPersistence as IPersistence,
    IValidator as IValidator,
    ILogicService as ILogicService,
    IExporter as IExporter,
)
from core.domain.utils_helpers import (
    NumericParsingError as NumericParsingError,
    to_decimal as to_decimal,
    to_decimal_percent as to_decimal_percent,
)
from core.domain.data_schemas import FinancialRowSchema as FinancialRowSchema
from core.domain.logic_service import (
    LogicService as LogicService,
    Validator as Validator,
)
```

APP\core\domain\data_loss_log.py

126 lines • 4,411 bytes

```
"""Data loss audit logger.

Records every place where user data is silently dropped or coerced so that
operators can spot patterns (broken template, wrong column name, network glitch).
"""
import json
import logging
import os
import re
import threading
from datetime import datetime, timezone
from pathlib import Path

from core.infra.utils_logging import scrub_path

_LOCK = threading.Lock()
_LOGGER_NAME = "data_loss"

_initialised = False

# Redact obvious monetary amounts (e.g. "1234.56", "99,50") before writing
# to disk so the log file does not accumulate financial PII in plaintext.
# Require 3+ digits before the separator to avoid false positives on dates
# (e.g. "12.05.2026"), versions ("v5.50"), or file names ("file_12.05.xlsx").
_AMOUNT_RE = re.compile(r"\d{3,}[.,]\d{2}")


def _project_root() -> Path:
    # Use CWD during pytest runs so that monkeypatch.chdir in tests works correctly.
    # Otherwise, use the package root (APP/) derived from this file's location to prevent
    # log files from being created in system directories (e.g. C:\Windows\System32) when
    # launched via Windows shortcuts.
    if "PYTEST_CURRENT_TEST" in os.environ:
        return Path(os.getcwd())
    return Path(__file__).resolve().parents[2]


def _get_logger() -> logging.Logger:
    global _initialised
    log = logging.getLogger(_LOGGER_NAME)
    if _initialised:
        return log
    log.setLevel(logging.INFO)
    log.propagate = False
    log_dir = _project_root() / "logs"
    try:
        log_dir.mkdir(parents=True, exist_ok=True)
        log_path = log_dir / "data_loss.log"
        fh = logging.FileHandler(log_path, encoding="utf-8")
        fh.setFormatter(
            logging.Formatter("%(asctime)s %(message)s", "%Y-%m-%dT%H:%M:%S")
        )
        log.addHandler(fh)
        # Restrict to owner-only — the log captures Excel-derived values
        # (paths, amounts) that must not be world-readable on shared PCs.
        try:
            os.chmod(log_path, 0o600)
        except OSError as chmod_err:
            logging.warning(
                "Could not restrict %s permissions: %s", log_path, chmod_err
            )
    except OSError:
        pass
    _initialised = True
    return log


def record_loss(
    *,
    source: str,
    kind: str,
    reason: str,
    row_index: int | None = None,
    sheet: str | None = None,
    field: str | None = None,
    original_value=None,
) -> None:
    """Append a structured data-loss event to logs/data_loss.log.

    Args:
        source: file path or pipeline stage identifier.
        kind: short category (e.g. ``"trash_filter"``, ``"numeric_coercion"``).
        reason: human-readable explanation.
```

```
        row_index: 1-based row number in the source sheet, if known.
        sheet: sheet name, if known.
        field: field/key being dropped, if known.
        original_value: the value that was lost (will be coerced to str).
    """
    payload = {
        "ts": datetime.now(timezone.utc).isoformat(timespec="seconds") + "Z",
        "source": source,
        "kind": kind,
        "reason": reason,
        "row": row_index,
        "sheet": sheet,
        "field": field,
        "value": str(original_value) if original_value is not None else None,
    }
    try:
        line = json.dumps(payload, ensure_ascii=False)
    except (TypeError, ValueError):
        line = json.dumps({k: str(v) for k, v in payload.items()}, ensure_ascii=False)
    # Defense-in-depth: redact Windows user paths / UNC shares and obvious
    # monetary amounts before the line hits disk. Does NOT change the JSON
    # structure or the kind/reason fields — only PII-bearing substrings.
    line = _AMOUNT_RE.sub("[AMOUNT]", scrub_path(line))
    log = _get_logger()
    with _LOCK:
        log.info(line)


def reset_for_testing() -> None:
    """Drop cached handler so the next call re-binds to the current CWD.

    Tests that change the working directory (e.g. via ``monkeypatch.chdir``)
    call this to make the logger pick up the new project root.
    """
    global _initialised
    log = logging.getLogger(_LOGGER_NAME)
    for handler in list(log.handlers):
        try:
            handler.close()
        except Exception:
            pass
        log.removeHandler(handler)
    _initialised = False
```

APP\core\domain\data_schemas.py

91 lines • 3,612 bytes

```
from decimal import Decimal
from typing import Optional, Any
from pydantic import BaseModel, Field, field_validator, ConfigDict
from datetime import datetime


class FinancialRowSchema(BaseModel):
    model_config = ConfigDict(coerce_numbers_to_str=True, extra="allow")

    date: Optional[str] = None
    customer: Optional[str] = Field(None, alias="customer")
    contractor: Optional[str] = Field(None, alias="contractor")
    amount: Decimal = Decimal("0.00")
    income: Decimal = Decimal("0.00")
    expense: Decimal = Decimal("0.00")
    total: Decimal = Decimal("0.00")
    transit_obligation: Decimal = Decimal("0.00")
    commission: Decimal = Decimal("0.00")
    percent_broker: Any = Decimal("0.00")
    broker1: Decimal = Decimal("0.00")
    broker2: Decimal = Decimal("0.00")
    percent_broker2: Any = Decimal("0.00")
    exchange_rate: Decimal = Decimal("0.00")
    currency_amount: Decimal = Decimal("0.00")
    currency: Optional[str] = ""
    filters: Optional[str] = ""

    _sheet_name: Optional[str] = None
    _row_index: Optional[int] = None
    _file_path: Optional[str] = None
    is_template_warning: bool = False

    @field_validator("date", mode="before")
    @classmethod
    def validate_date(cls, v):
        if v is None:
            return None
        try:
            datetime.strptime(str(v), "%d.%m.%Y")
            return str(v)
        except ValueError:
            # If it's not in DD.MM.YYYY, we don't fail, but we might want to log it.
            # For now, let's just return it as is if it's a string, or None if it's invalid.
            return str(v)

    @field_validator(
        "amount",
        "income",
        "expense",
        "total",
        "transit_obligation",
        "commission",
        "percent_broker",
        "broker1",
        "broker2",
        "percent_broker2",
        "exchange_rate",
        "currency_amount",
        mode="before",
    )
    @classmethod
    def to_decimal(cls, v):
        if v is None or v == "":
            return Decimal("0.00")
        if isinstance(v, Decimal):
            return v
        try:
            # Handle string with comma, spaces, etc.
            s = str(v).replace(" ", "").replace(",", ".")
            # Remove any non-numeric characters except . and -
            s_clean = "".join(c for c in s if c.isdigit() or c in ".-")
            if not s_clean or s_clean == "-":
                # Unparseable non-empty value (e.g. a text like "high").
                # Previously this branch returned the raw value, leaking a
                # str into Decimal-typed fields and breaking the type
                # contract. Downstream parsers (to_decimal_percent) already
                # coerce any input to Decimal, so returning 0 here is safe
                # and keeps the field type honest.
                return Decimal("0.00")
            # Sanitize: remove extra dots/minus signs to avoid invalid Decimal
            parts = s_clean.split(".")
            if len(parts) > 2:
                s_clean = parts[0] + "." + "".join(parts[1:])
```

```
        if s_clean.count("-") > 1:
            s_clean = ("-" if s_clean.startswith("-") else "") + s_clean.replace("-", "")
        return Decimal(s_clean)
    except (ValueError, TypeError, ArithmeticError):
        # Never return the raw value: that violated the Decimal contract
        # for typed fields. Log callers can still recover the original
        # from the source row if needed.
        return Decimal("0.00")
```

APP\core\domain\interfaces.py

150 lines • 4,575 bytes

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Optional
from pathlib import Path


class IDataSource(ABC):
    """Interface for scanning and reading source Excel reports."""

    @abstractmethod
    def scan_directory(self, root_path: Path) -> List[Dict[str, Any]]:
        """Scans directory and returns list of candidate file metadata."""
        pass

    @abstractmethod
    def read_file(
        self, file_path: Path, strict_io: bool = False
    ) -> List[Dict[str, Any]]:
        """Reads Excel file and returns raw rows.

        strict_io=True may raise I/O errors after retry exhaustion;
        strict_io=False should degrade to [] on transient read failures.
        """
        pass

    @abstractmethod
    def get_available_dates(
        self,
        file_path: Path,
        remote_mtime: float | None = None,
        remote_size: int | None = None,
        strict_io: bool = False,
    ) -> List[str]:
        """Returns detected sheet dates from a source workbook.

        Optional remote_mtime/remote_size allow callers to avoid duplicate
        network stat calls when metadata is already known.
        strict_io=True may raise read errors instead of degrading to [].
        """
        pass


class IPersistence(ABC):
    """Interface for database operations (SQLite)."""

    @abstractmethod
    def init_db(self) -> None:
        """Initializes database tables and indexes."""
        pass

    @abstractmethod
    def save_session(self, session_id: str, metadata: Dict[str, Any]) -> None:
        """Saves session start information."""
        pass

    @abstractmethod
    def save_file_metadata(
        self, session_id: str, files: List[Dict[str, Any]]
    ) -> List[int]:
        """Saves file metadata and returns generated IDs."""
        pass

    @abstractmethod
    def save_staging_data(self, file_id: int, rows: List[Dict[str, Any]]) -> None:
        """Saves raw rows into staging_data table."""
        pass

    @abstractmethod
    def save_template_error(self, session_id: str, error_info: Dict[str, Any]) -> None:
        """Saves a template violation or warning."""
        pass

    def save_template_errors_batch(
        self, session_id: str, errors: List[Dict[str, Any]]
    ) -> None:
        """Saves a batch of template violations or warnings."""
        for error_info in errors:
            self.save_template_error(session_id, error_info)



    @abstractmethod
    def get_template_errors(self, session_id: str) -> List[Dict[str, Any]]:
        """Retrieves all template errors for a session."""
```

```
        pass

    @abstractmethod
    def get_session_status(self, session_id: str) -> Dict[str, Any]:
        """Retrieves session progress and stats."""
        pass

    @abstractmethod
    def update_session_status(self, session_id: str, status: str) -> None:
        """Updates session lifecycle status."""
        pass

    @abstractmethod
    def update_file_status(
        self, file_id: int, status: str, error_message: Optional[str] = None
    ) -> None:
        """Updates file processing status and optional error message."""
        pass

    @abstractmethod
    def get_consolidated_data(self, session_id: str) -> List[Dict[str, Any]]:
        """Retrieves aggregated data grouped by employee and date."""
        pass

    @abstractmethod
    def close(self) -> None:
        """Closes database connections."""
        pass


class IValidator(ABC):
    """Interface for row-level validation and junk detection."""

    @abstractmethod
    def validate_row(self, row: Dict[str, Any]) -> bool:
        """Returns True if row is valid, False otherwise."""
        pass

    @abstractmethod
    def get_errors(self) -> List[str]:
        """Returns list of validation errors for the last processed row."""
        pass


class ILogicService(ABC):
    """Interface for financial transformations and consolidation."""

    @abstractmethod
    def apply_financial_rules(
        self, raw_rows: List[Dict[str, Any]]
    ) -> List[Dict[str, Any]]:
        """Applies commissions, returns, and other business rules."""
        pass

    @abstractmethod
    def consolidate(self, session_id: str) -> List[Dict[str, Any]]:
        """Performs SQL-based aggregation for the final report."""
        pass


class IExporter(ABC):
    """Interface for generating final Excel master reports."""

    @abstractmethod
    def write_report(self, data: List[Dict[str, Any]], output_path: Path) -> bool:
        """Writes consolidated data to the target Excel file with formatting."""
        pass
```

APP\core\domain\logic_service.py

515 lines • 21,830 bytes

```
"""Бизнес-логика: финансовые правила и валидация."""

import logging
import re
from decimal import Decimal, ROUND_HALF_UP
from typing import List, Dict, Any, Optional

from core.domain.interfaces import IValidator, ILogicService, IPersistence
from core.domain.utils_helpers import (
    to_decimal,
    to_decimal_percent,
    NumericParsingError,
)
from core.domain.data_schemas import FinancialRowSchema

logger = logging.getLogger(__name__)

# Default junk/header detection strings (configurable via config.validation)
_DEFAULT_JUNK_TOTALS = frozenset({"итого", "всего", "итог", "итог дня", "итоги дня"})
_DEFAULT_JUNK_MARKERS = ("заказчик", "исполнитель", "приход", "расход")
_DEFAULT_HEADER_WORDS = (
    "% посредника",
    "заказчик",
    "исполнитель",
    "сумма платежа",
    "комиссия",
    "итог",
    "итого",
    "всего",
    "фильтры",
    "посредник",
    "приход",
    "расход",
)


class Validator(IValidator):
    """Валидатор строк данных — отбрасывает мусор и заголовки."""

    def __init__(self, config: Dict[str, Any]) -> None:
        self.config = config
        v_cfg = config.get("validation", {})
        self._junk_totals = frozenset(v_cfg.get("junk_totals", _DEFAULT_JUNK_TOTALS))
        self._junk_markers = tuple(v_cfg.get("junk_markers", _DEFAULT_JUNK_MARKERS))
        self._header_words = v_cfg.get("header_words", _DEFAULT_HEADER_WORDS)

    @staticmethod
    def _is_header_like(s_norm: str, header_words) -> bool:
        """Check if a normalised string is a leaked column header.

        Uses word-boundary matching so company names like 'Приходько'
        don't match the header word 'приход'. Values containing digits
        (e.g. '100.00 (комиссия)') are treated as data, not headers.
        """
        if any(c.isdigit() for c in s_norm):
            return False
        for w in header_words:
            if re.search(r"(?<!\w)" + re.escape(w) + r"(?!\w)", s_norm):
                return True
        return False

    def validate_row(self, row: Dict[str, Any]) -> bool:
        """Return True if row contains valid financial data, False if junk/header."""
        customer, contractor, filters = (
            str(row.get("customer", "")).strip(),
            str(row.get("contractor", "")).strip(),
            str(row.get("filters", "")).strip(),
        )
        amount_val = row.get("amount")

        def _is_empty_zero(v: Any) -> bool:
            if v is None:
                return True
            return str(v).strip() in {"", "0", "0.00", "0.0", "0,00", "0,0"}

        if not customer and not contractor and not filters and _is_empty_zero(amount_val):
            return False

        # Service metadata fields (private keys starting with '_') carry
        # diagnostics like _template_warning_reason = 'unsupported header
        # name: % -> % посредника; ...'. Those descriptions legitimately
        # contain header words ('комиссия', 'фильтры', 'посредник'). The
        # header-word filter used to scan EVERY field, so it matched the
```

```
        # diagnostic text and rejected the whole row — which silently
        # zeroed out the Boris/Ivan curator batches in auto-reconciliation
        # (both files use aliased headers that trigger soft warnings).
        # Skip private fields in the junk/header heuristics below.
        def _is_service_field(key: str) -> bool:
            return isinstance(key, str) and key.startswith("_")

        for key, val in row.items():
            if _is_service_field(key) or val is None:
                continue
            s_norm = " ".join(str(val).lower().split())
            if self._is_header_like(s_norm, self._header_words):
                return False

        name_field_keys = ["customer", "contractor"]
        name_fields_filled = sum(
            1
            for k in name_field_keys
            if row.get(k) and str(row.get(k)).strip()
        )

        non_name_marker_hits = 0
        for k, v in row.items():
            if v is None or k in name_field_keys or _is_service_field(k):
                continue
            v_str = " ".join(str(v).lower().split())
            if any(m in v_str for m in self._junk_totals):
                non_name_marker_hits += 1

        if non_name_marker_hits >= 2:
            return False

        for key in ["amount", "income", "expense", "commission", "total"]:
            v = row.get(key)
            if v is None:
                continue
            v_str = " ".join(str(v).lower().split())
            if self._is_header_like(v_str, self._header_words):
                return False

        if name_fields_filled >= 2:
            row_str = " ".join(
                " ".join(str(v).lower().split())
                for k, v in row.items()
                if v is not None and not _is_service_field(k)
            )
            if sum(1 for m in self._junk_markers if m in row_str) >= 3:
                return False

        try:
            from pydantic import ValidationError
            FinancialRowSchema(**row)
            return True
        except ValidationError as e:
            logger.warning("Pydantic validation failed for data row: %s | Row: %s", e, row)
            return False

    def get_errors(self) -> List[str]:
        """Вернуть список ошибок валидации (пусто — заглушка)."""
        return []


# ─────────────────────────────────────────────────────────────
# Curator Strategy Registry (Item 7 — replaces hardcoded if/elif)
# ─────────────────────────────────────────────────────────────


class CuratorStrategy:
    """Base strategy — no extra broker calculations."""

    def apply(
        self,
        curator_config: Dict[str, Any],
        pb_val: Any,
        amt: Decimal,
        tot: Decimal,
        raw_b2: Decimal,
        pb2_raw: Any,
    ) -> tuple:
        """Return (pb_val, pb2_val, broker2_calc)."""
        return pb_val, Decimal("0.00"), Decimal("0.00")
```

```
class BorisCuratorStrategy(CuratorStrategy):
    """Boris-specific: keeps intermediary2 values from the source report."""

    def apply(self, curator_config, pb_val, amt, tot, raw_b2, pb2_raw):
        if raw_b2 == 0:
            return pb_val, Decimal("0.00"), Decimal("0.00")

        pb2_val = to_decimal_percent(pb2_raw or 0)
        if isinstance(pb2_val, Decimal):
            pb2_val = abs(pb2_val)
        else:
            pb2_val = abs(to_decimal(pb2_val))
        return pb_val, pb2_val, abs(raw_b2)


class IvanCuratorStrategy(CuratorStrategy):
    """Ivan-specific layout keeps manually entered intermediary values."""

    def apply(self, curator_config, pb_val, amt, tot, raw_b2, pb2_raw):
        return pb_val, Decimal("0.00"), Decimal("0.00")


# Global registry — add new curators here without modifying LogicService
CURATOR_STRATEGIES: Dict[str, CuratorStrategy] = {
    "boris": BorisCuratorStrategy(),
    "ivan": IvanCuratorStrategy(),
}

_DEFAULT_STRATEGY = CuratorStrategy()


class LogicService(ILogicService):
    """Сервис консолидации — применяет финансовые правила к данным."""

    def __init__(self, persistence: IPersistence, config: Dict[str, Any]) -> None:
        self.persistence = persistence
        self.config = config
        l_cfg = config.get("logic", {})
        self.inc_over = l_cfg.get("income_overrides", ["доп", "доп сф", "доп.сф", "% посред"])
        self.ret_kw = l_cfg.get("return_keywords", ["возврат"])
        self.commission_is_percent = l_cfg.get("commission_is_percent", True)
        self.exclude_kw = l_cfg.get(
            "exclude_keywords", ["недопустимо", "исключить", "исключение"]
        )
        # Pre-compile word-boundary patterns for income overrides so that
        # 'доп' matches 'доп оплата' but NOT 'недопустимый' (where 'доп'
        # is a substring of a completely different word).
        self._inc_over_patterns = [
            re.compile(r"(?<!\w)" + re.escape(kw))
            for kw in self.inc_over
        ]

    _SANITIZE_CHARS = frozenset(("=", "+", "-", "@", "\t", "\r", "\n"))

    @staticmethod
    def _abs_decimal(value: Any) -> Any:
        if isinstance(value, Decimal):
            return abs(value)
        return value

    def _sanitize(self, val: Any) -> str:
        """Sanitize cell value to prevent Excel formula injection."""
        s = str(val) if val is not None else ""
        s_stripped = s.strip()
        if s_stripped and (
            s_stripped[0] in self._SANITIZE_CHARS
            or any(c in s_stripped for c in ("\t", "\r", "\n"))
        ):
            return "'" + s
        return s

    def _build_error_row(
        self, r: Dict[str, Any], is_critical: bool = True
    ) -> Dict[str, Any]:
        """Build a row dict that marks a critical data integrity error.

        The row is included in the output so it appears in the final report
        highlighted in red, rather than being silently dropped.
        """
        return {
            "Дата": r.get("date", ""),
            "Источник": self._sanitize(r.get("_source_info", "Unknown")),
            "Сотрудник": self._sanitize(r.get("employee_name", "")),
```

```
            "ЗАКАЗЧИК (название проекта, наименование ЮЛ)": self._sanitize(
                r.get("customer", "")
            ),
            "ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)": self._sanitize(
                r.get("contractor", "")
            ),
            "сумма платежа ПРИХОД": Decimal("0.00"),
            "сумма платежа РАСХОД": Decimal("0.00"),
            "комиссия": Decimal("0.00"),
            "ИТОГ": Decimal("0.00"),
            "% посредника": Decimal("0.00"),
            "посредник 1(кэш)": Decimal("0.00"),
            "посредник 2": Decimal("0.00"),
            "% посредник 2": Decimal("0.00"),
            "Посредник 2 (сумма)": Decimal("0.00"),
            "ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ": Decimal("0.00"),
            "курс": Decimal("0.00"),
            "сумма": Decimal("0.00"),
            "валюта": self._sanitize(r.get("currency", "")),
            "фильтры": self._sanitize(r.get("filters", "")),
            "is_critical": is_critical,
            "is_template_warning": r.get("is_template_warning", False),
        }

    def _get_curator_strategy(self, curator: Optional[str]) -> CuratorStrategy:
        """Look up curator strategy from the registry."""
        if not curator:
            return _DEFAULT_STRATEGY
        return CURATOR_STRATEGIES.get(curator, _DEFAULT_STRATEGY)

    @staticmethod
    def _parse_decimal_or_warn(value: Any) -> tuple[Decimal, bool]:
        try:
            return to_decimal(value), False
        except (NumericParsingError, ValueError):
            return Decimal("0.00"), True

    @staticmethod
    def _has_transit_values(r: FinancialRowSchema) -> bool:
        values = [
            to_decimal(getattr(r, field, 0))
            for field in ("transit_obligation", "exchange_rate", "currency_amount")
        ]
        return any(value != 0 for value in values) or bool(str(r.currency or "").strip())

    @staticmethod
    def _resolve_flow(amount: Decimal, is_add: bool, is_ret: bool) -> tuple[Decimal, Decimal]:
        if is_add:
            return amount, Decimal("0.00")
        if is_ret:
            return Decimal("0.00"), -abs(amount)
        if amount < 0:
            return Decimal("0.00"), abs(amount)
        return amount, Decimal("0.00")

    def _resolve_commission(self, raw_value: Any) -> tuple[Decimal, bool]:
        c_raw = to_decimal_percent(raw_value or 0)
        try:
            if self.commission_is_percent and c_raw != 0 and abs(c_raw) <= 1:
                return (
                    abs(c_raw * 100).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP),
                    False,
                )
            return abs(c_raw).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP), False
        except (TypeError, ArithmeticError):
            try:
                return abs(to_decimal(c_raw)), False
            except (NumericParsingError, ValueError):
                return Decimal("0.00"), True

    def _build_result_row(
        self,
        r: Dict[str, Any],
        *,
        inc: Decimal,
        exp: Decimal,
        comm: Decimal,
        tot: Decimal,
        pb_val: Decimal,
        b1_val: Decimal,
        b2_val: Decimal,
        pb2_val: Decimal,
        broker2_calc: Decimal,
```

```
        is_critical: bool,
        is_warn: bool,
    ) -> Dict[str, Any]:
        # transit/rate/currency_amount may carry garbage (e.g. a header
        # label that leaked into a data cell like ' ОТПРАВКА ТРАНЗИТНЫХ
        # ОБЯЗАТЕЛЬСТВ'). The strict to_decimal() raises NumericParsingError
        # on such values, which is wrapped for amount/total/brokers above
        # but was NOT wrapped here — so one dirty transit cell crashed the
        # whole curator batch (and with it the Ivan sheet of auto-recon).
        # Use the same fail-soft helper the brokers use.
        transit_val, _ = self._parse_decimal_or_warn(r.get("transit_obligation", 0))
        rate_val, _ = self._parse_decimal_or_warn(r.get("exchange_rate", 0))
        cur_amt_val, _ = self._parse_decimal_or_warn(r.get("currency_amount", 0))
        return {
            "Дата": r.get("date", ""),
            "Источник": self._sanitize(r.get("_source_info", "Unknown")),
            "Сотрудник": self._sanitize(r.get("employee_name", "")),
            "ЗАКАЗЧИК (название проекта, наименование ЮЛ)": self._sanitize(
                r.get("customer", "")
            ),
            "ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)": self._sanitize(
                r.get("contractor", "")
            ),
            "сумма платежа ПРИХОД": inc,
            "сумма платежа РАСХОД": exp,
            "комиссия": comm,
            "ИТОГ": tot,
            "% посредника": pb_val,
            "посредник 1(кэш)": b1_val,
            "посредник 2": b2_val,
            "% посредник 2": pb2_val,
            "Посредник 2 (сумма)": broker2_calc,
            "ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ": transit_val,
            "курс": rate_val,
            "сумма": cur_amt_val,
            "валюта": self._sanitize(r.get("currency", "")),
            "фильтры": self._sanitize(r.get("filters", "")),
            "is_critical": is_critical,
            "is_template_warning": is_warn,
            "_template_warning_kind": r.get("_template_warning_kind"),
            "_template_warning_reason": r.get("_template_warning_reason"),
            "_source_file_path": r.get("_file_path"),
            "_source_sheet_name": r.get("_source_sheet_name"),
            "_source_row_index": r.get("_source_row_index"),
        }

    def apply_financial_rules(
        self, raw_rows: List[Dict[str, Any]], curator=None, curator_config=None
    ) -> List[Dict[str, Any]]:
        """Apply commissions, returns, and other business rules to raw rows."""
        res = []
        strategy = self._get_curator_strategy(curator)
        template_warning_count = 0

        for r in raw_rows:
            row_model = FinancialRowSchema(**r)
            f_val = str(row_model.filters or "").lower()
            is_add, is_ret = (
                any(p.search(f_val) for p in self._inc_over_patterns),
                any(kw in f_val for kw in self.ret_kw),
            )
            is_excluded = any(kw in f_val for kw in self.exclude_kw)
            is_critical = False
            is_warn = False

            source_info = row_model.model_extra.get("_source_info", "Unknown") if row_model.model_extra else "Unknown"

            try:
                amt = to_decimal(row_model.amount)
            except (NumericParsingError, ValueError) as e:
                logger.error(
                    f"DATA INTEGRITY ERROR (Logic): {e} | Source: {source_info}"
                )
                if isinstance(e, NumericParsingError):
                    # Critical parse error — record the row with is_critical flag
                    # but do NOT silently substitute zero
                    res.append(self._build_error_row(r, is_critical=True))
                    continue
                amt = Decimal("0.00")

            try:
                tot = to_decimal(row_model.total)
            except (NumericParsingError, ValueError) as e:
```

```
    logger.error(
        f"DATA INTEGRITY ERROR (Logic): {e} | Source: {source_info}"
    )
    # Critical parse error on total — record and skip
    res.append(self._build_error_row(r, is_critical=True))
    continue

raw_b1, broker1_warn = self._parse_decimal_or_warn(row_model.broker1)
raw_b2, broker2_warn = self._parse_decimal_or_warn(row_model.broker2)
is_warn = is_warn or broker1_warn or broker2_warn

has_broker_values = raw_b1 != 0 or raw_b2 != 0
try:
    has_transit_values = self._has_transit_values(row_model)
except (NumericParsingError, ValueError) as e:
    # Transit fields contain unparseable text (e.g. a header
    # label that leaked into a data cell).  Previously this
    # discarded the entire row as a critical error — but the
    # amount/total/broker fields may be perfectly valid.
    # Downgrade to a warning so the financial data is preserved.
    logger.warning(
        "Transit parse error (non-fatal): %s | Source: %s",
        e, source_info,
    )
    has_transit_values = True  # Assume transit might exist — don't drop
    is_warn = True

if (
    amt == 0
    and tot == 0
    and not has_broker_values
    and not has_transit_values
    and not is_add
    and not is_critical
):
    continue

if is_excluded:
    amt = Decimal("0.00")

inc, exp = self._resolve_flow(amt, is_add, is_ret)
comm, commission_warn = self._resolve_commission(row_model.commission)
is_warn = is_warn or commission_warn

pb_raw = row_model.percent_broker
pb_val = to_decimal_percent(pb_raw)
pb_val = self._abs_decimal(pb_val)

# Apply curator-specific broker logic via strategy
if curator_config:
    pb2_raw = row_model.percent_broker2
    pb_val, pb2_val, broker2_calc = strategy.apply(
        curator_config, pb_val, amt, tot, raw_b2, pb2_raw
    )
else:
    pb2_val = Decimal("0.00")
    broker2_calc = Decimal("0.00")

pb2_val = self._abs_decimal(pb2_val)
broker2_calc = self._abs_decimal(broker2_calc)

is_warn = bool(row_model.is_template_warning or is_warn)
if is_warn:
    template_warning_count += 1
b1_val = abs(raw_b1)
b2_val = abs(raw_b2)

res.append(
    self._build_result_row(
        r,
        inc=inc,
        exp=exp,
        comm=comm,
        tot=tot,
        pb_val=pb_val,
        b1_val=b1_val,
        b2_val=b2_val,
        pb2_val=pb2_val,
        broker2_calc=broker2_calc,
        is_critical=is_critical,
        is_warn=is_warn,
    )
)
```

```
        if template_warning_count:
            logger.warning("Template warnings: %s row(s) affected", template_warning_count)
        return res

    def consolidate(
        self, session_id: str, curator=None, curator_config=None
    ) -> List[Dict[str, Any]]:
        """Perform SQL-based aggregation for the final report.

        The curator/curator_config parameters enable separate curator-specific
        reports without affecting the standard single-report flow.
        """
        return self.apply_financial_rules(
            self.persistence.get_consolidated_data(session_id),
            curator=curator,
            curator_config=curator_config,
        )
```

APP\core\domain\utils_helpers.py

88 lines • 3,074 bytes

```
import re
from decimal import Decimal, ROUND_HALF_UP
from typing import Any


class NumericParsingError(ValueError):
    """Raised when numeric string cannot be parsed and isn't a known zero pattern."""

    pass


RE_NUMERIC_CLEAN = re.compile(r"[^\d., \-eEP]")
RE_NUMERIC_PERCENT_CLEAN = re.compile(r"[^\d.,\-]")
TRANS_REMOVE_SPACE = str.maketrans({" ": None, "\xa0": None})


def to_decimal(val: Any) -> Decimal:
    if val is None or val == "" or (isinstance(val, float) and val != val):
        return Decimal("0.00")
    if isinstance(val, (int, float, Decimal)):
        return Decimal(str(val)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

    s_raw = str(val).strip()
    if s_raw.lower() in ("nan", "null", "none", "#n/a", "#ref!", "#value!", "error"):
        raise NumericParsingError(f"Invalid numeric value: {s_raw}")

    try:
        s = s_raw.translate(TRANS_REMOVE_SPACE)
        s = RE_NUMERIC_CLEAN.sub("", s)

        if not s or s == "-":
            if s == "-" or not s_raw or any(c.isdigit() for c in s_raw):
                return Decimal("0.00")
            raise NumericParsingError(
                f"Silent data loss detected: '{val}' cleaned to empty or trash."
            )

        s = s.rstrip(".,")

        if "e" in s.lower():
            res = Decimal(s).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
            return -abs(res) if "(" in s_raw and ")" in s_raw else res

        if "," in s and "." in s:
            last_dot = s.rfind(".")
            last_comma = s.rfind(",")
            if last_comma > last_dot:
                s = s.replace(".", "").replace(",", ".")
            else:
                s = s.replace(",", "")
        elif "," in s:
            if s.count(",") == 1:
                s = s.replace(",", ".")
            else:
                s = s.replace(",", "")

        res = Decimal(s).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
        return -abs(res) if "(" in s_raw and ")" in s_raw else res
    except Exception as e:
        if isinstance(e, NumericParsingError):
            raise
        raise NumericParsingError(f"Could not parse numeric value {val}: {e!s}") from e


def to_decimal_percent(val: Any) -> Any:
    if val is None or val == "" or (isinstance(val, float) and val != val):
        return Decimal("0")
    if isinstance(val, (int, float, Decimal)):
        return Decimal(str(val))
    try:
        raw = str(val)
        s = raw.translate(TRANS_REMOVE_SPACE)
        if not all(c in "0123456789.,-" for c in s):
            s = RE_NUMERIC_PERCENT_CLEAN.sub("", s)
        if not s:
            return Decimal("0")
        res = Decimal(s.replace(",", "."))
        if "%" in raw and res > 1:
            res = res / Decimal("100")
        return res
    except (ValueError, TypeError, ArithmeticError):
        return Decimal("0")
```

```
def normalize_text(value: Any) -> str:
    if value is None:
        return ""
    return " ".join(str(value).replace("\xa0", " ").lower().split())
```

APP\core\infra__init__.py

13 lines • 620 bytes

```
from core.infra.utils import (
    open_path as open_path,
    is_file_writable as is_file_writable,
    get_local_copy as get_local_copy,
)
from core.infra.utils_logging import setup_gui_logging as setup_gui_logging
from core.infra.config_controller import ConfigController as ConfigController
from core.infra.export_writer import MasterWriter as MasterWriter
from core.infra.session_repo import SessionRepository as SessionRepository
from core.infra.discovery_service import DiscoveryService as DiscoveryService
from core.infra.path_resolver import (
    resolve_employee_path as resolve_employee_path,
)
```

APP\core\infra\config_controller.py

257 lines • 10,381 bytes

```
import os
import logging
import hashlib
import re
import threading
from pathlib import Path
from typing import List, Dict, Any
from ruamel.yaml import YAML

from core.infra.utils_logging import scrub_path


class ConfigController:
    def __init__(self, config_path: str):
        self.config_path = config_path
        self.yaml = YAML()
        self.yaml.preserve_quotes = True
        self.yaml.indent(mapping=2, sequence=4, offset=2)
        self.config = self.load_config()
        emp_path = Path(config_path).parent / "employees.yaml"
        self.employees_path = str(emp_path)
        self.employees = self._load_employees()
        self._employees_lock = threading.Lock()

    def _load_employees(self) -> Dict[str, Any]:
        try:
            if os.path.exists(self.employees_path):
                with open(self.employees_path, "r", encoding="utf-8") as f:
                    return self.yaml.load(f) or {}
        except Exception as e:
            logging.error(f"Error loading employees: {e}")
        return {}

    def _save_employees(self):
        try:
            with open(self.employees_path, "w", encoding="utf-8") as f:
                self.yaml.dump(self.employees, f)
            # Restrict to owner-only read/write — employees.yaml contains
            # network paths and file fingerprints that shouldn't be
            # world-readable on shared machines.
            try:
                os.chmod(self.employees_path, 0o600)
            except OSError as chmod_err:
                logging.warning(
                    "Could not restrict %s permissions: %s",
                    self.employees_path,
                    chmod_err,
                )
        except Exception as e:
            logging.error(f"Error saving employees: {e}")

    def load_config(self) -> Dict[str, Any]:
        """Loads the configuration from the YAML file."""
        try:
            if os.path.exists(self.config_path):
                with open(self.config_path, "r", encoding="utf-8") as f:
                    return self.yaml.load(f) or {}

            example_path = Path(self.config_path).with_name("config.example.yaml")
            if example_path.exists():
                logging.warning(
                    "Config file not found: %s. Creating it from %s",
                    self.config_path,
                    example_path,
                )
                config_path = Path(self.config_path)
                config_path.parent.mkdir(parents=True, exist_ok=True)
                config_path.write_text(
                    example_path.read_text(encoding="utf-8"),
                    encoding="utf-8",
                )
                # Restrict to owner-only — config.yaml may contain
                # network paths and operator-specific tuning.
                try:
                    os.chmod(config_path, 0o600)
                except OSError as chmod_err:
                    logging.warning(
                        "Could not restrict %s permissions: %s",
                        config_path,
                        chmod_err,
                    )
                with open(config_path, "r", encoding="utf-8") as f:
                    return self.yaml.load(f) or {}
```

```
            logging.warning(f"Config file not found: {self.config_path}")
            return {}
        except Exception as e:
            logging.error(f"Error loading config: {e}")
            return {}

    def save_config(self):
        """Saves the current configuration to the YAML file."""
        try:
            with open(self.config_path, "w", encoding="utf-8") as f:
                self.yaml.dump(self.config, f)
            # Restrict to owner-only read/write — see load_config.
            try:
                os.chmod(self.config_path, 0o600)
            except OSError as chmod_err:
                logging.warning(
                    "Could not restrict %s permissions: %s",
                    self.config_path,
                    chmod_err,
                )
        except Exception as e:
            logging.error(f"Error saving config: {e}")

    def _invalidate_discovery_cache(self):
        if isinstance(self.config, dict):
            self.config.pop("discovery_cache", None)

    def add_employee(self, name: str):
        name = name.strip()
        if not name:
            return
        with self._employees_lock:
            self.employees.setdefault(name, [])
        self._invalidate_discovery_cache()
        self._save_employees()

    def delete_employee(self, name: str):
        with self._employees_lock:
            if name in self.employees:
                del self.employees[name]
        self._invalidate_discovery_cache()
        self._save_employees()

    def attach_files(self, emp_name: str, files: List[str], append: bool = True):
        """Attaches files to an employee, updating metadata for smart tracking."""
        new_metas = []
        for fp in files:
            p = Path(fp).absolute()
            if not p.exists():
                continue
            try:
                stat = os.stat(p)
                with open(p, "rb") as f:
                    digest = hashlib.md5()
                    for chunk in iter(lambda: f.read(8192), b""):
                        digest.update(chunk)
                    h = digest.hexdigest()
                new_metas.append({"path": str(p), "id": stat.st_ino, "fp": h})
            except Exception as e:
                logging.error(f"Error generating metadata for {fp}: {e}")
                new_metas.append({"path": str(p)})

        current = self.employees.get(emp_name, [])
        if not isinstance(current, list):
            current = [current] if current else []

        combined = current + new_metas if append else new_metas

        final = []
        seen = set()
        for m in combined:
            p = m["path"] if isinstance(m, dict) else m
            if p not in seen:
                final.append(m)
                seen.add(p)

        with self._employees_lock:
            self.employees[emp_name] = final
        self._invalidate_discovery_cache()
        self._save_employees()

    def get_employees(self) -> List[str]:
        return sorted(self.employees.keys())
```

```
    def resolve_path(self, path_str: str) -> Path:
        config_dir = Path(self.config_path).parent.resolve()
        project_root = config_dir.parent.resolve()
        allow_roots = self._allowlist_roots(config_dir, project_root)
        if self._is_effectively_absolute(path_str):
            # Absolute / UNC / drive-letter paths previously bypassed the
            # traversal check entirely (returned as-is). Apply an allowlist
            # so only paths under a configured source_root, the config dir,
            # or the project root are accepted.
            try:
                resolved_abs = Path(path_str).resolve()
            except (OSError, ValueError) as exc:
                raise ValueError(
                    f"Path traversal detected: invalid path '{scrub_path(path_str)}'"
                ) from exc
            if not any(
                self._is_path_under(resolved_abs, root) for root in allow_roots
            ):
                raise ValueError(
                    f"Path traversal detected: '{scrub_path(path_str)}' resolves outside allowed roots"
                )
            return Path(path_str)
        p = Path(path_str)
        if p.is_absolute():
            # Defensive: should be unreachable because _is_effectively_absolute
            # already covers Posix absolute paths — kept for safety.
            return p
        resolved = (Path(self.config_path).parent / p).resolve()
        # Use the explicit separator-boundary check instead of a naive
        # str.startswith, which would match /app/config_evil against
        # /app/config. Allowlist also includes source_roots so legitimate
        # network paths under configured roots continue to work.
        if not any(self._is_path_under(resolved, root) for root in allow_roots):
            raise ValueError(
                f"Path traversal detected: '{scrub_path(path_str)}' resolves outside project root"
            )
        return resolved

    def _allowlist_roots(
        self, config_dir: Path, project_root: Path
    ) -> List[Path]:
        """Trusted roots: configured paths.source_roots + config dir + project root."""
        roots: List[Path] = [config_dir, project_root]
        paths_cfg = (
            self.config.get("paths", {}) if isinstance(self.config, dict) else {}
        )
        src_roots_raw = (
            paths_cfg.get("source_roots") or paths_cfg.get("source_root") or []
        )
        if isinstance(src_roots_raw, (str, Path)):
            src_roots_raw = [src_roots_raw]
        if not isinstance(src_roots_raw, list):
            return roots
        for raw in src_roots_raw:
            try:
                roots.append(Path(str(raw)).resolve())
            except (OSError, ValueError):
                continue
        return roots

    @staticmethod
    def _is_path_under(path: Path, root: Path) -> bool:
        """Return True if ``path`` equals or is nested under ``root``.

        Case-insensitive to match Windows file-system semantics and the
        pre-existing traversal-check behaviour. Uses an explicit separator
        boundary to avoid the ``/app/config`` vs ``/app/config_evil``
        prefix collision that a naive ``startswith`` would accept.
        """
        p = str(path).lower().rstrip("\\/")
        r = str(root).lower().rstrip("\\/")
        if not r:
            return False
        if p == r:
            return True
        return p.startswith(r + "\\") or p.startswith(r + "/")

    @staticmethod
    def _is_effectively_absolute(path_str: str) -> bool:
        p = Path(path_str)
        if p.is_absolute():
            return True
        if path_str.startswith("\\\\"):
```

```
            return True
        return bool(re.match(r"^[A-Za-z]:[\\/]", path_str))

    def get_paths(self) -> Dict[str, str]:
        """Returns the general paths configuration."""
        return self.config.get("paths", {})
```

APP\core\infra\customer_alias_repo.py

195 lines • 7,659 bytes

```
"""Repository for customer aliases — manages reconciliation name mappings.

Бухгалтеры вручную указывают соответствия между заказчиками со стороны
кураторов (master_customer) и заказчиками со стороны РСЧ (rsch_customer).
Эти алиасы используются сверкой для автоматического сопоставления строк,
где названия отличаются (сокращения, разные формы юр. лиц и т.д.).
"""

import logging
from typing import Dict, List, Optional, Set

from core.infra.db_connection import SQLiteConnectionManager

logger = logging.getLogger(__name__)


class CustomerAlias:
    """One alias entry: master_customer ↔ rsch_customer mapping."""

    __slots__ = ("id", "master_customer", "rsch_customer", "notes", "created_at", "updated_at")

    def __init__(
        self,
        id: Optional[int],
        master_customer: str,
        rsch_customer: str,
        notes: str = "",
        created_at: Optional[str] = None,
        updated_at: Optional[str] = None,
    ) -> None:
        self.id = id
        self.master_customer = master_customer.strip()
        self.rsch_customer = rsch_customer.strip()
        self.notes = notes or ""
        self.created_at = created_at
        self.updated_at = updated_at

    def __repr__(self) -> str:
        return f"CustomerAlias({self.master_customer!r} ↔ {self.rsch_customer!r})"


class CustomerAliasRepository:
    """CRUD repository for customer_aliases table.

    Caches aliases in memory after first load; call ``reload()`` after
    bulk edits to refresh the cache without restarting the app.
    """

    def __init__(self, connection_manager: SQLiteConnectionManager) -> None:
        self._conn_mgr = connection_manager
        self._cache: Optional[Dict[str, Set[str]]] = None

    # ── Cache management ──────────────────────────────────

    def reload(self) -> None:
        """Force reload of alias cache from DB."""
        self._cache = None
        self._ensure_cache()

    def _ensure_cache(self) -> Dict[str, Set[str]]:
        if self._cache is not None:
            return self._cache
        conn = self._conn_mgr.get_connection()
        try:
            rows = conn.execute(
                "SELECT master_customer, rsch_customer FROM customer_aliases"
            ).fetchall()
        finally:
            conn.close()
        cache: Dict[str, Set[str]] = {}
        for row in rows:
            master = row["master_customer"].strip().lower()
            rsch = row["rsch_customer"].strip().lower()
            cache.setdefault(master, set()).add(rsch)
            cache.setdefault(rsch, set()).add(master)
        self._cache = cache
        logger.info("Loaded %d unique alias keys (%d pairs)", len(cache), len(rows))
        return cache

    # ── Matching (used by reconciliation) ─────────────────

    def is_alias(self, master_customer: str, rsch_customer: str) -> bool:
        """Return True if an explicit alias exists between the two customers.
```

```
        Comparison is case-insensitive and whitespace-normalised.
        """
        cache = self._ensure_cache()
        if not master_customer or not rsch_customer:
            return False
        master_norm = master_customer.strip().lower()
        rsch_norm = rsch_customer.strip().lower()
        if master_norm == rsch_norm:
            return True
        aliases = cache.get(master_norm)
        return bool(aliases and rsch_norm in aliases)

    def get_all_aliases(self) -> List[CustomerAlias]:
        """Return all alias entries (for UI display)."""
        conn = self._conn_mgr.get_connection()
        try:
            rows = conn.execute(
                "SELECT id, master_customer, rsch_customer, notes, created_at, updated_at "
                "FROM customer_aliases ORDER BY master_customer COLLATE NOCASE"
            ).fetchall()
        finally:
            conn.close()
        return [
            CustomerAlias(
                id=row["id"],
                master_customer=row["master_customer"],
                rsch_customer=row["rsch_customer"],
                notes=row["notes"] or "",
                created_at=row["created_at"],
                updated_at=row["updated_at"],
            )
            for row in rows
        ]

    # ── CRUD ──────────────────────────────────────────────

    def add_alias(self, master_customer: str, rsch_customer: str, notes: str = "") -> int:
        """Insert a new alias. Returns the new row id.

        Raises ValueError if the alias already exists.
        """
        master = master_customer.strip()
        rsch = rsch_customer.strip()
        if not master or not rsch:
            raise ValueError("И заказчик куратора, и заказчик РСЧ должны быть заполнены")
        if master.lower() == rsch.lower():
            raise ValueError("Заказчики идентичны — алиас не нужен")
        conn = self._conn_mgr.get_connection()
        try:
            cursor = conn.execute(
                "INSERT INTO customer_aliases (master_customer, rsch_customer, notes) "
                "VALUES (?, ?, ?)",
                (master, rsch, notes),
            )
            conn.commit()
            self._cache = None  # invalidate cache
            logger.info("Added alias: %r ↔ %r", master, rsch)
            return int(cursor.lastrowid)
        except Exception as e:
            conn.rollback()
            if "UNIQUE" in str(e):
                raise ValueError(f"Алиас '{master}' ↔ '{rsch}' уже существует") from e
            raise
        finally:
            conn.close()

    def update_alias(
        self,
        alias_id: int,
        master_customer: str,
        rsch_customer: str,
        notes: str = "",
    ) -> None:
        """Update an existing alias by id."""
        master = master_customer.strip()
        rsch = rsch_customer.strip()
        if not master or not rsch:
            raise ValueError("И заказчик куратора, и заказчик РСЧ должны быть заполнены")
        conn = self._conn_mgr.get_connection()
        try:
            conn.execute(
                "UPDATE customer_aliases SET master_customer = ?, rsch_customer = ?, "
                "notes = ?, updated_at = CURRENT_TIMESTAMP WHERE id = ?",
```

```
                (master, rsch, notes, alias_id),
            )
            conn.commit()
            self._cache = None
            logger.info("Updated alias id=%d", alias_id)
        except Exception as e:
            conn.rollback()
            if "UNIQUE" in str(e):
                raise ValueError(
                    f"Алиас '{master}' ↔ '{rsch}' уже существует (дубликат)"
                ) from e
            raise
        finally:
            conn.close()

    def delete_alias(self, alias_id: int) -> None:
        """Delete an alias by id."""
        conn = self._conn_mgr.get_connection()
        try:
            conn.execute("DELETE FROM customer_aliases WHERE id = ?", (alias_id,))
            conn.commit()
            self._cache = None
            logger.info("Deleted alias id=%d", alias_id)
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()
```

APP\core\infra\db_connection.py

283 lines • 11,862 bytes

```
import sqlite3
import json
import time
import decimal
import datetime
import logging
import threading
from typing import Dict, Any
from pathlib import Path


class DecimalEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, decimal.Decimal):
            return str(obj)
        if hasattr(obj, "isoformat"):
            return obj.isoformat()
        return super().default(obj)


def sanitize_data(data: Any) -> Any:
    if isinstance(data, dict):
        return {str(k): sanitize_data(v) for k, v in data.items()}
    elif isinstance(data, list):
        return [sanitize_data(i) for i in data]
    elif isinstance(data, (decimal.Decimal, datetime.date, datetime.datetime)):
        return str(data) if isinstance(data, decimal.Decimal) else data.isoformat()
    return data


# Module-level adapter/converter registration. sqlite3.register_adapter
# and register_converter mutate global process state, so they must run
# exactly once — calling them in SQLiteConnectionManager.__init__
# re-registered (and re-wrapped) them on every connection manager. A
# lock guards the once-flag for import-time thread safety.
_ADAPTERS_REGISTERED = False
_ADAPTERS_LOCK = threading.Lock()


def _register_sqlite_adapters_once() -> None:
    global _ADAPTERS_REGISTERED
    if _ADAPTERS_REGISTERED:
        return
    with _ADAPTERS_LOCK:
        if _ADAPTERS_REGISTERED:
            return

        sqlite3.register_adapter(decimal.Decimal, str)

        def _convert_timestamp(v: bytes) -> str:
            try:
                return v.decode("utf-8")
            except UnicodeDecodeError:
                return ""

        sqlite3.register_converter("TIMESTAMP", _convert_timestamp)

        def _convert_numeric(v: bytes) -> decimal.Decimal:
            try:
                s = v.decode("utf-8").strip()
                return decimal.Decimal(s) if s else decimal.Decimal("0")
            except (decimal.InvalidOperation, UnicodeDecodeError):
                return decimal.Decimal("0")

        sqlite3.register_converter("NUMERIC", _convert_numeric)
        _ADAPTERS_REGISTERED = True


# Register on import so the first SQLiteConnectionManager is unaffected.
_register_sqlite_adapters_once()


class SQLiteConnectionManager:
    def __init__(self, db_path: Path, config: Dict[str, Any]):
        self.db_path = db_path
        self.config = config
        self.logger = logging.getLogger(__name__)
        self.retry_count = config.get("locking", {}).get("retry_count", 3)
        self.retry_backoff = config.get("locking", {}).get("retry_backoff", 1.0)

        # Adapters/converters are registered once at module import; the
        # per-instance registration that used to live here was redundant
        # and re-wrapped the globals on every connection manager.
```

```
    def get_connection(self) -> sqlite3.Connection:
        conn = sqlite3.connect(
            self.db_path,
            detect_types=sqlite3.PARSE_DECLTYPES,
            timeout=self.config.get("database", {}).get("timeout", 30.0),
        )
        try:
            conn.row_factory = sqlite3.Row
            conn.execute("PRAGMA foreign_keys = ON;")
            conn.execute("PRAGMA synchronous = NORMAL;")
            conn.execute("PRAGMA journal_mode = WAL;")
            conn.execute("PRAGMA temp_store = MEMORY;")
            conn.execute("PRAGMA cache_size = -64000;")
            conn.execute("PRAGMA mmap_size = 268435456;")
            conn.execute("PRAGMA busy_timeout = 5000;")
            # Overwrite deleted content with zeros so that subsequent VACUUM
            # genuinely removes the bytes from the file (defense-in-depth
            # for PII retention; no business-logic change).
            conn.execute("PRAGMA secure_delete = ON;")
            return conn
        except sqlite3.Error:
            conn.close()
            raise

    def check_integrity(self) -> tuple[bool, str | None]:
        """Run a quick PRAGMA quick_check and return (ok, error_message).

        Called at application startup; skips if no prior unclean-shutdown
        flag is found.  Returns (True, None) when the database is healthy.
        """
        flag_file = self.db_path.with_name(self.db_path.name + ".unclean")
        if not flag_file.exists():
            try:
                flag_file.write_text("unclean", encoding="utf-8")
            except Exception as e:
                self.logger.warning("Could not create unclean shutdown flag file: %s", e)
            return (True, None)

        self.logger.warning("Unclean shutdown detected. Running PRAGMA quick_check...")
        conn = self.get_connection()
        try:
            row = conn.execute("PRAGMA quick_check").fetchone()
            msg = row[0] if row else "unknown"
            return (msg == "ok", None if msg == "ok" else msg)
        except Exception as exc:
            return (False, str(exc))
        finally:
            conn.close()

    def init_db(self, schema_path: Path) -> None:
        if schema_path.exists():
            conn = self.get_connection()
            try:
                with open(schema_path, "r", encoding="utf-8") as f:
                    conn.executescript(f.read())
            finally:
                conn.close()
        self._run_migrations()
        self._cleanup_old_cache_entries()

    def _cleanup_old_cache_entries(self) -> None:
        """Delete stale rows from cache tables to keep cache.db bounded.

        row_cache / file_cache: entries older than 7 days.
        discovery_cache: entries older than 12 hours (matches the read TTL
        in source_file_repo.get_discovery_cache).
        """
        conn = self.get_connection()
        try:
            conn.execute("DELETE FROM row_cache WHERE created_at < datetime('now', '-7 days')")
            conn.execute("DELETE FROM file_cache WHERE last_updated < datetime('now', '-7 days')")
            conn.execute(
                "DELETE FROM discovery_cache WHERE scanned_at < CAST(strftime('%s','now') AS REAL) - 43200"
            )
            conn.commit()
            self.logger.info("Cache cleanup: removed stale row_cache, file_cache, discovery_cache entries")
        except sqlite3.Error as e:
            self.logger.warning("Cache cleanup failed (non-fatal): %s", e)
        finally:
            conn.close()

    def _backup_db(self) -> None:
        """Create a timestamped backup of the database file before migrations."""
```

```
        import shutil
        timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
        backup_path = self.db_path.parent / f"{self.db_path.name}.backup_{timestamp}"
        try:
            shutil.copy2(self.db_path, backup_path)
            self.logger.info("Database backed up to %s before migration", backup_path)
        except OSError as e:
            self.logger.warning("Could not create database backup: %s", e)

    def _run_migrations(self) -> None:
        migrations_dir = Path(__file__).resolve().parents[
            2
        ] / "migrations"
        if not migrations_dir.is_dir():
            return
        pending = sorted(
            p for p in migrations_dir.glob("*.sql") if p.is_file()
        )
        conn = self.get_connection()
        try:
            conn.execute("CREATE TABLE IF NOT EXISTS _migrations (filename TEXT PRIMARY KEY, applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)")
            if not pending:
                return
            applied = {
                row[0]
                for row in conn.execute(
                    "SELECT filename FROM _migrations"
                ).fetchall()
            }
            unapplied = [p for p in pending if p.name not in applied]
            if unapplied:
                self._backup_db()
            for migration_path in unapplied:
                name = migration_path.name
                # Apply each migration as one atomic unit: the DDL and the
                # _migrations row commit together. executescript() implicitly
                # commits any open transaction, so we drive the migration
                # statement-by-statement inside an explicit BEGIN IMMEDIATE
                # instead — that way a crash between 'DDL applied' and
                # 'migration recorded' can't leave a half-applied migration
                # that re-runs (and may fail) on the next start.
                sql_text = migration_path.read_text(encoding="utf-8")
                statements = self._split_sql_statements(sql_text)
                try:
                    conn.execute("BEGIN IMMEDIATE")
                    try:
                        for stmt in statements:
                            if stmt.strip():
                                conn.execute(stmt)
                        conn.execute(
                            "INSERT INTO _migrations (filename) VALUES (?)",
                            (name,),
                        )
                        conn.commit()
                    except Exception:
                        conn.rollback()
                        raise
                    self.logger.info("Applied migration: %s", name)
                except sqlite3.Error as exc:
                    # A failing migration must not prevent the rest of
                    # startup; surface it loudly but keep going so the
                    # app is at least usable for diagnosis.
                    self.logger.error(
                        "Migration %s failed and was rolled back: %s",
                        name, exc,
                    )
                    raise
        finally:
            conn.close()

    @staticmethod
    def _split_sql_statements(sql_text: str) -> list[str]:
        """Split a .sql script into individual statements.

        Keeps it simple: split on ';'. Multi-line DDL and comments are
        fine; the migration files in this project are plain CREATE
        TABLE / CREATE INDEX scripts without triggers or procedures, so
        a naive split is sufficient and avoids the implicit-commit
        behaviour of executescript().
        """
        statements: list[str] = []
        for raw in sql_text.split(";"):
            # Drop full-line and trailing comments so empty/comment-only
```

```
            # splits don't reach execute() as no-ops that some SQLite
            # builds reject.
            lines = [
                line for line in raw.splitlines()
                if not line.strip().startswith("--")
            ]
            stmt = "\n".join(lines).strip()
            if stmt:
                statements.append(stmt + ";")
        return statements

    def execute_with_retry(self, func, *args, **kwargs):
        """Execute a function with exponential backoff on transient SQLite errors."""
        last_err = None
        attempt_count = max(1, int(self.retry_count))
        for attempt in range(attempt_count):
            try:
                return func(*args, **kwargs)
            except sqlite3.OperationalError as e:
                if any(x in str(e).lower() for x in ["locked", "busy"]):
                    last_err = e
                    delay = self.retry_backoff * (2**attempt)
                    self.logger.warning(
                        f"SQLite retry {attempt + 1}/{attempt_count}: {e}, "
                        f"waiting {delay:.1f}s"
                    )
                    if attempt < attempt_count - 1:
                        time.sleep(delay)
                else:
                    raise
        if last_err is None:
            raise sqlite3.OperationalError("SQLite operation failed before execution")
        raise last_err
```

APP\core\infra\discovery_service.py

421 lines • 16,220 bytes

```
import hashlib
import logging
import time
import concurrent.futures
from pathlib import Path
from typing import List, Dict, Any, Optional

from core.infra.config_controller import ConfigController
from core.infra.ingestion_engine import IngestionEngine
from core.infra.path_resolver import resolve_employee_path
from core.infra.utils_logging import scrub_path
from core.infra.utils import path_exists_with_timeout


class DiscoveryService:
    _DEFAULT_CURATOR_ROOTS = (
        "Отчет кураторов 1",
        "Отчет кураторов 2",
        "Отчет кураторов 3",
    )

    def __init__(
        self,
        config_controller: ConfigController,
        ingestion: IngestionEngine,
        persistence: Optional[Any] = None,
    ) -> None:
        self.config_controller = config_controller
        self.ingestion = ingestion
        self.persistence = persistence
        self.rename_detected = False
        self.last_discovered_count = 0

    @staticmethod
    def _copy_config_entry(entry):
        return dict(entry) if isinstance(entry, dict) else entry

    @staticmethod
    def _append_unique(entries, entry, seen):
        path_value = entry["path"] if isinstance(entry, dict) else str(entry)
        if path_value not in seen:
            entries.append(entry)
            seen.add(path_value)

    def _detect_default_roots(self, config: Dict[str, Any]) -> List[Path]:
        network_cfg = config.get("network") or {}
        drive_letters = network_cfg.get("drive_letters", ["Q"])
        if not isinstance(drive_letters, list) or not drive_letters:
            drive_letters = ["Q"]

        roots: List[Path] = []
        for drive_letter in drive_letters:
            drive = str(drive_letter).strip().upper().rstrip(":\\/")
            if not drive:
                continue
            for folder_name in self._DEFAULT_CURATOR_ROOTS:
                candidate = Path(f"{drive}:/") / folder_name
                if path_exists_with_timeout(candidate):
                    roots.append(candidate)

        # dedupe while preserving order
        deduped: List[Path] = []
        seen = set()
        for root in roots:
            key = str(root)
            if key in seen:
                continue
            seen.add(key)
            deduped.append(root)
        return deduped

    def _resolve_roots_to_scan(self, config: Dict[str, Any]) -> List[Path]:
        paths_config = config.get("paths", {}) if hasattr(config, "get") else {}
        if hasattr(paths_config, "get"):
            sr_val = paths_config.get("source_roots") or paths_config.get("source_root")
        else:
            sr_val = None
        roots_to_scan = (
            [Path(r) for r in sr_val]
            if isinstance(sr_val, list)
            else [Path(sr_val)]
            if sr_val
            else []
```

```
        )

        if not roots_to_scan:
            auto_roots = self._detect_default_roots(config)
            if auto_roots:
                roots_to_scan = auto_roots
                logging.info(
                    "[NET] source_roots не заданы, использую автодетект: %s",
                    ", ".join(scrub_path(str(r)) for r in roots_to_scan),
                )
            return roots_to_scan

        is_legacy_reports = isinstance(sr_val, str) and sr_val.strip().upper().endswith(
            "REPORTS"
        )
        if not is_legacy_reports:
            return roots_to_scan

        unresolved_root = roots_to_scan[0]
        if path_exists_with_timeout(unresolved_root):
            return roots_to_scan

        auto_roots = self._detect_default_roots(config)
        if auto_roots:
            roots_to_scan = auto_roots
            logging.info(
                "[NET] Локальный source_root недоступен, переключаюсь на: %s",
                ", ".join(scrub_path(str(r)) for r in roots_to_scan),
            )
        return roots_to_scan

    @staticmethod
    def _roots_key(roots_to_scan: List[Path]) -> List[str]:
        return [str(root) for root in roots_to_scan]

    @staticmethod
    def _discovery_cache_ttl(config: Dict[str, Any]) -> float:
        network_cfg = config.get("network") or {}
        raw_ttl = network_cfg.get("discovery_cache_ttl_seconds", 12 * 60 * 60)
        try:
            return max(0.0, float(raw_ttl))
        except (TypeError, ValueError):
            return 12 * 60 * 60

    def _load_discovery_cache(
        self, config: Dict[str, Any], roots_to_scan: List[Path]
    ) -> List[Dict[str, Any]] | None:
        roots_key = hashlib.md5(
            str(self._roots_key(roots_to_scan)).encode()
        ).hexdigest()

        # Try SQLite cache first
        if self.persistence is not None:
            try:
                cached = self.persistence.get_discovery_cache(roots_key)
                if cached is not None:
                    return cached
            except Exception as e:
                logging.debug(f"SQLite discovery cache miss: {e}")

        # Fall back to YAML cache
        cache = config.get("discovery_cache") if hasattr(config, "get") else None
        if not isinstance(cache, dict):
            return None
        if cache.get("roots_key") != self._roots_key(roots_to_scan):
            return None

        scanned_at = cache.get("scanned_at")
        try:
            age = time.time() - float(scanned_at)
        except (TypeError, ValueError):
            return None
        if age > self._discovery_cache_ttl(config):
            return None

        files = cache.get("files")
        if not isinstance(files, list):
            return None
        return [dict(item) for item in files if isinstance(item, dict)]

    def _save_discovery_cache(
        self,
        config: Dict[str, Any],
```

```
        roots_to_scan: List[Path],
        discovered_files: List[Dict[str, Any]],
    ) -> None:
        roots_key = hashlib.md5(
            str(self._roots_key(roots_to_scan)).encode()
        ).hexdigest()

        # Save to SQLite if available
        if self.persistence is not None:
            try:
                self.persistence.save_discovery_cache(roots_key, discovered_files)
                logging.info("[CACHE] Discovery cache saved to SQLite")
            except Exception as e:
                logging.warning(f"Failed to save discovery cache to SQLite: {e}")

        # Also save to YAML for backward compatibility
        config["discovery_cache"] = {
            "roots_key": self._roots_key(roots_to_scan),
            "scanned_at": time.time(),
            "files": [dict(item) for item in discovered_files],
        }
        save_config = getattr(self.config_controller, "save_config", None)
        if callable(save_config):
            save_config()

    def _all_custom_paths_available(
        self, config_list: List[str], custom_paths: Dict[str, Any]
    ) -> bool:
        if not config_list or not all(name in custom_paths for name in config_list):
            return False

        for name in config_list:
            p_val = custom_paths[name]
            p_list = p_val if isinstance(p_val, list) else [p_val]
            for entry in p_list:
                path_str = entry["path"] if isinstance(entry, dict) else entry
                resolved, _, _ = resolve_employee_path(
                    path_str,
                    file_id=entry.get("id") if isinstance(entry, dict) else None,
                    file_hash=entry.get("fp") if isinstance(entry, dict) else None,
                    discovered_files=[],
                    config_controller=self.config_controller,
                )
                p_abs = Path(resolved)
                if not p_abs.is_absolute() or not path_exists_with_timeout(p_abs):
                    return False
        return True

    @staticmethod
    def _path_parts(path_str: str) -> List[str]:
        return [
            part.lower()
            for part in path_str.replace("\\", "/").split("/")
            if part not in ("", ".", "..")
        ]

    @classmethod
    def _matching_roots_for_path(
        cls, path_str: str, roots_to_scan: List[Path]
    ) -> List[Path]:
        normalized_path = path_str.replace("\\", "/").lower()
        path_parts = cls._path_parts(path_str)
        matches: List[Path] = []
        for root in roots_to_scan:
            normalized_root = str(root).replace("\\", "/").lower().rstrip("/")
            if normalized_path.startswith(normalized_root + "/"):
                matches.append(root)
                continue
            if root.name.lower() in path_parts:
                matches.append(root)
        return matches

    def _roots_requiring_scan(
        self,
        config_list: List[str],
        custom_paths: Dict[str, Any],
        roots_to_scan: List[Path],
        *,
        allow_network_scan: bool = True,
    ) -> List[Path]:
        if not allow_network_scan:
            return []
        if not config_list:
```

```
            return roots_to_scan
        if not roots_to_scan:
            return []
        if not all(name in custom_paths for name in config_list):
            return roots_to_scan

        missing_roots: List[Path] = []
        seen = set()
        for name in config_list:
            p_val = custom_paths[name]
            p_list = p_val if isinstance(p_val, list) else [p_val]
            for entry in p_list:
                path_str = entry["path"] if isinstance(entry, dict) else entry
                file_id = entry.get("id") if isinstance(entry, dict) else None
                file_hash = entry.get("fp") if isinstance(entry, dict) else None
                resolved, _, _ = resolve_employee_path(
                    path_str,
                    file_id=file_id,
                    file_hash=file_hash,
                    discovered_files=[],
                    config_controller=self.config_controller,
                )
                p_abs = Path(resolved)
                if p_abs.is_absolute() and path_exists_with_timeout(p_abs):
                    continue
                if file_id or file_hash:
                    return roots_to_scan

                matching_roots = self._matching_roots_for_path(
                    str(path_str), roots_to_scan
                )
                if not matching_roots:
                    return roots_to_scan
                for root in matching_roots:
                    key = str(root)
                    if key not in seen:
                        missing_roots.append(root)
                        seen.add(key)
        return missing_roots

    def _scan_discovered_files(
        self, roots_to_scan: List[Path], all_have_custom: bool
    ) -> List[Dict[str, Any]]:
        discovered_files: List[Dict[str, Any]] = []
        if not roots_to_scan:
            return discovered_files
        if all_have_custom:
            logging.info(
                "[NET] Все сотрудники привязаны, пропускаю полное сканирование сети."
            )
            return discovered_files

        def scan_root(root: Path) -> List[Dict[str, Any]]:
            try:
                logging.info("[NET] Сканирую сетевой путь: %s", scrub_path(str(root)))
                files = self.ingestion.scan_directory(root)
                logging.info("[NET] Найдено файлов в %s: %s", root.name, len(files))
                return files
            except Exception as e:
                logging.error("[NET] Ошибка доступа к %s: %s", scrub_path(str(root)), e)
                return []

        if len(roots_to_scan) == 1:
            return scan_root(roots_to_scan[0])

        max_workers = min(16, len(roots_to_scan))
        with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
            for files in executor.map(scan_root, roots_to_scan):
                discovered_files.extend(files)
        return discovered_files

    def _resolve_custom_paths_for_employee(
        self, target_name: str, custom_paths: Dict[str, Any], discovered_files: List[Dict[str, Any]]
    ) -> List[Dict[str, Any]]:
        if target_name not in custom_paths:
            return []

        paths_for_emp: List[Dict[str, Any]] = []
        seen_result_paths = set()

        p_val = custom_paths[target_name]
        p_list = p_val if isinstance(p_val, list) else [p_val]
        updated_paths = []
```

```
        config_changed = False

        for entry in p_list:
            path_str = entry["path"] if isinstance(entry, dict) else entry
            file_id = entry.get("id") if isinstance(entry, dict) else None
            file_hash = entry.get("fp") if isinstance(entry, dict) else None

            resolved, meta, renamed = resolve_employee_path(
                path_str,
                file_id=file_id,
                file_hash=file_hash,
                discovered_files=discovered_files,
                config_controller=self.config_controller,
            )

            if renamed:
                self.rename_detected = True
                config_changed = True
                updated_paths.append(meta if meta else {"path": resolved})
            elif meta:
                updated_paths.append(self._copy_config_entry(entry))
            else:
                p_abs = Path(resolved)
                if path_exists_with_timeout(p_abs):
                    entry_meta = {"path": str(p_abs)}
                    if file_id is not None:
                        entry_meta["id"] = file_id
                    if file_hash is not None:
                        entry_meta["fp"] = file_hash
                    updated_paths.append(self._copy_config_entry(entry))
                    meta = entry_meta

            if meta:
                self._append_unique(paths_for_emp, meta, seen_result_paths)

        deduped_paths = []
        seen_config_paths = set()
        for meta in updated_paths:
            self._append_unique(deduped_paths, meta, seen_config_paths)

        if config_changed and deduped_paths != self.config_controller.employees.get(
            target_name, []
        ):
            self.config_controller.employees[target_name] = deduped_paths
            self.config_controller._save_employees()

        return paths_for_emp

    def get_employees_map(
        self, *, allow_network_scan: bool = True
    ) -> Dict[str, List[Dict[str, Any]]]:
        config = self.config_controller.config
        custom_paths = self.config_controller.employees
        config_list = sorted(custom_paths.keys())

        roots_to_scan = self._resolve_roots_to_scan(config)
        roots_requiring_scan = self._roots_requiring_scan(
            config_list,
            custom_paths,
            roots_to_scan,
            allow_network_scan=allow_network_scan,
        )
        all_have_custom = not roots_requiring_scan
        discovered_files = (
            None
            if roots_requiring_scan
            else []
        )
        if roots_requiring_scan:
            discovered_files = self._load_discovery_cache(config, roots_requiring_scan)
        if discovered_files is None:
            discovered_files = self._scan_discovered_files(
                roots_requiring_scan, all_have_custom
            )
            if roots_requiring_scan:
                self._save_discovery_cache(
                    config, roots_requiring_scan, discovered_files
                )
        self.last_discovered_count = len(discovered_files or [])

        self.rename_detected = False

        result = {}
```

```
        for target_name in config_list:
            result[target_name] = self._resolve_custom_paths_for_employee(
                target_name, custom_paths, discovered_files
            )

        return result
```

APP\core\infra\event_bus.py

91 lines • 3,182 bytes

```
"""Thread-safe publish-subscribe event bus.

Events are published from any thread and drained from the GUI thread.
This decouples FileProcessor/AppController from GuiMaster — no more
4-layer callback chains for progress updates.
"""

import queue
import logging
import threading
from dataclasses import dataclass
from enum import Enum
from typing import Any, Callable, Dict, List


class EventType(Enum):
    PROGRESS_UPDATED = "progress_updated"
    OPERATION_STARTED = "operation_started"
    OPERATION_FINISHED = "operation_finished"
    ERROR_OCCURRED = "error_occurred"
    TITLE_REFRESH = "title_refresh"


@dataclass
class Event:
    type: EventType
    data: Any = None


class EventBus:
    _instance: "EventBus | None" = None
    _instance_lock = threading.Lock()

    def __init__(self) -> None:
        self._queue: queue.Queue[Event] = queue.Queue()
        self._listeners: Dict[EventType, List[Callable[[Event], None]]] = {}
        self._lock = threading.Lock()

    @classmethod
    def get(cls) -> "EventBus":
        """Return the process-wide singleton, creating it on first call.

        Uses double-checked locking so that two threads racing the
        initialisation still end up with the same instance (without a
        lock on every subsequent read). The previous unlocked version
        could build two independent buses if two workers both hit the
        ``_instance is None`` branch before either stored its result.
        """
        if cls._instance is None:
            with cls._instance_lock:
                if cls._instance is None:
                    cls._instance = cls()
        return cls._instance

    @classmethod
    def reset(cls) -> None:
        """Drop the singleton. Intended for tests; acquires the init lock
        so a concurrent get() can't observe a half-cleared state."""
        with cls._instance_lock:
            cls._instance = None

    def publish(self, event: Event) -> None:
        self._queue.put(event)

    def subscribe(self, event_type: EventType, callback: Callable[[Event], None]) -> None:
        with self._lock:
            self._listeners.setdefault(event_type, []).append(callback)

    def unsubscribe(self, event_type: EventType, callback: Callable[[Event], None]) -> None:
        with self._lock:
            listeners = self._listeners.get(event_type, [])
            if callback in listeners:
                listeners.remove(callback)

    def poll(self) -> None:
        """Drain the queue.  Must be called from the GUI thread."""
        try:
            while True:
                event = self._queue.get_nowait()
                with self._lock:
                    listeners = list(self._listeners.get(event.type, []))
                for cb in listeners:
                    try:
```

```
                cb(event)
            except Exception as e:
                logging.warning(
                    "EventBus callback error for %s: %s", event.type, e
                )
        self._queue.task_done()
except queue.Empty:
    pass
```

APP\core\infra\excel_backup.py

63 lines • 1,849 bytes

```
import logging
import shutil
from datetime import datetime
from pathlib import Path

logger = logging.getLogger(__name__)


def resolve_excel_backup_keep(value, default: int = 10) -> int:
    try:
        parsed = int(value)
    except (TypeError, ValueError):
        return default
    return max(1, parsed)


def backup_existing_excel_output(
    output_path: Path | str,
    backup_root: Path | str | None = None,
    *,
    keep: int = 10,
) -> Path | None:
    """Copy an existing Excel output before it is overwritten.

    Backups are best-effort: a backup failure must not silently overwrite the
    output, so the caller should abort saving when this function raises.
    """

    source = Path(output_path)
    if not source.exists() or not source.is_file():
        return None
    if source.name.startswith("~$"):
        return None

    root = Path(backup_root) if backup_root else source.parent / "backups"
    target_dir = root / "excel_outputs" / source.stem
    target_dir.mkdir(parents=True, exist_ok=True)

    stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    target = target_dir / f"{stamp}_{source.name}"
    shutil.copy2(source, target)
    _prune_backups(target_dir, source.suffix, keep)
    logger.info("Excel output backup created: %s", target)
    return target


def _prune_backups(target_dir: Path, suffix: str, keep: int) -> None:
    if keep <= 0:
        return
    backups = sorted(
        (
            path
            for path in target_dir.iterdir()
            if path.is_file() and path.suffix.lower() == suffix.lower()
        ),
        key=lambda path: path.stat().st_mtime,
        reverse=True,
    )
    for old_path in backups[keep:]:
        try:
            old_path.unlink()
        except OSError as exc:
            logger.warning("Could not prune old Excel backup %s: %s", old_path, exc)
```

APP\core\infra\excel_hyperlinks.py

68 lines • 1,802 bytes

```
from __future__ import annotations

from pathlib import Path
from typing import Any

from openpyxl.cell.cell import Cell
from openpyxl.styles import Font

_LINK_FONT = Font(color="FF0563C1", underline="single")


def build_excel_hyperlink(
    file_path: Any,
    sheet_name: Any = None,
    row_index: Any = None,
) -> str | None:
    if not file_path:
        return None
    try:
        return str(Path(str(file_path)).resolve())
    except (OSError, RuntimeError, ValueError):
        return str(file_path)


def extract_hyperlink_formula_parts(value: Any) -> tuple[str, str] | None:
    if not isinstance(value, str) or not value.startswith("=HYPERLINK("):
        return None
    inner = value.removeprefix("=HYPERLINK(").removesuffix(")")
    if not inner.startswith('"'):
        return None
    parts: list[str] = []
    curr: list[str] = []
    idx = 1
    while idx < len(inner):
        char = inner[idx]
        if char == '"':
            if idx + 1 < len(inner) and inner[idx + 1] == '"':
                curr.append('"')
                idx += 2
                continue
            parts.append("".join(curr))
            idx += 1
            if idx >= len(inner) or inner[idx] != ",":
                break
            idx += 1
            if idx >= len(inner) or inner[idx] != '"':
                break
            curr = []
            idx += 1
            continue
        curr.append(char)
        idx += 1
    if len(parts) != 2:
        return None
    return parts[0], parts[1]


def apply_cell_hyperlink(
    cell: Cell,
    file_path: Any,
    sheet_name: Any = None,
    row_index: Any = None,
) -> None:
    hyperlink = build_excel_hyperlink(file_path, sheet_name, row_index)
    if not hyperlink:
        return
    cell.hyperlink = hyperlink
    cell.font = _LINK_FONT
```

APP\core\infra\export_formatting.py

391 lines • 13,194 bytes

```
import logging
from decimal import Decimal
from openpyxl.utils import get_column_letter
from openpyxl.styles import PatternFill, Font, Alignment, Border, Side


class ExcelStyles:
    def __init__(self):
        self.clr_header_bg = "1F4E79"
        self.clr_header_text = "FFFFFF"
        self.clr_row_odd = "F2F2F2"
        self.clr_row_even = "FFFFFF"
        self.clr_income_bg = "C6EFCE"
        self.clr_income_text = "006100"
        self.clr_expense_bg = "FFC7CE"
        self.clr_expense_text = "9C0006"
        self.clr_total_bg = "D9E1F2"
        self.clr_warning_bg = "FFFF0000"

        self.fill_header = PatternFill(
            start_color=self.clr_header_bg,
            end_color=self.clr_header_bg,
            fill_type="solid",
        )
        self.fill_odd = PatternFill(
            start_color=self.clr_row_odd, end_color=self.clr_row_odd, fill_type="solid"
        )
        self.fill_even = PatternFill(
            start_color=self.clr_row_even,
            end_color=self.clr_row_even,
            fill_type="solid",
        )
        self.fill_income = PatternFill(
            start_color=self.clr_income_bg,
            end_color=self.clr_income_bg,
            fill_type="solid",
        )
        self.fill_expense = PatternFill(
            start_color=self.clr_expense_bg,
            end_color=self.clr_expense_bg,
            fill_type="solid",
        )
        self.fill_total = PatternFill(
            start_color=self.clr_total_bg,
            end_color=self.clr_total_bg,
            fill_type="solid",
        )
        self.fill_warning = PatternFill(
            start_color=self.clr_warning_bg,
            end_color=self.clr_warning_bg,
            fill_type="solid",
        )

        self.font_header = Font(color=self.clr_header_text, bold=True, size=11)
        self.font_standard = Font(size=10)
        self.font_bold = Font(bold=True, size=10)
        self.font_income = Font(color=self.clr_income_text, bold=True)
        self.font_expense = Font(color=self.clr_expense_text, bold=True)
        self.font_total = Font(bold=True, size=11)

        thin = Side(style="thin", color="000000")
        self.border_thin = Border(left=thin, right=thin, top=thin, bottom=thin)
        self.border_medium = Border(
            left=Side(style="medium"),
            right=Side(style="medium"),
            top=Side(style="medium"),
            bottom=Side(style="medium"),
        )
        self.align_center = Alignment(
            horizontal="center", vertical="center", wrap_text=True
        )
        self.align_left = Alignment(
            horizontal="left", vertical="center", wrap_text=True
        )
        self.align_right = Alignment(horizontal="right", vertical="center")


logger = logging.getLogger(__name__)


WARNING_FIELD_TO_HEADER = {
    "amount": ["сумма платежа"],
    "income": ["приход"],
```

```
    "expense": ["расход"],
    "total": ["итог"],
    "commission": ["комиссия"],
    "percent_broker": ["% посредника", "% посредник"],
    "broker1": ["посредник 1(кэш)", "посредник 1"],
    "broker2": ["посредник 2"],
}


def _get_header_and_source_col(ws):
    source_col = 1 if ws.cell(1, 1).value == "Источник" else 2
    return str(ws.cell(1, source_col).value or "").lower(), source_col


def _safe_float(value):
    try:
        return float(value or 0)
    except (TypeError, ValueError):
        return 0.0


def _extract_money_snapshot(row_cells, cols):
    income = _safe_float(row_cells[cols["приход"] - 1].value) if cols["приход"] > 0 else 0.0
    expense = _safe_float(row_cells[cols["расход"] - 1].value) if cols["расход"] > 0 else 0.0
    total = _safe_float(row_cells[cols["итог"] - 1].value) if cols["итог"] > 0 else 0.0
    commission = (
        abs(_safe_float(row_cells[cols["комиссия"] - 1].value)) if cols["комиссия"] > 0 else 0.0
    )
    return income, expense, total, commission


def _is_return_row(rows_data, row_idx, return_keywords):
    if row_idx >= len(rows_data):
        return False
    filters_val = str(rows_data[row_idx].get("фильтры", "")).lower()
    return any(kw in filters_val for kw in return_keywords)


def _is_math_error(is_return, income, expense, total, commission):
    if is_return:
        return False
    has_money = abs(income) > 0.02 or abs(expense) > 0.02
    has_total_only = not has_money and abs(total) > 0.02

    if has_money and abs(total) < 0.02:
        return True
    if (
        has_money
        and abs(commission) < 0.02
        and abs(total - (income - expense)) > 0.02
    ):
        return True
    return has_total_only


def _apply_base_cell_style(cell, is_critical, is_math_err, fill_row, st: ExcelStyles):
    fill_critical = PatternFill(patternType="solid", fgColor="FF9999")
    cell.border = st.border_thin
    if is_critical:
        cell.fill = fill_critical
    elif is_math_err:
        cell.fill = PatternFill(patternType="solid", fgColor="FFFF00")
    else:
        cell.fill = fill_row
    cell.font = st.font_standard


def _apply_non_numeric_warning_fill(
    cell,
    c_idx,
    is_warning,
    is_critical,
    warning_target_col,
    source_col,
    st: ExcelStyles,
):
    cell.alignment = st.align_left
    if is_critical or not is_warning:
        return
    if warning_target_col is None and c_idx == source_col:
        cell.fill = st.fill_warning
        return
    if c_idx == warning_target_col:
```

```
        cell.fill = st.fill_warning


def _get_warning_target_column(ws, warning_info):
    if not warning_info:
        return None

    kind = warning_info.get("kind")
    reason = str(warning_info.get("reason") or "")
    if kind in {"hard_structure", "soft_structure"}:
        return None

    prefix = "text in numeric field: "
    if not reason.startswith(prefix):
        return None

    field = reason[len(prefix) :].strip()
    patterns = WARNING_FIELD_TO_HEADER.get(field)
    if not patterns:
        return None

    for i, cell in enumerate(ws[1], 1):
        header = str(cell.value or "").lower()
        if any(pattern in header for pattern in patterns):
            return i
    return None


def detect_header_columns(ws) -> tuple:
    num_kw = ["приход", "расход", "комиссия", "итог", "посредник", "транзит"]
    cols = {kw: -1 for kw in num_kw}
    num_indices = set()
    percent_broker_cols = set()

    for i, cell in enumerate(ws[1], 1):
        val = str(cell.value or "").lower()
        if any(kw in val for kw in num_kw) or val == "сумма":
            num_indices.add(i)
        for kw in num_kw:
            if kw in val:
                if kw == "комиссия" and "посред" in val:
                    continue
                cols[kw] = i
        if val == "% посредника" or val.startswith("% посредник"):
            percent_broker_cols.add(i)

    return cols, num_indices, percent_broker_cols


def apply_cell_styling(
    ws,
    rows_data,
    warning_rows,
    cols,
    num_indices,
    percent_broker_cols,
    st: ExcelStyles,
    return_keywords=None,
    row_to_data_idx=None,
):
    return_keywords = [str(v).lower() for v in (return_keywords or ["возврат"])]
    _, source_col = _get_header_and_source_col(ws)
    warning_target_cache = {}
    row_to_data_idx = row_to_data_idx or {}

    for r_idx, row_cells in enumerate(ws.iter_rows(min_row=2, max_row=ws.max_row)):
        actual_row = r_idx + 2
        data_idx = row_to_data_idx.get(actual_row, r_idx)
        warning_info = warning_rows.get(actual_row)
        is_warning = warning_info is not None
        fill_row = st.fill_odd if (r_idx + 1) % 2 == 1 else st.fill_even
        is_return = _is_return_row(rows_data, data_idx, return_keywords)
        warning_target_col = None
        if warning_info:
            warning_cache_key = (
                str(warning_info.get("kind") or ""),
                str(warning_info.get("reason") or ""),
            )
            warning_target_col = warning_target_cache.get(warning_cache_key)
            if warning_cache_key not in warning_target_cache:
                warning_target_col = _get_warning_target_column(ws, warning_info)
                warning_target_cache[warning_cache_key] = warning_target_col
```

```
        i_val, e_val, t_val, c_val = _extract_money_snapshot(row_cells, cols)

        is_critical = (
            rows_data[data_idx].get("is_critical", False)
            if 0 <= data_idx < len(rows_data)
            else False
        )
        is_math_err = _is_math_error(is_return, i_val, e_val, t_val, c_val)

        for c_idx, cell in enumerate(row_cells, 1):
            _apply_base_cell_style(cell, is_critical, is_math_err, fill_row, st)

            if c_idx in num_indices:
                format_numeric_cell(
                    cell,
                    c_idx,
                    cols,
                    ws,
                    percent_broker_cols,
                    is_warning,
                    is_critical,
                    st,
                    warning_target_col,
                    warning_info.get("kind") if warning_info else None,
                    is_math_err,
                )
            else:
                _apply_non_numeric_warning_fill(
                    cell,
                    c_idx,
                    is_warning,
                    is_critical,
                    warning_target_col,
                    source_col,
                    st,
                )


def format_numeric_cell(
    cell,
    c_idx,
    cols,
    ws,
    percent_broker_cols,
    is_warning,
    is_critical,
    st: ExcelStyles,
    warning_target_col=None,
    warning_kind=None,
    is_math_err=False,
):
    val = cell.value or 0

    if c_idx == cols["приход"] and isinstance(val, (int, float, Decimal)) and val > 0.01:
        if not is_critical and not is_math_err:
            cell.fill = st.fill_income
        cell.font = st.font_income
    elif c_idx == cols["расход"] and isinstance(val, (int, float, Decimal)) and abs(val) > 0.01:
        if not is_critical and not is_math_err:
            cell.fill = st.fill_expense
        cell.font = st.font_expense

    is_percent_err = False
    if c_idx in percent_broker_cols:
        try:
            v_float = float(val)
            if abs(v_float) <= 1.0:
                cell.value = v_float
            elif abs(v_float) <= 100.0:
                cell.value = v_float / 100.0
            else:
                is_percent_err = True
                cell.value = v_float
        except (ValueError, TypeError):
            is_percent_err = True
            cell.value = val

        if is_percent_err:
            cell.fill = st.fill_warning

    h = str(ws.cell(1, c_idx).value or "").lower()
    if "%" in h and "посред" in h:
        fmt = (
```

```
            "General" if is_percent_err and isinstance(cell.value, str) else "#,##0.00"
        )
        if not is_percent_err:
            fmt = "0.##%" if "посредник 2" in h else "0.00%"
    elif "комиссия" in h:
        fmt = _commission_percent_format(cell.value)
    elif "транзит" in h:
        fmt = "#,##0"
    else:
        fmt = "#,##0.00"

    cell.alignment, cell.number_format = st.align_right, fmt

    if is_warning and not is_critical and warning_target_col == c_idx:
        cell.fill = st.fill_warning


def _commission_percent_format(value) -> str:
    try:
        numeric_value = float(value)
    except (TypeError, ValueError):
        return '0.##"%"'
    if abs(numeric_value - round(numeric_value)) < 0.0000001:
        return '0"%"'
    return '0.##"%"'


def build_total_row(ws, num_indices, st: ExcelStyles):
    total_row = ws.max_row + 1
    total_label = ws.cell(total_row, 1, "ИТОГО:")
    total_label.font, total_label.fill, total_label.border = (
        st.font_total,
        st.fill_total,
        st.border_medium,
    )

    last_data_row = total_row - 1
    for c in range(1, ws.max_column + 1):
        if c not in num_indices:
            continue
        h = str(ws.cell(1, c).value or "").lower()
        if (
            any(kw in h for kw in ["приход", "расход", "итог", "посредник"])
            and "%" not in h
        ):
            let = get_column_letter(c)
            cell = ws.cell(total_row, c)
            if last_data_row < 2:
                cell.value = 0
            else:
                cell.value = f"=SUM({let}2:{let}{last_data_row})"
            cell.fill, cell.font, cell.border, cell.number_format = (
                st.fill_total,
                st.font_total,
                st.border_medium,
                "#,##0.00",
            )

    return total_row
```

APP\core\infra\export_writer.py

648 lines • 28,846 bytes

```
import os
import logging
from datetime import datetime
from decimal import Decimal
from typing import List, Dict, Any, Optional
from pathlib import Path
import openpyxl
from openpyxl.utils import get_column_letter
from openpyxl.styles import PatternFill
from core.domain import IExporter
from core.domain.utils_helpers import to_decimal
from core.infra import is_file_writable
from core.infra.excel_backup import backup_existing_excel_output, resolve_excel_backup_keep
from core.infra.export_formatting import ExcelStyles
from core.infra.export_formatting import (
    detect_header_columns,
    apply_cell_styling,
    build_total_row,
)
from core.infra.excel_hyperlinks import apply_cell_hyperlink

logger = logging.getLogger(__name__)


class ExportPreflightError(RuntimeError):
    pass


def _safe_text(value: Any) -> Any:
    """Mitigate Excel formula injection (CWE-1236).

    Prepends a single quote to any string that starts with ``=``, ``+``,
    ``-`` or ``@`` so Excel treats it as text instead of evaluating it as
    a formula. Non-string values (Decimal, int, float, None) pass through
    unchanged. Mirrors the existing ``_excel_safe_text`` helpers in
    ``reconciliation/_utils.py`` and ``rsch/_format.py``.
    """
    if isinstance(value, str):
        stripped = value.lstrip()
        if stripped and stripped[0] in ("=", "+", "-", "@"):
            return "'" + value
    return value


def preflight_output_path(path: Path) -> None:
    output_path = Path(path)
    if not output_path.parent.exists():
        raise ExportPreflightError(f"Output directory does not exist: {output_path.parent}")
    if not os.access(output_path.parent, os.W_OK):
        raise ExportPreflightError(f"Output directory is not writable: {output_path.parent}")

DEFAULT_HEADERS = [
    "Источник",
    "ЗАКАЗЧИК",
    "ИСПОЛНИТЕЛЬ",
    "Приход",
    "Расход",
    "Комиссия",
    "ИТОГ",
    "ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ",
    "% посредника",
    "посредник 1(кэш)",
    "посредник 2",
    "Фильтры",
]


class MasterWriter(IExporter):
    def __init__(self, config: Dict[str, Any], config_path: str = ""):
        self.config = config
        self.config_dir = Path(config_path).parent if config_path else Path.cwd()
        logic_cfg = config.get("logic", {}) if isinstance(config, dict) else {}
        self.return_keywords = [
            str(v).lower() for v in logic_cfg.get("return_keywords", ["возврат"])
        ]
        p = (
            config.get("paths", {})
            if isinstance(config, dict)
            else getattr(config, "paths", {})
        )
        master = p.get("master_file", "Master_Report.xlsx")
        master_path = Path(master)
        if not master_path.is_absolute():
```

```
            master_path = self.config_dir / master_path
        self.master_path = str(master_path)
        self.st = ExcelStyles()

    def _apply_header_style(self, ws, row_idx=1):
        for cell in ws[row_idx]:
            cell.fill, cell.font, cell.alignment, cell.border = (
                self.st.fill_header,
                self.st.font_header,
                self.st.align_center,
                self.st.border_thin,
            )

    def _get_workbook(self, path=None):
        path_abs = os.path.abspath(path or self.master_path)
        if os.path.exists(path_abs):
            try:
                return openpyxl.load_workbook(path_abs)
            except (OSError, openpyxl.utils.exceptions.InvalidFileException) as e:
                logger.error("Failed to open workbook %s: %s", path_abs, e)
                return None
        wb = openpyxl.Workbook()
        if "Sheet" in wb.sheetnames:
            wb.remove(wb["Sheet"])
        return wb

    def _save_workbook(self, wb, path=None) -> bool:
        try:
            return self._save_workbook_inner(wb, path)
        finally:
            try:
                wb.close()
            except Exception:
                pass

    def _save_workbook_inner(self, wb, path=None) -> bool:
        SPECIAL_ORDER = ["ВСЕ ОПЕРАЦИИ", "ДАШБОРД", "АНАЛИТИКА", "СВОД"]

        def key(n):
            if "." in n:
                try:
                    p = n.split(".")
                    if len(p) == 3:
                        return (0, int(p[2]), int(p[1]), int(p[0]))
                    return (0, 9999, int(p[1]), int(p[0]))
                except (ValueError, IndexError):
                    return (0, 9999, 99, 99)
            return (1, 0, 0, SPECIAL_ORDER.index(n) if n in SPECIAL_ORDER else 99)

        if not wb.sheetnames:
            wb.create_sheet("СВОД")

        for i, n in enumerate(sorted(wb.sheetnames, key=key)):
            curr = wb.sheetnames.index(n)
            offset = i - curr
            if offset != 0:
                wb.move_sheet(n, offset=offset)

        path_abs = os.path.abspath(path or self.master_path)
        try:
            Path(path_abs).parent.mkdir(parents=True, exist_ok=True)
        except PermissionError as e:
            logger.error(
                f"Не удалось создать папку для отчёта (нет доступа): {path_abs} | {e}"
            )
            return False
        preflight_output_path(Path(path_abs))
        if os.path.exists(path_abs):
            backup_cfg = self.config.get("backup", {}) if isinstance(self.config, dict) else {}
            paths_cfg = self.config.get("paths", {}) if isinstance(self.config, dict) else {}
            backup_root = paths_cfg.get("archive_dir")
            if backup_root:
                backup_root = Path(backup_root)
                if not backup_root.is_absolute():
                    backup_root = self.config_dir / backup_root
            backup_keep = resolve_excel_backup_keep(backup_cfg.get("excel_keep"))
            try:
                backup_existing_excel_output(path_abs, backup_root, keep=backup_keep)
            except Exception as e:
                logger.error("Could not create backup before overwriting %s: %s", path_abs, e)
                return False
        tmp = path_abs + ".tmp"
```

```
        if os.name == "nt" and len(tmp) > 250:
            tmp = "\\\\?\\" + tmp
            path_abs = "\\\\?\\" + path_abs

        try:
            wb.save(tmp)
            if os.path.exists(path_abs):
                try:
                    os.replace(tmp, path_abs)
                except PermissionError:
                    # Do NOT delete the tmp file — the user can rename it
                    # manually after closing Excel.  Previously the tmp was
                    # deleted here, destroying a successfully generated
                    # report and forcing a full re-run.
                    logger.error(
                        "Файл занят: Закройте '%s' в Excel! "
                        "Временный файл сохранён: %s",
                        path or self.master_path, tmp,
                    )
                    return False
            else:
                os.rename(tmp, path_abs)
            return True
        except Exception as e:
            logger.error(f"Save failed: {e}")
            # Preserve tmp on error too — do not remove it in the finally
            # block unless the rename succeeded.
            return False

    def _auto_adjust(self, ws):
        numeric_keywords = [
            "приход",
            "расход",
            "комиссия",
            "итог",
            "посредник",
            "транзит",
            "сумма",
        ]
        wide_text_keywords = ["заказчик", "исполнитель", "источник", "комментар"]
        compact_text_min_widths = {
            "статус": 14,
            "фильтры": 16,
        }
        for i in range(1, ws.max_column + 1):
            max_len = 0
            for row in ws.iter_rows(
                min_row=1, max_row=min(20, ws.max_row), values_only=True
            ):
                if row[i - 1]:
                    max_len = max(max_len, len(str(row[i - 1])))
            if ws.max_row > 20:
                tail_value = ws.cell(ws.max_row, i).value
                if tail_value:
                    max_len = max(max_len, len(str(tail_value)))
            let = get_column_letter(i)
            header = str(ws.cell(1, i).value or "").lower()
            if any(kw in header for kw in numeric_keywords):
                ws.column_dimensions[let].width = max((max_len * 1.1) + 6, 18)
            elif any(kw in header for kw in wide_text_keywords):
                ws.column_dimensions[let].width = min(max(max_len + 4, 18), 60)
            else:
                min_width = max(
                    [width for key, width in compact_text_min_widths.items() if key in header],
                    default=10,
                )
                ws.column_dimensions[let].width = min(max(max_len + 4, min_width), 60)

    def _append_data_rows(self, ws, date, rows, curator=None):
        from collections import OrderedDict

        warning_rows = {}
        row_to_data_idx = {}

        def _safe_decimal(v: Any, default: Decimal = Decimal("0")) -> Decimal:
            """Coerce a financial cell value to Decimal without float loss.

            The consolidated rows already carry Decimal values (built by
            LogicService._build_result_row); float() here only hid that
            fact and risked precision loss for >15-significant-digit
            sums. Decimal passthrough is the honest contract — openpyxl
            writes it straight into the cell's numeric value. Garbage
            cells fall back to the domain parser and then to the default.
```

```
            """
            if v is None or v == "":
                return Decimal(default)
            if isinstance(v, Decimal):
                return v
            if isinstance(v, bool):
                # bool is an int subclass — guard before the int/float branch.
                return Decimal("1") if v else Decimal("0")
            if isinstance(v, (int, float)):
                return Decimal(str(v))
            try:
                return to_decimal(v)
            except Exception:
                logger.warning(
                    "Could not convert %r to Decimal; defaulting to 0. Sheet=%s, row=%s",
                    v,
                    ws.title,
                    ws.max_row + 1,
                )
                return Decimal(default)

        def _get_employee(r):
            if curator == "ivan":
                return str(r.get("Сотрудник", r.get("Источник", ""))).strip()
            return str(r.get("Источник", "")).strip()

        def _get_row_values(r):
            if curator == "boris":
                return [_safe_text(v) for v in [
                    r.get("Источник", ""),
                    r.get("ЗАКАЗЧИК (название проекта, наименование ЮЛ)", ""),
                    r.get("ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)", ""),
                    _safe_decimal(r.get("сумма платежа ПРИХОД", 0)),
                    _safe_decimal(r.get("сумма платежа РАСХОД", 0)),
                    _safe_decimal(r.get("комиссия", 0)),
                    _safe_decimal(r.get("ИТОГ", 0)),
                    _safe_decimal(r.get("% посредника", 0)),
                    _safe_decimal(r.get("посредник 1(кэш)", 0)),
                    _safe_decimal(r.get("% посредник 2", 0)),
                    _safe_decimal(r.get("посредник 2", 0)),
                    r.get("фильтры", ""),
                ]]
            elif curator == "ivan":
                return [_safe_text(v) for v in [
                    r.get("Сотрудник", r.get("Источник", "")),
                    r.get("ЗАКАЗЧИК (название проекта, наименование ЮЛ)", ""),
                    r.get("ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)", ""),
                    _safe_decimal(r.get("сумма платежа ПРИХОД", 0)),
                    _safe_decimal(r.get("сумма платежа РАСХОД", 0)),
                    _safe_decimal(r.get("комиссия", 0)),
                    _safe_decimal(r.get("ИТОГ", 0)),
                    _safe_decimal(r.get("ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ", 0)),
                    _safe_decimal(r.get("% посредника", 0)),
                    _safe_decimal(r.get("посредник 1(кэш)", 0)),
                    _safe_decimal(r.get("посредник 2", 0)),
                    _safe_decimal(r.get("курс", 0)),
                    _safe_decimal(r.get("сумма", 0)),
                    r.get("валюта", ""),
                    r.get("фильтры", ""),
                ]]
            else:
                return [_safe_text(v) for v in [
                    r.get("Источник", ""),
                    r.get("ЗАКАЗЧИК (название проекта, наименование ЮЛ)", ""),
                    r.get("ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)", ""),
                    _safe_decimal(r.get("сумма платежа ПРИХОД", 0)),
                    _safe_decimal(r.get("сумма платежа РАСХОД", 0)),
                    _safe_decimal(r.get("комиссия", 0)),
                    _safe_decimal(r.get("ИТОГ", 0)),
                    _safe_decimal(r.get("ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ", 0)),
                    _safe_decimal(r.get("% посредника", 0)),
                    _safe_decimal(r.get("посредник 1(кэш)", 0)),
                    _safe_decimal(r.get("посредник 2", 0)),
                    r.get("фильтры", ""),
                ]]

        if curator in ("boris", "ivan"):
            grouped = OrderedDict()
            for r in rows:
                emp = _get_employee(r)
                grouped.setdefault(emp, []).append(r)

            for emp_name, emp_rows in grouped.items():
```

```
            header_row_num = ws.max_row + 1
            ws.append([_safe_text(emp_name)] + [None] * (ws.max_column - 1))
            header_cell = ws.cell(header_row_num, 1)
            header_cell.font = self.st.font_bold
            header_cell.fill = PatternFill("solid", fgColor="D9E1F2")
            from openpyxl.styles import Border as _Border, Side as _Side
            med = _Side(style="medium", color="000000")
            header_cell.border = _Border(left=med, right=med, top=med, bottom=med)
            for c in range(2, ws.max_column + 1):
                ws.cell(header_row_num, c).fill = PatternFill("solid", fgColor="D9E1F2")
                ws.cell(header_row_num, c).border = _Border(left=med, right=med, top=med, bottom=med)

            for r in emp_rows:
                warning_kind = r.get("_template_warning_kind")
                suppress_structure_warning = curator in {"boris", "ivan"} and warning_kind in {
                    "hard_structure",
                    "soft_structure",
                }
                if r.get("is_template_warning") and not suppress_structure_warning:
                    warning_rows[ws.max_row + 1] = {
                        "kind": warning_kind,
                        "reason": r.get("_template_warning_reason"),
                    }
                ws.append(_get_row_values(r))
                data_row = ws.max_row
                try:
                    data_idx = rows.index(r)
                except ValueError:
                    data_idx = -1
                if data_idx >= 0:
                    row_to_data_idx[data_row] = data_idx
                apply_cell_hyperlink(
                    ws.cell(ws.max_row, 1),
                    r.get("_source_file_path"),
                    r.get("_source_sheet_name"),
                    r.get("_source_row_index"),
                )

        return warning_rows, row_to_data_idx

    for i, r in enumerate(rows):
        warning_kind = r.get("_template_warning_kind")
        if r.get("is_template_warning") and not (curator in {"boris", "ivan"} and warning_kind in {
            "hard_structure",
            "soft_structure",
        }):
            warning_rows[ws.max_row + 1] = {
                "kind": warning_kind,
                "reason": r.get("_template_warning_reason"),
            }
        if curator == "boris":
            ws.append(
                [_safe_text(v) for v in [
                    r.get("Источник", ""),
                    r.get("ЗАКАЗЧИК (название проекта, наименование ЮЛ)", ""),
                    r.get("ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)", ""),
                    _safe_decimal(r.get("сумма платежа ПРИХОД", 0)),
                    _safe_decimal(r.get("сумма платежа РАСХОД", 0)),
                    _safe_decimal(r.get("комиссия", 0)),
                    _safe_decimal(r.get("ИТОГ", 0)),
                    _safe_decimal(r.get("% посредника", 0)),
                    _safe_decimal(r.get("посредник 1(кэш)", 0)),
                    _safe_decimal(r.get("% посредник 2", 0)),
                    _safe_decimal(r.get("посредник 2", 0)),
                    r.get("фильтры", ""),
                ]]
            )
        elif curator == "ivan":
            ws.append(
                [_safe_text(v) for v in [
                    r.get("Сотрудник", r.get("Источник", "")),
                    r.get("ЗАКАЗЧИК (название проекта, наименование ЮЛ)", ""),
                    r.get("ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)", ""),
                    _safe_decimal(r.get("сумма платежа ПРИХОД", 0)),
                    _safe_decimal(r.get("сумма платежа РАСХОД", 0)),
                    _safe_decimal(r.get("комиссия", 0)),
                    _safe_decimal(r.get("ИТОГ", 0)),
                    _safe_decimal(r.get("ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ", 0)),
                    _safe_decimal(r.get("% посредника", 0)),
                    _safe_decimal(r.get("посредник 1(кэш)", 0)),
                    _safe_decimal(r.get("посредник 2", 0)),
                    _safe_decimal(r.get("курс", 0)),
                    _safe_decimal(r.get("сумма", 0)),
```

```
                    r.get("валюта", ""),
                    r.get("фильтры", ""),
                ]]
            )
        else:
            ws.append(
                [_safe_text(v) for v in [
                    r.get("Источник", ""),
                    r.get("ЗАКАЗЧИК (название проекта, наименование ЮЛ)", ""),
                    r.get("ИСПОЛНИТЕЛЬ (наименование ЮЛ, банк)", ""),
                    _safe_decimal(r.get("сумма платежа ПРИХОД", 0)),
                    _safe_decimal(r.get("сумма платежа РАСХОД", 0)),
                    _safe_decimal(r.get("комиссия", 0)),
                    _safe_decimal(r.get("ИТОГ", 0)),
                    _safe_decimal(r.get("ОТПРАВКА ТРАНЗИТНЫХ ОБЯЗАТЕЛЬСТВ", 0)),
                    _safe_decimal(r.get("% посредника", 0)),
                    _safe_decimal(r.get("посредник 1(кэш)", 0)),
                    _safe_decimal(r.get("посредник 2", 0)),
                    r.get("фильтры", ""),
                ]]
            )
        data_row = ws.max_row
        row_to_data_idx[data_row] = i
        apply_cell_hyperlink(
            ws.cell(ws.max_row, 1),
            r.get("_source_file_path"),
            r.get("_source_sheet_name"),
            r.get("_source_row_index"),
        )
    return warning_rows, row_to_data_idx

def _write_template_errors_sheet(self, wb, template_errors: List[Dict[str, Any]]):
    sheet_name = "ОШИБКИ ШАБЛОНА"
    if sheet_name in wb.sheetnames:
        del wb[sheet_name]
    # Template errors are stored in DB/session metadata only.
    # Do not render a separate workbook sheet.
    return

def write_daily_data(
    self, date, rows, wb=None, sh_name_override=None, curator=None, headers=None
):
    should_save = False
    owns_wb = False
    if wb is None:
        wb = self._get_workbook()
        owns_wb = True
        should_save = True

    try:
        sh_name = sh_name_override or date.strftime("%d.%m.%Y")
        if sh_name in wb.sheetnames:
            del wb[sh_name]
        ws = wb.create_sheet(sh_name)
        # Make the newly created sheet active so wb.active access works as expected
        wb.active = wb.sheetnames.index(sh_name)

        ws.append(headers or DEFAULT_HEADERS)
        self._apply_header_style(ws)
        ws.freeze_panes = "A2"

        warning_rows, row_to_data_idx = self._append_data_rows(ws, date, rows, curator=curator)

        cols, num_indices, percent_broker_cols = detect_header_columns(ws)

        apply_cell_styling(
            ws,
            rows,
            warning_rows,
            cols,
            num_indices,
            percent_broker_cols,
            self.st,
            self.return_keywords,
            row_to_data_idx=row_to_data_idx,
        )

        build_total_row(ws, num_indices, self.st)

        self._auto_adjust(ws)
        if should_save:
            if not self._save_workbook(wb):
                logger.error("Failed to save workbook in write_daily_data()")
```

```
        finally:
            # When we opened the workbook ourselves, guarantee it is
            # closed even if an exception aborted the build before the
            # save branch ran (otherwise the handle stays locked and a
            # retry hits "file is busy" on Windows).
            if owns_wb:
                try:
                    wb.close()
                except Exception:
                    pass

    def write_empty_daily_report(
        self,
        path: Path,
        date,
        curator=None,
        curator_config=None,
    ) -> bool:
        headers = None
        if curator_config and "headers" in curator_config:
            headers = curator_config["headers"]
        wb = self._get_workbook(path=str(path))
        if wb is None:
            return False
        try:
            self.write_daily_data(
                date,
                [],
                wb=wb,
                curator=curator,
                headers=headers,
            )
            return self._save_workbook(wb, path=str(path))
        finally:
            # _save_workbook closes wb on success; close explicitly on the
            # error path so a failed empty-report build doesn't leave the
            # file handle locked.
            try:
                wb.close()
            except Exception:
                pass

    def write_report(
        self,
        data: List[Dict[str, Any]],
        path: Path,
        template_errors: Optional[List[Dict[str, Any]]] = None,
        curator=None,
        curator_config=None,
    ) -> bool:
        if not is_file_writable(str(path)):
            logger.error(
                f"Файл занят: {os.path.abspath(str(path))}. Закройте его в Excel перед началом!"
            )
            return False

        headers = None
        if curator_config and "headers" in curator_config:
            headers = curator_config["headers"]

        try:
            today = datetime.now().date()
            dbd = {}
            for r in data:
                if r is None:
                    logger.warning(
                        "TEMPORAL FILTER: None row encountered and will be skipped."
                    )
                    continue
                d = r.get("Дата")
                if not d:
                    logger.warning(
                        "TEMPORAL FILTER: Row from %s has no date and will be skipped.",
                        r.get("Источник"),
                    )
                    continue

                if hasattr(d, "strftime") and hasattr(d, "date"):
                    d_str = d.strftime("%d.%m.%Y")
                    dt = d
                else:
                    try:
                        dt = datetime.strptime(str(d), "%d.%m.%Y")
```

```
                    d_str = dt.strftime("%d.%m.%Y")
                except ValueError:
                    try:
                        dt = datetime.strptime(str(d)[:10], "%Y-%m-%d")
                        d_str = dt.strftime("%d.%m.%Y")
                    except ValueError:
                        logger.warning(
                            "TEMPORAL FILTER: Row from %s has invalid date '%s' and will be skipped.",
                            r.get("Источник"),
                            d,
                        )
                        continue

            if dt.date() > today:
                logger.warning(
                    "TEMPORAL FILTER: Future date %s from %s will be skipped.",
                    d,
                    r.get("Источник"),
                )
                try:
                    from core.domain.data_loss_log import record_loss
                    record_loss(
                        source=str(path),
                        kind="future_date",
                        reason=f"date {d} is in the future",
                        field="Дата",
                        original_value=d,
                    )
                except Exception:
                    pass
                continue

            dbd.setdefault(d_str, []).append(r)

        if not dbd:
            logger.error("Write error: no valid dated rows after temporal filtering")
            return False

        wb = self._get_workbook(path=str(path))
        if wb is None:
            return False
        try:
            for d_str, rows in sorted(dbd.items()):
                try:
                    dt = datetime.strptime(d_str, "%d.%m.%Y")
                    self.write_daily_data(
                        dt,
                        rows,
                        wb=wb,
                        curator=curator,
                        headers=headers,
                    )
                except Exception as e:
                    logger.error(f"Date error {d_str}: {e}")
            self._write_template_errors_sheet(wb, template_errors or [])
            return self._save_workbook(wb, path=str(path))
        finally:
            # _save_workbook closes wb on success; this guarantees
            # close on the error paths too (exception raised between
            # open and save left the handle locked on disk).
            try:
                wb.close()
            except Exception:
                pass
    except Exception as e:
        logger.error(f"Write error: {e}")
        return False
```

APP\core\infra\ingestion_engine.py

464 lines • 17,963 bytes

```
import os
import logging
import time
import json
import subprocess
import sys
import threading
import zipfile
from pathlib import Path
from typing import List, Dict, Any
import xml.etree.ElementTree as ET
from python_calamine import CalamineWorkbook

from core.domain import IDataSource
from core.infra.utils import get_local_copy
from core.infra.ingestion_reader import CalamineReader, extract_date_from_sheet

logger = logging.getLogger(__name__)

_FILE_READ_RETRIES = 3
_FILE_READ_BACKOFF = 1.0
_APP_ROOT = Path(__file__).resolve().parents[2]
_IGNORED_DATE_SHEETS = {"АНАЛИТИКА", "СВОД"}
_WORKBOOK_XML = "xl/workbook.xml"
_SPREADSHEET_NS = "{http://schemas.openxmlformats.org/spreadsheetml/2006/main}"


def _extract_dates_from_sheet_names(sheet_names: List[str]) -> List[str]:
    dates = set()
    for sheet_name in sheet_names:
        if sheet_name.startswith("_") or sheet_name in _IGNORED_DATE_SHEETS:
            continue
        sheet_date = extract_date_from_sheet(sheet_name)
        if sheet_date:
            dates.add(sheet_date)
    return sorted(dates)


def _get_xlsx_sheet_dates_from_metadata(target_path: str | Path) -> List[str]:
    with zipfile.ZipFile(target_path) as archive:
        workbook_xml = archive.read(_WORKBOOK_XML)
    root = ET.fromstring(workbook_xml)
    sheet_names = [
        str(sheet.attrib.get("name", ""))
        for sheet in root.findall(f".//{_SPREADSHEET_NS}sheet")
    ]
    return _extract_dates_from_sheet_names(sheet_names)


def _try_get_xlsx_sheet_dates_from_metadata(target_path: str | Path) -> List[str] | None:
    if not str(target_path).lower().endswith(".xlsx"):
        return None
    try:
        return _get_xlsx_sheet_dates_from_metadata(target_path)
    except (OSError, KeyError, zipfile.BadZipFile, ET.ParseError) as e:
        logger.debug(
            "Fast xlsx sheet metadata read failed for %s: %s",
            target_path,
            e,
        )
        return None


def _run_calamine_worker(
    mode: str,
    target_path: str,
    markers,
    aliases,
    strict_io: bool = False,
    target_date_str: str | None = None,
    start_date_str: str | None = None,
    end_date_str: str | None = None,
    cancel_event: threading.Event | None = None,
) -> Dict[str, Any]:
    script = r"""
import json
import importlib.util
import sys
import types
from pathlib import Path

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
```

```
if hasattr(sys.stderr, "reconfigure"):
    sys.stderr.reconfigure(encoding="utf-8", errors="replace")

app_root = Path(sys.argv[6])


def _load_module(name, file_path):
    spec = importlib.util.spec_from_file_location(name, file_path)
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    return module


core_pkg = types.ModuleType("core")
core_pkg.__path__ = [str(app_root / "core")]
sys.modules.setdefault("core", core_pkg)

domain_pkg = types.ModuleType("core.domain")
domain_pkg.__path__ = [str(app_root / "core" / "domain")]
sys.modules["core.domain"] = domain_pkg

infra_pkg = types.ModuleType("core.infra")
infra_pkg.__path__ = [str(app_root / "core" / "infra")]
sys.modules["core.infra"] = infra_pkg

_load_module("core.domain.utils_helpers", app_root / "core" / "domain" / "utils_helpers.py")
_load_module("core.infra.utils", app_root / "core" / "infra" / "utils.py")
reader_mod = _load_module("core.infra.ingestion_reader", app_root / "core" / "infra" / "ingestion_reader.py")

from python_calamine import CalamineWorkbook

CalamineReader = reader_mod.CalamineReader
extract_date_from_sheet = reader_mod.extract_date_from_sheet


mode = sys.argv[1]
target_path = sys.argv[2]
markers = json.loads(sys.argv[3])
aliases = json.loads(sys.argv[4])
strict_io = json.loads(sys.argv[5])
target_date_str = json.loads(sys.argv[7])
start_date_str = json.loads(sys.argv[8])
end_date_str = json.loads(sys.argv[9])

if mode == "dates":
    dates = set()
    wb = CalamineWorkbook.from_path(target_path)
    for sheet_name in wb.sheet_names:
        if sheet_name.startswith("_") or sheet_name in ("АНАЛИТИКА", "СВОД"):
            continue
        sheet_date = extract_date_from_sheet(sheet_name)
        if sheet_date:
            dates.add(sheet_date)
    print(json.dumps({"ok": True, "result": sorted(dates)}, ensure_ascii=False))
elif mode == "rows":
    reader = CalamineReader(markers, aliases)
    rows = reader.read(
        Path(target_path),
        strict_io=strict_io,
        target_date_str=target_date_str,
        start_date_str=start_date_str,
        end_date_str=end_date_str,
    )
    print(json.dumps({"ok": True, "result": rows}, ensure_ascii=False, default=str))
else:
    print(json.dumps({"ok": False, "error": f"Unknown mode: {mode}"}, ensure_ascii=False))
    raise SystemExit(2)
"""

    cmd = [
        sys.executable,
        "-c",
        script,
        mode,
        target_path,
        json.dumps(markers, ensure_ascii=False),
        json.dumps(aliases, ensure_ascii=False),
        json.dumps(strict_io),
        str(_APP_ROOT),
        json.dumps(target_date_str),
        json.dumps(start_date_str),
        json.dumps(end_date_str),
```

```
    ]
    cwd = str(Path(__file__).resolve().parents[3])
    env = {**os.environ, "PYTHONIOENCODING": "utf-8"}
    timeout = 300

    # Use Popen instead of subprocess.run so we can poll the cancel_event
    # while the worker is running.  A synchronous subprocess.run with
    # timeout=300 blocks for up to 5 minutes on a hung network drive (Q:\),
    # ignoring the user's Cancel button the entire time.
    proc = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
        encoding="utf-8",
        errors="replace",
        cwd=cwd,
        env=env,
    )
    try:
        deadline = time.monotonic() + timeout
        while proc.poll() is None:
            if cancel_event is not None and cancel_event.is_set():
                proc.terminate()
                try:
                    proc.wait(timeout=5)
                except subprocess.TimeoutExpired:
                    proc.kill()
                    proc.wait(timeout=3)
                return {"ok": False, "error": "cancelled by user"}
            remaining = deadline - time.monotonic()
            if remaining <= 0:
                proc.terminate()
                try:
                    proc.wait(timeout=5)
                except subprocess.TimeoutExpired:
                    proc.kill()
                    proc.wait(timeout=3)
                return {"ok": False, "error": "worker timed out"}
            # Poll at 0.5 s intervals — fast enough for cancel, cheap on CPU
            try:
                proc.wait(timeout=min(0.5, remaining))
            except subprocess.TimeoutExpired:
                continue
    except Exception as exc:
        try:
            proc.kill()
        except OSError:
            pass
        return {"ok": False, "error": str(exc)}

    stdout = (proc.stdout or "").strip()
    stderr = (proc.stderr or "").strip()
    if proc.returncode != 0:
        return {
            "ok": False,
            "error": stderr or stdout or f"worker exited with code {proc.returncode}",
        }

    try:
        payload = json.loads(stdout)
    except json.JSONDecodeError:
        return {
            "ok": False,
            "error": stderr or stdout or "worker returned invalid json",
        }

    if stderr:
        payload["stderr"] = stderr
    return payload


def _should_isolate_calamine(file_path: Path | str) -> bool:
    normalized = str(file_path).replace("\\", "/").lower()
    if "/reports/" not in f"/{normalized.strip('/')}":
        return False
    if "crash" in normalized:
        return True
    try:
        if os.path.exists(file_path):
            return os.path.getsize(file_path) > 15 * 1024 * 1024
    except Exception:
        pass
```

```
    return True


class IngestionEngine(IDataSource):
    def __init__(
        self, config_markers: Dict[str, str], drive_letters: tuple = ("Q", "Z")
    ):
        self.markers = config_markers
        self.drive_letters = tuple(d.upper() for d in drive_letters)
        self.config = {}
        self.aliases = {
            "amount": ["сумма платежа", "бн рф руб", "бн", "приход", "расход"],
            "income": ["приход"],
            "expense": ["расход"],
            "customer": ["заказчик", "агент"],
            "contractor": ["исполнитель", "вторая сторона"],
            "total": ["итог", "итоги дня", "итог дня", "формула"],
            "transit_obligation": ["отправка транзитных обязательств"],
            "commission": ["комиссия", "ставка/цена"],
            "percent_broker": ["% посредника", "%"],
            "broker1": [
                "посредник 1(кэш)",
                "посредник 1",
                "комиссия посредника 1",
                "приходный кассовый ордер",
            ],
            "broker2": ["посредник 2", "комиссия посредника 2", "%.1"],
            "exchange_rate": ["курс"],
            "currency_amount": ["сумма"],
            "currency": ["валюта"],
            "filters": ["фильтры", "фильтр", "filter"],
        }
        self.reader = CalamineReader(self.markers, self.aliases, self.drive_letters)

    def _should_isolate_calamine(self, file_path: Path | str) -> bool:
        if self.config.get("performance", {}).get("isolate_calamine") is False:
            return False
        return _should_isolate_calamine(file_path)

    def _get_available_dates_direct(self, target_path: str) -> List[str]:
        wb = CalamineWorkbook.from_path(target_path)
        return _extract_dates_from_sheet_names(list(wb.sheet_names))

    def get_available_dates(
        self,
        file_path: Path,
        remote_mtime: float | None = None,
        remote_size: int | None = None,
        strict_io: bool = False,
    ) -> List[str]:
        dates = set()
        f_path_abs = os.path.abspath(str(file_path))
        should_isolate = self._should_isolate_calamine(file_path)

        target_path, temp_file = get_local_copy(
            f_path_abs,
            self.drive_letters,
            remote_mtime=remote_mtime,
            remote_size=remote_size,
        )

        try:
            if should_isolate:
                metadata_dates = _try_get_xlsx_sheet_dates_from_metadata(target_path)
                if metadata_dates is not None:
                    return metadata_dates
            if should_isolate:
                payload = _run_calamine_worker(
                    "dates", target_path, self.markers, self.aliases
                )
                if not payload.get("ok"):
                    raise RuntimeError(
                        payload.get("error", "unknown calamine worker error")
                    )
                for d in payload.get("result", []):
                    if d:
                        dates.add(d)
                if payload.get("stderr"):
                    logger.warning(payload["stderr"])
            else:
                for d in self._get_available_dates_direct(target_path):
                    dates.add(d)
        except BaseException as e:
```

```
            if isinstance(e, (KeyboardInterrupt, SystemExit, GeneratorExit)):
                raise
            logger.error(f"Error getting dates from {file_path}: {e}")
            if strict_io:
                raise
        finally:
            if temp_file and os.path.exists(temp_file):
                try:
                    os.remove(temp_file)
                except OSError:
                    pass
        return sorted(list(dates))

    def scan_directory(self, root_path: Path) -> List[Dict[str, Any]]:
        res = []
        if not root_path.exists():
            logger.warning(f"[SCAN] Папка не найдена: {root_path}")
            return res
        logger.info(f"[SCAN] Сканирую папку: {root_path}...")
        scan_root = os.path.abspath(os.fspath(root_path))
        roots_to_visit = [scan_root]
        while roots_to_visit:
            current_root = roots_to_visit.pop()
            subdirs = []
            try:
                with os.scandir(current_root) as entries:
                    for entry in entries:
                        try:
                            if entry.is_dir(follow_symlinks=False):
                                subdirs.append(entry.path)
                                continue
                            if not entry.is_file(follow_symlinks=False):
                                continue
                            f = entry.name
                            if f.startswith("~$") or not f.lower().endswith(
                                (".xlsx", ".xlsb")
                            ):
                                continue
                            stat = entry.stat(follow_symlinks=False)
                            # СКИПИРУЕМ хэширование при сканировании (это ОЧЕНЬ медленно по сети)
                            res.append(
                                {
                                    "path": str(entry.path),
                                    "id": stat.st_ino,
                                    "fp": None,  # Хэш будет вычислен только при необходимости
                                    "size": stat.st_size,
                                    "mtime": stat.st_mtime,
                                }
                            )
                        except Exception as e:
                            logger.error(f"Error scanning file {entry.path}: {e}")
                roots_to_visit.extend(reversed(subdirs))
            except Exception as e:
                logger.error(f"Error scanning directory {current_root}: {e}")
                continue
        return res

    def read_file(
        self,
        file_path: Path,
        strict_io: bool = False,
        target_date_str: str | None = None,
        start_date_str: str | None = None,
        end_date_str: str | None = None,
    ) -> List[Dict[str, Any]]:
        """Read Excel file with retry logic for transient network errors."""
        if not self._should_isolate_calamine(file_path) and not file_path.exists():
            if strict_io:
                raise FileNotFoundError(f"Не найден файл отчета: {file_path}")
            return []
        last_err = None
        for attempt in range(_FILE_READ_RETRIES):
            temp_file = None
            try:
                if self._should_isolate_calamine(file_path):
                    f_path_abs = os.path.abspath(str(file_path))
                    target_path, temp_file = get_local_copy(
                        f_path_abs,
                        self.drive_letters,
                    )
                    try:
                        payload = _run_calamine_worker(
                            "rows",
```

```
                    target_path,
                    self.markers,
                    self.aliases,
                    strict_io=strict_io,
                    target_date_str=target_date_str,
                    start_date_str=start_date_str,
                    end_date_str=end_date_str,
                )
                if not payload.get("ok"):
                    raise RuntimeError(
                        payload.get("error", "unknown calamine worker error")
                    )
                if payload.get("stderr"):
                    logger.warning(payload["stderr"])
                return payload.get("result", [])
            finally:
                if temp_file is not None:
                    try:
                        Path(temp_file).unlink(missing_ok=True)
                    except OSError:
                        pass
        if target_date_str or (start_date_str and end_date_str):
            return self.reader.read(
                file_path,
                strict_io=strict_io,
                target_date_str=target_date_str,
                start_date_str=start_date_str,
                end_date_str=end_date_str,
            )
        return self.reader.read(file_path, strict_io=strict_io)
    except (OSError, IOError, PermissionError) as e:
        last_err = e
        logger.warning(
            f"File read retry {attempt + 1}/{_FILE_READ_RETRIES} for "
            f"{file_path}: {e}"
        )
        time.sleep(min(60, _FILE_READ_BACKOFF * (2**attempt)))
    except RuntimeError as e:
        last_err = e
        logger.error(f"Failed to read {file_path}: {e}")
        if strict_io:
            raise
        return []
if strict_io and last_err is not None:
    raise last_err
logger.error(
    f"Failed to read {file_path} after {_FILE_READ_RETRIES} retries: {last_err}"
)
return []
```

APP\core\infra\ingestion_reader.py

1118 lines • 41,158 bytes

```python
"""Ingestion Core — движок чтения Excel файлов.

Объединяет:
- CalamineReader — парсинг Excel через python-calamine
- extract_date_from_sheet — извлечение даты из имени листа
"""

import os
import re
import logging
from pathlib import Path
from datetime import datetime, date
from decimal import Decimal
from typing import List, Dict, Any, Optional

from python_calamine import CalamineWorkbook

from core.domain.utils_helpers import to_decimal, to_decimal_percent
from core.infra.utils import get_local_copy

logger = logging.getLogger(__name__)
MAX_HEADER_SCAN_ROWS = 6000
_TRASH_KEYWORDS = (
    "c16",
    "d16",
    "название клиента",
    "согласно платежному",
    "процентном формате",
    "итого",
    "всего",
    "итог",
    "итог дня",
    "итоги дня",
)
# Pre-compiled trash-row detector: _is_trash_row used to rebuild this
# regex (re.escape + re.search) once per cell × keyword — the profiler
# showed it dominated ingestion (~86% of wall time on a 10-file batch,
# 2.27M re.escape calls). Compiling the alternation once collapses
# that to a single regex search per cell. Word boundaries are kept via
# the same lookarounds so matching semantics are unchanged.
_TRASH_PATTERN = re.compile(
    r"(?<![a-яёa-z0-9])(?:"
    + "|".join(re.escape(tk) for tk in _TRASH_KEYWORDS if tk)
    + r")(?![a-яёa-z0-9])"
)
_NUMERIC_FIELDS = frozenset(
    {
        "amount",
        "income",
        "expense",
        "total",
        "transit_obligation",
        "commission",
        "percent_broker",
        "percent_broker2",
        "broker1",
        "broker2",
        "exchange_rate",
        "currency_amount",
    }
)
_PREFERRED_HEADER_ORDER = (
    "customer",
    "contractor",
    "amount",
    "income",
    "expense",
    "commission",
    "total",
    "transit_obligation",
    "percent_broker",
    "broker1",
    "percent_broker2",
    "broker2",
    "exchange_rate",
    "currency_amount",
    "currency",
    "filters",
)
# ponytail: тот же список, другое имя. _PREFERRED задаёт порядок дисплея
# заголовков в каноническом выводе, _STRICT — контракт шаблона для валидации
# структуры (required keys, column order, unexpected columns). Сейчас
# идентичны; разойдутся, если формат шаблона изменится — тогда развести.
```

```
_STRICT_TEMPLATE_ORDER = _PREFERRED_HEADER_ORDER
_STRICT_OPTIONAL_KEYS = frozenset(
    {
        "filters",
        "transit_obligation",
        "percent_broker2",
        "exchange_rate",
        "currency_amount",
        "currency",
        "income",
        "expense",
    }
)
_STRICT_SOFT_MISSING_KEYS = frozenset({"filters"})
_STRICT_HELPER_HEADERS = frozenset(
    {
        "итог дня",
        "итоги дня",
        "итоги дня (данные колонка должна быть скрытой)",
        "итоги дня (данная колонка должна быть скрытой)",
        "оплата постащику, выплата зарплатного реестра",
        "список р/с",
    }
)


def _normalize_header_text(value: Any) -> str:
    if value is None:
        return ""
    text = str(value).lower()
    text = text.replace("\xa0", " ").replace("\u200b", "").replace("\ufeff", "")
    text = "".join(ch for ch in text if ch.isprintable())
    text = re.sub(r"\s+", " ", text).strip()
    return text.rstrip(": ")


def extract_date_from_sheet(sheet_name: str, **kwargs) -> Optional[str]:
    """Извлечь дату из имени листа.

    Поддерживаемые форматы: DD.MM.YYYY, DD.MM.YY, DD,MM,YY и т.д.
    Если год не указан — лист считается некорректным и игнорируется.

    Если передан ``with_year_hint=True``, возвращает словарь
    ``{"date": "DD.MM.YYYY", "year_hint": <int>}`` — год выносится наружу,
    чтобы вызывающий код мог использовать его как fallback для строк без
    явной даты (см. ``_unpack_sheet_date_meta``).
    """
    if not sheet_name:
        return None
    # Очистка невидимых и непечатаемых Unicode символов
    clean_name = "".join(c for c in str(sheet_name) if c.isprintable()).strip()
    clean_name = clean_name.replace("\u200b", "").replace("\ufeff", "")

    pat = re.compile(r"(^|[^0-9])(\d{1,2})[.,](\d{1,2})(?:[.,](\d{2}|\d{4}))?([^0-9]|$)")
    m = pat.search(clean_name)
    if m:
        try:
            d, month = int(m.group(2)), int(m.group(3))
            year_token = m.group(4)
            # Бизнес-правило: лист без явного года (напр. "13.04") невалиден.
            # Реальная фильтрация дублируется в file_processor._has_explicit_year.
            if year_token is None:
                return None
            y = int(year_token)
            if len(year_token) == 2:
                # Y2K pivot (Excel default): 00-29 → 20XX, 30-99 → 19XX.
                # Without this, "01.01.99" became 2099 (future) and the
                # whole sheet's rows were silently dropped by the date filter.
                y += 2000 if y < 30 else 1900
            normalized = datetime(y, month, d).strftime("%d.%m.%Y")
            if kwargs.get("with_year_hint"):
                return {
                    "date": normalized,
                    "year_hint": y,
                }
            return normalized
        except Exception:
            logger.warning(
                "INVALID DATE in sheet '%s'. Date will be skipped.",
                sheet_name,
                exc_info=True,
            )
            return None
```

```
    return None


def _parse_scope_date(value: Optional[str]) -> Optional[date]:
    if not value:
        return None
    try:
        return datetime.strptime(value, "%d.%m.%Y").date()
    except ValueError:
        return None


def _sheet_date_matches_scope(
    sheet_date: str,
    target_date_str: Optional[str] = None,
    start_date_str: Optional[str] = None,
    end_date_str: Optional[str] = None,
) -> bool:
    target = _parse_scope_date(target_date_str)
    start = _parse_scope_date(start_date_str)
    end = _parse_scope_date(end_date_str)
    if target is None and (start is None or end is None):
        return True

    if len(sheet_date) != 10:
        return False
    try:
        sheet_dt = datetime.strptime(sheet_date, "%d.%m.%Y").date()
    except ValueError:
        return False
    if target is not None:
        return sheet_dt == target
    return start <= sheet_dt <= end


class CalamineReader:
    """Чтение Excel файлов через python-calamine с маппингом колонок.

    Оптимизации:
    - Локальная копия для сетевых дисков
    - Двухпроходный алгоритм (поиск заголовков → чтение данных)
    - Пропуск Pydantic model_dump для производительности
    """

    def __init__(
        self,
        markers: Dict[str, Any],
        aliases: Dict[str, List[str]],
        drive_letters: tuple = ("Q", "Z"),
    ) -> None:
        self.markers = markers
        self.aliases = aliases
        self.drive_letters = tuple(d.upper() for d in drive_letters)
        self._canonical_order = [
            key for key in _PREFERRED_HEADER_ORDER if key in markers
        ] + [key for key in markers if key not in _PREFERRED_HEADER_ORDER]
        required_strict_keys = set(_STRICT_TEMPLATE_ORDER) - _STRICT_OPTIONAL_KEYS
        self._strict_template_enabled = required_strict_keys.issubset(set(markers))
        self._normalized_markers = {
            key: [
                _normalize_header_text(v)
                for v in (val if isinstance(val, list) else [val])
            ]
            for key, val in markers.items()
        }
        self._normalized_aliases = {
            key: [_normalize_header_text(v) for v in values]
            for key, values in aliases.items()
        }
        self._header_search_terms = []
        for key in self.markers:
            terms = []
            terms.extend(self._normalized_markers.get(key, []))
            terms.extend(self._normalized_aliases.get(key, []))
            seen = set()
            uniq = []
            for term in terms:
                if term and term not in seen:
                    seen.add(term)
                    uniq.append(term)
            self._header_search_terms.append(tuple(uniq))

    def _preferred_header_name(self, key: str) -> str:
```

```
        values = self.markers.get(key, [])
        if isinstance(values, list):
            return str(values[0]) if values else key
        return str(values)

    def _matches_any_header_token(self, header_value: str, tokens: List[str]) -> bool:
        return any(token and token in header_value for token in tokens)

    def _classify_header_structure(
        self, header_row: List[Any], col_map: Dict[str, int]
    ) -> tuple[Optional[str], Optional[str]]:
        if not self._strict_template_enabled:
            return None, None

        normalized_headers = [_normalize_header_text(v) for v in header_row]
        mapped_by_index = {}
        for key in _STRICT_TEMPLATE_ORDER:
            col_idx = col_map.get(key)
            if (
                col_idx is not None
                and col_idx < len(normalized_headers)
                and col_idx not in mapped_by_index
            ):
                mapped_by_index[col_idx] = key
        if not mapped_by_index:
            return None, None

        missing_keys = [
            key for key in _STRICT_TEMPLATE_ORDER if key not in mapped_by_index.values()
        ]
        hard_missing = [key for key in missing_keys if key not in _STRICT_OPTIONAL_KEYS]
        if hard_missing:
            return (
                "hard_structure",
                "missing columns: "
                + ", ".join(self._preferred_header_name(key) for key in hard_missing),
            )

        unexpected_header_items = [
            (idx, str(header_row[idx]).strip())
            for idx, value in enumerate(normalized_headers)
            if value and idx not in mapped_by_index
        ]
        helper_headers = [
            header
            for idx, header in unexpected_header_items
            if self._is_soft_extra_header(idx, header, mapped_by_index)
        ]
        real_unexpected_headers = [
            header
            for idx, header in unexpected_header_items
            if not self._is_soft_extra_header(idx, header, mapped_by_index)
        ]
        if real_unexpected_headers:
            return (
                "hard_structure",
                f"unexpected columns: {', '.join(real_unexpected_headers)}",
            )

        present_keys = [key for _, key in sorted(mapped_by_index.items())]
        expected_order = [key for key in _STRICT_TEMPLATE_ORDER if key in present_keys]
        if present_keys != expected_order:
            return (
                "hard_structure",
                "column order mismatch: " + " -> ".join(present_keys),
            )

        alias_only_headers = []
        for idx, key in sorted(mapped_by_index.items()):
            header_value = normalized_headers[idx]
            if self._matches_any_header_token(
                header_value, self._normalized_markers.get(key, [])
            ):
                continue
            alias_match = next(
                (
                    alias
                    for alias in self._normalized_aliases.get(key, [])
                    if alias and alias in header_value
                ),
                None,
            )
            if alias_match is None and key == "broker1" and header_value == "посредник":
```

```
            alias_match = header_value
        if alias_match:
            alias_only_headers.append(
                f"{str(header_row[idx]).strip()} -> {self._preferred_header_name(key)}"
            )

    if alias_only_headers:
        return (
            "soft_structure",
            "unsupported header name: " + "; ".join(alias_only_headers),
        )

    soft_reasons = []
    if helper_headers:
        soft_reasons.append("helper columns: " + ", ".join(helper_headers))
    soft_missing = [
        self._preferred_header_name(key)
        for key in missing_keys
        if key in _STRICT_SOFT_MISSING_KEYS
    ]
    if soft_missing:
        soft_reasons.append("missing optional columns: " + ", ".join(soft_missing))
    if soft_reasons:
        return ("soft_structure", "; ".join(soft_reasons))

    return None, None

@staticmethod
def _should_skip_commission_match(key: str, value: str) -> bool:
    return key == "commission" and "посредник" in value

@staticmethod
def _should_skip_currency_amount_match(key: str, value: str) -> bool:
    return key == "currency_amount" and "сумма платеж" in value

@staticmethod
def _is_boris_second_percent_header(
    idx: int,
    normalized_header: str,
    mapped_by_index: Dict[int, str],
) -> bool:
    if normalized_header != "%":
        return False

    positions = {key: col_idx for col_idx, key in mapped_by_index.items()}
    percent_idx = positions.get("percent_broker")
    broker1_idx = positions.get("broker1")
    broker2_idx = positions.get("broker2")
    if percent_idx is None or broker1_idx is None or broker2_idx is None:
        return False

    return percent_idx < broker1_idx < idx < broker2_idx

def _is_soft_extra_header(
    self,
    idx: int,
    header: str,
    mapped_by_index: Dict[int, str],
) -> bool:
    normalized_header = _normalize_header_text(header)
    return (
        normalized_header in _STRICT_HELPER_HEADERS
        or self._is_boris_second_percent_header(
            idx,
            normalized_header,
            mapped_by_index,
        )
    )

def _find_header_index(
    self,
    header_values: List[str],
    tokens: List[str],
    key: str,
    *,
    exact: bool,
) -> Optional[int]:
    for idx, value in enumerate(header_values):
        if self._should_skip_commission_match(key, value):
            continue
        if self._should_skip_currency_amount_match(key, value):
            continue
        if exact and value in tokens:
```

```
                return idx
            if not exact and any(token in value for token in tokens):
                return idx
        return None

    def _find_alias_index(
        self,
        header_values: List[str],
        alias_lowers: List[str],
        key: str,
    ) -> Optional[int]:
        found_idx = self._find_header_index(
            header_values, alias_lowers, key, exact=True
        )
        if found_idx is not None:
            return found_idx

        for alias in alias_lowers:
            for idx, value in enumerate(header_values):
                if self._should_skip_commission_match(key, value):
                    continue
                if self._should_skip_currency_amount_match(key, value):
                    continue
                if alias in value or (
                    key == "broker1"
                    and value == "посредник"
                    and "посредник 2" not in value
                ):
                    return idx
        return None

    def _map_columns(self, header_row: List[Any]) -> tuple[Dict[str, int], bool]:
        """Сопоставить колонки заголовка с известными маркерами.

        Returns:
            (col_map, used_aliases) — словарь маппинга и флаг использования алиасов.
        """
        col_map = {}
        used_aliases = False
        header_values = [_normalize_header_text(val) for val in header_row]

        for key in self.markers:
            marker_lowers = self._normalized_markers.get(key, [])
            found_idx = self._find_header_index(
                header_values, marker_lowers, key, exact=True
            )
            if found_idx is None:
                found_idx = self._find_header_index(
                    header_values, marker_lowers, key, exact=False
                )

            if found_idx is None and key in self.aliases:
                alias_lowers = self._normalized_aliases.get(key, [])
                found_idx = self._find_alias_index(header_values, alias_lowers, key)
                if found_idx is not None:
                    used_aliases = True

            if found_idx is not None:
                col_map[key] = found_idx

        second_percent_idx = self._find_boris_second_percent_index(header_values, col_map)
        if second_percent_idx is not None:
            col_map["percent_broker2"] = second_percent_idx

        return col_map, used_aliases

    @classmethod
    def _find_boris_second_percent_index(
        cls,
        header_values: List[str],
        col_map: Dict[str, int],
    ) -> Optional[int]:
        mapped_by_index = {col_idx: key for key, col_idx in col_map.items()}
        for idx, value in enumerate(header_values):
            if cls._is_boris_second_percent_header(idx, value, mapped_by_index):
                return idx
        return None

    @staticmethod
    def _row_has_values(row: List[Any]) -> bool:
        return any(v is not None and v != "" for v in row)

    @staticmethod
```

```
def _is_trash_row(row: List[Any]) -> bool:
    # One pre-compiled alternation search per cell (see _TRASH_PATTERN).
    # The previous version rebuilt re.escape+re.search per cell×keyword.
    for v in row:
        if v is None:
            continue
        if _TRASH_PATTERN.search(_normalize_header_text(v)):
            return True
    return False

@staticmethod
def _record_data_loss(
    source: str,
    kind: str,
    reason: str,
    row_index: Optional[int] = None,
    sheet: Optional[str] = None,
    field: Optional[str] = None,
    original_value: Any = None,
) -> None:
    from core.domain.data_loss_log import record_loss

    try:
        record_loss(
            source=source,
            kind=kind,
            reason=reason,
            row_index=row_index,
            sheet=sheet,
            field=field,
            original_value=original_value,
        )
    except Exception:
        logger.warning(
            "Failed to record data loss for source=%s kind=%s reason=%s",
            source, kind, reason, exc_info=True,
        )

@staticmethod
def _should_skip_sheet(sheet_name: str) -> bool:
    return sheet_name.startswith("_") or sheet_name in ("АНАЛИТИКА", "СВОД")

@staticmethod
def _unpack_sheet_date_meta(sheet_name: str) -> tuple[Optional[str], Optional[int]]:
    sheet_date_meta = extract_date_from_sheet(sheet_name, with_year_hint=True)
    if not sheet_date_meta:
        return None, None
    if isinstance(sheet_date_meta, dict):
        return sheet_date_meta.get("date"), sheet_date_meta.get("year_hint")
    return sheet_date_meta, None

def _load_sheet_sample_rows(
    self,
    sheet,
    sheet_name: str,
    strict_io: bool = False,
) -> tuple[Optional[List[Any]], bool]:
    use_to_python = False
    try:
        rows_sample = []
        for idx, row in enumerate(sheet.iter_rows()):
            rows_sample.append(row)
            if idx >= MAX_HEADER_SCAN_ROWS - 1:
                break
        return rows_sample, use_to_python
    except Exception as iter_err:
        logger.debug(
            f"iter_rows failed for sheet {sheet_name} with {type(iter_err).__name__}: {iter_err}. "
            f"Falling back to to_python()."
        )

    use_to_python = True
    try:
        rows_sample = sheet.to_python()[:MAX_HEADER_SCAN_ROWS]
        return rows_sample, use_to_python
    except Exception as e:
        if strict_io:
            raise
        logger.debug(f"Could not read sheet {sheet_name}: {e}")
        return None, use_to_python

@staticmethod
def _header_match_score(row_str: str, header_search_terms: List[tuple[str, ...]]) -> int:
```

```
        return sum(1 for terms in header_search_terms if any(alias in row_str for alias in terms))

    @staticmethod
    def _is_valid_header_map(map_res: Dict[str, int]) -> bool:
        return "amount" in map_res and ("customer" in map_res or "contractor" in map_res)

    def _find_best_header_candidate(
        self,
        rows_sample: List[Any],
        header_search_terms: List[tuple[str, ...]],
    ) -> tuple[Optional[int], Optional[Dict[str, int]], Optional[str], Optional[str], set, int]:
        max_score = 0
        standard_keys = set()
        best_header_idx = None
        best_col_map = None
        best_structure_kind = None
        best_structure_reason = None

        for idx, row in enumerate(rows_sample[:MAX_HEADER_SCAN_ROWS]):
            if not self._row_has_values(row):
                continue

            row_str = " ".join(_normalize_header_text(v) for v in row if v is not None)
            score = self._header_match_score(row_str, header_search_terms)
            if score < 2 or score <= max_score:
                continue

            map_res, _ = self._map_columns(row)
            if not self._is_valid_header_map(map_res):
                continue

            structure_kind, structure_reason = self._classify_header_structure(row, map_res)
            max_score = score
            standard_keys = set(map_res.keys())
            best_header_idx = idx
            best_col_map = map_res
            best_structure_kind = structure_kind
            best_structure_reason = structure_reason

            if score >= 5 or len(standard_keys) >= 5:
                break

        return (
            best_header_idx,
            best_col_map,
            best_structure_kind,
            best_structure_reason,
            standard_keys,
            max_score,
        )

    def _activate_header_state(
        self,
        row: List[Any],
        i: int,
        sheet_name: str,
        best_header_idx: Optional[int],
        best_col_map: Optional[Dict[str, int]],
        best_structure_kind: Optional[str],
        best_structure_reason: Optional[str],
    ) -> Optional[tuple[Dict[str, int], tuple, Optional[int], Optional[int], Optional[int], bool, Optional[str], Optional[str]]]:
        if best_header_idx is None or i != best_header_idx:
            return None

        tmp_map, _used_aliases = self._map_columns(row)
        active_col_map = tmp_map or best_col_map
        if not active_col_map:
            return None

        active_col_items = tuple(active_col_map.items())
        amount_idx = active_col_map.get("amount")
        commission_idx = active_col_map.get("commission")
        total_idx = active_col_map.get("total")
        logger.debug(f"DEBUG: Found header in {sheet_name} row {i + 1}: {active_col_map}")

        structure_kind, structure_reason = self._classify_header_structure(row, active_col_map)
        current_warning_kind = structure_kind or best_structure_kind
        current_warning_reason = structure_reason or best_structure_reason
        current_is_warning = current_warning_kind == "hard_structure"

        return (
            active_col_map,
            active_col_items,
```

```
            amount_idx,
            commission_idx,
            total_idx,
            current_is_warning,
            current_warning_kind,
            current_warning_reason,
        )

    def _coerce_numeric_cell(self, key: str, val: Any) -> tuple[Any, bool, bool]:
        if isinstance(val, bool):
            return Decimal("0.00"), False, False
        if isinstance(val, (int, float)):
            d_val = Decimal(str(val))
            return d_val, d_val != 0, False

        try:
            d_val = (
                to_decimal_percent(val)
                if key in {"percent_broker", "percent_broker2"}
                else to_decimal(val)
            )
            return d_val, d_val != 0, False
        except Exception:
            logger.debug(
                "Decimal conversion failed for key=%s val=%s, falling back to text match",
                key, repr(val), exc_info=True,
            )
            normalized_value = _normalize_header_text(val)
            header_terms = {
                term
                for term in (
                    self._normalized_markers.get(key, [])
                    + self._normalized_aliases.get(key, [])
                )
                if term
            }
            if normalized_value in header_terms:
                return val, False, False
            self._record_data_loss(
                source=getattr(self, "_current_file_path", "") or "",
                kind="numeric_coercion",
                reason=f"could not parse {key!r} value as number",
                field=key,
                original_value=val,
            )
            if key == "amount":
                return val, False, True
            return None, False, True

    @staticmethod
    def _apply_shifted_columns_fix(
        raw_data: Dict[str, Any],
        row: List[Any],
        row_len: int,
        amount_idx: Optional[int],
        commission_idx: Optional[int],
        total_idx: Optional[int],
        row_has_numeric_text_warning: bool,
        row_warning_reason: Optional[str],
    ) -> tuple[Dict[str, Any], bool, Optional[str]]:
        amount_val = raw_data.get("amount")
        commission_val = raw_data.get("commission")
        total_val = raw_data.get("total")
        next_total_val = (
            row[total_idx + 1] if total_idx is not None and total_idx + 1 < row_len else None
        )

        if (
            isinstance(amount_val, str)
            and commission_idx is not None
            and total_idx is not None
            and amount_idx is not None
            and commission_idx == amount_idx + 1
            and total_idx == commission_idx + 1
            and isinstance(commission_val, Decimal)
            and isinstance(total_val, Decimal)
            and abs(total_val) <= Decimal("1")
            and isinstance(next_total_val, (int, float))
        ):
            raw_data["amount"] = commission_val
            raw_data["commission"] = total_val
            raw_data["total"] = Decimal(str(next_total_val))
            row_has_numeric_text_warning = True
```

```
            row_warning_reason = "shifted columns after amount"

        return raw_data, row_has_numeric_text_warning, row_warning_reason

    @staticmethod
    def _attach_warning_metadata(
        raw_data: Dict[str, Any],
        row_has_numeric_text_warning: bool,
        row_warning_reason: Optional[str],
        current_is_warning: bool,
        current_warning_kind: Optional[str],
        current_warning_reason: Optional[str],
    ) -> None:
        if row_has_numeric_text_warning:
            raw_data["is_template_warning"] = True
            raw_data["_template_warning_kind"] = "template_warning"
            raw_data["_template_warning_reason"] = (
                row_warning_reason or "numeric text in numeric field"
            )
            return

        raw_data["is_template_warning"] = current_is_warning
        if current_warning_kind:
            raw_data["_template_warning_kind"] = current_warning_kind
            raw_data["_template_warning_reason"] = current_warning_reason

    def _build_data_row_payload(
        self,
        row: List[Any],
        i: int,
        sheet_name: str,
        f_path_abs: str,
        sheet_date: str,
        sheet_year_hint: Optional[int],
        active_col_items: tuple,
        amount_idx: Optional[int],
        commission_idx: Optional[int],
        total_idx: Optional[int],
        current_is_warning: bool,
        current_warning_kind: Optional[str],
        current_warning_reason: Optional[str],
        file_basename: str,
    ) -> Optional[Dict[str, Any]]:
        raw_data = {
            "_sheet_name": sheet_name,
            "_row_index": i + 1,
            "_file_path": f_path_abs,
            "date": sheet_date,
            "_sheet_year_hint": sheet_year_hint,
        }

        has_data = False
        has_financial = False
        row_has_numeric_text_warning = False
        row_warning_reason = None
        row_len = len(row)

        for key, col_idx in active_col_items:
            if col_idx >= row_len:
                continue
            val = row[col_idx]
            if val is None or val == "":
                continue

            has_data = True
            if key in _NUMERIC_FIELDS:
                parsed_val, has_financial_value, numeric_text_warning = self._coerce_numeric_cell(
                    key, val
                )
                raw_data[key] = parsed_val
                if has_financial_value:
                    has_financial = True
                if numeric_text_warning:
                    row_has_numeric_text_warning = True
                    if row_warning_reason is None:
                        row_warning_reason = f"text in numeric field: {key}"
            else:
                raw_data[key] = val

        raw_data, row_has_numeric_text_warning, row_warning_reason = self._apply_shifted_columns_fix(
            raw_data,
            row,
            row_len,
```

```
            amount_idx,
            commission_idx,
            total_idx,
            row_has_numeric_text_warning,
            row_warning_reason,
        )

        if not (has_data and (has_financial or row_has_numeric_text_warning)):
            if has_data:
                logger.debug(
                    f"Row {i + 1} in {sheet_name} has data but no financial values"
                )
            return None

        if row_has_numeric_text_warning and not has_financial:
            unique_texts = set()
            for val in row:
                if val is None or val == "":
                    continue
                text = str(val).strip().strip("\"'").strip()
                if text:
                    unique_texts.add(text.lower())
            if len(unique_texts) <= 1:
                logger.debug(
                    f"Row {i + 1} in {sheet_name} looks like a section header "
                    f"(text in numeric column, no real numbers)"
                )
                return None
            customer_idx = None
            for key, ci in active_col_items:
                if key == "customer":
                    customer_idx = ci
                    break
            if customer_idx is not None and customer_idx < len(row):
                cust_text = str(row[customer_idx]).strip() if row[customer_idx] is not None else ""
                if cust_text.startswith('"') and len(cust_text) < 30:
                    logger.debug(
                        f"Row {i + 1} in {sheet_name} looks like a section header "
                        f"(quoted customer '{cust_text}', no financial data)"
                    )
                    return None

        self._attach_warning_metadata(
            raw_data,
            row_has_numeric_text_warning,
            row_warning_reason,
            current_is_warning,
            current_warning_kind,
            current_warning_reason,
        )
        raw_data["_source_info"] = f"{file_basename} | {sheet_name} | Row {i + 1}"
        return raw_data

    def _iter_sheet_rows_safe(
        self,
        sheet,
        use_to_python: bool,
        sheet_name: str,
        strict_io: bool = False,
    ):
        row_iterator = enumerate(sheet.to_python() if use_to_python else sheet.iter_rows())
        while True:
            try:
                yield next(row_iterator)
            except StopIteration:
                return
            except Exception as second_pass_err:
                if strict_io:
                    raise
                logger.warning(
                    f"Error during second pass in sheet {sheet_name}: "
                    f"{type(second_pass_err).__name__} - {second_pass_err}"
                )
                return

    def _process_workbook_sheet(
        self,
        wb,
        sheet_name: str,
        f_path_abs: str,
        file_basename: str,
        header_search_terms: List[tuple[str, ...]],
        all_rows: List[Dict[str, Any]],
```

```
        target_date_str: Optional[str] = None,
        start_date_str: Optional[str] = None,
        end_date_str: Optional[str] = None,
        strict_io: bool = False,
    ) -> None:
        if self._should_skip_sheet(sheet_name):
            return

        sheet_date, sheet_year_hint = self._unpack_sheet_date_meta(sheet_name)
        if not sheet_date:
            logger.debug(f"Skipping sheet {sheet_name}: no date found")
            return
        if not _sheet_date_matches_scope(
            sheet_date,
            target_date_str=target_date_str,
            start_date_str=start_date_str,
            end_date_str=end_date_str,
        ):
            return

        try:
            sheet = wb.get_sheet_by_name(sheet_name)
        except Exception as e:
            if strict_io:
                raise
            logger.debug(f"Could not load sheet {sheet_name}: {e}")
            return

        rows_sample, use_to_python = self._load_sheet_sample_rows(
            sheet,
            sheet_name,
            strict_io=strict_io,
        )
        if not rows_sample:
            return

        (
            best_header_idx,
            best_col_map,
            best_structure_kind,
            best_structure_reason,
            standard_keys,
            max_score,
        ) = self._find_best_header_candidate(rows_sample, header_search_terms)

        if not standard_keys:
            logger.debug(f"No headers found in sheet {sheet_name} (max score: {max_score})")
            return

        active_col_map = None
        active_col_items = ()
        amount_idx = None
        commission_idx = None
        total_idx = None
        current_is_warning = False
        current_warning_kind = None
        current_warning_reason = None

        for i, row in self._iter_sheet_rows_safe(
            sheet,
            use_to_python,
            sheet_name,
            strict_io=strict_io,
        ):
            if not self._row_has_values(row):
                continue

            if not active_col_map:
                header_state = self._activate_header_state(
                    row,
                    i,
                    sheet_name,
                    best_header_idx,
                    best_col_map,
                    best_structure_kind,
                    best_structure_reason,
                )
                if header_state:
                    (
                        active_col_map,
                        active_col_items,
                        amount_idx,
                        commission_idx,
```

```
                    total_idx,
                    current_is_warning,
                    current_warning_kind,
                    current_warning_reason,
                ) = header_state
            continue

        if self._is_trash_row(row):
            self._record_data_loss(
                source=f_path_abs,
                kind="trash_filter",
                reason="row matched trash keyword",
                row_index=i + 1,
                sheet=sheet_name,
            )
            continue

        payload = self._build_data_row_payload(
            row,
            i,
            sheet_name,
            f_path_abs,
            sheet_date,
            sheet_year_hint,
            active_col_items,
            amount_idx,
            commission_idx,
            total_idx,
            current_is_warning,
            current_warning_kind,
            current_warning_reason,
            file_basename,
        )
        if payload is None:
            continue
        try:
            all_rows.append(payload)
        except Exception as ve:
            logger.debug(f"Row {i + 1} in {sheet_name} failed: {ve}")

@staticmethod
def _cleanup_temp_file(temp_file: Optional[str]) -> None:
    if temp_file and os.path.exists(temp_file):
        try:
            os.remove(temp_file)
        except Exception:
            logger.debug("Failed to remove temp file %s", temp_file, exc_info=True)

def read(
    self,
    file_path: Path,
    strict_io: bool = False,
    target_date_str: Optional[str] = None,
    start_date_str: Optional[str] = None,
    end_date_str: Optional[str] = None,
) -> List[Dict[str, Any]]:
    """Прочитать все листы файла и вернуть список строк.

    Алгоритм:
    1. Для сетевых файлов — создать локальную копию
    2. Первый проход (первые 6000 строк) — найти заголовки
    3. Второй проход — извлечь данные с маппингом
    """
    all_rows: List[Dict[str, Any]] = []
    f_path_abs = os.path.abspath(str(file_path))
    file_basename = os.path.basename(f_path_abs)
    self._current_file_path = f_path_abs
    header_search_terms = self._header_search_terms

    try:
        target_path, temp_file = get_local_copy(f_path_abs, self.drive_letters)
    except (OSError, IOError, PermissionError) as e:
        if strict_io:
            raise
        logger.error(f"Calamine error {f_path_abs}: {e}")
        return all_rows
    except Exception as e:
        if strict_io:
            raise
        logger.error(f"Calamine error {f_path_abs}: {e}")
        return all_rows

    # Validate file signature (Magic Bytes)
```

```
        try:
            with open(target_path, "rb") as f:
                header_bytes = f.read(4)
            if header_bytes != b"PK\x03\x04":
                raise ValueError("Invalid file signature: expected Excel ZIP archive (XLSX)")
        except Exception as sig_err:
            self._cleanup_temp_file(temp_file)
            if strict_io:
                raise
            logger.error(f"File signature validation failed for {f_path_abs}: {sig_err}")
            return all_rows

        try:
            wb = CalamineWorkbook.from_path(target_path)
            try:
                for sheet_name in wb.sheet_names:
                    self._process_workbook_sheet(
                        wb,
                        sheet_name,
                        f_path_abs,
                        file_basename,
                        header_search_terms,
                        all_rows,
                        target_date_str=target_date_str,
                        start_date_str=start_date_str,
                        end_date_str=end_date_str,
                        strict_io=strict_io,
                    )
            finally:
                wb.close()

        except Exception as e:
            if strict_io:
                raise
            logger.error(f"Calamine error {f_path_abs}: {e}")
        finally:
            self._cleanup_temp_file(temp_file)

        return all_rows
```

APP\core\infra\metrics.py

78 lines • 2,516 bytes

```
"""Simple in-process metrics counters for monitoring consolidation runs."""

import logging
import threading
import time
from typing import Dict, Any

logger = logging.getLogger(__name__)


class Metrics:
    """Thread-safe counters for tracking consolidation pipeline metrics.

    Usage:
        metrics = Metrics()
        metrics.increment("rows_processed", 100)
        metrics.increment("files_read")
        metrics.set_gauge("active_threads", 3)
        metrics.log_summary()
    """

    def __init__(self) -> None:
        self._lock = threading.Lock()
        self._counters: Dict[str, int] = {}
        self._gauges: Dict[str, float] = {}
        self._timers: Dict[str, float] = {}
        self._start_time: float = time.monotonic()

    def increment(self, name: str, amount: int = 1) -> None:
        """Increment a counter by the given amount."""
        with self._lock:
            self._counters[name] = self._counters.get(name, 0) + amount

    def set_gauge(self, name: str, value: float) -> None:
        """Set a gauge to a specific value."""
        with self._lock:
            self._gauges[name] = value

    def start_timer(self, name: str) -> None:
        """Start a named timer."""
        with self._lock:
            self._timers[name] = time.monotonic()

    def stop_timer(self, name: str) -> float:
        """Stop a named timer and return elapsed seconds."""
        with self._lock:
            start = self._timers.pop(name, None)
            if start is None:
                return 0.0
            elapsed = time.monotonic() - start
            self._gauges[f"{name}_elapsed_s"] = elapsed
            return elapsed

    def get_snapshot(self) -> Dict[str, Any]:
        """Return a snapshot of all metrics."""
        with self._lock:
            return {
                "counters": dict(self._counters),
                "gauges": dict(self._gauges),
                "uptime_s": round(time.monotonic() - self._start_time, 2),
            }

    def log_summary(self) -> None:
        """Log current metrics as a structured summary."""
        snap = self.get_snapshot()
        logger.info(
            f"[METRICS] counters={snap['counters']} "
            f"gauges={snap['gauges']} "
            f"uptime={snap['uptime_s']}s"
        )

    def reset(self) -> None:
        """Reset all metrics."""
        with self._lock:
            self._counters.clear()
            self._gauges.clear()
            self._timers.clear()
            self._start_time = time.monotonic()
```

APP\core\infra\path_resolver.py

176 lines • 5,610 bytes

```
import logging
from pathlib import Path
from typing import List, Dict, Any, Optional


logger = logging.getLogger(__name__)


def _normalize_path_parts(path_str: str) -> List[str]:
    return [part for part in path_str.replace("\\", "/").split("/") if part not in ("", ".", "..")]


def _resolve_reports_alias(path_str: str, config_controller) -> Optional[Path]:
    if not (config_controller and hasattr(config_controller, "get_paths")):
        return None

    parts = _normalize_path_parts(path_str)
    if not parts or parts[0].lower() != "reports" or len(parts) < 3:
        return None

    get_paths = config_controller.get_paths
    if not callable(get_paths):
        return None

    try:
        paths_cfg = get_paths() or {}
    except Exception:
        return None

    roots_raw = paths_cfg.get("source_roots") or paths_cfg.get("source_root") or []
    if isinstance(roots_raw, (str, Path)):
        roots = [Path(str(roots_raw))]
    else:
        roots = [Path(str(root)) for root in roots_raw]

    curator_root_name = parts[1]
    suffix_parts = parts[2:]

    for root in roots:
        root_parts = _normalize_path_parts(str(root))
        if not root_parts:
            continue
        if root_parts[-1].lower() != curator_root_name.lower():
            continue
        candidate = Path(root, *suffix_parts)
        if candidate.exists():
            return candidate

    return None


def _path_suffix_candidates(path_str: str) -> List[str]:
    norm = path_str.replace("\\", "/").lower().strip()
    if not norm:
        return []

    candidates = [norm]
    reports_marker = "/reports/"
    idx = norm.find(reports_marker)
    if idx != -1:
        candidates.append(norm[idx + len(reports_marker) :])

    parts = [part for part in norm.split("/") if part not in ("", ".", "..")]
    if parts:
        candidates.append("/".join(parts))
    if len(parts) >= 2:
        candidates.append("/".join(parts[-2:]))
    if parts:
        candidates.append(parts[-1])

    deduped = []
    seen = set()
    for candidate in candidates:
        if candidate and candidate not in seen:
            deduped.append(candidate)
            seen.add(candidate)
    return deduped


def _resolve_with_config_controller(path_str: str, config_controller) -> Path:
    p_abs = Path(path_str)
    if not (config_controller and hasattr(config_controller, "resolve_path")):
        return p_abs
```

```
    reports_alias = _resolve_reports_alias(path_str, config_controller)
    if reports_alias is not None:
        return reports_alias

    resolver = config_controller.resolve_path
    if not callable(resolver):
        return p_abs

    try:
        candidate = resolver(path_str)
    except (TypeError, OSError) as e:
        # NOTE: ValueError is deliberately NOT caught here. resolve_path
        # raises ValueError to signal a path-traversal rejection; catching
        # it and falling back to the raw ``p_abs`` would silently re-open
        # the very bypass resolve_path is meant to close. Let it propagate.
        logger.debug("resolve_path failed, using original path value: %s", e)
        return p_abs

    if isinstance(candidate, (str, Path)):
        return Path(candidate)
    return p_abs


def _find_discovered_meta(
    path_str: str,
    discovered_files: List[Dict[str, Any]],
    file_id=None,
    file_hash=None,
):
    suffixes = _path_suffix_candidates(path_str)
    for df in discovered_files:
        df_norm = df["path"].replace("\\", "/").lower()
        if any(df_norm.endswith(suffix) for suffix in suffixes):
            return df

    if not (file_id or file_hash):
        return None

    for df in discovered_files:
        if file_id and df.get("id") == file_id:
            return df
        if file_hash and df.get("fp") and df.get("fp") == file_hash:
            return df
    return None


def _build_meta(path_value: str, file_id=None, file_hash=None) -> Dict[str, Any]:
    meta = {"path": path_value}
    if file_id is not None:
        meta["id"] = file_id
    if file_hash is not None:
        meta["fp"] = file_hash
    return meta


def _resolve_relative_from_config(p_abs: Path, config_controller) -> Path:
    if p_abs.is_absolute():
        return p_abs
    if not (config_controller and hasattr(config_controller, "config_path")):
        return p_abs
    return (Path(config_controller.config_path).parent / p_abs).resolve()


def resolve_employee_path(
    path_str: str,
    file_id=None,
    file_hash=None,
    discovered_files: Optional[List[Dict[str, Any]]] = None,
    config_controller=None,
) -> tuple:
    """Резолвит путь к файлу сотрудника с обнаружением переименований.

    Returns:
        (resolved_path_str, updated_meta, rename_detected)
    """
    discovered_files = discovered_files or []
    p_abs = _resolve_with_config_controller(path_str, config_controller)
    found_meta = _find_discovered_meta(path_str, discovered_files, file_id, file_hash)
    if found_meta:
        return found_meta["path"], found_meta, True

    if p_abs.is_absolute() and p_abs.exists():
        return str(p_abs), _build_meta(str(p_abs), file_id, file_hash), False
```

```
    alt_p = _resolve_relative_from_config(p_abs, config_controller)
    if alt_p.exists():
        p_abs = alt_p

    if p_abs.exists():
        return str(p_abs), _build_meta(str(p_abs), file_id, file_hash), False

    return path_str, None, False
```

APP\core\infra\reconciliation__init__.py

30 lines • 984 bytes

```
"""Reconciliation report package — decomposed from reconciliation_report.py.

Layout:
    _types.py   — data classes (MasterReconciliationRow, RschReconciliationRow, ReconciliationRow)
    _utils.py   — pure utility functions, constants, _to_decimal, _excel_safe_text
    _resolve.py — date/sheet resolution, executor/customer matching
    _collect.py — reading from Excel: _collect_master_rows, _collect_rsch_rows
    _build.py   — writing to Excel: _build_workbook, build_reconciliation_report, build_combined
"""

from ._types import (
    MasterReconciliationRow,
    RschReconciliationRow,
    ReconciliationRow,
)
from ._utils import _row_id

from ._build import (
    build_reconciliation_report,
    build_combined_reconciliation_report,
)

__all__ = [
    "MasterReconciliationRow",
    "RschReconciliationRow",
    "ReconciliationRow",
    "_row_id",
    "build_reconciliation_report",
    "build_combined_reconciliation_report",
]
```

APP\core\infra\reconciliation_build.py

429 lines • 16,918 bytes

```
from __future__ import annotations
from datetime import date
from pathlib import Path
from typing import Any, Iterable, Mapping, Optional
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Alignment, PatternFill
from openpyxl.comments import Comment
from openpyxl.utils import get_column_letter
from openpyxl.worksheet.worksheet import Worksheet
from core.infra.excel_hyperlinks import apply_cell_hyperlink
from core.infra.excel_backup import backup_existing_excel_output
from core.infra.utils import atomic_save_workbook
from ._types import RschReconciliationRow, ReconciliationRow
from ._utils import (_COMBINED_SHEET_PRIORITY,
    _is_numeric_header, _is_percent_header,
    _normalize_text, _HEADER_FILL, _HEADER_FONT, _THIN_BORDER,
    _STATUS_OK_FILL, _STATUS_REVIEW_FILL, _STATUS_ERROR_FILL, _STATUS_FONT,
    _MASTER_EXPENSE_FILL, STATUS_VERIFIED, STATUS_REVIEW, REASON_HEADER, STATUS_HEADER, REASON_NO_CURATOR, RSCH_HEADERS, _SUMMARY_REASON_LABELS)
from ._resolve import (_resolve_target_date, _resolve_combined_target_date,
    _find_sheet, _local_read_path)
from ._collect import (_collect_master_rows, _collect_rsch_rows, _master_values, _rsch_values,
    _build_reconciliation_rows,
    _suppress_rsch_rows_matched_by_excluded_master_filters)

def _reason_label(reason: str) -> str:
    return _SUMMARY_REASON_LABELS.get(reason, reason)


def _write_headers(ws: Worksheet, headers: list[str]) -> None:
    ws.append(headers)
    for col_idx in range(1, len(headers) + 1):
        cell = ws.cell(1, col_idx)
        cell.fill = _HEADER_FILL
        cell.font = _HEADER_FONT
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        cell.border = _THIN_BORDER


def _update_column_widths(widths: dict[int, int], values: list[Any]) -> None:
    for col_idx, value in enumerate(values, start=1):
        widths[col_idx] = max(
            widths.get(col_idx, 0),
            len(str(value)) if value is not None else 0,
        )


def _apply_column_widths(ws: Worksheet, widths: dict[int, int]) -> None:
    wide_text_headers = ("заказчик", "исполнитель", "контрагент", "комментар", "источник")
    compact_min_widths = {
        "статус": 14,
        "причина": 18,
    }
    for col_idx in range(1, ws.max_column + 1):
        header = _normalize_text(ws.cell(1, col_idx).value)
        max_len = widths.get(col_idx, 0)
        min_width = 18 if _is_numeric_header(header) else 10
        for key, width in compact_min_widths.items():
            if key in header:
                min_width = max(min_width, width)
        max_width = 70 if any(key in header for key in wide_text_headers) else 45
        ws.column_dimensions[get_column_letter(col_idx)].width = max(
            min_width,
            min(max_width, max_len + 2),
        )


def _append_reconciliation_row(
    ws: Worksheet,
    row: ReconciliationRow,
    master_headers: list[str],
    column_widths: dict[int, int] | None = None,
) -> None:
    headers = [*master_headers, STATUS_HEADER, *RSCH_HEADERS, REASON_HEADER]
    status_col = len(master_headers) + 1
    reason = _reason_label(row.reason) if row.reason else None
    values = [
        *_master_values(row, master_headers),
        row.status,
        *_rsch_values(row.rsch),
        reason,
    ]
    if column_widths is not None:
        _update_column_widths(column_widths, values)
```

```
    ws.append(values)
    row_idx = ws.max_row
    is_even = (row_idx % 2 == 0)
    row_fill = PatternFill("solid", fgColor="FFF2F2F2") if is_even else PatternFill("solid", fgColor="FFFFFFFF")
    for col_idx, header in enumerate(headers, start=1):
        cell = ws.cell(row_idx, col_idx)
        cell.border = _THIN_BORDER
        cell.alignment = Alignment(
            horizontal="right" if _is_numeric_header(header) else "left",
            vertical="top",
            wrap_text=True,
        )
        if _is_numeric_header(header):
            cell.number_format = "#,##0.00"
        if _is_percent_header(header):
            cell.number_format = "0.##%"
        if col_idx != status_col:
            cell.fill = row_fill
    status_cell = ws.cell(row_idx, status_col)
    status_cell.fill = {
        STATUS_VERIFIED: _STATUS_OK_FILL,
        STATUS_REVIEW: _STATUS_REVIEW_FILL,
    }.get(row.status, _STATUS_ERROR_FILL)
    status_cell.font = _STATUS_FONT
    status_cell.alignment = Alignment(horizontal="center", vertical="center")
    if row.reason:
        status_cell.comment = Comment(_reason_label(row.reason), "Master-Report")
    source_link = row.master.source_link if row.master is not None else None
    if source_link is None and row.rsch is not None:
        source_link = row.rsch.source_link
    if source_link:
        apply_cell_hyperlink(ws.cell(row_idx, 1), source_link)
    for col_idx, header in enumerate(headers, start=1):
        if _normalize_text(header) == "расход" and ws.cell(row_idx, col_idx).value not in (
            None,
            "",
            0,
        ):
            ws.cell(row_idx, col_idx).fill = _MASTER_EXPENSE_FILL


def _autosize_columns(ws: Worksheet) -> None:
    for col_idx in range(1, ws.max_column + 1):
        max_len = 0
        for row in ws.iter_rows(min_col=col_idx, max_col=col_idx):
            value = row[0].value
            max_len = max(max_len, len(str(value)) if value is not None else 0)
        ws.column_dimensions[get_column_letter(col_idx)].width = max(
            10,
            min(45, max_len + 2),
        )


def _build_workbook(
    target_date: date,
    rows: Iterable[ReconciliationRow],
    master_headers: list[str],
) -> Workbook:
    wb = Workbook()
    ws = wb.active
    ws.title = target_date.strftime("%d.%m.%Y")
    _write_reconciliation_sheet(ws, rows, master_headers)
    return wb


def _reconciliation_sort_key(row: ReconciliationRow) -> tuple:
    status_order = {STATUS_VERIFIED: 0, STATUS_REVIEW: 1}
    source = ""
    customer = ""
    executor = ""
    if row.master is not None:
        source = _normalize_text(row.master.source)
        customer = _normalize_text(row.master.customer)
        executor = _normalize_text(row.master.executor)
    elif row.rsch is not None:
        source = _normalize_text(row.rsch.executor)
        customer = _normalize_text(row.rsch.customer)
        executor = _normalize_text(row.rsch.executor)
    return (status_order.get(row.status, 2), source, customer, executor)


def _write_reconciliation_sheet(
    ws: Worksheet,
```

```
    rows: Iterable[ReconciliationRow],
    master_headers: list[str],
) -> None:
    row_list = sorted(rows, key=_reconciliation_sort_key)
    headers = [*master_headers, STATUS_HEADER, *RSCH_HEADERS, REASON_HEADER]
    _write_headers(ws, headers)

    column_widths: dict[int, int] = {}
    _update_column_widths(column_widths, headers)
    for row in row_list:
        _append_reconciliation_row(ws, row, master_headers, column_widths)
    ws.freeze_panes = "A2"
    ws.auto_filter.ref = f"A1:{get_column_letter(len(headers))}{ws.max_row}"
    _apply_column_widths(ws, column_widths)


def _build_reconciliation_sheet_data(
    master_wb,
    rsch_wb,
    target_date: date | None,
    alias_repo: Optional[Any] = None,
) -> tuple[date, list[str], list[ReconciliationRow]]:
    effective_date = _resolve_target_date(master_wb, rsch_wb, target_date)
    master_ws = _find_sheet(master_wb, effective_date)
    rsch_ws = _find_sheet(rsch_wb, effective_date)
    master_headers, master_rows, excluded_master_rows = _collect_master_rows(master_ws)
    rsch_rows = _suppress_rsch_rows_matched_by_excluded_master_filters(
        _collect_rsch_rows(rsch_ws),
        excluded_master_rows,
    )
    rows = _build_reconciliation_rows(master_rows, rsch_rows, alias_repo=alias_repo)
    return effective_date, master_headers, rows


def _matched_rsch_row_ids(rows: Iterable[ReconciliationRow]) -> set[str]:
    return {
        row.rsch.row_id
        for row in rows
        if row.status in {STATUS_VERIFIED, STATUS_REVIEW}
        and row.master is not None
        and row.rsch is not None
    }


def _is_rsch_only_no_curator_row(row: ReconciliationRow) -> bool:
    return (
        row.master is None
        and row.rsch is not None
        and row.reason == REASON_NO_CURATOR
    )


def _build_curator_routing(sheet_names: Iterable[str]) -> dict[str, str]:
    routing: dict[str, str] = {}
    for name in sheet_names:
        norm = _normalize_text(name)
        if norm and norm not in ("общий", "general", "all"):
            routing[norm] = name
    return routing


def _infer_rsch_row_owner(
    rsch_row: RschReconciliationRow,
    curator_routing: dict[str, str],
) -> str | None:
    comment_norm = _normalize_text(rsch_row.comment or "")
    if not comment_norm:
        return None
    for key, sheet in curator_routing.items():
        if key in comment_norm:
            return sheet
    return None


def _suppress_combined_rsch_only_rows(
    rows: Iterable[ReconciliationRow],
    matched_rsch_ids: set[str],
    seen_unmatched_rsch_ids: set[str],
    current_sheet: str,
    rsch_only_owners: dict[str, str],
) -> list[ReconciliationRow]:
    filtered = []
    for row in rows:
```

```
        if not _is_rsch_only_no_curator_row(row):
            filtered.append(row)
            continue
        assert row.rsch is not None
        row_id = row.rsch.row_id
        if row_id in matched_rsch_ids or row_id in seen_unmatched_rsch_ids:
            continue
        owner = rsch_only_owners.get(row_id)
        if owner is not None and owner != current_sheet:
            continue
        seen_unmatched_rsch_ids.add(row_id)
        filtered.append(row)
    return filtered


def _ordered_combined_master_items(
    master_paths: Mapping[str, Path | str],
) -> list[tuple[str, Path | str]]:
    original_order = {name: idx for idx, name in enumerate(master_paths)}
    return sorted(
        master_paths.items(),
        key=lambda item: (
            _COMBINED_SHEET_PRIORITY.get(item[0], len(_COMBINED_SHEET_PRIORITY)),
            original_order[item[0]],
        ),
    )


def build_reconciliation_report(
    master_path: Path | str,
    rsch_path: Path | str,
    output_path: Path | str,
    *,
    target_date: date | None = None,
    drive_letters: tuple[str, ...] = ("Q", "Z"),
    alias_repo: Optional[Any] = None,
) -> int:
    master_path = Path(master_path)
    rsch_path = Path(rsch_path)
    output_path = Path(output_path)

    if not master_path.exists():
        raise FileNotFoundError(f"Не найден мастер-отчет: {master_path}")
    if not rsch_path.exists():
        raise FileNotFoundError(f"Не найден РСЧ-отчет: {rsch_path}")

    with (
        _local_read_path(master_path, drive_letters) as master_read_path,
        _local_read_path(rsch_path, drive_letters) as rsch_read_path,
    ):
        master_wb = None
        rsch_wb = None
        wb_out = None
        try:
            master_wb = load_workbook(master_read_path, data_only=True)
            rsch_wb = load_workbook(rsch_read_path, data_only=True)
            effective_date, master_headers, rows = _build_reconciliation_sheet_data(
                master_wb,
                rsch_wb,
                target_date,
                alias_repo=alias_repo,
            )
            wb_out = _build_workbook(effective_date, rows, master_headers)
            output_path.parent.mkdir(parents=True, exist_ok=True)
            backup_existing_excel_output(output_path)
            atomic_save_workbook(wb_out, output_path)
            return len(rows)
        finally:
            if master_wb is not None:
                master_wb.close()
            if rsch_wb is not None:
                rsch_wb.close()
            if wb_out is not None:
                wb_out.close()


def build_combined_reconciliation_report(
    master_paths: Mapping[str, Path | str],
    rsch_path: Path | str,
    output_path: Path | str,
    *,
    target_date: date | None = None,
    drive_letters: tuple[str, ...] = ("Q", "Z"),
```

```
    alias_repo: Optional[Any] = None,
) -> dict[str, int]:
    rsch_path = Path(rsch_path)
    output_path = Path(output_path)
    if not master_paths:
        raise ValueError("Не выбраны мастер-отчеты для сверки")
    if not rsch_path.exists():
        raise FileNotFoundError(f"Не найден РСЧ-отчет: {rsch_path}")

    ordered_master_paths = _ordered_combined_master_items(master_paths)
    with _local_read_path(rsch_path, drive_letters) as rsch_read_path:
        rsch_wb = None
        wb_out = None
        try:
            wb_out = Workbook()
            default_ws = wb_out.active
            counts: dict[str, int] = {}
            sheet_rows: list[tuple[str, list[str], list[ReconciliationRow]]] = []
            rsch_wb = load_workbook(rsch_read_path, data_only=True)
            effective_target_date = _resolve_combined_target_date(
                ordered_master_paths,
                rsch_wb,
                target_date,
                drive_letters,
            )
            for sheet_name, master_path_value in ordered_master_paths:
                master_path = Path(master_path_value)
                if not master_path.exists():
                    raise FileNotFoundError(f"Не найден мастер-отчет {sheet_name}: {master_path}")
                with _local_read_path(master_path, drive_letters) as master_read_path:
                    master_wb = None
                    try:
                        master_wb = load_workbook(master_read_path, data_only=True)
                        (
                            _effective_date,
                            master_headers,
                            rows,
                        ) = _build_reconciliation_sheet_data(
                            master_wb,
                            rsch_wb,
                            effective_target_date,
                            alias_repo=alias_repo,
                        )
                    finally:
                        if master_wb is not None:
                            master_wb.close()
                sheet_rows.append((sheet_name, master_headers, rows))

            matched_rsch_ids = {
                row_id
                for _sheet_name, _master_headers, rows in sheet_rows
                for row_id in _matched_rsch_row_ids(rows)
            }
            rsch_only_owners: dict[str, str] = {}
            curator_routing = _build_curator_routing(master_paths.keys())
            if curator_routing:
                for _sheet_name, _master_headers, rows in sheet_rows:
                    for row in rows:
                        if _is_rsch_only_no_curator_row(row) and row.rsch is not None:
                            owner = _infer_rsch_row_owner(row.rsch, curator_routing)
                            if owner:
                                rsch_only_owners[row.rsch.row_id] = owner
            seen_unmatched_rsch_ids: set[str] = set()
            for idx, (sheet_name, master_headers, rows) in enumerate(sheet_rows):
                rows = _suppress_combined_rsch_only_rows(
                    rows,
                    matched_rsch_ids,
                    seen_unmatched_rsch_ids,
                    sheet_name,
                    rsch_only_owners,
                )
                rows = sorted(rows, key=_reconciliation_sort_key)

                ws = default_ws if idx == 0 else wb_out.create_sheet()
                # Excel sheet names limited to 31 chars; deduplicate if truncated name collides
                truncated = sheet_name[:31]
                if truncated in wb_out.sheetnames:
                    n = 2
                    while True:
                        alt = f"{truncated[:28]}({n})"
                        if alt not in wb_out.sheetnames:
                            truncated = alt
                            break
```

```
                n += 1
        ws.title = truncated
        _write_reconciliation_sheet(ws, rows, master_headers)
        counts[sheet_name] = len(rows)

    output_path.parent.mkdir(parents=True, exist_ok=True)
    backup_existing_excel_output(output_path)
    atomic_save_workbook(wb_out, output_path)
    return counts
finally:
    if rsch_wb is not None:
        rsch_wb.close()
    if wb_out is not None:
        wb_out.close()
```

APP\core\infra\reconciliation_collect.py

715 lines • 27,334 bytes

```
from __future__ import annotations
import re
from decimal import Decimal
from typing import Any, Callable, Optional
from openpyxl.worksheet.worksheet import Worksheet
from core.domain.data_loss_log import record_loss
from core.infra.excel_hyperlinks import extract_hyperlink_formula_parts
from ._types import MasterReconciliationRow, RschReconciliationRow, ReconciliationRow
from ._utils import (_to_decimal, _normalize_text, _is_total_row,
    _is_excluded_customer, _is_filter_header, _is_excluded_master_filter,
    _is_excluded_master_header, _is_excluded_rsch_comment, _excel_safe_text, STATUS_VERIFIED, STATUS_REVIEW, STATUS_ERROR,
    REASON_NO_CURATOR, REASON_NO_RSCH, REASON_AMOUNT_MISMATCH,
    REASON_EXECUTOR_MISMATCH, REASON_CUSTOMER_MISMATCH,
    _MASTER_HEADER_ALIASES,
    _text_tokens, _compact_text, _extract_canonical_customer,
    _customer_prefix_match)
from ._resolve import (_executor_matches, _row_amount_matches,
    _rsch_customer_matches, _rsch_customer_matches_weak)


def _customer_exact_or_alias_match(
    master_customer: str,
    rsch_row: RschReconciliationRow,
    alias_repo: Optional[Any] = None,
) -> bool:
    """Strict customer match for VERIFIED status.

    Returns True if any of the following holds:
    1. Exact text match (case-insensitive, whitespace-normalised) between
       master customer and any of rsch customer/contractor/comment, OR
    2. The canonical prefix of the master customer (name extracted before
       trailing noise — project number, INN, amount) equals the normalised
       rsch candidate. Handles curator cells like
       «Анастасия рекрутинг 529. ... ИНН ... сумма ...».
    3. An explicit alias exists in the alias repository — looked up by the
       canonical prefix (so one alias «анастасия рекрутинг» covers every
       noisy variant of that customer), falling back to the raw master text.

    Loose/substring matches do NOT qualify — those get REVIEW status
    so accountants can review and add aliases if appropriate.
    """
    if not master_customer:
        return False
    master_norm = _normalize_text(master_customer)
    if not master_norm:
        return False

    # Уровень 1 — точное совпадение нормализованного текста.
    for candidate in (rsch_row.customer, rsch_row.contractor, rsch_row.comment):
        if not candidate:
            continue
        cand_norm = _normalize_text(candidate)
        if cand_norm and cand_norm == master_norm:
            return True

    # Уровень 2 — авто-извлечение канонического префикса из «мусорной» строки.
    canonical = _extract_canonical_customer(master_customer)
    if canonical and canonical != master_norm:
        for candidate in (rsch_row.customer, rsch_row.contractor, rsch_row.comment):
            if not candidate:
                continue
            cand_norm = _normalize_text(candidate)
            # 2a — точное совпадение извлечённого префикса с rsch-кандидатом.
            if cand_norm and canonical == cand_norm:
                return True
            # 2b — rsch-имя является префиксом из целых токенов канонического
            # master (имя заказчика + хвост из юр.формы/контрагента). Покрывает
            # «Анастасия рекрутинг ООО Интернет Решения ИНН ...» ↔ «анастасия
            # рекрутинг».
            if cand_norm and _customer_prefix_match(canonical, candidate):
                return True

    # Уровень 3 — алиас по каноническому префиксу, затем по исходному тексту.
    if alias_repo is not None:
        keys = [canonical, master_customer] if canonical else [master_customer]
        for key in keys:
            for candidate in (rsch_row.customer, rsch_row.contractor):
                if candidate and alias_repo.is_alias(key, candidate):
                    return True
    return False


def _to_decimal_soft(
```

```
    value: Any,
    *,
    sheet: str,
    row_index: int,
    field: str,
    source: str,
) -> Decimal:
    """Fail-soft amount parsing for reconciliation row collection.

    Mirrors :func:`_to_decimal` but, instead of raising on a dirty cell,
    logs the loss to ``data_loss.log`` and returns ``Decimal("0")`` so a
    single bad amount never aborts the whole sheet. Returning 0 lets the
    surrounding ``income == 0 and expense == 0`` filter drop the row
    cleanly when both amounts are unusable.
    """
    try:
        return _to_decimal(value)
    except (ValueError, ArithmeticError):
        record_loss(
            source=source,
            kind="reconciliation_amount_parse",
            reason="Некорректное числовое значение в сверке; строка пропущена",
            row_index=row_index,
            sheet=sheet,
            field=field,
            original_value=value,
        )
        return Decimal("0")


def _sheet_source(ws: Worksheet) -> str:
    """Stable source identifier for a worksheet.

    openpyxl does not retain the on-disk file path after ``load_workbook``
    (``Workbook.path`` is the in-archive path ``/xl/workbook.xml``), so we
    key data-loss entries on the sheet title. The reconciliation callers
    already pass the on-disk paths to ``build_*_reconciliation_report``,
    which is where the broader context lives.
    """
    return f"reconciliation:{ws.title}"



def _header_map(ws: Worksheet, *, prefer_last: bool = False) -> dict[str, int]:
    headers: dict[str, int] = {}
    for col_idx in range(1, ws.max_column + 1):
        value = ws.cell(1, col_idx).value
        if value in (None, ""):
            continue
        header = str(value).strip()
        if prefer_last or header not in headers:
            headers[header] = col_idx
    return headers


def _master_output_headers(ws: Worksheet) -> list[str]:
    headers: list[str] = []
    for col_idx in range(1, ws.max_column + 1):
        value = ws.cell(1, col_idx).value
        if value in (None, "") or _is_filter_header(value) or _is_excluded_master_header(value):
            continue
        headers.append(_excel_safe_text(str(value).strip()))
    return headers


def _header_index(
    headers: dict[str, int],
    header: str,
    *,
    required: bool = True,
) -> int | None:
    aliases = _MASTER_HEADER_ALIASES.get(header, (header,))
    for alias in aliases:
        if alias in headers:
            return headers[alias]

    normalized_headers = {_normalize_text(key): value for key, value in headers.items()}
    for alias in aliases:
        normalized_alias = _normalize_text(alias)
        if normalized_alias in normalized_headers:
            return normalized_headers[normalized_alias]

    if required:
```

```
        raise KeyError(header)
    return None


def _master_cell_value(
    ws: Worksheet,
    row_idx: int,
    headers: dict[str, int],
    header: str,
) -> Any:
    col_idx = _header_index(headers, header, required=False)
    if col_idx is None:
        return None
    return _cell_display_value(ws, row_idx, col_idx)


def _cell_hyperlink_target(ws: Worksheet, row_idx: int, col_idx: int) -> str | None:
    formula_parts = extract_hyperlink_formula_parts(ws.cell(row_idx, col_idx).value)
    if formula_parts is not None:
        return formula_parts[0]
    hyperlink = ws.cell(row_idx, col_idx).hyperlink
    if hyperlink is None:
        return None
    target = hyperlink.target or hyperlink.location
    if target is None:
        return None
    return target.split("#", 1)[0]


def _cell_display_value(ws: Worksheet, row_idx: int, col_idx: int) -> Any:
    value = ws.cell(row_idx, col_idx).value
    formula_parts = extract_hyperlink_formula_parts(value)
    if formula_parts is not None:
        return formula_parts[1]
    return value


def _collect_master_rows(
    ws: Worksheet,
) -> tuple[list[str], list[MasterReconciliationRow], list[MasterReconciliationRow]]:
    headers = _header_map(ws)
    output_headers = _master_output_headers(ws)
    source_col = _header_index(headers, "Источник")
    customer_col = _header_index(headers, "ЗАКАЗЧИК")
    executor_col = _header_index(headers, "ИСПОЛНИТЕЛЬ")
    income_col = _header_index(headers, "Приход")
    expense_col = _header_index(headers, "Расход")
    filter_col = _header_index(headers, "Фильтры", required=False)
    rows: list[MasterReconciliationRow] = []
    excluded_rows: list[MasterReconciliationRow] = []
    for row_idx in range(2, ws.max_row + 1):
        source = _cell_display_value(ws, row_idx, source_col)
        if _is_total_row(source):
            continue
        customer = str(ws.cell(row_idx, customer_col).value or "").strip()
        executor = str(ws.cell(row_idx, executor_col).value or "").strip()
        if not customer and not executor:
            continue
        income = _to_decimal_soft(
            ws.cell(row_idx, income_col).value,
            sheet=ws.title,
            row_index=row_idx,
            field="Приход",
            source=_sheet_source(ws),
        )
        expense = _to_decimal_soft(
            ws.cell(row_idx, expense_col).value,
            sheet=ws.title,
            row_index=row_idx,
            field="Расход",
            source=_sheet_source(ws),
        )
        if income == 0 and expense == 0:
            continue
        values = [
            _master_cell_value(ws, row_idx, headers, header) for header in output_headers
        ]
        row = MasterReconciliationRow(
            values=values,
            source=source,
            source_link=_cell_hyperlink_target(ws, row_idx, source_col),
            customer=customer,
            executor=executor,
```

```
            income=income,
            expense=expense,
        )
        if filter_col is not None and _is_excluded_master_filter(
            _cell_display_value(ws, row_idx, filter_col)
        ):
            excluded_rows.append(row)
        else:
            rows.append(row)
    return output_headers, rows, excluded_rows


def _collect_rsch_rows(ws: Worksheet) -> list[RschReconciliationRow]:
    headers = _header_map(ws, prefer_last=True)
    rows: list[RschReconciliationRow] = []
    for row_idx in range(2, ws.max_row + 1):
        first = ws.cell(row_idx, 1).value
        if _is_total_row(first):
            continue
        income = _to_decimal_soft(
            ws.cell(row_idx, headers["Приход"]).value,
            sheet=ws.title,
            row_index=row_idx,
            field="Приход",
            source=_sheet_source(ws),
        )
        expense = _to_decimal_soft(
            ws.cell(row_idx, headers["Расход"]).value,
            sheet=ws.title,
            row_index=row_idx,
            field="Расход",
            source=_sheet_source(ws),
        )
        if income == 0 and expense == 0:
            continue
        customer = str(ws.cell(row_idx, headers["ЗАКАЗЧИК"]).value or "").strip()
        if _is_excluded_customer(customer):
            continue
        comment = str(ws.cell(row_idx, headers["Комментарий"]).value or "").strip()
        if _is_excluded_rsch_comment(comment):
            continue
        rows.append(
            RschReconciliationRow(
                row_id=f"{ws.title}:{row_idx}",
                executor=str(ws.cell(row_idx, headers["ИСПОЛНИТЕЛЬ"]).value or "").strip(),
                customer=customer,
                contractor=str(ws.cell(row_idx, headers["Контрагент"]).value or "").strip(),
                comment=comment,
                income=income,
                expense=expense,
                source_link=_cell_hyperlink_target(ws, row_idx, headers.get("Источник", 1)),
            )
        )
    return rows


def _find_best_rsch_match(
    master_row: MasterReconciliationRow,
    customer_candidates: list[tuple[int, RschReconciliationRow]],
    used_indexes: set[int],
) -> tuple[Optional[int], Optional[RschReconciliationRow]]:
    amount_candidates = [
        (idx, row)
        for idx, row in customer_candidates
        if idx not in used_indexes and _row_amount_matches(master_row, row)
    ]
    for idx, row in amount_candidates:
        if _executor_matches(master_row.executor, row.executor):
            return idx, row
    return None, None


def _find_likely_executor_mismatch(
    master_row: MasterReconciliationRow,
    customer_candidates: list[tuple[int, RschReconciliationRow]],
    used_indexes: set[int],
) -> tuple[Optional[int], Optional[RschReconciliationRow]]:
    for idx, row in customer_candidates:
        if idx not in used_indexes and _row_amount_matches(master_row, row):
            return idx, row
    return None, None
```

```
def _find_likely_rsch_mismatch(
    master_row: MasterReconciliationRow,
    customer_candidates: list[tuple[int, RschReconciliationRow]],
    used_indexes: set[int],
) -> tuple[Optional[int], Optional[RschReconciliationRow]]:
    for idx, row in customer_candidates:
        if idx not in used_indexes and _executor_matches(master_row.executor, row.executor):
            return idx, row
    return None, None


def _weak_customer_review_candidate_matches(
    master_row: MasterReconciliationRow,
    rsch_row: RschReconciliationRow,
) -> bool:
    return (
        _row_amount_matches(master_row, rsch_row)
        and _executor_matches(master_row.executor, rsch_row.executor)
        and _rsch_customer_matches_weak(master_row.customer, rsch_row)
    )


def _find_likely_customer_mismatch(
    master_idx: int,
    master_count: int,
    get_weak_candidates: Callable[[int], list[tuple[int, RschReconciliationRow]]],
    used_indexes: set[int],
    exact_master_indexes: set[int],
) -> tuple[Optional[int], Optional[RschReconciliationRow]]:
    candidates = [
        (idx, row)
        for idx, row in get_weak_candidates(master_idx)
        if idx not in used_indexes
    ]
    if len(candidates) != 1:
        return None, None

    candidate_idx, candidate = candidates[0]
    competing_masters = [
        idx
        for idx in range(master_count)
        if idx != master_idx
        and idx not in exact_master_indexes
        and any(
            other_candidate_idx == candidate_idx
            for other_candidate_idx, _ in get_weak_candidates(idx)
        )
    ]
    if competing_masters:
        return None, None
    return candidate_idx, candidate


def _surname_tokens(value: str) -> set[str]:
    raw = re.split(r"[^a-za-яёA-ZA-ЯЁ]+", str(value))
    return {
        w.lower() for w in raw
        if len(w) >= 3 and w[0].isalpha() and w[0].upper() == w[0]
    }


def _customer_surname_matches(
    master_customer: str,
    rsch_row: RschReconciliationRow,
) -> bool:
    master_surnames = _surname_tokens(master_customer)
    if not master_surnames:
        return False
    rsch_text = f"{rsch_row.comment or ''} {rsch_row.contractor or ''}".lower()
    return any(surname in rsch_text for surname in master_surnames)


def _find_surname_rsch_match(
    master_row: MasterReconciliationRow,
    rsch_rows: list[RschReconciliationRow],
    used_indexes: set[int],
    amount_to_rsch: dict[Decimal, list[tuple[int, RschReconciliationRow]]],
) -> tuple[int | None, RschReconciliationRow | None]:
    target_amounts = set()
    if master_row.income != 0:
        target_amounts.add(abs(master_row.income).quantize(Decimal("0.01")))
    if master_row.expense != 0:
        target_amounts.add(abs(master_row.expense).quantize(Decimal("0.01")))
```

```
    amount_candidates = set()
    for amt in target_amounts:
        if amt in amount_to_rsch:
            for idx, _ in amount_to_rsch[amt]:
                amount_candidates.add(idx)

    candidates = [
        (idx, rsch_rows[idx])
        for idx in sorted(amount_candidates)
        if idx not in used_indexes
        and _row_amount_matches(master_row, rsch_rows[idx])
        and _executor_matches(master_row.executor, rsch_rows[idx].executor)
        and _customer_surname_matches(master_row.customer, rsch_rows[idx])
    ]
    if not candidates:
        return None, None
    return candidates[0]


def _build_reconciliation_rows(
    master_rows: list[MasterReconciliationRow],
    rsch_rows: list[RschReconciliationRow],
    alias_repo: Optional[Any] = None,
) -> list[ReconciliationRow]:
    result: list[ReconciliationRow] = []
    used_rsch_indexes: set[int] = set()
    exact_matches: dict[int, int] = {}

    from collections import defaultdict
    token_to_rsch: dict[str, list[tuple[int, RschReconciliationRow]]] = defaultdict(list)
    compact_to_rsch: dict[str, list[tuple[int, RschReconciliationRow]]] = defaultdict(list)
    norm_to_rsch: dict[str, list[tuple[int, RschReconciliationRow]]] = defaultdict(list)
    amount_to_rsch: dict[Decimal, list[tuple[int, RschReconciliationRow]]] = defaultdict(list)

    for idx, row in enumerate(rsch_rows):
        if row.income != 0:
            amount_to_rsch[abs(row.income).quantize(Decimal("0.01"))].append((idx, row))
        if row.expense != 0:
            amount_to_rsch[abs(row.expense).quantize(Decimal("0.01"))].append((idx, row))

        for candidate in (row.customer, row.contractor, row.comment):
            if not candidate:
                continue
            norm = _normalize_text(candidate)
            if norm:
                norm_to_rsch[norm].append((idx, row))
            comp = _compact_text(candidate)
            if comp:
                compact_to_rsch[comp].append((idx, row))
            for tok in _text_tokens(candidate):
                token_to_rsch[tok].append((idx, row))

    customer_candidates_by_master = []
    for master_row in master_rows:
        candidate_indices = set()
        master_tokens = _text_tokens(master_row.customer)
        for tok in master_tokens:
            if tok in token_to_rsch:
                for idx, _ in token_to_rsch[tok]:
                    candidate_indices.add(idx)

        master_comp = _compact_text(master_row.customer)
        if len(master_comp) >= 6:
            for comp_key, rows in compact_to_rsch.items():
                if len(comp_key) >= 6 and (master_comp in comp_key or comp_key in master_comp):
                    for idx, _ in rows:
                        candidate_indices.add(idx)

        master_norm = _normalize_text(master_row.customer)
        if master_norm:
            for norm_key, rows in norm_to_rsch.items():
                if norm_key and (master_norm in norm_key or norm_key in master_norm):
                    for idx, _ in rows:
                        candidate_indices.add(idx)

        cands = [
            (idx, rsch_rows[idx])
            for idx in sorted(candidate_indices)
            if _rsch_customer_matches(master_row.customer, rsch_rows[idx])
        ]
        customer_candidates_by_master.append(cands)

    weak_candidates_by_master: dict[int, list[tuple[int, RschReconciliationRow]]] = {}
```

```
    def get_weak_candidates(master_idx: int) -> list[tuple[int, RschReconciliationRow]]:
        candidates = weak_candidates_by_master.get(master_idx)
        if candidates is None:
            master_row = master_rows[master_idx]
            target_amounts = set()
            if master_row.income != 0:
                target_amounts.add(abs(master_row.income).quantize(Decimal("0.01")))
            if master_row.expense != 0:
                target_amounts.add(abs(master_row.expense).quantize(Decimal("0.01")))

            amount_candidates = set()
            for amt in target_amounts:
                if amt in amount_to_rsch:
                    for idx, _ in amount_to_rsch[amt]:
                        amount_candidates.add(idx)

            candidates = [
                (idx, rsch_rows[idx])
                for idx in sorted(amount_candidates)
                if _weak_customer_review_candidate_matches(master_row, rsch_rows[idx])
            ]
            weak_candidates_by_master[master_idx] = candidates
        return candidates

    for master_idx, master_row in enumerate(master_rows):
        match_idx, rsch_row = _find_best_rsch_match(
            master_row,
            customer_candidates_by_master[master_idx],
            used_rsch_indexes,
        )
        if rsch_row is not None and match_idx is not None:
            used_rsch_indexes.add(match_idx)
            exact_matches[master_idx] = match_idx

    for master_idx, master_row in enumerate(master_rows):
        if master_idx in exact_matches:
            rsch_row = rsch_rows[exact_matches[master_idx]]
            executor_ok = _executor_matches(master_row.executor, rsch_row.executor)
            # VERIFIED requires strict customer match (exact or alias)
            # + amount (already guaranteed by exact_matches) + executor.
            # Loose customer match downgrades to REVIEW so accountants
            # can add an alias if appropriate.
            customer_strict = _customer_exact_or_alias_match(
                master_row.customer, rsch_row, alias_repo
            )
            if customer_strict and executor_ok:
                status, reason = STATUS_VERIFIED, ""
            elif not customer_strict:
                status, reason = STATUS_REVIEW, REASON_CUSTOMER_MISMATCH
            else:
                status, reason = STATUS_REVIEW, REASON_EXECUTOR_MISMATCH
            result.append(
                ReconciliationRow(
                    master=master_row,
                    rsch=rsch_row,
                    status=status,
                    reason=reason,
                )
            )
            continue

        match_idx, rsch_row = _find_likely_executor_mismatch(
            master_row,
            customer_candidates_by_master[master_idx],
            used_rsch_indexes,
        )
        if rsch_row is not None and match_idx is not None:
            used_rsch_indexes.add(match_idx)
            result.append(
                ReconciliationRow(
                    master=master_row,
                    rsch=rsch_row,
                    status=STATUS_REVIEW,
                    reason=REASON_EXECUTOR_MISMATCH,
                )
            )
            continue

        match_idx, rsch_row = _find_likely_rsch_mismatch(
            master_row,
            customer_candidates_by_master[master_idx],
            used_rsch_indexes,
        )
```

```
        if rsch_row is not None and match_idx is not None:
            used_rsch_indexes.add(match_idx)
            result.append(
                ReconciliationRow(
                    master=master_row,
                    rsch=rsch_row,
                    status=STATUS_REVIEW,
                    reason=REASON_AMOUNT_MISMATCH,
                )
            )
            continue

        match_idx, rsch_row = _find_likely_customer_mismatch(
            master_idx,
            len(master_rows),
            get_weak_candidates,
            used_rsch_indexes,
            set(exact_matches),
        )
        if rsch_row is not None and match_idx is not None:
            used_rsch_indexes.add(match_idx)
            result.append(
                ReconciliationRow(
                    master=master_row,
                    rsch=rsch_row,
                    status=STATUS_REVIEW,
                    reason=REASON_CUSTOMER_MISMATCH,
                )
            )
            continue

        match_idx, rsch_row = _find_surname_rsch_match(
            master_row,
            rsch_rows,
            used_rsch_indexes,
            amount_to_rsch,
        )
        if rsch_row is not None and match_idx is not None:
            used_rsch_indexes.add(match_idx)
            result.append(
                ReconciliationRow(
                    master=master_row,
                    rsch=rsch_row,
                    status=STATUS_REVIEW,
                    reason=REASON_CUSTOMER_MISMATCH,
                )
            )
            continue

        result.append(
            ReconciliationRow(
                master=master_row,
                rsch=None,
                status=STATUS_ERROR,
                reason=REASON_NO_RSCH,
            )
        )

    for idx, row in enumerate(rsch_rows):
        if idx in used_rsch_indexes:
            continue
        result.append(
            ReconciliationRow(
                master=None,
                rsch=row,
                status=STATUS_ERROR,
                reason=REASON_NO_CURATOR,
            )
        )
    return result


def _suppress_rsch_rows_matched_by_excluded_master_filters(
    rsch_rows: list[RschReconciliationRow],
    excluded_master_rows: list[MasterReconciliationRow],
) -> list[RschReconciliationRow]:
    if not excluded_master_rows:
        return rsch_rows

    suppressed_indexes: set[int] = set()
    for master_row in excluded_master_rows:
        candidates = [
            (idx, row)
```

```
            for idx, row in enumerate(rsch_rows)
            if idx not in suppressed_indexes
            and _rsch_customer_matches(master_row.customer, row)
        ]
        match_idx, _rsch_row = _find_best_rsch_match(
            master_row,
            candidates,
            suppressed_indexes,
        )
        if match_idx is not None:
            suppressed_indexes.add(match_idx)

    if not suppressed_indexes:
        return rsch_rows
    return [
        row
        for idx, row in enumerate(rsch_rows)
        if idx not in suppressed_indexes
    ]


def _rsch_values(row: Optional[RschReconciliationRow]) -> list[Any]:
    if row is None:
        return [None, None, None, None, None, None]
    return [
        _excel_safe_text(row.executor),
        _excel_safe_text(row.customer),
        _excel_safe_text(row.contractor),
        _excel_safe_text(row.comment),
        row.income if row.income != 0 else None,
        row.expense if row.expense != 0 else None,
    ]


def _master_values(row: ReconciliationRow, master_headers: list[str]) -> list[Any]:
    target_len = len(master_headers)
    if row.master is not None:
        vals = row.master.values
        if len(vals) < target_len:
            vals = list(vals) + [None] * (target_len - len(vals))
        elif len(vals) > target_len:
            vals = vals[:target_len]
        return [_excel_safe_text(value) for value in vals]
    values: list[Any] = [None for _ in master_headers]
    if values:
        values[0] = _excel_safe_text(row.reason)
    return values
```

APP\core\infra\reconciliation_resolve.py

232 lines • 8,901 bytes

```
from __future__ import annotations
from contextlib import contextmanager
from datetime import date, datetime
from decimal import Decimal
from pathlib import Path
from openpyxl import load_workbook
from openpyxl.worksheet.worksheet import Worksheet
from core.infra.utils import get_local_copy
from ._types import MasterReconciliationRow, RschReconciliationRow
from ._utils import (_compact_text, _normalize_text,
    _canonical_match_text, _executor_alias_keys, _executor_tokens,
    _text_tokens, _weak_customer_tokens)

def _amount_equal(left: Decimal, right: Decimal) -> bool:
    return abs(left - right) <= Decimal("0.01")


def _row_amount_matches(master: MasterReconciliationRow, rsch: RschReconciliationRow) -> bool:
    income_matches = master.income != 0 and _amount_equal(master.income, rsch.income)
    expense_matches = master.expense != 0 and _amount_equal(master.expense, rsch.expense)
    if income_matches or expense_matches:
        return True
    if master.income < 0 and _amount_equal(abs(master.income), rsch.expense):
        return True
    if master.expense < 0 and _amount_equal(abs(master.expense), rsch.income):
        return True
    if rsch.income < 0 and _amount_equal(master.income, abs(rsch.income)):
        return True
    if rsch.expense < 0 and _amount_equal(master.expense, abs(rsch.expense)):
        return True
    return False


def _executor_matches(master_executor: str, rsch_executor: str) -> bool:
    left = _canonical_match_text(master_executor)
    right = _canonical_match_text(rsch_executor)
    if not left or not right:
        return False
    if left == right or _compact_text(left) == _compact_text(right):
        return True
    if _executor_alias_keys(left) & _executor_alias_keys(right):
        return True
    left_tokens = {t for t in _executor_tokens(left) if len(t) >= 4}
    right_tokens = {t for t in _executor_tokens(right) if len(t) >= 4}
    if not left_tokens or not right_tokens:
        return False
    return left_tokens.issubset(right_tokens) or right_tokens.issubset(left_tokens)


def _customer_matches(master_customer: str, rsch_customer: str) -> bool:
    left = _normalize_text(master_customer)
    right = _normalize_text(rsch_customer)
    return bool(left and right and (left in right or right in left))


def _customer_matches_loose(left_value: str, right_value: str) -> bool:
    if _customer_matches(left_value, right_value):
        return True
    left = _compact_text(left_value)
    right = _compact_text(right_value)
    if len(left) >= 6 and len(right) >= 6 and (left in right or right in left):
        return True
    common_tokens = _text_tokens(left_value) & _text_tokens(right_value)
    return len(common_tokens) >= 2


def _rsch_customer_matches(master_customer: str, rsch_row: RschReconciliationRow) -> bool:
    return any(
        _customer_matches_loose(master_customer, candidate)
        for candidate in (rsch_row.customer, rsch_row.contractor, rsch_row.comment)
    )


def _customer_matches_weak(left_value: str, right_value: str) -> bool:
    if _customer_matches_loose(left_value, right_value):
        return False
    common_tokens = _weak_customer_tokens(left_value) & _weak_customer_tokens(right_value)
    return bool(common_tokens)


def _rsch_customer_matches_weak(master_customer: str, rsch_row: RschReconciliationRow) -> bool:
    return any(
        _customer_matches_weak(master_customer, candidate)
```

```
        for candidate in (rsch_row.customer, rsch_row.contractor, rsch_row.comment)
    )


def _sheet_candidates(target_date: date) -> tuple[str, str]:
    return target_date.strftime("%d.%m.%Y"), target_date.strftime("%d.%m")


def _coerce_date(value: date) -> date:
    if isinstance(value, datetime):
        return value.date()
    return value


def _parse_full_sheet_date(sheet_title: str) -> date | None:
    try:
        return datetime.strptime(sheet_title, "%d.%m.%Y").date()
    except ValueError:
        return None


def _parse_day_month_sheet(sheet_title: str) -> tuple[int, int] | None:
    try:
        parsed = datetime.strptime(sheet_title, "%d.%m")
    except ValueError:
        return None
    return parsed.day, parsed.month


def _has_sheet_for_date(wb, target_date: date) -> bool:
    candidates = set(_sheet_candidates(target_date))
    return any(ws.title in candidates for ws in wb.worksheets)


def _workbook_sheet_dates(wb) -> tuple[set[date], set[tuple[int, int]]]:
    full_dates: set[date] = set()
    day_months: set[tuple[int, int]] = set()
    for ws in wb.worksheets:
        full_date = _parse_full_sheet_date(ws.title)
        if full_date is not None:
            full_dates.add(full_date)
            continue
        day_month = _parse_day_month_sheet(ws.title)
        if day_month is not None:
            day_months.add(day_month)
    return full_dates, day_months


def _common_sheet_dates(master_wb, rsch_wb) -> list[date]:
    master_dates, master_day_months = _workbook_sheet_dates(master_wb)
    rsch_dates, rsch_day_months = _workbook_sheet_dates(rsch_wb)
    common = set(master_dates & rsch_dates)
    common.update(
        d for d in master_dates if (d.day, d.month) in rsch_day_months
    )
    common.update(
        d for d in rsch_dates if (d.day, d.month) in master_day_months
    )
    return sorted(common)


def _format_dates(dates: list[date]) -> str:
    return ", ".join(d.strftime("%d.%m.%Y") for d in dates)


def _resolve_target_date(master_wb, rsch_wb, target_date: date | None) -> date:
    if target_date is not None:
        target_date = _coerce_date(target_date)
        if _has_sheet_for_date(master_wb, target_date) and _has_sheet_for_date(
            rsch_wb,
            target_date,
        ):
            return target_date
        raise ValueError(
            f"Не найден лист за дату {target_date.strftime('%d.%m.%Y')} "
            "в выбранной паре мастер- и РСЧ-файлов"
        )

    common_dates = _common_sheet_dates(master_wb, rsch_wb)
    if len(common_dates) == 1:
        return common_dates[0]

    if not common_dates:
```

```
        raise ValueError("Не найдена общая дата в выбранных мастер- и РСЧ-файлах")

    raise ValueError(
        "В выбранных файлах найдено несколько общих дат: "
        f"{_format_dates(common_dates)}. Выберите нужную дату в панели слева."
    )


def _find_sheet(wb, target_date: date) -> Worksheet:
    target_date = _coerce_date(target_date)
    candidates = set(_sheet_candidates(target_date))
    for ws in wb.worksheets:
        if ws.title in candidates:
            return ws
    raise ValueError(
        f"Не найден лист за дату {target_date.strftime('%d.%m.%Y')}"
    )


def _resolve_combined_target_date(
    master_paths: list[tuple[str, Path | str]],
    rsch_wb,
    target_date: date | None,
    drive_letters: tuple[str, ...],
) -> date | None:
    if target_date is not None:
        return _coerce_date(target_date)

    common_dates: set[date] | None = None
    for sheet_name, master_path_value in master_paths:
        master_path = Path(master_path_value)
        if not master_path.exists():
            raise FileNotFoundError(f"Не найден мастер-отчет {sheet_name}: {master_path}")
        with _local_read_path(master_path, drive_letters) as master_read_path:
            master_wb = None
            try:
                master_wb = load_workbook(master_read_path, data_only=True)
                master_dates = _common_sheet_dates(master_wb, rsch_wb)
                if common_dates is None:
                    common_dates = set(master_dates)
                else:
                    common_dates &= set(master_dates)
            finally:
                if master_wb is not None:
                    master_wb.close()

    if common_dates and len(common_dates) == 1:
        return next(iter(common_dates))
    if not common_dates:
        raise ValueError("Не найдена общая дата в выбранных мастер- и РСЧ-файлах")
    raise ValueError(
        "В выбранных файлах найдено несколько общих дат: "
        f"{_format_dates(sorted(common_dates))}. Выберите нужную дату в панели слева."
    )


@contextmanager
def _local_read_path(path: Path, drive_letters: tuple[str, ...] = ("Q", "Z")):
    target_path, temp_file = get_local_copy(str(path), drive_letters)
    try:
        yield target_path
    finally:
        if temp_file:
            Path(temp_file).unlink(missing_ok=True)
```

APP\core\infra\reconciliation_types.py

35 lines • 754 bytes

```
from __future__ import annotations
from dataclasses import dataclass
from decimal import Decimal
from typing import Any, Optional


@dataclass(frozen=True)
class MasterReconciliationRow:
    values: list[Any]
    source: Any
    source_link: str | None
    customer: str
    executor: str
    income: Decimal
    expense: Decimal


@dataclass(frozen=True)
class RschReconciliationRow:
    row_id: str
    executor: str
    customer: str
    contractor: str
    comment: str
    income: Decimal
    expense: Decimal
    source_link: str | None


@dataclass(frozen=True)
class ReconciliationRow:
    master: Optional[MasterReconciliationRow]
    rsch: Optional[RschReconciliationRow]
    status: str
    reason: str = ""
```

APP\core\infra\reconciliation_utils.py

432 lines • 19,489 bytes

```
from __future__ import annotations

import re
from decimal import Decimal, InvalidOperation
from functools import lru_cache
from typing import Any

from openpyxl.styles import Border, Font, PatternFill, Side

from ._types import ReconciliationRow


MASTER_HEADERS = [
    "Источник",
    "ЗАКАЗЧИК",
    "ИСПОЛНИТЕЛЬ",
    "Приход",
    "Расход",
    "Комиссия",
    "ИТОГ",
    "% посредника",
    "посредник 1(кэш)",
    "посредник 2",
]
_MASTER_HEADER_ALIASES = {
    "Источник": ("Источник", "Сотрудник"),
    "% посредника": ("% посредника", "% посредник 1", "% посредника 1"),
    "посредник 1(кэш)": ("посредник 1(кэш)", "посредник 1"),
}
RSCH_HEADERS = [
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
]
REASON_HEADER = "Причина"
STATUS_HEADER = "Статус"
RECONCILIATION_HEADERS = [*MASTER_HEADERS, STATUS_HEADER, *RSCH_HEADERS, REASON_HEADER]
REASON_NO_CURATOR = "ОШИБКА НЕТУ У КУРАТОРОВ"
REASON_NO_RSCH = "НЕТ В РСЧ"
REASON_AMOUNT_MISMATCH = "НЕ СОВПАЛА СУММА"
REASON_EXECUTOR_MISMATCH = "НЕ СОВПАЛ ИСПОЛНИТЕЛЬ"
REASON_CUSTOMER_MISMATCH = "ПОХОЖИЙ ЗАКАЗЧИК"
REASON_AMOUNT_ONLY_MATCH = "СОВПАЛА СУММА"
STATUS_VERIFIED = "СВЕРЕНО"
STATUS_REVIEW = "ПРОВЕРИТЬ"
STATUS_ERROR = "ОШИБКА"
_MATCH_TOKEN_STOPWORDS = {
    "альфа",
    "сбер",
    "сби",
    "сти",
    "стинг",
    "ука",
    "реестр",
}
_COMBINED_SHEET_PRIORITY = {"Общий": 0, "Борис": 1, "Иван": 2}
_EXCLUDED_MASTER_FILTER_KEYWORDS = (
    "доп сф",
    "доп сы",
    "касса",
    "зачет",
    "посредник",
    "корректировка",
)
_SUMMARY_REASON_LABELS = {
    REASON_NO_CURATOR: "Нет у кураторов",
    REASON_NO_RSCH: "Нет в РСЧ",
    REASON_AMOUNT_MISMATCH: "Не совпала сумма",
    REASON_EXECUTOR_MISMATCH: "Не совпал исполнитель",
    REASON_CUSTOMER_MISMATCH: "Похожий заказчик",
    REASON_AMOUNT_ONLY_MATCH: "Совпала сумма",
}

_HEADER_FILL = PatternFill("solid", fgColor="FF1F4E79")
_STATUS_OK_FILL = PatternFill("solid", fgColor="FF00B050")
_STATUS_REVIEW_FILL = PatternFill("solid", fgColor="FFFFEB9C")
_STATUS_ERROR_FILL = PatternFill("solid", fgColor="FFFFC7CE")
_MASTER_EXPENSE_FILL = PatternFill("solid", fgColor="FFFFC7CE")
_HEADER_FONT = Font(color="FFFFFFFF", bold=True)
_STATUS_FONT = Font(color="FF000000", bold=True)
```

```
_THIN_SIDE = Side(style="thin", color="FF000000")
_THIN_BORDER = Border(
    left=_THIN_SIDE,
    right=_THIN_SIDE,
    top=_THIN_SIDE,
    bottom=_THIN_SIDE,
)



def _normalize_text(value: Any) -> str:
    if value is None:
        return ""
    return _normalize_text_cached(str(value))


@lru_cache(maxsize=65536)
def _normalize_text_cached(text: str) -> str:
    return " ".join(text.replace("\xa0", " ").replace("ё", "е").lower().split())


def _canonical_match_text(value: Any) -> str:
    return _canonical_match_text_cached(_normalize_text(value))


@lru_cache(maxsize=65536)
def _canonical_match_text_cached(text: str) -> str:
    text = re.sub(r"([а-я])([a-z])", r"\1 \2", text)
    text = re.sub(r"([a-z])([а-я])", r"\1 \2", text)
    text = text.replace("gabriel", "габриэль")
    # УКА: "УК Альянс", "УКАльянс", "УК А", "УКА_Альфа", "УКА_Сбер", "УКА_ВТБ", "УК_ВТБ", "УК_Альфа"
    # Note: input is already lowercased+collapsed by _normalize_text, but `_` survives
    # and is a word char in Python regex (\b does NOT trigger after "с" in "сбер" when
    # followed by "_"). So we use a lookbehind for non-cyrillic instead of \b.
    text = re.sub(r"(?<![а-я])ук(?:а_альфа|а_сбер|а_втб|альянс|альфа|втб)", "ука", text)
    text = re.sub(r"\bук[\s_]+альянс\b", "ука", text)
    text = re.sub(r"\bукальянс\b", "ука", text)
    text = re.sub(r"(?<![а-я])укалинс(?![а-я])", "ука", text)  # common typo
    text = re.sub(r"\бул[\s_]+альянс\b", "ука", text)  # "Ул альянс" = УК Альянс
    text = re.sub(r"\бук[\s_]+а\b", "ука", text)
    text = re.sub(r"\бук[\s_]+аф\b", "ука", text)  # "УК Аф..." (shorthand for УК А + bank)
    text = re.sub(r"\бук[\s_]+(?=(альфа|сбер|втб|сб)\b)", "ука ", text)
    # Catch-all: any word starting with ука+letters (optionally _bank suffix)
    text = re.sub(r"(?<![а-я])ука[а-я]*(?:_[а-я]+)?(?![а-я])", "ука", text)
    # "Сбер_РН" used as a synonym for УК Альянс routed via Sber settlement account
    text = re.sub(r"(?<![а-я])сбер[\s_]*рн(?![а-я])", "ука", text)
    # Tinkoff: "Тина", "Тинадар", "Тинадар", "Тина Сбер"
    # Note: re.sub replaces the matched text verbatim; "[\s_]*" with trailing \b
    # would eat a trailing space. Anchor with explicit lookahead or use \b before/after.
    text = re.sub(r"\втина(?:[\s_]*(?:дар|да|бизнес))?(?![а-я])", "тина", text)
    text = re.sub(r"\втина[\s_]+сбер\b", "тина", text)
    # Billtrade: "Билл", "Билтрейд", "Биллтрейд"
    text = re.sub(r"\ббилл?трейд\b", "биллтрейд", text)
    text = re.sub(r"\ббилл(?!\w)", "биллтрейд", text)
    text = re.sub(r"\ббилтрейд\b", "биллтрейд", text)
    # АНО Техникум: "АНО ТСК", "АНО ТЕХНИКУМ"
    text = re.sub(r"\вано[\s_]+тск\b", "анотехникум", text)
    text = re.sub(r"\вано[\s_]+техникум\b", "анотехникум", text)
    text = re.sub(r"(?<![а-я])тск(?:(?:[\s_]+сбер)|(?:[\s_]+альфа)|(?:[\s_]+втб)|(?:[\s_]+сб))?(?![а-я])", "анотехникум", text)
    # СВ Защита: "СБ ЗАЩИТА", "Защита РРБ", "Защита"
    text = re.sub(r"\всб[\s_]+защита\b", "сбзащита", text)
    text = re.sub(r"\взащита[\s_]+ррб\b", "сбзащита", text)
    # Стинг: "Стип", "СтипСбер", "Стинг", "СТИ Сбер"
    text = re.sub(r"\встип[\s_]*сбер\b", "стинг", text)
    text = re.sub(r"(?<![а-я])стип(?![а-я])", "стинг", text)
    text = re.sub(r"\встинг[\s_]*сбер\b", "стинг сбер", text)
    text = re.sub(r"(?<![а-я])сти(?:[\s_]+сбер|[\s_]+альфа|[\s_]+втб|[\s_]+сб)?(?![а-я])", "стинг", text)
    # Прайм = Прессбук (один банк)
    text = re.sub(r"\впрайм\b", "прессбук", text)
    # ПБ = Прессбук (compact form)
    text = re.sub(r"(?<![а-я])пб(?:[\s_]+альфа|[\s_]+сбер|[\s_]+втб)?(?![а-я])", "прессбук", text)
    # Fuzzy typos for Прессбук: преесбук, прссбук, перссбук, прессбук
    text = re.sub(r"(?<![а-я])пр[её]+?ссбук(?![а-я])", "прессбук", text)
    text = re.sub(r"(?<![а-я])пе?рссбук(?![а-я])", "прессбук", text)
    # Пиплкрафт: "Пип", "ПипКрафт", "Пипл Сбер", "Пип крафт Альфа"
    # NOTE: (?!л) prevents matching "пип" inside "пипл" which would
    # leave trailing characters (e.g. "пипл крафт" → "пипл крафтл крафт").
    text = re.sub(r"(?<![а-я])пип(?!л)(?:[\s_]*крафт)?(?:[\s_]+альфа|[\s_]+сбер|[\s_]+втб)?", "пипл крафт", text)
    # Normalise "Пипл" (the full short form) with a bank suffix to canonical form.
    # "Пипл сбер" → "пипл крафт сбер", "Пипл Альфа" → "пипл крафт альфа"
    text = re.sub(r"(?<![а-я])пипл(?:[\s_]+крафт)?[\s_]+(альфа|сбер|втб|сб)\b", r"пипл крафт \1", text)
    # Standalone "пипл" without банк suffix: keep as is, tokens will match via alias keys
    # ПУ = Пиплкрафт (ПК в одном контексте)
```

```
    text = re.sub(r"\bпу[\s_]*(альфа|сбер|втб)\b", r"пипл крафт \1", text)
    text = re.sub(r"\bпу\b", "пиплкрафт", text)
    # ПК = Пиплкрафт
    text = re.sub(r"\bпк[\s_]*(альфа|сбер|втб|аф)?\b", r"пипл крафт", text)
    # СБ = СБИ (СтройБизнесИнвест)
    text = re.sub(r"(?<![а-я])стройбизнесинвест(?![а-я])", "сби", text)
    # СТ = Спецстаффсервис
    text = re.sub(r"\bст[\s_]*(втб|альфа|сбер|бизнес)\b", r"ссс \1", text)
    text = re.sub(r"\bст\b", "ссс", text)
    # ВоркБрайт = ВБ (Slot WorkBright)
    text = re.sub(r"(?<![а-я])слот[\s_]*воркбрайт(?![а-я])", "вб", text)
    text = re.sub(r"(?<![а-я])воркбрайт(?![а-я])", "вб", text)
    return text


def _extract_canonical_customer(value: Any) -> str:
    """Выделить короткое имя заказчика из «мусорной» строки куратора.

    Со стороны кураторов в колонку «Заказчик» часто пишут не только имя, но и
    номер проекта, ИНН, сумму, выходной банк — всё в одну ячейку:
        «Анастасия рекрутинг 529. ... ИНН 280128132624, сумма 70 605,00»
    Эта функция отрезает имя по первому разделителю (2+ цифры / точка / запятая
    / маркеры «инн», «сумма», «выход») и возвращает чистый префикс.

    Возвращает "" для пустого/короткого результата — тогда вызывающий код
    оставляет исходную строку и работает прежний алгоритм сравнения.
    """
    return _extract_canonical_customer_cached(_normalize_text(value))


@lru_cache(maxsize=65536)
def _extract_canonical_customer_cached(text: str) -> str:
    if not text:
        return ""
    # Пропустить ведущий номер проекта: «511. Икарус ...» → «икарус ...»,
    # «530_рубашка ...» → «рубашка ...». Только 1-4 цифры в самом начале,
    # за которыми следует разделитель (точка/подчёркивание/пробел).
    text = re.sub(r"^\d{1,4}[._\s]+", "", text)
    # Разделители: 2+ цифр подряд, точка, запятая, либо слова-маркеры мусора.
    parts = re.split(r"\s*\d{2,}|\s*[.,]|\s+(?:инн|сумма|выход)\b", text, maxsplit=1)
    canonical = parts[0].strip()
    if len(canonical) < 3:
        return ""
    if len(canonical) > 40:
        canonical = canonical[:40].rsplit(" ", 1)[0]
    return canonical


def _customer_prefix_match(canonical_master: str, rsch_candidate: str) -> bool:
    """Проверить, что rsch-имя — это префикс из целых токенов канонического master.

    Реальные строки куратора часто содержат после имени заказчика юр. форму и
    контрагента: «Анастасия рекрутинг ООО Интернет Решения ИНН ...». После
    _extract_canonical_customer остаётся «анастасия рекрутинг ооо интернет
    решения», а в РСЧ записано коротко «анастасия рекрутинг». Сравнение по
    целым токенам (rsch должен полностью укладываться в начало master-токенов)
    безопаснее, чем подстрока/compact-startswith: оно не разрывает слова и не
    путает «гетворк мерч» с «гетворк анкор мерчандайзинг».
    """
    if not canonical_master or not rsch_candidate:
        return False
    master_tokens = _name_prefix_tokens(canonical_master)
    rsch_tokens = _name_prefix_tokens(rsch_candidate)
    if not master_tokens or not rsch_tokens:
        return False
    if len(rsch_tokens) > len(master_tokens):
        return False
    return master_tokens[: len(rsch_tokens)] == rsch_tokens


@lru_cache(maxsize=65536)
def _name_prefix_tokens(text: str) -> tuple[str, ...]:
    """Токены имени для префиксного сравнения (канонизированные, ≥4 символов)."""
    canon = _canonical_match_text(text)
    return tuple(
        w for w in re.split(r"[^0-9a-za-я]+", canon)
        if len(w) >= 4 and not w.isdigit()
    )


def _compact_text(value: Any) -> str:
    return _compact_text_cached(_canonical_match_text(value))
```

```
@lru_cache(maxsize=65536)
def _compact_text_cached(text: str) -> str:
    return "".join(ch for ch in text if ch.isalnum())


def _text_tokens(value: Any) -> set[str]:
    return set(_text_tokens_cached(_canonical_match_text(value)))


@lru_cache(maxsize=65536)
def _text_tokens_cached(text: str) -> frozenset[str]:
    tokens = set()
    for token in re.split(r"[^0-9a-za-я]+", text):
        if len(token) >= 4 and not token.isdigit() and token not in _MATCH_TOKEN_STOPWORDS:
            tokens.add(token)
    return frozenset(tokens)


def _weak_customer_tokens(value: Any) -> set[str]:
    ignored = {
        "готово",
        "корр",
        "проект",
        "труд",
        "нетруд",
        "выплата",
        "зарплатного",
    }
    return {token for token in _text_tokens(value) if token not in ignored}


def _executor_tokens(value: Any) -> list[str]:
    return list(_executor_tokens_cached(_canonical_match_text(value)))


@lru_cache(maxsize=65536)
def _executor_tokens_cached(text: str) -> tuple[str, ...]:
    return tuple(token for token in re.split(r"[^0-9a-za-я]+", text) if len(token) >= 2)


def _executor_alias_keys(value: Any) -> set[str]:
    compact = _compact_text(value)
    tokens = set(_executor_tokens(value))
    aliases = {compact} if compact else set()

    if compact in {"мосальфа", "моссбер", "мосвтб"} or "мос" in tokens:
        aliases.add("мос")
    if (
        compact in {"пк", "пкальфа", "пксбер", "пксб"}
        or compact.startswith(("пиплкрафт", "пилпкрафт"))
        or {"пипл", "крафт"}.issubset(tokens)
        or {"пилп", "крафт"}.issubset(tokens)
    ):
        aliases.add("пиплкрафт")
    if (
        compact in {"пб", "пбальфа", "пбсбер", "пбсб"}
        or compact.startswith(("прессбук", "пресс", "прес"))
    ):
        aliases.add("прессбук")
    if compact.startswith(("ссс", "спецстафф", "спецстаф")):
        aliases.add("спецстаффсервис")
    if compact.startswith("воркбрайт") or compact == "вб" or "вб" in tokens:
        aliases.add("вб")
    if compact.startswith("ук") and len(compact) >= 3:
        aliases.add("ук")
    return aliases


def _is_filter_header(value: Any) -> bool:
    return _normalize_text(value) in {"фильтр", "фильтры", "filter", "filters"}


def _is_excluded_master_filter(value: Any) -> bool:
    normalized = _normalize_text(value)
    collapsed = normalized.replace(".", " ").replace(",", " ")
    collapsed = " ".join(collapsed.split())
    return any(
        keyword in collapsed
        for keyword in _EXCLUDED_MASTER_FILTER_KEYWORDS
    ) or any(keyword in normalized for keyword in _EXCLUDED_MASTER_FILTER_KEYWORDS)


def _is_excluded_master_header(value: Any) -> bool:
```

```
    return _normalize_text(value) in {
        "отправка транзитных обязательств",
        "курс",
        "сумма",
        "валюта",
    }


def _is_excluded_customer(value: Any) -> bool:
    normalized = _normalize_text(value)
    return any(
        keyword in normalized
        for keyword in (
            "налог",
            "расход",
            "% по депозиту",
            "по депозиту",
            "% по выплате",
            "по выплате",
            "комиссия",
            "взносы",
        )
    )


def _is_excluded_rsch_comment(value: Any) -> bool:
    return "перевод" in re.split(r"[^0-9a-za-я]+", _normalize_text(value))


def _is_numeric_header(value: Any) -> bool:
    normalized = _normalize_text(value)
    return any(
        keyword in normalized
        for keyword in (
            "приход",
            "расход",
            "комиссия",
            "итог",
            "посредник",
            "транзит",
            "сумма",
        )
    )


def _is_percent_header(value: Any) -> bool:
    return "%" in str(value or "")


def _to_decimal(value: Any) -> Decimal:
    if value is None or value == "":
        return Decimal("0")
    if isinstance(value, bool):
        return Decimal("1") if value else Decimal("0")
    if isinstance(value, Decimal):
        return value
    if isinstance(value, (int, float)):
        return Decimal(str(value))
    text = str(value).replace(" ", "").replace(",", ".").strip()
    if not text or text.startswith("="):
        return Decimal("0")
    try:
        return Decimal(text)
    except (InvalidOperation, ValueError):
        raise ValueError(f"Некорректное числовое значение в сверке: {value}") from None


def _is_total_row(value: Any) -> bool:
    return _normalize_text(value).startswith("итого")


def _excel_safe_text(value: Any) -> Any:
    if isinstance(value, str):
        stripped = value.lstrip()
        if stripped and stripped[0] in ("=", "+", "-", "@"):
            return "'" + value
    return value


def _row_id(row: ReconciliationRow) -> str:
    """Stable ID for a reconciliation row based on master/rsch content."""
    parts: list[str] = []
    if row.master is not None:
```

```
        parts.extend([
            _normalize_text(row.master.customer),
            _normalize_text(row.master.executor),
            str(row.master.income),
            str(row.master.expense),
        ])
    if row.rsch is not None:
        parts.extend([
            _normalize_text(row.rsch.customer),
            _normalize_text(row.rsch.executor),
            str(row.rsch.income),
            str(row.rsch.expense),
        ])
    return "|".join(parts) if parts else f"status:{row.status}:{row.reason}"
```

APP\core\infra\rsch__init__.py

12 lines • 454 bytes

```
"""RSCH report package — decomposed from rsch_report.py.

Layout:
    _format.py   — constants, pure utilities (_normalize_text, _to_decimal, _excel_safe_text, ...)
    _collect.py  — reading from workbooks (calamine, openpyxl)
    _template.py — template handling, style constants
    _build.py    — writing to Excel, public build_rsch_report entry point
"""

from ._build import build_rsch_report

__all__ = ["build_rsch_report"]
```

APP\core\infra\rsch_build.py

552 lines • 20,374 bytes

```
from __future__ import annotations

import logging
from copy import copy
from datetime import date
from decimal import Decimal
from pathlib import Path
from typing import Any, Dict, List, Optional, Sequence

import openpyxl
from openpyxl import Workbook
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter
from openpyxl.worksheet.worksheet import Worksheet

from core.infra.excel_hyperlinks import apply_cell_hyperlink
from core.infra.excel_backup import backup_existing_excel_output
from core.infra.utils import atomic_save_workbook

from ._format import _display_len, _excel_safe_text, _is_problem_row
from ._collect import _collect_rows_from_workbook, _copy_style, _normalize_source_paths
from ._template import _create_builtin_template_workbook, _find_total_row, _resolve_template_style_layout

logger = logging.getLogger(__name__)

RSCH_HEADERS = [
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
    "р/с",
    "Источник",
]
RSCH_TEMPLATE_HEADERS = RSCH_HEADERS

RSCH_TABLE_HEADERS = [
    "Дата",
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
    "р/с",
    "Источник",
]

_REPO_ROOT = Path(__file__).resolve().parents[3]
DEFAULT_RSCH_TEMPLATE_PATH = _REPO_ROOT / "РСЧ" / "шаблон отчета.xlsx"

_INCOME_FILL = PatternFill(
    start_color="FFC6EFCE", end_color="FFC6EFCE", fill_type="solid"
)
_EXPENSE_FILL = PatternFill(
    start_color="FFFFC7CE", end_color="FFFFC7CE", fill_type="solid"
)
_WARNING_FILL = PatternFill(
    start_color="FFFFFF00", end_color="FFFFFF00", fill_type="solid"
)
_INCOME_FONT = Font(color="FF006100", bold=True)
_EXPENSE_FONT = Font(color="FF9C0006", bold=True)
_HEADER_FILL = PatternFill(start_color="FF1F4E79", end_color="FF1F4E79", fill_type="solid")
_TOTAL_FILL = PatternFill(start_color="FFD9E1F2", end_color="FFD9E1F2", fill_type="solid")
_HEADER_FONT = Font(color="FFFFFFFF", bold=True)
_THIN_SIDE = Side(style="thin", color="FF000000")
_MEDIUM_SIDE = Side(style="medium", color="FF000000")
_THIN_BORDER = Border(
    left=_THIN_SIDE,
    right=_THIN_SIDE,
    top=_THIN_SIDE,
    bottom=_THIN_SIDE,
)
_TOTAL_BORDER = Border(
    left=_MEDIUM_SIDE,
    right=_MEDIUM_SIDE,
    top=_MEDIUM_SIDE,
    bottom=_MEDIUM_SIDE,
)
_HEADER_ALIGNMENT = Alignment(horizontal="center", vertical="center", wrap_text=True)
_TEXT_ALIGNMENT = Alignment(horizontal="left", vertical="top", wrap_text=True)
```

```
_ACCOUNT_ALIGNMENT = Alignment(horizontal="left", vertical="center")
_AMOUNT_ALIGNMENT = Alignment(horizontal="right", vertical="center")
_DATE_ALIGNMENT = Alignment(horizontal="left", vertical="center")
_DEFAULT_AMOUNT_NUMBER_FORMAT = "#,##0.00"
_SPECIAL_CUSTOMER_MARKERS = (
    "% по депозиту",
    "% по выплате",
    "комиссия",
    "налог",
    "взносы",
)
_NO_ACCOUNT_MARKERS = (
    "факторинг",
    "факторнинг",
    "вф",
    "вафт фокс",
    "гетворк",
)



def _ensure_header_style(ws: Worksheet, force_normalize: bool, is_table: bool = False) -> None:
    if not force_normalize:
        return
    headers = RSCH_TABLE_HEADERS if is_table else RSCH_HEADERS
    for col_idx in range(1, len(headers) + 1):
        cell = ws.cell(1, col_idx)
        cell.fill = copy(_HEADER_FILL)
        cell.font = copy(_HEADER_FONT)
        cell.alignment = copy(_HEADER_ALIGNMENT)
        cell.border = copy(_THIN_BORDER)


def _ensure_comment_cell_style(cell) -> None:
    cell.number_format = "General"
    cell.alignment = copy(_TEXT_ALIGNMENT)
    cell.border = copy(_THIN_BORDER)


def _ensure_account_cell_style(cell) -> None:
    cell.number_format = "General"
    cell.alignment = copy(_ACCOUNT_ALIGNMENT)
    cell.border = copy(_THIN_BORDER)


def _ensure_amount_cell_style(cell, number_format: str) -> None:
    cell.number_format = number_format
    cell.alignment = copy(_AMOUNT_ALIGNMENT)
    cell.border = copy(_THIN_BORDER)


def _ensure_date_cell_style(cell) -> None:
    cell.alignment = copy(_DATE_ALIGNMENT)
    cell.border = copy(_THIN_BORDER)


def _normalize_legacy_data_row(
    ws_out: Worksheet, row_idx: int, amount_formats: Dict[int, str], is_table: bool = False
) -> None:
    if is_table:
        _ensure_date_cell_style(ws_out.cell(row_idx, 1))
        _ensure_comment_cell_style(ws_out.cell(row_idx, 5))
        _ensure_amount_cell_style(ws_out.cell(row_idx, 6), amount_formats[6])
        _ensure_amount_cell_style(ws_out.cell(row_idx, 7), amount_formats[7])
        _ensure_account_cell_style(ws_out.cell(row_idx, 8))
        _ensure_comment_cell_style(ws_out.cell(row_idx, 9))
    else:
        _ensure_comment_cell_style(ws_out.cell(row_idx, 4))
        _ensure_amount_cell_style(ws_out.cell(row_idx, 5), amount_formats[5])
        _ensure_amount_cell_style(ws_out.cell(row_idx, 6), amount_formats[6])
        _ensure_account_cell_style(ws_out.cell(row_idx, 7))
        _ensure_comment_cell_style(ws_out.cell(row_idx, 8))


def _ensure_total_row_style(
    ws_out: Worksheet,
    row_idx: int,
    amount_formats: Dict[int, str],
    *,
    force_normalize: bool,
    is_table: bool = False,
) -> None:
    if not force_normalize:
```

```
        return

    headers = RSCH_TABLE_HEADERS if is_table else RSCH_HEADERS
    income_col = 6 if is_table else 5
    expense_col = 7 if is_table else 6

    label = ws_out.cell(row_idx, 1)
    label.fill = copy(_TOTAL_FILL)
    label.font = Font(bold=True)
    label.border = copy(_TOTAL_BORDER)

    non_amount_cols = [c for c in range(2, len(headers) + 1) if c not in (income_col, expense_col)]
    for col_idx in non_amount_cols:
        cell = ws_out.cell(row_idx, col_idx)
        cell.fill = copy(_TOTAL_FILL)
        cell.font = Font(bold=True)
        cell.border = copy(_TOTAL_BORDER)
        cell.number_format = "General"

    for col_idx in (income_col, expense_col):
        cell = ws_out.cell(row_idx, col_idx)
        cell.fill = copy(_TOTAL_FILL)
        cell.font = Font(bold=True)
        cell.border = copy(_TOTAL_BORDER)
        cell.number_format = amount_formats[col_idx]
        cell.alignment = copy(_AMOUNT_ALIGNMENT)

def _update_column_widths(widths: dict[int, int], values: list[Any]) -> None:
    for col_idx, value in enumerate(values, start=1):
        widths[col_idx] = max(widths.get(col_idx, 0), _display_len(value))


def _apply_column_widths(
    ws_out: Worksheet,
    widths: dict[int, int],
    min_width: int = 10,
    max_width: int = 60,
) -> None:
    for col_idx in range(1, ws_out.max_column + 1):
        max_len = widths.get(col_idx, 0)
        if col_idx in (5, 6):
            max_len = max(max_len, 14)

        ws_out.column_dimensions[get_column_letter(col_idx)].width = max(
            min_width,
            min(max_width, max_len + 2),
        )


def _autosize_all_columns(
    ws_out: Worksheet, min_width: int = 10, max_width: int = 60
) -> None:
    for col_idx in range(1, ws_out.max_column + 1):
        col_letter = get_column_letter(col_idx)
        max_len = 0
        for row_idx in range(1, ws_out.max_row + 1):
            max_len = max(max_len, _display_len(ws_out.cell(row_idx, col_idx).value))

        if col_idx in (5, 6):
            max_len = max(max_len, 14)

        ws_out.column_dimensions[col_letter].width = max(
            min_width,
            min(max_width, max_len + 2),
        )


def _apply_amount_highlighting(
    ws_out: Worksheet,
    row_idx: int,
    income: Decimal,
    expense: Decimal,
    is_table: bool = False,
) -> None:
    income_col = 6 if is_table else 5
    expense_col = 7 if is_table else 6
    income_cell = ws_out.cell(row_idx, income_col)
    expense_cell = ws_out.cell(row_idx, expense_col)

    if income != 0:
        income_cell.fill = _INCOME_FILL
        income_cell.font = copy(_INCOME_FONT)
```

```
    if expense != 0:
        expense_cell.fill = _EXPENSE_FILL
        expense_cell.font = copy(_EXPENSE_FONT)


def _apply_warning_row_highlighting(ws_out: Worksheet, row_idx: int, is_table: bool = False) -> None:
    headers = RSCH_TABLE_HEADERS if is_table else RSCH_HEADERS
    for col_idx in range(1, len(headers) + 1):
        ws_out.cell(row_idx, col_idx).fill = copy(_WARNING_FILL)



def _write_rsch_sheet(
    wb_out: Workbook,
    template_ws: Worksheet,
    sheet_name: str,
    rows: List[Dict[str, Any]],
    *,
    first_sheet: bool,
    is_table: bool = False,
) -> None:
    ws_out = wb_out.active if first_sheet else wb_out.create_sheet()
    ws_out.title = sheet_name

    style_layout = _resolve_template_style_layout(template_ws, is_table)
    headers = RSCH_TABLE_HEADERS if is_table else RSCH_HEADERS
    ws_out.append(headers)
    column_widths: dict[int, int] = {}
    _update_column_widths(column_widths, headers)

    for col_idx, _ in enumerate(headers, start=1):
        header_source = template_ws.cell(1, style_layout["header_cols"][col_idx])
        _copy_style(header_source, ws_out.cell(1, col_idx))
        col = get_column_letter(col_idx)
        width_source_col = get_column_letter(style_layout["width_cols"][col_idx])
        src_dim = template_ws.column_dimensions.get(width_source_col)
        if src_dim and src_dim.width:
            ws_out.column_dimensions[col].width = src_dim.width

    ws_out.freeze_panes = template_ws.freeze_panes
    ws_out.sheet_view.zoomScale = template_ws.sheet_view.zoomScale
    ws_out.sheet_view.showGridLines = template_ws.sheet_view.showGridLines
    ws_out.sheet_format.defaultRowHeight = template_ws.sheet_format.defaultRowHeight
    header_height = template_ws.row_dimensions[1].height
    if header_height:
        ws_out.row_dimensions[1].height = header_height

    _ensure_header_style(ws_out, style_layout["legacy_template"], is_table)

    template_data_row_idx = 2 if template_ws.max_row >= 2 else 1
    data_style_sources = {
        col_idx: template_ws.cell(template_data_row_idx, style_layout["data_cols"][col_idx])
        for col_idx in range(1, len(headers) + 1)
    }

    income_col = 6 if is_table else 5
    expense_col = 7 if is_table else 6
    hyperlink_source_col = 9 if is_table else 8

    for row in rows:
        if is_table:
            values = [
                row["date"],
                _excel_safe_text(row["executor"]),
                _excel_safe_text(row["customer"]),
                _excel_safe_text(row["contractor"]),
                _excel_safe_text(row["comment"]),
                row["income"],
                row["expense"],
                _excel_safe_text(row["account"]),
                _excel_safe_text(row["source"]),
            ]
        else:
            values = [
                _excel_safe_text(row["executor"]),
                _excel_safe_text(row["customer"]),
                _excel_safe_text(row["contractor"]),
                _excel_safe_text(row["comment"]),
                row["income"],
                row["expense"],
                _excel_safe_text(row["account"]),
                _excel_safe_text(row["source"]),
            ]
```

```
        _update_column_widths(column_widths, values)
        ws_out.append(values)
        curr_idx = ws_out.max_row
        for col_idx in range(1, len(headers) + 1):
            _copy_style(
                data_style_sources[col_idx],
                ws_out.cell(curr_idx, col_idx),
            )

        if is_table:
            ws_out.cell(curr_idx, 1).number_format = "dd.mm.yyyy"
            ws_out.cell(curr_idx, 1).alignment = copy(_DATE_ALIGNMENT)

        if style_layout["legacy_template"]:
            _normalize_legacy_data_row(
                ws_out, curr_idx, style_layout["detail_amount_formats"], is_table
            )
        if _is_problem_row(row):
            _apply_warning_row_highlighting(ws_out, curr_idx, is_table)
        _apply_amount_highlighting(ws_out, curr_idx, row["income"], row["expense"], is_table)
        apply_cell_hyperlink(
            ws_out.cell(curr_idx, hyperlink_source_col),
            row.get("source_path"),
            row.get("source_sheet"),
            row.get("source_row"),
        )

    _apply_column_widths(ws_out, column_widths)

    total_row_idx = ws_out.max_row + 1
    ws_out.cell(total_row_idx, 1, "ИТОГО:")

    income_letter = get_column_letter(income_col)
    expense_letter = get_column_letter(expense_col)

    if total_row_idx > 2:
        ws_out.cell(total_row_idx, income_col, f"=SUM({income_letter}2:{income_letter}{total_row_idx - 1})")
        ws_out.cell(total_row_idx, expense_col, f"=SUM({expense_letter}2:{expense_letter}{total_row_idx - 1})")
    else:
        ws_out.cell(total_row_idx, income_col, Decimal("0"))
        ws_out.cell(total_row_idx, expense_col, Decimal("0"))

    template_total_row_idx = _find_total_row(template_ws)
    if template_total_row_idx:
        for col_idx in range(1, len(headers) + 1):
            _copy_style(
                template_ws.cell(
                    template_total_row_idx,
                    style_layout["total_cols"][col_idx],
                ),
                ws_out.cell(total_row_idx, col_idx),
            )
    _ensure_total_row_style(
        ws_out,
        total_row_idx,
        style_layout["total_amount_formats"],
        force_normalize=style_layout["legacy_template"],
        is_table=is_table,
    )
    ws_out.auto_filter.ref = (
        f"A1:{get_column_letter(len(headers))}{max(1, total_row_idx - 1)}"
    )


def build_rsch_report(
    source_path: Path | str | Sequence[Path | str],
    output_path: Path,
    *,
    target_date: Optional[date] = None,
    start_date: Optional[date] = None,
    end_date: Optional[date] = None,
    template_path: Optional[Path] = None,
    issues_sink: Optional[List[str]] = None,
    drive_letters: tuple[str, ...] = ("Q", "Z"),
) -> int:
    source_paths = _normalize_source_paths(source_path)
    output_path = Path(output_path)

    # BUG 5 guard: without a date, the sheet-naming branch below would
    # try `None.strftime(...)` (AttributeError) or silently fall back to
    # the template sheet title. Fail fast with a clear message instead.
    if target_date is None and start_date is None:
        raise ValueError("build_rsch_report: требуется target_date или start_date")
```

```
template_to_use = (
    Path(template_path) if template_path else DEFAULT_RSCH_TEMPLATE_PATH
)

if not source_paths:
    raise ValueError("Не выбраны файлы РСЧ")
for src_path in source_paths:
    if not src_path.exists():
        raise FileNotFoundError(f"Не найден файл РСЧ: {src_path}")
if template_to_use.exists():
    wb_template = openpyxl.load_workbook(template_to_use)
elif template_path is not None:
    raise FileNotFoundError(f"Не найден шаблон РСЧ: {template_to_use}")
else:
    warning = (
        f"Шаблон РСЧ не найден: {template_to_use}. "
        "Используется встроенный шаблон."
    )
    logger.warning(warning)
    if issues_sink is not None:
        issues_sink.append(warning)
    wb_template = _create_builtin_template_workbook()

template_ws = wb_template[wb_template.sheetnames[0]]

rows: List[Dict[str, Any]] = []
issues: List[str] = []
try:
    for src_path in source_paths:
        rows.extend(
            _collect_rows_from_workbook(
                src_path,
                target_date,
                issues,
                start_date=start_date,
                end_date=end_date,
                drive_letters=drive_letters,
            )
        )

    if issues_sink is not None:
        issues_sink.extend(issues)

    if issues:
        logger.warning(
            "РСЧ: обнаружены проблемы в листах (%s). Сборка продолжена по валидным листам.",
            len(issues),
        )
        if not rows:
            preview = "; ".join(issues[:3])
            if len(issues) > 3:
                preview += " ..."
            raise RuntimeError(
                "РСЧ: не удалось собрать строки из выбранных файлов. "
                f"Проблемы: {preview}"
            )

    if not rows:
        logger.info(
            "RSC report skipped: no rows matched requested date/range (output=%s, sources=%s)",
            output_path,
            len(source_paths),
        )
        return 0

    use_period = start_date is not None and end_date is not None
    wb_out = Workbook()
    try:
        if use_period and rows:
            rows_by_date: Dict[date, List[Dict[str, Any]]] = {}
            for row in rows:
                rows_by_date.setdefault(row["date"], []).append(row)
            sorted_dates = sorted(rows_by_date)
            # Имена листов всегда с годом (%d.%m.%Y) — совпадает с
            # форматом мастер-отчёта и исключает коллизию при периоде,
            # охватывающем январь двух лет (31.12.2024 + 01.01.2025).
            # Раньше был guard на %d.%m без года + ValueError при конфликте;
            # с годом конфликт невозможен, guard убран.
            for idx, row_date in enumerate(sorted_dates, start=1):
                _write_rsch_sheet(
                    wb_out,
                    template_ws,
                    row_date.strftime("%d.%m.%Y"),
```

```
                    rows_by_date[row_date],
                    first_sheet=idx == 1,
                )
            # Сводная таблица за весь период на последнем листе
            sorted_all_rows = sorted(rows, key=lambda r: r["date"])
            _write_rsch_sheet(
                wb_out,
                template_ws,
                "Период",
                sorted_all_rows,
                first_sheet=False,
                is_table=True,
            )
        else:
            sheet_date = target_date or start_date
            sheet_name = (
                sheet_date.strftime("%d.%m.%Y") if sheet_date else template_ws.title
            )
            _write_rsch_sheet(
                wb_out,
                template_ws,
                sheet_name,
                rows,
                first_sheet=True,
            )

        output_path.parent.mkdir(parents=True, exist_ok=True)
        backup_existing_excel_output(output_path)
        atomic_save_workbook(wb_out, output_path)
    finally:
        # Guarantee the output workbook handle is released even if
        # writing aborts mid-sheet, otherwise a retry on Windows
        # fails with a locked-file error.
        try:
            wb_out.close()
        except Exception:
            pass

    logger.info(
        "RSC report generated: %s (rows=%s, sources=%s)",
        output_path,
        len(rows),
        len(source_paths),
    )
    return len(rows)
finally:
    # The template workbook is only needed during the build; close
    # it once the sheets are written so we never leak the handle
    # when an exception aborts the build early.
    try:
        wb_template.close()
    except Exception:
        pass
```

APP\core\infra\rsch_collect.py

433 lines • 14,645 bytes

```
from __future__ import annotations

import logging
from copy import copy
from datetime import date
from decimal import Decimal
from pathlib import Path
from typing import Any, Dict, List, Optional

import openpyxl
from openpyxl.worksheet.worksheet import Worksheet
from python_calamine import CalamineWorkbook

from core.infra.utils import get_local_copy

from ._format import (_normalize_text, _to_decimal, _as_date, _clean_executor_name,
    _cell_text_from_row, _pick_text, _format_source_label, _is_rsch_sheet_title)

logger = logging.getLogger(__name__)


def _copy_style(src_cell, dst_cell) -> None:
    dst_cell.font = copy(src_cell.font)
    dst_cell.fill = copy(src_cell.fill)
    dst_cell.border = copy(src_cell.border)
    dst_cell.alignment = copy(src_cell.alignment)
    dst_cell.number_format = src_cell.number_format
    dst_cell.protection = copy(src_cell.protection)


def _find_header_row(sheet_data: List[tuple]) -> Optional[int]:
    max_scan = min(len(sheet_data), 30)
    for row_idx_0 in range(max_scan):
        row_values = sheet_data[row_idx_0]
        values = []
        for col_idx_0 in range(32):
            val = row_values[col_idx_0] if col_idx_0 < len(row_values) else None
            values.append(_normalize_text(val))
        has_date = any(v == "дата" for v in values)
        has_income = any("приход" in v and "рсч" in v for v in values)
        has_expense = any("расход" in v and "рсч" in v for v in values)
        if has_date and has_income and has_expense:
            return row_idx_0 + 1
    return None


def _detect_columns(sheet_data: List[tuple], header_row_idx: int, sheet_title: str) -> Dict[str, Optional[int]]:
    row_values = sheet_data[header_row_idx - 1]
    max_scan = min(len(row_values), 128)
    header_values = [
        _normalize_text(row_values[col_idx_0])
        for col_idx_0 in range(max_scan)
    ]
    if len(header_values) < 128:
        header_values.extend([""] * (128 - len(header_values)))

    project_cols = [i + 1 for i, v in enumerate(header_values) if "проект" in v]
    contractor_cols = [i + 1 for i, v in enumerate(header_values) if "контрагент" in v]

    def first_index(predicate) -> Optional[int]:
        for i, v in enumerate(header_values, start=1):
            if predicate(v):
                return i
        return None

    col_map = {
        "date": first_index(lambda v: v == "дата"),
        "income": first_index(lambda v: "приход" in v and "рсч" in v),
        "expense": first_index(lambda v: "расход" in v and "рсч" in v),
        "account": first_index(lambda v: "р/с" in v),
        "project_left": project_cols[0] if project_cols else None,
        "project_right": project_cols[1]
        if len(project_cols) > 1
        else (project_cols[0] if project_cols else None),
        "contractor_left": contractor_cols[0] if contractor_cols else None,
        "contractor_right": contractor_cols[1]
        if len(contractor_cols) > 1
        else (contractor_cols[0] if contractor_cols else None),
    }

    required = (
        "date",
```

```
        "income",
        "expense",
        "project_left",
        "contractor_left",
    )
    missing = [key for key in required if not col_map.get(key)]
    if missing:
        raise ValueError(
            f"Header columns not found in sheet '{sheet_title}': {', '.join(missing)}"
        )
    return col_map



def _build_row(
    row_date: date,
    executor: str,
    account: str,
    source: str,
    source_path: Path,
    source_sheet: str,
    source_row: int,
    customer_primary: str,
    customer_secondary: str,
    contractor_primary: str,
    contractor_secondary: str,
    comment: str,
    income: Decimal,
    expense: Decimal,
) -> Dict[str, Any]:
    return {
        "date": row_date,
        "contractor": _pick_text(contractor_primary, contractor_secondary),
        "customer": _pick_text(customer_primary, customer_secondary),
        "executor": executor,
        "comment": comment,
        "income": income,
        "expense": expense,
        "account": account,
        "source": source,
        "source_path": str(source_path),
        "source_sheet": source_sheet,
        "source_row": source_row,
    }



def _collect_rows_from_sheet(
    ws: Worksheet,
    target_date: Optional[date],
    source_name: str,
    source_path: Path,
    start_date: Optional[date] = None,
    end_date: Optional[date] = None,
) -> List[Dict[str, Any]]:
    sheet_data = list(ws.iter_rows(values_only=True))
    if not sheet_data:
        return []

    header_row_idx = _find_header_row(sheet_data)
    if header_row_idx is None:
        return []
    col_map = _detect_columns(sheet_data, header_row_idx, ws.title)
    executor = _clean_executor_name(ws.title)
    rows: List[Dict[str, Any]] = []
    current_row_date: Optional[date] = None

    for row_idx in range(header_row_idx + 1, len(sheet_data) + 1):
        row_values = sheet_data[row_idx - 1]

        date_col = col_map["date"]
        raw_date = row_values[date_col - 1] if date_col and date_col <= len(row_values) else None

        explicit_row_date = _as_date(raw_date)
        if explicit_row_date is not None:
            current_row_date = explicit_row_date
        elif raw_date not in (None, "") and str(raw_date).strip():
            current_row_date = None
        if current_row_date is None:
            continue
        row_date = current_row_date
        if target_date and row_date != target_date:
            continue
```

```
if start_date and end_date and not (start_date <= row_date <= end_date):
    continue

income_col = col_map["income"]
raw_income = row_values[income_col - 1] if income_col and income_col <= len(row_values) else None
income = _to_decimal(raw_income)

expense_col = col_map["expense"]
raw_expense = row_values[expense_col - 1] if expense_col and expense_col <= len(row_values) else None
expense = _to_decimal(raw_expense)

account = _cell_text_from_row(row_values, col_map["account"])
expense_comment = _cell_text_from_row(row_values, 10)
source = _format_source_label(source_name, ws.title, row_idx)

customer_left = _cell_text_from_row(row_values, col_map["project_left"])
customer_right = _cell_text_from_row(row_values, col_map["project_right"])
contractor_left = _cell_text_from_row(row_values, col_map["contractor_left"])
contractor_right = _cell_text_from_row(row_values, col_map["contractor_right"])
has_business_text = any(
    (
        expense_comment,
        customer_left,
        customer_right,
        contractor_left,
        contractor_right,
    )
)
if (
    income == 0
    and expense == 0
    and not has_business_text
):
    continue

if income != 0 and expense != 0:
    rows.append(
        _build_row(
            row_date,
            executor,
            account,
            source,
            source_path,
            ws.title,
            row_idx,
            customer_left,
            customer_right,
            contractor_left,
            contractor_right,
            expense_comment,
            income,
            expense,
        )
    )
elif income != 0:
    rows.append(
        _build_row(
            row_date,
            executor,
            account,
            source,
            source_path,
            ws.title,
            row_idx,
            customer_left,
            customer_right,
            contractor_left,
            contractor_right,
            "",
            income,
            Decimal("0"),
        )
    )
elif expense != 0:
    rows.append(
        _build_row(
            row_date,
            executor,
            account,
            source,
            source_path,
            ws.title,
            row_idx,
```

```
                    customer_right,
                    customer_left,
                    contractor_right,
                    contractor_left,
                    expense_comment,
                    Decimal("0"),
                    expense,
                )
            )
        else:
            rows.append(
                _build_row(
                    row_date,
                    executor,
                    account,
                    source,
                    source_path,
                    ws.title,
                    row_idx,
                    customer_left,
                    customer_right,
                    contractor_left,
                    contractor_right,
                    expense_comment,
                    Decimal("0"),
                    Decimal("0"),
                )
            )

    return rows



def _normalize_source_paths(source_paths: Any) -> List[Path]:
    if isinstance(source_paths, (str, Path)):
        raw_paths = [source_paths]
    else:
        raw_paths = list(source_paths or [])

    normalized: List[Path] = []
    seen: set[str] = set()
    for raw_path in raw_paths:
        if raw_path is None:
            continue
        path = Path(raw_path)
        key = str(path)
        if not key or key in seen:
            continue
        seen.add(key)
        normalized.append(path)
    return normalized


class _CalamineWorksheetAdapter:
    def __init__(self, title: str, sheet: Any) -> None:
        self.title = title
        self._sheet = sheet

    def iter_rows(self, values_only: bool = True):
        return self._sheet.iter_rows()


def _collect_rows_from_openpyxl_workbook(
    target_path: str,
    source_path: Path,
    target_date: Optional[date],
    issues_sink: Optional[List[str]] = None,
    start_date: Optional[date] = None,
    end_date: Optional[date] = None,
) -> List[Dict[str, Any]]:
    wb_source = None
    try:
        wb_source = openpyxl.load_workbook(target_path, read_only=True, data_only=True)
        rows: List[Dict[str, Any]] = []
        for ws in wb_source.worksheets:
            if ws.title.startswith("_"):
                continue
            if not _is_rsch_sheet_title(ws.title):
                continue
            try:
                rows.extend(
                    _collect_rows_from_sheet(
                        ws,
```

```
                        target_date,
                        source_path.name,
                        source_path,
                        start_date=start_date,
                        end_date=end_date,
                    )
                )
            except Exception as e:
                issue = (
                    f"Лист '{ws.title}' в файле '{source_path.name}' пропущен: {e}"
                )
                logger.warning(issue)
                if issues_sink is not None:
                    issues_sink.append(issue)
        return rows
    finally:
        if wb_source is not None:
            wb_source.close()


def _collect_rows_from_calamine_workbook(
    target_path: str,
    source_path: Path,
    target_date: Optional[date],
    issues_sink: Optional[List[str]] = None,
    start_date: Optional[date] = None,
    end_date: Optional[date] = None,
) -> List[Dict[str, Any]]:
    wb_source = None
    try:
        wb_source = CalamineWorkbook.from_path(target_path)
        rows: List[Dict[str, Any]] = []
        for sheet_name in wb_source.sheet_names:
            if sheet_name.startswith("_"):
                continue
            if not _is_rsch_sheet_title(sheet_name):
                continue
            try:
                sheet = wb_source.get_sheet_by_name(sheet_name)
                rows.extend(
                    _collect_rows_from_sheet(
                        _CalamineWorksheetAdapter(sheet_name, sheet),
                        target_date,
                        source_path.name,
                        source_path,
                        start_date=start_date,
                        end_date=end_date,
                    )
                )
            except Exception as e:
                issue = f"Лист '{sheet_name}' в файле '{source_path.name}' пропущен: {e}"
                logger.warning(issue)
                if issues_sink is not None:
                    issues_sink.append(issue)
        return rows
    finally:
        # Guard: if CalamineWorkbook.from_path raised before assignment,
        # `wb_source` is still None — calling .close() here would raise
        # UnboundLocalError and mask the original cause.
        if wb_source is not None:
            wb_source.close()


def _collect_rows_from_workbook(
    source_path: Path,
    target_date: Optional[date],
    issues_sink: Optional[List[str]] = None,
    start_date: Optional[date] = None,
    end_date: Optional[date] = None,
    drive_letters: tuple[str, ...] = ("Q", "Z"),
) -> List[Dict[str, Any]]:
    target_path, temp_file = get_local_copy(str(source_path), drive_letters)
    try:
        try:
            return _collect_rows_from_calamine_workbook(
                target_path,
                source_path,
                target_date,
                issues_sink=issues_sink,
                start_date=start_date,
                end_date=end_date,
            )
        except Exception as e:
```

```
            logger.warning(
                "Calamine failed for RSC source '%s', falling back to openpyxl: %s",
                source_path,
                e,
            )
            return _collect_rows_from_openpyxl_workbook(
                target_path,
                source_path,
                target_date,
                issues_sink=issues_sink,
                start_date=start_date,
                end_date=end_date,
            )
    finally:
        if temp_file:
            Path(temp_file).unlink(missing_ok=True)
```

APP\core\infra\rsch_format.py

225 lines • 6,794 bytes

```
from __future__ import annotations

import logging
from datetime import date, datetime
from decimal import Decimal, InvalidOperation
from pathlib import Path
from typing import Any, Dict, Optional

from openpyxl.styles import Alignment, Border, Font, PatternFill, Side


logger = logging.getLogger(__name__)

RSCH_HEADERS = [
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
    "р/с",
    "Источник",
]
RSCH_TEMPLATE_HEADERS = RSCH_HEADERS

RSCH_TABLE_HEADERS = [
    "Дата",
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
    "р/с",
    "Источник",
]

_REPO_ROOT = Path(__file__).resolve().parents[3]
DEFAULT_RSCH_TEMPLATE_PATH = _REPO_ROOT / "РСЧ" / "шаблон отчета.xlsx"

_INCOME_FILL = PatternFill(
    start_color="FFC6EFCE", end_color="FFC6EFCE", fill_type="solid"
)
_EXPENSE_FILL = PatternFill(
    start_color="FFFFC7CE", end_color="FFFFC7CE", fill_type="solid"
)
_WARNING_FILL = PatternFill(
    start_color="FFFFFF00", end_color="FFFFFF00", fill_type="solid"
)
_INCOME_FONT = Font(color="FF006100", bold=True)
_EXPENSE_FONT = Font(color="FF9C0006", bold=True)
_HEADER_FILL = PatternFill(start_color="FF1F4E79", end_color="FF1F4E79", fill_type="solid")
_TOTAL_FILL = PatternFill(start_color="FFD9E1F2", end_color="FFD9E1F2", fill_type="solid")
_HEADER_FONT = Font(color="FFFFFFFF", bold=True)
_THIN_SIDE = Side(style="thin", color="FF000000")
_MEDIUM_SIDE = Side(style="medium", color="FF000000")
_THIN_BORDER = Border(
    left=_THIN_SIDE,
    right=_THIN_SIDE,
    top=_THIN_SIDE,
    bottom=_THIN_SIDE,
)
_TOTAL_BORDER = Border(
    left=_MEDIUM_SIDE,
    right=_MEDIUM_SIDE,
    top=_MEDIUM_SIDE,
    bottom=_MEDIUM_SIDE,
)
_HEADER_ALIGNMENT = Alignment(horizontal="center", vertical="center", wrap_text=True)
_TEXT_ALIGNMENT = Alignment(horizontal="left", vertical="top", wrap_text=True)
_ACCOUNT_ALIGNMENT = Alignment(horizontal="left", vertical="center")
_AMOUNT_ALIGNMENT = Alignment(horizontal="right", vertical="center")
_DATE_ALIGNMENT = Alignment(horizontal="left", vertical="center")
_DEFAULT_AMOUNT_NUMBER_FORMAT = "#,##0.00"
_SPECIAL_CUSTOMER_MARKERS = (
    "% по депозиту",
    "% по выплате",
    "комиссия",
    "налог",
    "взносы",
)
_NO_ACCOUNT_MARKERS = (
```

```
    "факторинг",
    "факторнинг",
    "вф",
    "вафт фокс",
    "гетворк",
)


def _normalize_text(value: Any) -> str:
    if value is None:
        return ""
    return " ".join(str(value).replace("\xa0", " ").lower().split())


def _is_rsch_sheet_title(sheet_title: str) -> bool:
    normalized = _normalize_text(sheet_title)
    return "рсч" in normalized and "шаблон" not in normalized


def _to_decimal(value: Any) -> Decimal:
    if value is None or value == "":
        return Decimal("0")
    if isinstance(value, bool):
        return Decimal("1") if value else Decimal("0")
    if isinstance(value, Decimal):
        return value
    if isinstance(value, (int, float)):
        return Decimal(str(value))
    text = str(value).strip().replace(" ", "").replace("\xa0", "").replace(",", ".")
    if not text:
        return Decimal("0")
    try:
        return Decimal(text)
    except (InvalidOperation, ValueError):
        raise ValueError(f"Некорректное числовое значение в РСЧ: {value}") from None


def _as_date(value: Any) -> Optional[date]:
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    if isinstance(value, str):
        stripped = value.strip()
        if not stripped:
            return None
        for fmt in ("%d.%m.%Y", "%d.%m.%y", "%Y-%m-%d"):
            try:
                return datetime.strptime(stripped, fmt).date()
            except ValueError:
                continue
    return None


def _clean_executor_name(sheet_name: str) -> str:
    tokens = [
        token for token in str(sheet_name).strip().split() if token.lower() != "рсч"
    ]
    cleaned = " ".join(tokens).strip()
    return (cleaned or str(sheet_name).strip()).upper()



def _cell_text_from_row(row_values: tuple, col_idx: Optional[int]) -> str:
    if not col_idx:
        return ""
    value = row_values[col_idx - 1] if col_idx <= len(row_values) else None
    return "" if value is None else str(value).strip()


def _pick_text(primary: str, secondary: str) -> str:
    return primary if primary else secondary


def _is_special_customer_label(customer: str) -> bool:
    normalized = _normalize_text(customer)
    return any(marker in normalized for marker in _SPECIAL_CUSTOMER_MARKERS)


def _is_account_optional_row(row: Dict[str, Any]) -> bool:
    searchable = " ".join(
        _normalize_text(row.get(key))
        for key in ("executor", "customer", "contractor", "comment")
```

```
    )
    return any(marker in searchable for marker in _NO_ACCOUNT_MARKERS)


def _is_problem_row(row: Dict[str, Any]) -> bool:
    income = row["income"]
    expense = row["expense"]
    contractor = str(row["contractor"] or "").strip()
    customer = str(row["customer"] or "").strip()
    executor = str(row["executor"] or "").strip()
    account = str(row["account"] or "").strip()

    if income != 0 and expense != 0:
        return True

    is_getwork_customer = "гетворк" in _normalize_text(customer)
    if (
        not contractor
        and customer
        and not _is_special_customer_label(customer)
        and not is_getwork_customer
    ):
        return True

    if not customer or not executor:
        return True

    if not account and not _is_account_optional_row(row):
        return True

    return income == 0 and expense == 0



def _format_source_label(source_name: str, sheet_name: str, row_idx: int) -> str:
    return f"{source_name} | {sheet_name} | строка {row_idx}"



def _display_len(value: Any) -> int:
    if value is None:
        return 0
    if isinstance(value, datetime):
        return 10
    if isinstance(value, Decimal):
        return len(f"{value:,.2f}")
    if isinstance(value, (int, float)):
        return len(f"{value:,.2f}")
    return len(str(value))


def _excel_safe_text(value: Any) -> Any:
    if isinstance(value, str):
        stripped = value.lstrip()
        if stripped and stripped[0] in ("=", "+", "-", "@"):
            return "'" + value
    return value
```

APP\core\infra\rsch_template.py

371 lines • 13,448 bytes

```
from __future__ import annotations

import logging
from copy import copy
from pathlib import Path
from typing import Any, Dict, List, Optional, Sequence

from openpyxl import Workbook
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter
from openpyxl.worksheet.worksheet import Worksheet


from ._format import _normalize_text
from ._collect import _copy_style

logger = logging.getLogger(__name__)

RSCH_HEADERS = [
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
    "р/с",
    "Источник",
]
RSCH_TEMPLATE_HEADERS = RSCH_HEADERS

RSCH_TABLE_HEADERS = [
    "Дата",
    "ИСПОЛНИТЕЛЬ",
    "ЗАКАЗЧИК",
    "Контрагент",
    "Комментарий",
    "Приход",
    "Расход",
    "р/с",
    "Источник",
]

_REPO_ROOT = Path(__file__).resolve().parents[3]
DEFAULT_RSCH_TEMPLATE_PATH = _REPO_ROOT / "РСЧ" / "шаблон отчета.xlsx"

_INCOME_FILL = PatternFill(
    start_color="FFC6EFCE", end_color="FFC6EFCE", fill_type="solid"
)
_EXPENSE_FILL = PatternFill(
    start_color="FFFFC7CE", end_color="FFFFC7CE", fill_type="solid"
)
_WARNING_FILL = PatternFill(
    start_color="FFFFFF00", end_color="FFFFFF00", fill_type="solid"
)
_INCOME_FONT = Font(color="FF006100", bold=True)
_EXPENSE_FONT = Font(color="FF9C0006", bold=True)
_HEADER_FILL = PatternFill(start_color="FF1F4E79", end_color="FF1F4E79", fill_type="solid")
_TOTAL_FILL = PatternFill(start_color="FFD9E1F2", end_color="FFD9E1F2", fill_type="solid")
_HEADER_FONT = Font(color="FFFFFFFF", bold=True)
_THIN_SIDE = Side(style="thin", color="FF000000")
_MEDIUM_SIDE = Side(style="medium", color="FF000000")
_THIN_BORDER = Border(
    left=_THIN_SIDE,
    right=_THIN_SIDE,
    top=_THIN_SIDE,
    bottom=_THIN_SIDE,
)
_TOTAL_BORDER = Border(
    left=_MEDIUM_SIDE,
    right=_MEDIUM_SIDE,
    top=_MEDIUM_SIDE,
    bottom=_MEDIUM_SIDE,
)
_HEADER_ALIGNMENT = Alignment(horizontal="center", vertical="center", wrap_text=True)
_TEXT_ALIGNMENT = Alignment(horizontal="left", vertical="top", wrap_text=True)
_ACCOUNT_ALIGNMENT = Alignment(horizontal="left", vertical="center")
_AMOUNT_ALIGNMENT = Alignment(horizontal="right", vertical="center")
_DATE_ALIGNMENT = Alignment(horizontal="left", vertical="center")
_DEFAULT_AMOUNT_NUMBER_FORMAT = "#,##0.00"
_SPECIAL_CUSTOMER_MARKERS = (
    "% по депозиту",
    "% по выплате",
```

```
    "комиссия",
    "налог",
    "взносы",
)
_NO_ACCOUNT_MARKERS = (
    "факторинг",
    "факторнинг",
    "вф",
    "вафт фокс",
    "гетворк",
)



def _find_total_row(template_ws: Worksheet) -> Optional[int]:
    for row_idx in range(1, template_ws.max_row + 1):
        label = _normalize_text(template_ws.cell(row_idx, 1).value)
        if label.startswith("итого"):
            return row_idx
    return None


def _prepare_output_sheet(
    template_ws: Worksheet, sheet_name: str
) -> tuple[Workbook, Worksheet, Dict[str, Any]]:
    wb = Workbook()
    ws = wb.active
    ws.title = sheet_name

    style_layout = _resolve_template_style_layout(template_ws)

    ws.append(RSCH_HEADERS)

    for col_idx, _ in enumerate(RSCH_HEADERS, start=1):
        header_source = template_ws.cell(1, style_layout["header_cols"][col_idx])
        _copy_style(header_source, ws.cell(1, col_idx))
        col = get_column_letter(col_idx)
        source_col = get_column_letter(style_layout["width_cols"][col_idx])
        src_dim = template_ws.column_dimensions.get(source_col)
        if src_dim and src_dim.width:
            ws.column_dimensions[col].width = src_dim.width

    ws.freeze_panes = template_ws.freeze_panes
    ws.sheet_view.zoomScale = template_ws.sheet_view.zoomScale
    ws.sheet_view.showGridLines = template_ws.sheet_view.showGridLines
    ws.sheet_format.defaultRowHeight = template_ws.sheet_format.defaultRowHeight
    header_height = template_ws.row_dimensions[1].height
    if header_height:
        ws.row_dimensions[1].height = header_height

    _ensure_header_style(ws, style_layout["legacy_template"])
    return wb, ws, style_layout


def _template_header_values(template_ws: Worksheet, max_scan: int) -> List[str]:
    return [
        _normalize_text(template_ws.cell(1, col_idx).value)
        for col_idx in range(1, max_scan + 1)
    ]


def _first_matching_column(header_values: Sequence[str], predicate) -> Optional[int]:
    for idx, value in enumerate(header_values, start=1):
        if predicate(value):
            return idx
    return None


def _resolve_template_semantic_columns(header_values: Sequence[str]) -> Dict[str, int]:
    fallback_col = _first_matching_column(header_values, lambda v: bool(v)) or 1
    date_col = _first_matching_column(header_values, lambda v: v == "дата") or 1
    executor_col = _first_matching_column(header_values, lambda v: "исполнитель" in v)
    customer_col = _first_matching_column(header_values, lambda v: "заказчик" in v)
    contractor_col = _first_matching_column(header_values, lambda v: "контрагент" in v)
    comment_col = _first_matching_column(header_values, lambda v: "комментар" in v)
    income_col = _first_matching_column(header_values, lambda v: "приход" in v)
    expense_col = _first_matching_column(header_values, lambda v: "расход" in v)
    account_col = _first_matching_column(header_values, lambda v: "р/с" in v)
    text_col = contractor_col or executor_col or customer_col or fallback_col
    comment_style_col = comment_col or contractor_col or customer_col or text_col

    return {
        "fallback": fallback_col,
```

```
        "date": date_col,
        "executor": executor_col or text_col,
        "customer": customer_col or text_col,
        "contractor": contractor_col or text_col,
        "comment_style": comment_style_col,
        "income": income_col or fallback_col,
        "expense": expense_col or fallback_col,
        "account": account_col or text_col,
        "text": text_col,
    }


def _find_blank_total_col(
    template_ws: Worksheet, max_scan: int, total_row_idx: Optional[int]
) -> int:
    default_col = 2 if template_ws.max_column >= 2 else 1
    if total_row_idx is None:
        return default_col
    for col_idx in range(1, max_scan + 1):
        if template_ws.cell(total_row_idx, col_idx).value in (None, ""):
            return col_idx
    return default_col


def _build_header_style_columns(cols: Dict[str, int], is_table: bool = False) -> Dict[int, int]:
    if is_table:
        return {
            1: cols["date"],
            2: cols["executor"],
            3: cols["customer"],
            4: cols["contractor"],
            5: cols["comment_style"],
            6: cols["income"],
            7: cols["expense"],
            8: cols["account"],
            9: cols["comment_style"],
        }
    return {
        1: cols["executor"],
        2: cols["customer"],
        3: cols["contractor"],
        4: cols["comment_style"],
        5: cols["income"],
        6: cols["expense"],
        7: cols["account"],
        8: cols["comment_style"],
    }


def _build_data_style_columns(cols: Dict[str, int], blank_total_col: int, is_table: bool = False) -> Dict[int, int]:
    if is_table:
        return {
            1: cols["date"],
            2: cols["executor"],
            3: cols["customer"],
            4: cols["contractor"],
            5: cols["comment_style"],
            6: cols["income"] or blank_total_col,
            7: cols["expense"] or blank_total_col,
            8: cols["account"],
            9: cols["comment_style"],
        }
    return {
        1: cols["executor"],
        2: cols["customer"],
        3: cols["contractor"],
        4: cols["comment_style"],
        5: cols["income"] or blank_total_col,
        6: cols["expense"] or blank_total_col,
        7: cols["account"],
        8: cols["comment_style"],
    }


def _build_total_style_columns(cols: Dict[str, int], blank_total_col: int, is_table: bool = False) -> Dict[int, int]:
    if is_table:
        return {
            1: 1,
            2: blank_total_col,
            3: blank_total_col,
            4: blank_total_col,
            5: blank_total_col,
            6: cols["income"] or blank_total_col,
```

```
            7: cols["expense"] or blank_total_col,
            8: blank_total_col,
            9: blank_total_col,
        }
    return {
        1: 1,
        2: blank_total_col,
        3: blank_total_col,
        4: blank_total_col,
        5: cols["income"] or blank_total_col,
        6: cols["expense"] or blank_total_col,
        7: blank_total_col,
        8: blank_total_col,
    }


def _is_legacy_template(header_values: Sequence[str]) -> bool:
    expected = [_normalize_text(value) for value in RSCH_TEMPLATE_HEADERS]
    expected_with_date = ["дата", *expected]
    prefix = list(header_values)
    return not (
        prefix[: len(expected)] == expected
        or prefix[: len(expected_with_date)] == expected_with_date
    )


def _resolve_template_style_layout(template_ws: Worksheet, is_table: bool = False) -> Dict[str, Any]:
    max_scan = max(8, min(template_ws.max_column, 32))
    header_values = _template_header_values(template_ws, max_scan)
    cols = _resolve_template_semantic_columns(header_values)
    total_row_idx = _find_total_row(template_ws)
    blank_total_col = _find_blank_total_col(template_ws, max_scan, total_row_idx)
    data_cols = _build_data_style_columns(cols, blank_total_col, is_table)
    total_cols = _build_total_style_columns(cols, blank_total_col, is_table)

    income_col_idx = 6 if is_table else 5
    expense_col_idx = 7 if is_table else 6

    return {
        "header_cols": _build_header_style_columns(cols, is_table),
        "width_cols": data_cols,
        "data_cols": data_cols,
        "total_cols": total_cols,
        "legacy_template": _is_legacy_template(header_values),
        "detail_amount_formats": {
            income_col_idx: _resolve_amount_number_format(template_ws, data_cols[income_col_idx], 2, total_row_idx, 1),
            expense_col_idx: _resolve_amount_number_format(template_ws, data_cols[expense_col_idx], 2, total_row_idx, 1),
        },
        "total_amount_formats": {
            income_col_idx: _resolve_amount_number_format(template_ws, total_cols[income_col_idx], total_row_idx, 2, 1),
            expense_col_idx: _resolve_amount_number_format(template_ws, total_cols[expense_col_idx], total_row_idx, 2, 1),
        },
    }


def _resolve_amount_number_format(
    template_ws: Worksheet, col_idx: int, *row_indexes: Optional[int]
) -> str:
    for row_idx in row_indexes:
        if row_idx is None or row_idx > template_ws.max_row:
            continue
        fmt = template_ws.cell(row_idx, col_idx).number_format
        if fmt and fmt != "General":
            return fmt
    return _DEFAULT_AMOUNT_NUMBER_FORMAT



def _create_builtin_template_workbook() -> Workbook:
    wb = Workbook()
    ws = wb.active
    ws.title = "РСЧ"
    ws.append(RSCH_HEADERS)
    ws.append(["", "", "", "", 0, 0, "", ""])
    ws.append(["ИТОГО:", None, None, None, "=SUM(E2:E2)", "=SUM(F2:F2)", None, None])

    for col_idx in range(1, len(RSCH_HEADERS) + 1):
        cell = ws.cell(1, col_idx)
        cell.fill = copy(_HEADER_FILL)
        cell.font = copy(_HEADER_FONT)
        cell.alignment = copy(_HEADER_ALIGNMENT)
        cell.border = copy(_THIN_BORDER)
```

```
    for ref in ("A2", "B2", "C2", "D2", "H2"):
        ws[ref].alignment = copy(_TEXT_ALIGNMENT)
        ws[ref].border = copy(_THIN_BORDER)
        ws[ref].number_format = "General"
    ws["G2"].alignment = copy(_ACCOUNT_ALIGNMENT)
    ws["G2"].border = copy(_THIN_BORDER)
    ws["G2"].number_format = "General"
    for ref in ("E2", "F2"):
        ws[ref].number_format = _DEFAULT_AMOUNT_NUMBER_FORMAT
        ws[ref].alignment = copy(_AMOUNT_ALIGNMENT)
        ws[ref].border = copy(_THIN_BORDER)

    for col_idx in range(1, len(RSCH_HEADERS) + 1):
        cell = ws.cell(3, col_idx)
        cell.fill = copy(_TOTAL_FILL)
        cell.font = Font(bold=True)
        cell.border = copy(_TOTAL_BORDER)
    for ref in ("B3", "C3", "D3", "G3", "H3"):
        ws[ref].number_format = "General"
    ws["E3"].number_format = _DEFAULT_AMOUNT_NUMBER_FORMAT
    ws["F3"].number_format = _DEFAULT_AMOUNT_NUMBER_FORMAT
    ws["E3"].alignment = copy(_AMOUNT_ALIGNMENT)
    ws["F3"].alignment = copy(_AMOUNT_ALIGNMENT)
    ws.freeze_panes = "A2"
    return wb


def _ensure_header_style(ws: Worksheet, force_normalize: bool, is_table: bool = False) -> None:
    if not force_normalize:
        return
    headers = RSCH_TABLE_HEADERS if is_table else RSCH_HEADERS
    for col_idx in range(1, len(headers) + 1):
        cell = ws.cell(1, col_idx)
        cell.fill = copy(_HEADER_FILL)
        cell.font = copy(_HEADER_FONT)
        cell.alignment = copy(_HEADER_ALIGNMENT)
        cell.border = copy(_THIN_BORDER)
```

APP\core\infra\session_repo.py

290 lines • 10,918 bytes

```
import json
import logging
from typing import List, Dict, Any, Optional
from pathlib import Path
from core.domain import IPersistence
from core.infra.db_connection import SQLiteConnectionManager, DecimalEncoder
from core.infra.db_connection import sanitize_data as sanitize_data
from core.infra.source_file_repo import SourceFileRepository
from core.infra.staging_repo import DBWriterThread as DBWriterThread
from core.infra.staging_repo import StagingDataRepository


def _log_wal_size(db_path: Path, logger: logging.Logger) -> None:
    wal_path = Path(str(db_path) + "-wal")
    if wal_path.exists():
        size_mb = wal_path.stat().st_size / (1024 * 1024)
        if size_mb > 1:
            logger.info("WAL file size: %.1f MB", size_mb)
        if size_mb > 100:
            logger.warning(
                "WAL file is %.1f MB — exceeds 100 MB threshold. "
                "Consider increasing wal_autocheckpoint or running "
                "manual checkpoints more frequently.",
                size_mb,
            )


class SessionRepository(IPersistence):
    def __init__(self, db_path: str, schema_path: str, config: Dict[str, Any]):
        self.connection_manager = SQLiteConnectionManager(Path(db_path), config)
        self.connection_manager.init_db(Path(schema_path))

        self.source_file_repo = SourceFileRepository(self.connection_manager)
        self.staging_repo = StagingDataRepository(self.connection_manager)
        self.logger = logging.getLogger(__name__)

        self.db_path = Path(db_path)
        self.writer_thread = self.staging_repo.writer_thread
        self.write_queue = self.staging_repo.write_queue

    def init_db(self) -> None:
        pass

    def _get_connection(self):
        return self.connection_manager.get_connection()

    def _execute_with_retry(self, func, *args, **kwargs):
        return self.connection_manager.execute_with_retry(func, *args, **kwargs)

    def save_session(self, session_id: str, metadata: Dict[str, Any]) -> None:
        def _save():
            conn = self._get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.execute(
                    "INSERT INTO sessions (session_id, metadata) VALUES (?, ?)",
                    (
                        session_id,
                        json.dumps(metadata, ensure_ascii=False, cls=DecimalEncoder),
                    ),
                )
                conn.commit()
            except Exception:
                conn.rollback()
                raise
            finally:
                conn.close()

        self._execute_with_retry(_save)

    def save_template_error(self, session_id: str, error_info: Dict[str, Any]) -> None:
        def _save():
            conn = self._get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.execute(
                    """INSERT INTO template_errors
                       (session_id, file_path, sheet_name, row_index, error_type, details, raw_data)
                       VALUES (?, ?, ?, ?, ?, ?, ?)""",
                    (
                        session_id,
                        error_info.get("file_path"),
                        error_info.get("sheet_name"),
```

```
                    error_info.get("row_index"),
                    error_info.get("error_type"),
                    error_info.get("details"),
                    json.dumps(
                        error_info.get("raw_data"),
                        ensure_ascii=False,
                        cls=DecimalEncoder,
                    ),
                ),
            )
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()

    self._execute_with_retry(_save)

def save_template_errors_batch(
    self, session_id: str, errors: List[Dict[str, Any]]
) -> None:
    if not errors:
        return

    def _save():
        conn = self._get_connection()
        try:
            conn.execute("BEGIN IMMEDIATE")
            conn.executemany(
                """INSERT INTO template_errors
                   (session_id, file_path, sheet_name, row_index, error_type, details, raw_data)
                   VALUES (?, ?, ?, ?, ?, ?, ?)""",
                [
                    (
                        session_id,
                        error_info.get("file_path"),
                        error_info.get("sheet_name"),
                        error_info.get("row_index"),
                        error_info.get("error_type"),
                        error_info.get("details"),
                        json.dumps(
                            error_info.get("raw_data"),
                            ensure_ascii=False,
                            cls=DecimalEncoder,
                        ),
                    )
                    for error_info in errors
                ],
            )
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()

    self._execute_with_retry(_save)

def get_template_errors(self, session_id: str) -> List[Dict[str, Any]]:
    conn = self._get_connection()
    try:
        return [
            dict(r)
            for r in conn.execute(
                "SELECT * FROM template_errors WHERE session_id = ? ORDER BY file_path, row_index",
                (session_id,),
            )
        ]
    finally:
        conn.close()

def get_session_status(self, session_id: str) -> Dict[str, Any]:
    conn = self._get_connection()
    try:
        row = conn.execute(
            "SELECT * FROM sessions WHERE session_id = ?", (session_id,)
        ).fetchone()
        if not row:
            return {}
        stats = conn.execute(
            "SELECT status, COUNT(*) as cnt FROM source_files WHERE session_id = ? GROUP BY status",
            (session_id,),
```

```
            ).fetchall()
            return {
                "session_id": row["session_id"],
                "status": row["status"],
                "created_at": row["created_at"],
                "file_stats": {s["status"]: s["cnt"] for s in stats},
            }
        finally:
            conn.close()

    def update_session_status(self, session_id: str, status: str) -> None:
        def _update():
            conn = self._get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.execute(
                    "UPDATE sessions SET status = ? WHERE session_id = ?",
                    (status, session_id),
                )
                conn.commit()
            except Exception:
                conn.rollback()
                raise
            finally:
                conn.close()

        self._execute_with_retry(_update)

    def update_file_status(
        self, file_id: int, status: str, error_message: Optional[str] = None
    ) -> None:
        def _update():
            conn = self._get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.execute(
                    "UPDATE source_files SET status = ?, error_message = ? WHERE id = ?",
                    (status, error_message, file_id),
                )
                conn.commit()
            except Exception:
                conn.rollback()
                raise
            finally:
                conn.close()

        self._execute_with_retry(_update)

    def save_file_metadata(
        self, session_id: str, files: List[Dict[str, Any]]
    ) -> List[int]:
        return self.source_file_repo.save_file_metadata(session_id, files)

    def save_staging_data(self, file_id: int, rows: List[Dict[str, Any]]) -> None:
        self.staging_repo.save_staging_data(file_id, rows)

    def get_consolidated_data(self, session_id: str) -> List[Dict[str, Any]]:
        return self.staging_repo.get_consolidated_data(session_id)

    def wait_for_writes(self) -> None:
        self.source_file_repo.wait_for_writes()
        self.staging_repo.wait_for_writes()

    def close(self) -> None:
        self.logger.info("Closing database manager.")
        # Log WAL file size before checkpoint
        _log_wal_size(self.db_path, self.logger)
        # Run a WAL checkpoint to flush the WAL file back into the main
        # database — keeps the WAL from growing unboundedly between runs.
        try:
            conn = self._get_connection()
            try:
                conn.execute("PRAGMA wal_checkpoint(TRUNCATE)")
            finally:
                conn.close()
        except Exception as e:
            self.logger.warning("WAL checkpoint failed (non-fatal): %s", e)
        self.source_file_repo.close()
        self.staging_repo.close()
        try:
            unclean_file = self.db_path.with_name(self.db_path.name + ".unclean")
            if unclean_file.exists():
                unclean_file.unlink()
```

```
        except Exception as e:
            self.logger.warning("Failed to remove unclean shutdown flag: %s", e)

    def get_file_cache(self, file_path: str) -> Optional[Dict[str, Any]]:
        return self.source_file_repo.get_file_cache(file_path)

    def save_file_cache(
        self,
        file_path: str,
        mtime: float,
        size: int,
        dates: List[str],
        fingerprint: str | None = None,
    ) -> None:
        self.source_file_repo.save_file_cache(
            file_path, mtime, size, dates, fingerprint=fingerprint
        )

    def get_row_cache(
        self, file_path: str, mtime: float, size: int, date_key: str
    ) -> Optional[List[Dict[str, Any]]]:
        return self.source_file_repo.get_row_cache(file_path, mtime, size, date_key)

    def save_row_cache(
        self,
        file_path: str,
        mtime: float,
        size: int,
        date_key: str,
        rows: List[Dict[str, Any]],
    ) -> None:
        self.source_file_repo.save_row_cache(file_path, mtime, size, date_key, rows)

    def get_discovery_cache(self, roots_key: str) -> Optional[List[Dict[str, Any]]]:
        return self.source_file_repo.get_discovery_cache(roots_key)

    def save_discovery_cache(self, roots_key: str, files: List[Dict[str, Any]]) -> None:
        self.source_file_repo.save_discovery_cache(roots_key, files)


SQLiteManager = SessionRepository
```

APP\core\infra\source_file_repo.py

263 lines • 9,681 bytes

```
import json
import logging
import threading
import queue
from typing import List, Dict, Any, Optional
from core.infra.db_connection import SQLiteConnectionManager


class SourceFileRepository:
    def __init__(self, connection_manager: SQLiteConnectionManager):
        self.connection_manager = connection_manager
        self.write_queue = queue.Queue()
        self._write_error: Optional[Exception] = None
        self._write_error_lock = threading.Lock()
        self.writer_thread = threading.Thread(
            target=self._writer_loop, name="SourceFileWriter", daemon=True
        )
        self.writer_thread.start()
        self._logger = logging.getLogger(__name__)

    def _record_write_error(self, error: Exception) -> None:
        """Keep the first write error so callers can surface it.

        Mirrors StagingDataRepository: the writer thread must not raise
        (it would kill the loop and deadlock wait_for_writes), but it
        also must not silently swallow failures — record the first one
        so the controller can report that cache writes are degraded.
        """
        with self._write_error_lock:
            if self._write_error is None:
                self._write_error = error

    @property
    def write_error(self) -> Optional[Exception]:
        with self._write_error_lock:
            return self._write_error

    def _writer_loop(self):
        while True:
            try:
                item = self.write_queue.get(timeout=0.1)
                if item is None:
                    self.write_queue.task_done()
                    break
                fn, args = item
                self.connection_manager.execute_with_retry(fn, *args)
                self.write_queue.task_done()
            except queue.Empty:
                continue
            except Exception as e:
                self._logger.error(f"SourceFileWriter error: {e}", exc_info=True)
                self._record_write_error(e)
                self.write_queue.task_done()

    def save_file_metadata(
        self, session_id: str, files: List[Dict[str, Any]]
    ) -> List[int]:
        def _save():
            ids = []
            conn = self.connection_manager.get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                for f in files:
                    file_hash = f.get("hash")
                    if file_hash is None:
                        file_hash = f.get("fp")

                    inode = f.get("inode")
                    if inode is None:
                        inode = f.get("id")

                    existing = conn.execute(
                        "SELECT id FROM source_files WHERE session_id = ? AND file_path = ?",
                        (session_id, f["path"]),
                    ).fetchone()
                    if existing is not None:
                        ids.append(existing[0])
                        continue

                    cur = conn.execute(
                        "INSERT INTO source_files (session_id, file_path, file_hash, inode, status) VALUES (?, ?, ?, ?, ?)",
                        (
                            session_id,
```

```
                        f["path"],
                        file_hash,
                        inode,
                        f.get("status", "pending"),
                    ),
                )
                ids.append(cur.lastrowid)
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()
        return ids

    return self.connection_manager.execute_with_retry(_save)

def get_file_cache(self, file_path: str) -> Optional[Dict[str, Any]]:
    conn = self.connection_manager.get_connection()
    try:
        row = conn.execute(
            "SELECT mtime, size, dates_json FROM file_cache WHERE file_path = ?",
            (file_path,),
        ).fetchone()
        if row:
            raw_payload = json.loads(row["dates_json"])
            if isinstance(raw_payload, dict):
                dates = raw_payload.get("dates", [])
                cache_version = int(raw_payload.get("cache_version", 1) or 1)
                fingerprint = raw_payload.get("fingerprint")
            else:
                dates = raw_payload
                cache_version = 1
                fingerprint = None
            if not isinstance(dates, list):
                dates = []
            return {
                "mtime": row["mtime"],
                "size": row["size"],
                "dates": dates,
                "cache_version": cache_version,
                "fingerprint": fingerprint if isinstance(fingerprint, str) else None,
            }
        return None
    finally:
        conn.close()

def save_file_cache(
    self,
    file_path: str,
    mtime: float,
    size: int,
    dates: List[str],
    fingerprint: str | None = None,
) -> None:
    def _save():
        conn = self.connection_manager.get_connection()
        try:
            conn.execute("BEGIN IMMEDIATE")
            conn.execute(
                "INSERT OR REPLACE INTO file_cache (file_path, mtime, size, dates_json, last_updated) VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP)",
                (
                    file_path,
                    mtime,
                    size,
                    json.dumps(
                        {
                            "cache_version": 4,
                            "dates": dates,
                            "fingerprint": fingerprint,
                        },
                        ensure_ascii=False,
                    ),
                ),
            )
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()

    self.write_queue.put((_save, ()))
```

```
    def get_row_cache(
        self, file_path: str, mtime: float, size: int, date_key: str
    ) -> Optional[List[Dict[str, Any]]]:
        conn = self.connection_manager.get_connection()
        try:
            row = conn.execute(
                "SELECT rows_json FROM row_cache WHERE file_path = ? AND mtime = ? AND size = ? AND date_key = ?",
                (file_path, mtime, size, date_key),
            ).fetchone()
            if row:
                return json.loads(row["rows_json"])
            return None
        finally:
            conn.close()

    def save_row_cache(
        self,
        file_path: str,
        mtime: float,
        size: int,
        date_key: str,
        rows: List[Dict[str, Any]],
    ) -> None:
        def _save():
            conn = self.connection_manager.get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.execute(
                    "INSERT OR REPLACE INTO row_cache (file_path, mtime, size, date_key, rows_json, row_count, created_at) VALUES (?, ?, ?, ?, ?, ?
                    (
                        file_path,
                        mtime,
                        size,
                        date_key,
                        json.dumps(rows, ensure_ascii=False, default=str),
                        len(rows),
                    ),
                )
                conn.commit()
            except Exception:
                conn.rollback()
                raise
            finally:
                conn.close()

        self.write_queue.put((_save, ()))

    def get_discovery_cache(self, roots_key: str) -> Optional[List[Dict[str, Any]]]:
        conn = self.connection_manager.get_connection()
        try:
            row = conn.execute(
                "SELECT files_json, scanned_at FROM discovery_cache WHERE roots_key = ?",
                (roots_key,),
            ).fetchone()
            if row:
                import time as _time
                scanned_at = float(row["scanned_at"])
                ttl = 12 * 3600  # 12 hours default
                if _time.time() - scanned_at < ttl:
                    return json.loads(row["files_json"])
            return None
        finally:
            conn.close()

    def save_discovery_cache(
        self, roots_key: str, files: List[Dict[str, Any]]
    ) -> None:
        def _save():
            import time as _time
            conn = self.connection_manager.get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.execute(
                    "INSERT OR REPLACE INTO discovery_cache (roots_key, scanned_at, files_json) VALUES (?, ?, ?)",
                    (
                        roots_key,
                        _time.time(),
                        json.dumps(files, ensure_ascii=False, default=str),
                    ),
                )
                conn.commit()
            except Exception:
                conn.rollback()
```

```
                raise
            finally:
                conn.close()

        self.write_queue.put((_save, ()))

    def close(self):
        self.wait_for_writes()
        self.write_queue.put(None)
        self.writer_thread.join()

    def wait_for_writes(self):
        self.write_queue.join()
```

APP\core\infra\staging_repo.py

219 lines • 8,295 bytes

```
import json
import logging
import decimal
import threading
import queue
from typing import List, Dict, Any
from pathlib import Path
from core.infra.db_connection import SQLiteConnectionManager, sanitize_data


class DBWriterThread(threading.Thread):
    """Background thread that batches staging data writes to SQLite."""

    BATCH_SIZE = 500
    BATCH_TIMEOUT = 0.5

    def __init__(
        self,
        connection_manager: SQLiteConnectionManager,
        q: queue.Queue,
        error_callback=None,
    ):
        super().__init__(name="DBWriterThread", daemon=True)
        self.connection_manager = connection_manager
        self.q = q
        self.running = True
        self.logger = logging.getLogger(__name__)
        self.error_callback = error_callback

    _WRITE_RETRIES = 1  # One retry on transient failure before giving up

    def run(self):
        while self.running or not self.q.empty():
            batch = []
            try:
                item = self.q.get(timeout=self.BATCH_TIMEOUT)
                if item is None:
                    self.q.task_done()
                    break
                batch.append(item)
                while len(batch) < self.BATCH_SIZE:
                    try:
                        next_item = self.q.get_nowait()
                        if next_item is None:
                            self.q.task_done()
                            # Process remaining batch before exiting
                            if batch:
                                self._process_batch_with_retry(batch)
                                for _ in batch:
                                    self.q.task_done()
                            return
                        batch.append(next_item)
                    except queue.Empty:
                        break
                self._process_batch_with_retry(batch)
                for _ in batch:
                    self.q.task_done()
            except queue.Empty:
                continue
            except Exception as e:
                self.logger.error(f"Error in DBWriterThread: {e}", exc_info=True)
                if self.error_callback is not None:
                    self.error_callback(e)
                # Mark batch items as done to avoid blocking join()
                for _ in batch:
                    try:
                        self.q.task_done()
                    except ValueError:
                        pass

    def _process_batch_with_retry(self, batch):
        """Try to process a batch, retrying once on transient errors.

        Previously a single _process_batch failure silently dropped the
        entire batch — rows were lost even though task_done() was called,
        making wait_for_writes() return as if everything succeeded.
        The retry gives transient SQLite lock errors a second chance before
        the data is declared lost.
        """
        try:
            self._process_batch(batch)
            return
        except Exception as first_err:
```

```
            self.logger.warning(
                "Batch write failed (attempt 1/%d): %s. Retrying...",
                self._WRITE_RETRIES + 1, first_err,
            )
        # Retry once
        try:
            self._process_batch(batch)
            self.logger.info("Batch write succeeded on retry.")
            return
        except Exception as retry_err:
            self.logger.critical(
                "Batch write failed on retry too. %d rows LOST: %s",
                sum(len(rows) for _, rows in batch), retry_err,
                exc_info=True,
            )
            if self.error_callback is not None:
                self.error_callback(retry_err)
            raise

    def _process_batch(self, batch):
        sql = "INSERT INTO staging_data (file_id, row_index, raw_content_json, date_val, amount, employee_name, source_type) VALUES (?, ?, ?, ?, ?,
        all_data = []
        for file_id, rows in batch:
            for r in rows:
                amt_raw = r.get("amount", 0)
                try:
                    amt_val = decimal.Decimal(str(amt_raw))
                except (decimal.InvalidOperation, TypeError, ValueError):
                    amt_val = decimal.Decimal("0.00")
                    try:
                        from core.domain.data_loss_log import record_loss
                        record_loss(
                            source=f"staging_repo:file_id={file_id}",
                            kind="numeric_coercion",
                            reason=f"amount field could not be parsed: {amt_raw!r}",
                            row_index=r.get("_row_index"),
                            field="amount",
                            original_value=amt_raw,
                        )
                    except Exception:
                        pass

                all_data.append(
                    (
                        file_id,
                        r.get("_row_index"),
                        json.dumps(sanitize_data(r), ensure_ascii=False),
                        r.get("date"),
                        amt_val,
                        r.get("employee_name"),
                        r.get("source_type"),
                    )
                )

        if not all_data:
            return

        def _save():
            conn = self.connection_manager.get_connection()
            try:
                conn.execute("BEGIN IMMEDIATE")
                conn.executemany(sql, all_data)
                conn.commit()
            except Exception:
                conn.rollback()
                raise
            finally:
                conn.close()

        self.connection_manager.execute_with_retry(_save)

    def stop(self):
        self.running = False
        self.q.put(None)


class StagingDataRepository:
    def __init__(self, connection_manager: SQLiteConnectionManager):
        self.connection_manager = connection_manager
        self.write_queue = queue.Queue()
        self._write_error = None
        self._write_error_lock = threading.Lock()
        self.writer_thread = DBWriterThread(
```

```
            self.connection_manager,
            self.write_queue,
            error_callback=self._record_write_error,
        )
        self.writer_thread.start()

    def _record_write_error(self, error: Exception) -> None:
        with self._write_error_lock:
            if self._write_error is None:
                self._write_error = error

    def save_staging_data(self, file_id: int, rows: List[Dict[str, Any]]) -> None:
        if not rows:
            return
        self.write_queue.put((file_id, rows))

    def get_consolidated_data(self, session_id: str) -> List[Dict[str, Any]]:
        sql = """
            SELECT staging_data.raw_content_json, staging_data.employee_name,
                   staging_data.date_val, staging_data.row_index, source_files.file_path
            FROM staging_data
            JOIN source_files ON staging_data.file_id = source_files.id
            WHERE source_files.session_id = ?
            ORDER BY staging_data.employee_name, staging_data.date_val
        """
        results = []
        conn = self.connection_manager.get_connection()
        try:
            for row in conn.execute(sql, (session_id,)):
                data = json.loads(row["raw_content_json"])
                data.update(
                    {
                        "employee_name": row["employee_name"],
                        "date": row["date_val"],
                        "_source_info": f"{Path(row['file_path']).name} : Строка {row['row_index']}",
                    }
                )
                results.append(data)
        finally:
            conn.close()
        return results

    def close(self):
        self.writer_thread.stop()
        self.writer_thread.join()

    def wait_for_writes(self):
        self.write_queue.join()
        with self._write_error_lock:
            if self._write_error is not None:
                err = self._write_error
                self._write_error = None
                raise err
```

APP\core\infra\utils.py

265 lines • 9,022 bytes

```
import os
import subprocess
import platform
import hashlib
import logging
import shutil
import tempfile
import threading
import time
from types import SimpleNamespace
from pathlib import Path


def open_path(path: str):
    if platform.system() == "Windows":
        os.startfile(path)
    elif platform.system() == "Darwin":
        subprocess.run(["open", path], check=False)
    else:
        subprocess.run(["xdg-open", path], check=False)


def atomic_save_workbook(wb, output_path: Path) -> None:
    """Save an openpyxl workbook to output_path atomically.

    Writes to a sibling temp file then os.replace's it over the target, so a
    crash or window-close mid-save never leaves a truncated/half-written
    workbook. The backup_existing_excel_output call (done by the caller)
    protects the previous version; this guards the live file.

    Mirrors the proven pattern in export_writer._save_workbook_inner, but as
    a free function so rsch/ and reconciliation/ can share one implementation.
    """
    output_path = Path(output_path)
    tmp = str(output_path) + ".tmp"
    # Long Windows paths need the \\?\ prefix.
    if os.name == "nt" and len(tmp) > 250:
        tmp = "\\\\?\\" + tmp
        target = "\\\\?\\" + str(output_path)
    else:
        target = str(output_path)
    try:
        wb.save(tmp)
        os.replace(tmp, target)
    except Exception:
        # Clean up the partial temp file so a retry does not collide.
        try:
            if os.path.exists(tmp):
                os.remove(tmp)
        except OSError:
            pass
        raise


def is_file_writable(path: str) -> bool:
    p = Path(path)
    if not p.exists():
        return True
    try:
        with open(str(p), "r+b"):
            pass
        return True
    except (IOError, PermissionError, OSError):
        return False


_CACHE_DIR = Path(tempfile.gettempdir()) / "master_report_cache"
_CACHE_CLEANUP_LOCK = threading.Lock()
_CACHE_CLEANUP_DONE = False
_CACHE_COPY_LOCK = threading.Lock()  # threading.Lock is sufficient for ThreadPoolExecutor; multiprocessing.Lock was overkill and could cause pickl
_CACHE_MIN_DELETE_AGE_SECONDS = 30 * 60
_DIGEST_CHUNK_SIZE = 1024 * 1024


def _is_configured_drive_path(path: str, drive_letters: tuple) -> bool:
    if len(path) < 2 or path[1] != ":":
        return False

    configured_drives = {
        str(letter).strip().upper().rstrip(":\\/")
        for letter in drive_letters
        if str(letter).strip()
    }
```

```
    return path[0].upper() in configured_drives


def _is_network_path(path: str, drive_letters: tuple) -> bool:
    return path.startswith("\\\\") or _is_configured_drive_path(path, drive_letters)


def file_content_fingerprint(path: str | Path) -> str | None:
    digest = hashlib.sha256()
    try:
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(_DIGEST_CHUNK_SIZE), b""):
                digest.update(chunk)
    except OSError:
        return None
    return digest.hexdigest()


def _cache_copy_is_fresh(local_path: Path, remote_path: str, remote_stat) -> bool:
    try:
        cached_stat = os.stat(local_path)
    except OSError:
        return False

    remote_mtime_ns = getattr(
        remote_stat, "st_mtime_ns", int(float(remote_stat.st_mtime) * 1_000_000_000)
    )
    cached_mtime_ns = getattr(
        cached_stat, "st_mtime_ns", int(float(cached_stat.st_mtime) * 1_000_000_000)
    )
    if cached_stat.st_size != remote_stat.st_size or cached_mtime_ns < remote_mtime_ns:
        return False
    return True


def cleanup_local_copy_cache(
    max_age_days: int = 14,
    max_total_mb: int = 512,
    min_delete_age_seconds: int = _CACHE_MIN_DELETE_AGE_SECONDS,
) -> None:
    """Remove old local network-file cache entries without touching fresh files."""
    try:
        if not _CACHE_DIR.exists():
            return

        now = time.time()
        max_age_seconds = max(0, max_age_days) * 24 * 60 * 60
        max_total_bytes = max(0, max_total_mb) * 1024 * 1024
        entries: list[tuple[float, int, Path]] = []

        for entry in _CACHE_DIR.iterdir():
            try:
                if entry.is_symlink() or not entry.is_file():
                    continue
                stat = entry.stat()
            except OSError:
                continue

            age = now - stat.st_mtime
            if age < min_delete_age_seconds:
                entries.append((stat.st_mtime, stat.st_size, entry))
                continue
            if max_age_seconds and age > max_age_seconds:
                entry.unlink(missing_ok=True)
                continue
            entries.append((stat.st_mtime, stat.st_size, entry))

        if max_total_bytes <= 0:
            return

        total_size = sum(size for _mtime, size, _entry in entries)
        for mtime, size, entry in sorted(entries, key=lambda item: item[0]):
            if total_size <= max_total_bytes:
                break
            if now - mtime < min_delete_age_seconds:
                continue
            try:
                entry.unlink(missing_ok=True)
                total_size -= size
            except OSError:
                logging.debug("Could not remove cache file %s", entry, exc_info=True)
    except OSError:
        logging.debug("Could not cleanup local copy cache %s", _CACHE_DIR, exc_info=True)
```

```
def _maybe_cleanup_local_copy_cache() -> None:
    global _CACHE_CLEANUP_DONE
    if _CACHE_CLEANUP_DONE:
        return
    with _CACHE_CLEANUP_LOCK:
        if _CACHE_CLEANUP_DONE:
            return
        cleanup_local_copy_cache()
        _CACHE_CLEANUP_DONE = True


def get_local_copy(
    network_path: str,
    drive_letters: tuple = ("Q", "Z"),
    remote_mtime: float | None = None,
    remote_size: int | None = None,
) -> tuple:
    """Get a local cached copy of a network file.

    Cache is invalidated when mtime or size changes on the remote file.
    Returns (local_path, temp_file_to_cleanup_or_None).
    """
    if not network_path:
        return network_path, None
    is_network = _is_network_path(network_path, drive_letters)
    if not is_network:
        return network_path, None

    if remote_mtime is not None and remote_size is not None:
        try:
            stat = SimpleNamespace(st_mtime=float(remote_mtime), st_size=int(remote_size))
            if stat.st_size < 0:
                raise ValueError("remote_size must be non-negative")
        except (TypeError, ValueError):
            try:
                stat = os.stat(network_path)
            except OSError:
                stat = SimpleNamespace(st_mtime=0, st_size=0)
    else:
        try:
            stat = os.stat(network_path)
        except OSError:
            stat = SimpleNamespace(st_mtime=0, st_size=0)


    stat_mtime_ns = getattr(stat, "st_mtime_ns", int(float(stat.st_mtime) * 1_000_000_000))
    key = hashlib.sha256(
        f"{network_path}_{stat_mtime_ns}_{stat.st_size}".encode()
    ).hexdigest()[:32]
    suffix = Path(network_path).suffix
    local_path = _CACHE_DIR / f"{key}{suffix}"

    with _CACHE_COPY_LOCK:
        if local_path.exists():
            if _cache_copy_is_fresh(local_path, network_path, stat):
                return str(local_path), None
            try:
                os.remove(local_path)
            except OSError:
                pass

        _maybe_cleanup_local_copy_cache()
        _CACHE_DIR.mkdir(parents=True, exist_ok=True)
        try:
            shutil.copy2(network_path, str(local_path))
            return str(local_path), None
        except Exception:
            fd, temp_file = tempfile.mkstemp(suffix=suffix)
            os.close(fd)
            try:
                shutil.copy2(network_path, temp_file)
                return temp_file, temp_file
            except Exception as exc:
                raise IOError(f"Network copy failed for {network_path} — no local fallback available") from exc


def path_exists_with_timeout(path: str | Path, timeout: float = 1.0) -> bool:
    """Check if a path exists without blocking long on slow or offline network paths."""
    p = Path(path)
    try:
        drive = p.drive
        if drive:
            import os
```

```
        if hasattr(os, "listdrives"):
            available_drives = {d.upper().rstrip(":\\") for d in os.listdrives()}
            if drive.upper().rstrip(":\\") not in available_drives:
                return False
except Exception:
    pass

import concurrent.futures
with concurrent.futures.ThreadPoolExecutor(max_workers=1) as executor:
    future = executor.submit(p.exists)
    try:
        return future.result(timeout=timeout)
    except (concurrent.futures.TimeoutError, Exception):
        return False
```

APP\core\infra\utils_logging.py

94 lines • 3,456 bytes

```
"""
Module: utils_logging
Description: Part of the Master-Report v5.5 architecture.
This module provides specialized functionality for the financial consolidation system.
Maintained for high modularity, testability, and structural integrity.
"""
import logging
import queue
import hashlib
import re
from pathlib import Path, PureWindowsPath

try:
    import tkinter as tk
except ModuleNotFoundError:  # pragma: no cover - headless environments
    tk = None


WINDOWS_USER_RE = re.compile(r"C:\\Users\\[^\\]+", re.IGNORECASE)
SMB_RE = re.compile(r"\\\\[^\\]+\\[^\\]+")


def scrub_path(value: str) -> str:
    value = WINDOWS_USER_RE.sub(r"C:\\Users\\<user>", str(value))
    return SMB_RE.sub(r"\\\\<server>\<share>", value)


def safe_file_label(path: str | Path) -> str:
    raw = str(path)
    name = PureWindowsPath(raw).name or Path(raw).name or "<file>"
    digest = hashlib.sha256(raw.encode("utf-8", errors="replace")).hexdigest()[:10]
    return f"{name}#{digest}"

class TextHandler(logging.Handler):
    def __init__(self, text_widget):
        super().__init__()
        self.text_widget = text_widget
        self.msg_queue = queue.Queue(maxsize=10000)

    def emit(self, record):
        msg = self.format(record)
        try:
            self.msg_queue.put_nowait(msg)
        except queue.Full:
            try:
                self.msg_queue.get_nowait()
                self.msg_queue.put_nowait(msg)
            except queue.Empty:
                pass

    def _process_queue(self):
        try:
            self.text_widget.configure(state='normal')
            count = 0
            while not self.msg_queue.empty() and count < 50:
                try:
                    msg = self.msg_queue.get_nowait()
                    self.text_widget.insert(tk.END, msg + '\n')
                    count += 1
                except queue.Empty:
                    break
            self.text_widget.configure(state='disabled')
            self.text_widget.yview(tk.END)
        finally:
            # ALWAYS reschedule the next poll. Previously this was gated on
            # `if not self.msg_queue.empty()`, so once the queue drained the
            # polling loop died and any subsequent records emitted from
            # worker threads sat in the queue forever, never reaching the
            # GUI. The 100 ms delay throttles the loop — it is NOT a tight
            # spin.
            try:
                self.text_widget.after(100, self._process_queue)
            except Exception:
                # Widget may have been destroyed during shutdown; ignore.
                pass

def setup_gui_logging(log_widget):
    if tk is None:
        raise RuntimeError("tkinter is not available in this environment")
    handler = TextHandler(log_widget)
    handler.setFormatter(logging.Formatter('%(asctime)s - %(message)s', datefmt='%H:%M:%S'))
    root = logging.getLogger()
    # Prevent duplicate handlers
```

```
for h in root.handlers[:]:
    if isinstance(h, TextHandler):
        root.removeHandler(h)
root.addHandler(handler)
root.setLevel(logging.INFO)
# Start the queue polling loop from the GUI thread
if handler.msg_queue.empty():
    # Initial empty queue — still schedule first poll so future messages are picked up
    pass
log_widget.after(100, handler._process_queue)
return handler
```

APP\core\services__init__.py

5 lines • 243 bytes

```
"""Application services — business orchestration extracted from AppController.

Each service owns one functional area (RSЧ, reconciliation, consolidation)
while AppController delegates, preserving its public API for the GUI layer.
"""
```

APP\core\services\reconciliation_service.py

73 lines • 2,918 bytes

```
"""Reconciliation report service.

Extracted from AppController. ``run_reconciliation_report`` builds a
discrepancy report from already-generated master and RSЧ files — it does
not depend on the consolidation pipeline, so it is safe to isolate here.

``run_auto_reconciliation_day`` stays in AppController because it calls
``_run_auto_master_report`` → ``_run_consolidation_blocking``, which
couples it to the consolidation core.
"""

import logging
import threading
from pathlib import Path
from typing import Callable, Dict, Optional

from core.infra.reconciliation import build_combined_reconciliation_report


class ReconciliationService:
    """Builds discrepancy reports comparing master reports against RSЧ."""

    def __init__(
        self,
        start_thread_fn: Callable[[Callable], None],
        get_drive_letters_fn: Callable[[], tuple[str, ...]],
        alias_repo=None,
    ) -> None:
        self._start_thread = start_thread_fn
        self._get_drive_letters = get_drive_letters_fn
        self._alias_repo = alias_repo

    def run_reconciliation_report(
        self,
        master_paths: Dict[str, Path],
        rsch_path: Path,
        output_path: Path,
        selected_date,
        on_complete: Callable[[bool, str, Optional[Path]], None],
        cancel_event: Optional[threading.Event] = None,
    ) -> None:
        """Build a reconciliation report from already selected report and RSC files."""

        def task() -> None:
            try:
                from core.app_controller import _is_cancelled, _cancel_message

                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, output_path)
                    return
                target_date = (
                    selected_date.date() if hasattr(selected_date, "date") else selected_date
                )
                counts = build_combined_reconciliation_report(
                    {label: Path(path) for label, path in master_paths.items()},
                    Path(rsch_path),
                    Path(output_path),
                    target_date=target_date,
                    drive_letters=self._get_drive_letters(),
                    alias_repo=self._alias_repo,
                )
                msg = (
                    "Отчет расхождений собран: "
                    f"Общий {counts.get('Общий', 0)}, "
                    f"Борис {counts.get('Борис', 0)}, "
                    f"Иван {counts.get('Иван', 0)} строк."
                )
                on_complete(True, msg, Path(output_path))
            except Exception as e:
                logging.exception("Reconciliation report error: %s", e)
                on_complete(False, f"Ошибка сверки отчетов: {e}", None)

        self._start_thread(task)
```

APP\core\services\rsch_service.py

299 lines • 12,310 bytes

```
"""РЗЧ (bank account control) report service.

Extracted from AppController to reduce its size. All four run_rsch_*
variants live here; AppController delegates, keeping its public API
unchanged so the GUI layer needs no changes.
"""

import logging
import threading
from datetime import timedelta
from pathlib import Path
from typing import Callable, List, Optional, Sequence

from core.infra.rsch import build_rsch_report
from core.app.message_formatting import build_issue_preview


class RschService:
    """Builds РЗЧ control workbooks from source Excel files.

    Each method starts a background thread (via the injected
    ``start_thread_fn``) so the GUI stays responsive. Cancellation is
    cooperative: pass a ``threading.Event`` and the worker checks it
    at every checkpoint.
    """

    def __init__(
        self,
        start_thread_fn: Callable[[Callable], None],
        get_drive_letters_fn: Callable[[], tuple[str, ...]],
    ) -> None:
        self._start_thread = start_thread_fn
        self._get_drive_letters = get_drive_letters_fn

    def run_rsch_report(
        self,
        source_paths: Path | Sequence[Path],
        output_path: Path,
        selected_date,
        on_complete: Callable[[bool, str, Optional[Path]], None],
        cancel_event: Optional[threading.Event] = None,
    ) -> None:
        """Build an RSC control report from selected workbooks."""

        def task() -> None:
            try:
                target_date = selected_date
                if hasattr(selected_date, "date"):
                    target_date = selected_date.date()

                from core.app_controller import _is_cancelled, _cancel_message

                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, output_path)
                    return

                if isinstance(source_paths, (str, Path)):
                    normalized_sources = [Path(source_paths)]
                else:
                    normalized_sources = [Path(p) for p in source_paths]

                issues: List[str] = []

                written_rows = build_rsch_report(
                    normalized_sources,
                    output_path,
                    target_date=target_date,
                    issues_sink=issues,
                    drive_letters=self._get_drive_letters(),
                )
                if written_rows == 0:
                    raise ValueError("РСЧ: нет строк за выбранную дату")

                if issues:
                    msg = (
                        f"РСЧ отчет собран: {written_rows} строк из {len(normalized_sources)} файлов. "
                        f"Есть проблемы с листами: {build_issue_preview(issues)}"
                    )
                else:
                    msg = (
                        f"РСЧ отчет собран: {written_rows} строк из {len(normalized_sources)} файлов"
                    )
```

```
            on_complete(
                True,
                msg,
                output_path,
            )
        except Exception as e:
            logging.exception("RSC build error: %s", e)
            on_complete(False, f"Ошибка сборки РСЧ: {e}", None)

    self._start_thread(task)

def run_rsch_report_period(
    self,
    source_paths: Path | Sequence[Path],
    output_path: Path,
    start_date,
    end_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
    """Build one RSC control workbook with one sheet per row date in range."""

    def task() -> None:
        try:
            start = start_date.date() if hasattr(start_date, "date") else start_date
            end = end_date.date() if hasattr(end_date, "date") else end_date

            from core.app_controller import _is_cancelled, _cancel_message

            if _is_cancelled(cancel_event):
                _cancel_message(on_complete, output_path)
                return

            if isinstance(source_paths, (str, Path)):
                normalized_sources = [Path(source_paths)]
            else:
                normalized_sources = [Path(p) for p in source_paths]

            issues: List[str] = []
            written_rows = build_rsch_report(
                normalized_sources,
                output_path,
                start_date=start,
                end_date=end,
                issues_sink=issues,
                drive_letters=self._get_drive_letters(),
            )
            if written_rows == 0:
                raise ValueError("РСЧ: нет строк за выбранный период")

            if issues:
                msg = (
                    f"РСЧ отчет за период собран: {written_rows} строк из {len(normalized_sources)} файлов. "
                    f"Есть проблемы с листами: {build_issue_preview(issues)}"
                )
            else:
                msg = (
                    f"РСЧ отчет за период собран: {written_rows} строк из "
                    f"{len(normalized_sources)} файлов"
                )

            on_complete(True, msg, output_path)
        except Exception as e:
            logging.exception("RSC period build error: %s", e)
            on_complete(False, f"Ошибка сборки РСЧ за период: {e}", None)

    self._start_thread(task)

def run_rsch_report_period_batch(
    self,
    source_paths: Path | Sequence[Path],
    output_dir: Path,
    start_date,
    end_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
    """Build separate RSC workbooks for each date in the selected range."""

    def task() -> None:
        try:
            start = start_date.date() if hasattr(start_date, "date") else start_date
            end = end_date.date() if hasattr(end_date, "date") else end_date
```

```
            from core.app_controller import _is_cancelled, _cancel_message

            if _is_cancelled(cancel_event):
                _cancel_message(on_complete, output_dir)
                return

            if isinstance(source_paths, (str, Path)):
                normalized_sources = [Path(source_paths)]
            else:
                normalized_sources = [Path(p) for p in source_paths]

            output_dir_path = Path(output_dir)
            output_dir_path.mkdir(parents=True, exist_ok=True)

            current = start
            total_rows = 0
            files_written = 0
            issues: List[str] = []
            while current <= end:
                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, output_dir_path)
                    return
                output_path = output_dir_path / f"РСЧ_отчет_{current.strftime('%d.%m.%Y')}.xlsx"
                written_rows = build_rsch_report(
                    normalized_sources,
                    output_path,
                    target_date=current,
                    issues_sink=issues,
                    drive_letters=self._get_drive_letters(),
                )
                if written_rows:
                    total_rows += written_rows
                    files_written += 1
                current += timedelta(days=1)

            if files_written == 0:
                raise ValueError("РСЧ: нет строк за выбранный период")

            if issues:
                msg = (
                    f"РСЧ отчеты за период собраны: {files_written} файлов, {total_rows} строк. "
                    f"Есть проблемы с листами: {build_issue_preview(issues)}"
                )
            else:
                msg = (
                    f"РСЧ отчеты за период собраны: {files_written} файлов, "
                    f"{total_rows} строк"
                )

            on_complete(True, msg, output_dir_path)
        except Exception as e:
            logging.exception("RSC period batch build error: %s", e)
            on_complete(False, f"Ошибка пакетной сборки РСЧ за период: {e}", None)

    self._start_thread(task)

def run_rsch_report_batch(
    self,
    source_paths: Sequence[Path],
    output_paths: Sequence[Path],
    selected_date,
    on_complete: Callable[[bool, str, Optional[Path]], None],
    cancel_event: Optional[threading.Event] = None,
) -> None:
    """Build separate RSC reports per source workbook."""

    def task() -> None:
        try:
            target_date = selected_date
            if hasattr(selected_date, "date"):
                target_date = selected_date.date()

            from core.app_controller import _is_cancelled, _cancel_message

            if _is_cancelled(cancel_event):
                _cancel_message(on_complete, *output_paths)
                return

            normalized_sources = [Path(p) for p in source_paths]
            normalized_outputs = [Path(p) for p in output_paths]
            if len(normalized_sources) != len(normalized_outputs):
                raise ValueError("Sources and outputs count mismatch")
            if len({str(path) for path in normalized_outputs}) != len(
```

```
                normalized_outputs
            ):
                raise ValueError("Duplicate output paths")

            total_rows = 0
            issues: List[str] = []
            for src, out in zip(normalized_sources, normalized_outputs, strict=True):
                if _is_cancelled(cancel_event):
                    _cancel_message(on_complete, *normalized_outputs)
                    return
                written_rows = build_rsch_report(
                    src,
                    out,
                    target_date=target_date,
                    issues_sink=issues,
                    drive_letters=self._get_drive_letters(),
                )
                if written_rows:
                    total_rows += written_rows
                elif out.exists():
                    out.unlink()

            output_dir = (
                normalized_outputs[0].parent if normalized_outputs else None
            )
            if total_rows == 0:
                raise ValueError("РСЧ: нет строк за выбранную дату")

            if issues:
                msg = (
                    f"РСЧ отчеты собраны: {len(normalized_outputs)} файлов, {total_rows} строк. "
                    f"Есть проблемы с листами: {build_issue_preview(issues)}"
                )
            else:
                msg = (
                    f"РСЧ отчеты собраны: {len(normalized_outputs)} файлов, "
                    f"{total_rows} строк"
                )

            on_complete(
                True,
                msg,
                output_dir,
            )
        except Exception as e:
            logging.exception("RSC batch build error: %s", e)
            on_complete(False, f"Ошибка пакетной сборки РСЧ: {e}", None)

    self._start_thread(task)
```

APP\core\ui__init__.py

0 lines • 0 bytes

APP\core\ui\dashboard_state.py

489 lines • 17,263 bytes

```
import os
import json
import sys
from dataclasses import dataclass
from datetime import datetime
from pathlib import Path
from typing import Callable, Iterable


@dataclass(frozen=True)
class PathInfo:
    label: str
    path: Path | None
    exists: bool
    kind: str
    mtime: str = "-"
    size: str = "-"


@dataclass(frozen=True)
class DiagnosticItem:
    label: str
    status: str
    detail: str
    path: Path | None = None


@dataclass(frozen=True)
class RuntimeResult:
    label: str
    ok: bool
    message: str
    path: Path | None = None
    rows: int | None = None
    created_at: datetime | None = None


STATUS_MARKERS = {
    "ok": " OK",
    "warning": " Внимание",
    "error": " Ошибка",
}


def format_file_size(path: Path | None) -> str:
    if not path:
        return "-"
    try:
        size = path.stat().st_size
    except OSError:
        return "-"

    units = ("B", "KB", "MB", "GB")
    value = float(size)
    unit = units[0]
    for unit in units:
        if value < 1024 or unit == units[-1]:
            break
        value /= 1024
    if unit == "B":
        return f"{int(value)} B"
    return f"{value:.1f} {unit}"


def format_mtime(path: Path | None) -> str:
    if not path:
        return "-"
    try:
        stamp = datetime.fromtimestamp(path.stat().st_mtime)
    except OSError:
        return "-"
    return stamp.strftime("%d.%m.%Y %H:%M")


def find_latest_file(directory: Path | str | None, patterns: Iterable[str]) -> Path | None:
    if not directory:
        return None
    root = Path(directory)
    if not root.exists() or not root.is_dir():
        return None

    candidates: list[Path] = []
    for pattern in patterns:
```

```
        candidates.extend(path for path in root.glob(pattern) if path.is_file())
    if not candidates:
        return None
    return max(candidates, key=lambda path: path.stat().st_mtime)


def safe_open_existing_path(
    path: Path | str | None,
    opener: Callable[[str], object],
    on_missing: Callable[[str], object] | None = None,
) -> bool:
    if not path:
        if on_missing:
            on_missing("Путь не задан")
        return False

    resolved = Path(path)
    if not resolved.exists():
        if on_missing:
            on_missing(f"Путь не найден: {resolved}")
        return False

    opener(str(resolved))
    return True


def status_text(status: str) -> str:
    return STATUS_MARKERS.get(status, f" {status}")


def load_runtime_results(state_path: Path | str) -> list[RuntimeResult]:
    path = Path(state_path)
    try:
        raw = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return []
    if not isinstance(raw, list):
        return []

    results: list[RuntimeResult] = []
    for item in raw[:10]:
        if not isinstance(item, dict):
            continue
        label = str(item.get("label") or "").strip()
        message = str(item.get("message") or "").strip()
        if not label or not message:
            continue
        created_at = None
        raw_created = item.get("created_at")
        if isinstance(raw_created, str) and raw_created:
            try:
                created_at = datetime.fromisoformat(raw_created)
            except ValueError:
                created_at = None
        raw_path = item.get("path")
        raw_rows = item.get("rows")
        rows = raw_rows if isinstance(raw_rows, int) else None
        results.append(
            RuntimeResult(
                label=label,
                ok=bool(item.get("ok")),
                message=message,
                path=Path(str(raw_path)) if raw_path and str(raw_path).strip() else None,
                rows=rows,
                created_at=created_at,
            )
        )
    return results


def save_runtime_results(
    state_path: Path | str,
    results: Iterable[RuntimeResult],
) -> None:
    path = Path(state_path)
    payload = []
    for result in list(results)[:10]:
        payload.append(
            {
                "label": result.label,
                "ok": result.ok,
                "message": result.message,
                "path": str(result.path) if result.path else "",
```

```
                "rows": result.rows,
                "created_at": result.created_at.isoformat()
                if result.created_at
                else "",
            }
        )
    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(
            json.dumps(payload, ensure_ascii=False, indent=2),
            encoding="utf-8",
        )
    except OSError:
        return


def build_diagnostic_summary(
    diagnostics: Iterable[DiagnosticItem | tuple[str, str, str, Path | None]],
    results: Iterable[RuntimeResult],
    log_errors: Iterable[str],
) -> str:
    lines = ["Диагностика Master-Report", ""]
    lines.append("Система:")
    for item in diagnostics:
        if isinstance(item, DiagnosticItem):
            label = item.label
            state = status_text(item.status)
            detail = item.detail
            path = item.path
        else:
            label, state, detail, path = item
        suffix = f" | {path}" if path else ""
        lines.append(f"- {label}: {state} | {detail}{suffix}")

    lines.extend(["", "Последние результаты:"])
    result_list = list(results)
    if result_list:
        for result in result_list[:10]:
            state = " успех" if result.ok else " ошибка"
            time_text = (
                result.created_at.strftime("%d.%m.%Y %H:%M")
                if result.created_at
                else "-"
            )
            path_text = f" | {result.path}" if result.path else ""
            lines.append(f"- {time_text} | {result.label}: {state} | {result.message}{path_text}")
    else:
        lines.append("- нет сохраненных результатов")

    lines.extend(["", "Последние ошибки логов:"])
    errors = list(log_errors)
    if errors:
        lines.extend(f"- {line}" for line in errors)
    else:
        lines.append("- не найдены")
    return "\n".join(lines)


def describe_path(label: str, path: Path | str | None, kind: str = "file") -> PathInfo:
    resolved = Path(path) if path else None
    exists = bool(resolved and resolved.exists())
    return PathInfo(
        label=label,
        path=resolved,
        exists=exists,
        kind=kind,
        mtime=format_mtime(resolved) if exists else "-",
        size=format_file_size(resolved) if exists and resolved.is_file() else "-",
    )


def _resolve_config_path(config_controller, value: str | Path | None) -> Path | None:
    if not value:
        return None
    path = Path(str(value))
    if path.is_absolute() or str(value).startswith("\\\\"):
        return path
    resolver = getattr(config_controller, "resolve_path", None)
    if callable(resolver):
        return Path(resolver(str(value)))
    config_path = Path(getattr(config_controller, "config_path", "."))
    return (config_path.parent / path).resolve()
```

```
def _config(config_controller) -> dict:
    raw = getattr(config_controller, "config", {})
    return raw if isinstance(raw, dict) else {}


def collect_report_paths(config_controller, app_root: Path | None = None) -> list[PathInfo]:
    cfg = _config(config_controller)
    app_root = app_root or Path(getattr(config_controller, "config_path", ".")).parent
    paths_cfg = cfg.get("paths", {}) if isinstance(cfg.get("paths"), dict) else {}
    rsch_cfg = cfg.get("rsch", {}) if isinstance(cfg.get("rsch"), dict) else {}
    curators_cfg = cfg.get("curators", {}) if isinstance(cfg.get("curators"), dict) else {}

    master_path = _resolve_config_path(
        config_controller, paths_cfg.get("master_file", "../Master_Report.xlsx")
    )
    rsch_dir = _resolve_config_path(config_controller, rsch_cfg.get("last_output_dir"))
    backup_dir = _resolve_config_path(config_controller, paths_cfg.get("archive_dir", "../backups"))
    benchmarks_dir = app_root.parent / "benchmarks"

    items = [
        describe_path("Master_Report.xlsx", master_path),
        describe_path(
            "boris_report.xlsx",
            _resolve_config_path(
                config_controller,
                curators_cfg.get("boris", {}).get("output_path")
                if isinstance(curators_cfg.get("boris"), dict)
                else None,
            ),
        ),
        describe_path(
            "ivan_report.xlsx",
            _resolve_config_path(
                config_controller,
                curators_cfg.get("ivan", {}).get("output_path")
                if isinstance(curators_cfg.get("ivan"), dict)
                else None,
            ),
        ),
        describe_path(
            "Последний РСЧ",
            find_latest_file(rsch_dir, ("РСЧ_отчет*.xlsx", "РСЧ*.xlsx")),
        ),
        describe_path(
            "Последняя сверка",
            find_latest_file(rsch_dir, ("Отчет_расхождений*.xlsx",)),
        ),
        describe_path("Backup", backup_dir, kind="folder"),
        describe_path(
            "Последний benchmark",
            find_latest_file(benchmarks_dir, ("timing_*.md", "timing_*.json")),
        ),
    ]
    return items


def collect_settings_lines(config_controller) -> list[str]:
    cfg = _config(config_controller)
    paths_cfg = cfg.get("paths", {}) if isinstance(cfg.get("paths"), dict) else {}
    network_cfg = cfg.get("network", {}) if isinstance(cfg.get("network"), dict) else {}
    performance_cfg = cfg.get("performance", {}) if isinstance(cfg.get("performance"), dict) else {}
    backup_cfg = cfg.get("backup", {}) if isinstance(cfg.get("backup"), dict) else {}
    rsch_cfg = cfg.get("rsch", {}) if isinstance(cfg.get("rsch"), dict) else {}

    drive_letters = network_cfg.get("drive_letters", [])
    if isinstance(drive_letters, list):
        drive_text = ", ".join(str(item) for item in drive_letters) or "-"
    else:
        drive_text = str(drive_letters)

    return [
        f"performance.max_workers: {performance_cfg.get('max_workers') or 'auto'}",
        f"backup.excel_keep: {backup_cfg.get('excel_keep', 10)}",
        f"paths.master_file: {paths_cfg.get('master_file', '-')}",
        f"paths.archive_dir: {paths_cfg.get('archive_dir', '-')}",
        f"rsch.last_output_dir: {rsch_cfg.get('last_output_dir', '-')}",
        f"network.drive_letters: {drive_text}",
    ]


def read_recent_log_errors(log_paths: Iterable[Path | str], limit: int = 5) -> list[str]:
    lines: list[str] = []
    for raw_path in log_paths:
```

```
        path = Path(raw_path)
        if not path.exists() or not path.is_file():
            continue
        try:
            # Read only the tail of the file — these logs can grow to
            # multiple MB and the previous implementation loaded each
            # one fully into memory on every refresh. 64 KB is enough
            # to capture the most recent errors we display.
            size = path.stat().st_size
            with open(path, "rb") as f:
                f.seek(max(0, size - 65536))
                tail = f.read().decode("utf-8", errors="replace")
            for line in tail.splitlines():
                upper = line.upper()
                if "ERROR" in upper or "CRITICAL" in upper or "ОШИБ" in upper:
                    lines.append(line.strip())
        except OSError:
            continue
    return lines[-limit:]


def read_recent_logs(log_paths: Iterable[Path | str], limit: int = 150) -> list[str]:
    lines: list[str] = []
    for raw_path in log_paths:
        path = Path(raw_path)
        if not path.exists() or not path.is_file():
            continue
        try:
            # Read the last 128KB of the file
            size = path.stat().st_size
            with open(path, "rb") as f:
                f.seek(max(0, size - 131072))
                tail = f.read().decode("utf-8", errors="replace")
            for line in tail.splitlines():
                if line.strip():
                    lines.append(line.strip())
        except OSError:
            continue
    return lines[-limit:]


def run_light_diagnostics(
    config_controller,
    app_root: Path | None = None,
    include_network: bool = True,
) -> list[DiagnosticItem]:
    cfg = _config(config_controller)
    app_root = app_root or Path(getattr(config_controller, "config_path", ".")).parent
    repo_root = app_root.parent
    paths_cfg = cfg.get("paths", {}) if isinstance(cfg.get("paths"), dict) else {}
    db_cfg = cfg.get("database", {}) if isinstance(cfg.get("database"), dict) else {}
    network_cfg = cfg.get("network", {}) if isinstance(cfg.get("network"), dict) else {}

    config_path = Path(getattr(config_controller, "config_path", app_root / "config.yaml"))
    employees_path = Path(getattr(config_controller, "employees_path", app_root / "employees.yaml"))
    db_path = _resolve_config_path(config_controller, db_cfg.get("path", "cache.db"))
    backup_path = _resolve_config_path(config_controller, paths_cfg.get("archive_dir", "../backups"))
    master_path = _resolve_config_path(config_controller, paths_cfg.get("master_file", "../Master_Report.xlsx"))

    diagnostics = [
        DiagnosticItem("Python", "ok", sys.version.split()[0], Path(sys.executable)),
        DiagnosticItem(
            ".venv",
            "ok" if Path(sys.prefix).name == ".venv" else "warning",
            str(sys.prefix),
            Path(sys.prefix),
        ),
        _path_diagnostic("config.yaml", config_path, require_file=True),
        _path_diagnostic("employees.yaml", employees_path, require_file=True),
        _writable_diagnostic("cache.db", db_path),
        _writable_diagnostic("Master output", master_path.parent if master_path else None, require_dir=True),
        _path_diagnostic(
            "Backup folder",
            backup_path,
            require_file=False,
            missing_status="warning",
        ),
        _path_diagnostic("Logs", repo_root / "startup_log.txt", require_file=True),
        _path_diagnostic(
            "Benchmarks",
            repo_root / "benchmarks",
            require_file=False,
            missing_status="warning",
```

```
        ),
    ]

    if include_network:
        source_roots = paths_cfg.get("source_roots") or [paths_cfg.get("source_root")]
        for raw in source_roots:
            if not raw:
                continue
            path = Path(str(raw))
            diagnostics.append(
                DiagnosticItem(
                    str(raw),
                    "ok" if path.exists() else "warning",
                    "доступен" if path.exists() else "не найден или недоступен",
                    path,
                )
            )

        for raw_letter in network_cfg.get("drive_letters", []):
            letter = str(raw_letter).strip().rstrip(":\\/")
            if not letter:
                continue
            root = Path(f"{letter}:\\")
            label = f"{letter}:\\"
            if any(item.label == label for item in diagnostics):
                continue
            diagnostics.append(
                DiagnosticItem(
                    label,
                    "ok" if root.exists() else "warning",
                    "доступен" if root.exists() else "не найден или недоступен",
                    root,
                )
            )
    return diagnostics


def _path_diagnostic(
    label: str,
    path: Path | None,
    require_file: bool,
    missing_status: str | None = None,
) -> DiagnosticItem:
    if not path:
        return DiagnosticItem(label, "error", "путь не задан", None)
    exists = path.is_file() if require_file else path.exists()
    if missing_status is None:
        missing_status = "error" if require_file else "warning"
    return DiagnosticItem(
        label,
        "ok" if exists else missing_status,
        "найден" if exists else "не найден",
        path,
    )


def _writable_diagnostic(
    label: str,
    path: Path | None,
    require_dir: bool = False,
) -> DiagnosticItem:
    if not path:
        return DiagnosticItem(label, "error", "путь не задан", None)
    target = path if require_dir else path.parent
    if not target.exists():
        return DiagnosticItem(label, "warning", f"папка не найдена: {target}", path)
    writable = os.access(target, os.W_OK)
    return DiagnosticItem(
        label,
        "ok" if writable else "error",
        "доступен для записи" if writable else "нет доступа на запись",
        path,
    )
```

APP\core\ui\debounce.py

78 lines • 2,366 bytes

```
"""Tiny debounce helper for tkinter variable traces.

The employee search box fires ``filter_tree`` on every keystroke via a
``StringVar`` trace. For a list of a few hundred employees that's still
cheap, but the goal is to keep the GUI snappy under the maximum data
load and avoid re-running the filter while the user is still typing.

The implementation schedules the callback via ``widget.after`` so it
runs on the same Tk thread — no locking, no extra dependency.
"""
from __future__ import annotations

from typing import Callable, Optional


class Debouncer:
    """Call ``fn`` no more than once per ``delay_ms`` milliseconds.

    A new call resets the timer. The scheduled ``after`` id is held on
    the instance, so cancelling the widget (or replacing the callback)
    also cancels any pending call.

    Usage::

        debouncer = Debouncer(self.search_var, self.controller.filter_tree, 300)
        self.search_var.trace_add("write", lambda *_: debouncer())

        # When the widget is destroyed:
        debouncer.cancel()
    """

    def __init__(
        self,
        widget,
        fn: Callable[[], None],
        delay_ms: int = 300,
    ) -> None:
        self._widget = widget
        self._fn = fn
        self._delay_ms = int(delay_ms)
        self._after_id: Optional[str] = None

    def __call__(self) -> None:
        """Schedule ``fn`` to run after ``delay_ms``.

        Resets the timer if a call is already pending. Safe to call
        from inside a trace callback (the trace fires synchronously,
        the scheduled call runs later).
        """
        self.cancel()
        try:
            self._after_id = self._widget.after(self._delay_ms, self._fire)
        except Exception:
            self._after_id = None

    def cancel(self) -> None:
        """Cancel any pending call. Idempotent."""
        if self._after_id is None:
            return
        try:
            self._widget.after_cancel(self._after_id)
        except Exception:
            pass
        self._after_id = None

    def _fire(self) -> None:
        self._after_id = None
        try:
            self._fn()
        except Exception:
            pass

    @property
    def pending(self) -> bool:
        return self._after_id is not None


__all__ = ["Debouncer"]
```

APP\core\ui\geometry.py

116 lines • 3,991 bytes

```
"""Persist and restore the main window geometry between launches.

The geometry is stored in a single line in ``~/.master_report_geometry``
using the format ``WIDTHxHEIGHT+X+Y`` (the same string Tk returns
from ``geometry()``). We deliberately do not store width/height only:
position is also part of the user-visible state.

If the saved geometry is off-screen (e.g. a previously-attached
external monitor is no longer connected) we fall back to a sensible
default. The check is intentionally permissive: any single coordinate
outside the current virtual screen is enough to discard the saved
value.
"""
from __future__ import annotations

import logging
import re
from pathlib import Path
from typing import Optional, Tuple

_logger = logging.getLogger(__name__)

_DEFAULT_FILE = Path.home() / ".master_report_geometry"
_GEOM_RE = re.compile(r"^(\d+)x(\d+)([+-]\d+)([+-]\d+)$")


def _parse_geometry(geom: str) -> Optional[Tuple[int, int, int, int]]:
    m = _GEOM_RE.match(geom.strip())
    if not m:
        return None
    w, h, x, y = (int(m.group(i)) for i in range(1, 5))
    return w, h, x, y


def _is_visible(
    width: int, height: int, x: int, y: int, screen_w: int, screen_h: int
) -> bool:
    """Reject saved geometries that no longer fit on the current display.

    The check is deliberately permissive about multi-monitor setups:
    ``winfo_screenwidth/height`` returns the *primary* monitor's
    geometry on some platforms, so a window legitimately placed on a
    secondary monitor (x > screen_w) would be rejected by a strict
    bound. We only discard the saved value when the top-left corner is
    more than ~half a screen beyond any edge of the virtual desktop.
    """
    if width < 100 or height < 100:
        return False
    if x < -screen_w * 0.5 or y < -screen_h * 0.5:
        return False
    if x > screen_w * 1.5 or y > screen_h * 1.5:
        return False
    return True


def load_geometry(file_path: Optional[Path] = None) -> Optional[str]:
    """Return a usable geometry string, or None when none is stored.

    The caller (GuiMaster) is expected to merge this with its own
    default-size hint: if the saved geometry is invalid or off-screen
    we return None and the GUI falls back to its hard-coded default.
    """
    path = file_path or _DEFAULT_FILE
    try:
        content = path.read_text(encoding="utf-8").strip()
    except FileNotFoundError:
        return None
    except OSError as e:
        _logger.debug("Geometry file not readable: %s", e)
        return None

    parsed = _parse_geometry(content)
    if parsed is None:
        _logger.info("Discarded malformed saved geometry: %r", content)
        return None
    w, h, x, y = parsed
    # Without a screen reference we can't validate visibility; return
    # the value and let the caller check after Tk is up.
    return f"{w}x{h}+{x}+{y}"


def is_geometry_visible(
    geometry: str, screen_w: int, screen_h: int
```

```
) -> bool:
    parsed = _parse_geometry(geometry)
    if parsed is None:
        return False
    return _is_visible(*parsed, screen_w=screen_w, screen_h=screen_h)


def save_geometry(geometry: str, file_path: Optional[Path] = None) -> None:
    """Persist the geometry for the next launch.

    Failures are logged but never raised — the GUI should still be
    able to close cleanly even if the disk is full or read-only.
    """
    path = file_path or _DEFAULT_FILE
    if _parse_geometry(geometry) is None:
        _logger.debug("Refusing to save malformed geometry: %r", geometry)
        return
    try:
        path.write_text(geometry, encoding="utf-8")
    except OSError as e:
        _logger.warning("Failed to persist window geometry: %s", e)


def default_geometry_path() -> Path:
    return _DEFAULT_FILE


__all__ = [
    "load_geometry",
    "save_geometry",
    "is_geometry_visible",
    "default_geometry_path",
]
```

APP\core\ui\gui_commands__init__.py

82 lines • 3,165 bytes

```
"""GuiCommands package.

Composes the four feature mixins (employee / consolidation / rsch /
reconciliation) plus the shared :class:`CommandContext`. The original
monolith was 734 lines; each feature is now a separate module so they
can evolve independently.

Layout:
    gui_commands/
        __init__.py        - this file, public surface
        _base.py           - CommandContext, finish() helper
        _employee.py       - refresh + employee table actions
        _consolidation.py  - 3 consolidation commands
        _rsch.py           - RSCH commands + helpers
        _reconciliation.py - reconciliation commands
"""
from core.ui.gui_commands._base import CommandContext
from core.ui.gui_commands._consolidation import ConsolidationCommandsMixin
from core.ui.gui_commands._employee import EmployeeCommandsMixin
from core.ui.gui_commands._reconciliation import ReconciliationCommandsMixin


class GuiCommands(
    EmployeeCommandsMixin,
    ConsolidationCommandsMixin,
    ReconciliationCommandsMixin,
):
    # ReconciliationCommandsMixin transitively provides RschCommandsMixin
    # (it inherits from it for the source-file resolution helpers), so
    # we don't list RschCommandsMixin here. Adding it would create a
    # diamond that Python refuses to linearise.
    pass
    """Single dispatch surface for the GUI.

    The class holds no real behaviour of its own — it stitches the
    four feature mixins together and exposes ``_record_dashboard_result``
    so each mixin's ``finish()`` call can update the dashboard tab.
    Public methods are reachable from ``GuiMaster`` via plain attribute
    access (e.g. ``self.commands.run_consolidation()``).
    """

    def __init__(
        self,
        root,
        config_controller,
        app_controller,
        employee_vm,
        consolidation_vm,
        sidebar,
        employee_table,
        stats_panel,
        dashboard_tabs=None,
        operation_state=None,
        toasts=None,
    ):
        self.ctx = CommandContext(
            root=root,
            config_controller=config_controller,
            app_controller=app_controller,
            employee_vm=employee_vm,
            consolidation_vm=consolidation_vm,
            sidebar=sidebar,
            employee_table=employee_table,
            stats_panel=stats_panel,
            dashboard_tabs=dashboard_tabs,
            operation_state=operation_state,
            toasts=toasts,
        )
        # Compatibility shim: the original code stored the operation
        # state on both ``self`` and ``self.ctx``. New code should
        # read ``self.ctx.operation_state``; old tests may still poke
        # ``self.operation_state``. Both point at the same object.
        self.operation_state = self.ctx.operation_state
        self.toasts = self.ctx.toasts
        self._apply_saved_rsch_mode()

    def _record_dashboard_result(self, label, ok, msg, path) -> None:
        if self.ctx.dashboard_tabs is not None:
            self.ctx.dashboard_tabs.record_result(label, ok, msg, path)


__all__ = ["GuiCommands", "CommandContext"]
```

APP\core\ui\gui_commands_base.py

146 lines • 5,628 bytes

```
"""Shared context and helpers for the GuiCommands modules.

The dispatching ``GuiCommands`` class is split by feature into
``_employee``, ``_consolidation``, ``_rsch`` and ``_reconciliation``;
every feature module needs the same handful of collaborators
(``root``, ``operation_state``, ``toasts``…). Those collaborators are
collected in a single ``CommandContext`` dataclass so each module can
ask for ``self.ctx`` instead of repeating the same constructor
signature.
"""
from __future__ import annotations

import logging
from dataclasses import dataclass
from pathlib import Path
from tkinter import messagebox
from typing import Callable, Optional

from core.infra import open_path
from core.ui.operation_state import OperationState
from core.ui.toast import ToastLevel, ToastManager


@dataclass
class CommandContext:
    """Bag of shared collaborators for every GuiCommands feature module.

    Held as a single attribute (``self.ctx``) on the ``GuiCommands``
    class. The fields are exactly the ones the original monolith took
    in its constructor; adding a new one here is the only place that
    needs to change.
    """

    root: object
    config_controller: object
    app_controller: object
    employee_vm: object
    consolidation_vm: object
    sidebar: object
    employee_table: object
    stats_panel: object
    dashboard_tabs: object
    operation_state: OperationState
    toasts: Optional[ToastManager] = None

    def post(self, fn: Callable[[], None]) -> None:
        """Schedule ``fn`` to run on the Tk main loop.

        Every async completion handler in the project wraps its work
        in ``self.root.after(0, fin)``; the helper saves the
        boilerplate and reads better.
        """
        self.root.after(0, fn)


class CancelledByUser(str):
    """Marker signalling a user-initiated cancellation.

    Subclasses ``str`` (rather than ``Exception``) so it flows unchanged
    through every existing callback that expects a string message —
    ``logging.info("%s", msg)``, ``messagebox.showerror(..., msg)``,
    the toast manager, and the duplicate substring check in
    ``gui_master._on_*_complete``. Detection in :func:`finish` uses
    ``isinstance(msg, CancelledByUser)`` rather than substring matching
    on the Russian phrase, so a reworded message or a localized UI no
    longer breaks the cancelled-vs-error branch.

    The cancellation site (``app_controller._cancel_message``) constructs
    this marker instead of a bare ``str``; both paths produce identical
    string output, but only the marker is reliably detectable here.
    """

    __slots__ = ()


def _is_cancelled_message(msg) -> bool:
    """True if ``msg`` signals a 'cancelled by user' completion.

    Detection is type-based (``isinstance``) instead of Russian substring
    matching so the cancelled-vs-error branch in :func:`finish` survives
    a reworded message or a localized UI. The legacy substring fallback
    covers any cancellation site that still passes a plain ``str``
    (e.g. ``gui_master`` re-routes completions through its own path).
```

```
    """
    if isinstance(msg, CancelledByUser):
        return True
    return "отменена пользователем" in str(msg or "").lower()


def finish(
    ctx: CommandContext,
    ok: bool,
    msg: str,
    path: Optional[Path],
    label: str,
    ask_open_text: Optional[str] = None,
    record_result: Optional[Callable[[str, bool, str, Optional[Path]], None]] = None,
    success_duration_ms: int = 4000,
    error_duration_ms: int = 6000,
) -> None:
    """Shared completion handler for the build/reconciliation commands.

    The original monolith had four near-identical 20-line
    ``_on_*_complete`` methods; they collapsed to one helper plus a
    one-line caller. The behaviour is:

    * ``ok``         → success toast + optional 'open file?' prompt
    * cancelled      → warning toast (NOT an error)
    * error          → error toast (or ``messagebox.showerror`` when
                       no toast manager is available)
    """
    # Release the operation slot BEFORE showing any toasts or messageboxes.
    # Otherwise the sidebar buttons stay locked during the dialog, and if
    # the user closes the dialog without clicking through, the slot is
    # never freed — permanent button freeze.
    ctx.operation_state.end()

    cancelled = (not ok) and _is_cancelled_message(msg)
    if ok:
        logging.info("Успешно (%s): %s", label, msg)
        if record_result is not None:
            record_result(label, True, msg, path)
        if path and ask_open_text and messagebox.askyesno("Успех", ask_open_text):
            open_path(str(path))
        if ctx.toasts:
            ctx.toasts.show(ToastLevel.SUCCESS, label, msg, success_duration_ms)
        return

    if cancelled:
        logging.info("Отменено (%s): %s", label, msg)
        if record_result is not None:
            record_result(label, False, msg, path)
        if ctx.toasts:
            ctx.toasts.show(ToastLevel.WARNING, f"{label}: отменено", msg, success_duration_ms)
        return

    logging.error("Ошибка (%s): %s", label, msg)
    if record_result is not None:
        record_result(label, ok, msg, path)
    if ctx.toasts:
        ctx.toasts.show(ToastLevel.ERROR, f"{label}: ошибка", msg, error_duration_ms)
    else:
        messagebox.showerror("Ошибка", msg)


__all__ = ["CancelledByUser", "CommandContext", "finish"]
```

APP\core\ui\gui_commands_consolidation.py

83 lines • 3,116 bytes

```
"""Consolidation commands (day / all dates / month).

The three methods share the same skeleton: validate, confirm, ask
the state machine for a slot, then hand off to the ViewModel. Kept
as a small module so the build-vs-filter distinction is obvious to
anyone reading the package.
"""
from __future__ import annotations

from tkinter import messagebox

from core.ui.operation_state import OperationType
from core.ui.gui_commands._base import CommandContext


class ConsolidationCommandsMixin:
    """Mixin providing the three consolidation entry points.

    Combines with :class:`GuiCommands` which supplies ``self.ctx``.
    """

    ctx: CommandContext

    def run_consolidation(self) -> None:
        if not self.ctx.operation_state.begin(OperationType.CONSOLIDATION_DAY):
            return

        missing_names = self.ctx.employee_vm.get_missing_names()
        if missing_names:
            msg = f"Не сдано отчетов: {len(missing_names)}. Продолжить?"
            if not messagebox.askyesno("Предупреждение", msg):
                self.ctx.operation_state.end()
                return

        curator = self.ctx.sidebar.get_curator()
        self.ctx.consolidation_vm.run_consolidation(
            self.ctx.app_controller,
            self.ctx.sidebar.get_date(),
            curator=curator,
        )

    def run_consolidation_all_dates(self) -> None:
        if not self.ctx.operation_state.begin(OperationType.CONSOLIDATION_ALL):
            return

        missing_names = self.ctx.employee_vm.get_missing_names()
        if missing_names:
            msg = f"Не сдано отчетов: {len(missing_names)}. Продолжить?"
            if not messagebox.askyesno("Предупреждение", msg):
                self.ctx.operation_state.end()
                return

        curator = self.ctx.sidebar.get_curator()
        self.ctx.consolidation_vm.run_consolidation_all_dates(
            self.ctx.app_controller,
            curator=curator,
        )

    def run_consolidation_month(self) -> None:
        start_date, end_date = self.ctx.sidebar.get_month_range()
        if start_date > end_date:
            messagebox.showerror(
                "Ошибка", "Дата начала должна быть раньше даты окончания"
            )
            return

        if not self.ctx.operation_state.begin(OperationType.CONSOLIDATION_MONTH):
            return

        missing_names = self.ctx.employee_vm.get_missing_names()
        if missing_names:
            msg = f"Не сдано отчетов: {len(missing_names)}. Продолжить?"
            if not messagebox.askyesno("Предупреждение", msg):
                self.ctx.operation_state.end()
                return

        curator = self.ctx.sidebar.get_curator()
        self.ctx.consolidation_vm.run_consolidation_month(
            self.ctx.app_controller,
            start_date,
            end_date,
            curator=curator,
        )
```

APP\core\ui\gui_commands_employee.py

190 lines • 7,200 bytes

```
"""Refresh, employee-table sync, and one-off employee actions.

Everything the user can do *to the employee list* — add, attach, open
folder, delete, refresh — lives here. The consolidation / RSCH /
reconciliation commands are bigger and get their own modules.
"""
from __future__ import annotations

from tkinter import messagebox
from pathlib import Path
from typing import List

from core.infra import open_path
from core.ui.operation_state import OperationType
from core.ui.gui_commands._base import CommandContext


class EmployeeCommandsMixin:
    """Mixin providing refresh + employee table actions.

    The mixin is meant to be combined with :class:`GuiCommands`, which
    supplies the ``ctx`` and ``_record_dashboard_result`` methods
    referenced below. Splitting along this seam keeps the file size
    manageable: the employee list interactions change together, and
    the build-report commands change together.
    """

    ctx: CommandContext

    # --- refresh / status sync ----------------------------------------

    def refresh_status(self) -> None:
        """Trigger a full employee re-scan and update the table."""
        if not self.ctx.operation_state.begin(OperationType.REFRESH):
            return
        self.ctx.employee_table.clear_tree()
        self.ctx.stats_panel.set_readiness(0)
        self.ctx.stats_panel.set_operation(0)

        def on_refresh_complete() -> None:
            self.ctx.operation_state.end()

        def on_refresh_error(message: str) -> None:
            def show() -> None:
                if self.ctx.toasts:
                    self.ctx.toasts.error(
                        "Ошибка обновления", message, duration_ms=6000
                    )
                else:
                    messagebox.showerror("Ошибка", message)
            self.ctx.root.after(0, show)

        self.ctx.employee_vm.refresh(
            self.ctx.app_controller,
            self.ctx.sidebar.get_date(),
            self._on_consolidation_progress,
            on_complete=on_refresh_complete,
            on_error=on_refresh_error,
        )

    def _on_consolidation_progress(self, progress: float) -> None:
        self.ctx.post(lambda: self.ctx.stats_panel.set_operation(progress))

    def _sync_employee_ui(self) -> None:
        self.ctx.employee_table.clear_tree()
        for emp in self.ctx.employee_vm.get_filtered_employees():
            self.ctx.employee_table.insert_row(
                (emp.name, emp.status_text, emp.path_text), emp.tag
            )
        self.ctx.stats_panel.update_stats(
            self.ctx.employee_vm.get_ready_count(),
            self.ctx.employee_vm.get_total_count(),
        )
        self.ctx.stats_panel.set_readiness(
            self.ctx.employee_vm.get_progress_percent()
        )
        self.ctx.sidebar.set_refresh_state("normal")
        self._refresh_dashboard()

    def _refresh_dashboard(self) -> None:
        if self.ctx.dashboard_tabs is not None:
            self.ctx.dashboard_tabs.refresh()
```

```
    # --- add / attach / open / delete ----------------------------------

    def add_employee_dialog(self) -> None:
        from core.ui.ui_dialogs import ask_employee_name

        existing = list(self.ctx.employee_vm.get_all_names())
        name = ask_employee_name(existing_names=existing)
        if name:
            self.ctx.employee_vm.add_employee(self.ctx.config_controller, name)
            self.refresh_status()

    def attach_file_dialog(self) -> None:
        from core.ui.ui_dialogs import show_attach_file_dialog

        def on_attach(name, paths, append) -> None:
            self.ctx.employee_vm.attach_files(
                self.ctx.config_controller, name, paths, append=append
            )
            self.refresh_status()

        show_attach_file_dialog(
            self.ctx.root,
            self.ctx.employee_vm.get_all_names() or self.ctx.config_controller.get_employees(),
            on_attach,
        )

    def open_master_folder(self) -> None:
        p = self.ctx.employee_vm.get_master_path(self.ctx.config_controller)
        if p and p.parent.exists():
            open_path(str(p.parent))

    def copy_missing_to_clipboard(self) -> None:
        missing = self.ctx.employee_vm.get_missing_names()
        if missing:
            date_str = self.ctx.sidebar.get_date().strftime("%d.%m")
            text = "Должники за " + date_str + ":\n" + "\n".join(
                f"- {m}" for m in missing
            )
            self.ctx.root.clipboard_clear()
            self.ctx.root.clipboard_append(text)
            if self.ctx.toasts:
                self.ctx.toasts.info(
                    "Должники",
                    "Список скопирован в буфер обмена",
                    duration_ms=2500,
                )
            else:
                messagebox.showinfo(
                    "Успех", "Список должников скопирован в буфер обмена"
                )

    def filter_tree(self) -> None:
        self.ctx.employee_vm.search_query = self.ctx.employee_table.get_search_query()
        self._sync_employee_ui()

    def open_employee_folder(self) -> None:
        name = self.ctx.employee_table.get_selected_employee()
        paths = self.ctx.employee_vm.get_selected_employee_paths(name)
        if not paths:
            return

        unique_parents: List[str] = []
        seen: set = set()
        for raw_path in paths:
            parent = str(Path(raw_path).parent)
            if parent in seen:
                continue
            seen.add(parent)
            unique_parents.append(parent)

        for parent in unique_parents:
            open_path(parent)

    def open_employee_file(self) -> None:
        name = self.ctx.employee_table.get_selected_employee()
        paths = self.ctx.employee_vm.get_selected_employee_paths(name)
        if not paths:
            return
        if len(paths) == 1:
            open_path(paths[0])
            return

        action = messagebox.askyesnocancel(
```

```
            "Несколько файлов",
            (
                f"У сотрудника '{name}' найдено {len(paths)} файлов.\n\n"
                "Да - открыть все файлы\n"
                "Нет - открыть папки с файлами\n"
                "Отмена - ничего не делать"
            ),
        )
        if action is None:
            return
        if action:
            for path in paths:
                open_path(path)
            return

        self.open_employee_folder()

    def delete_employee(self) -> None:
        name = self.ctx.employee_table.get_selected_employee()
        if name and messagebox.askyesno(
            "Удалить", f"Удалить сотрудника '{name}'?"
        ):
            self.ctx.employee_vm.delete_employee(self.ctx.config_controller, name)
            self.refresh_status()
```

APP\core\ui\gui_commands_reconciliation.py

149 lines • 5,661 bytes

```
"""Reconciliation commands and their completion handlers.

Two operations here: the manual reconciliation (the user picks
already-built reports) and the automatic one (the controller
rebuilds the master reports itself before comparing them to the
РСЧ). Both share the source-file resolution flow with RSCH but
diverge in the file-picker UX.
"""
from __future__ import annotations

from pathlib import Path
from tkinter import filedialog
from typing import Optional

from core.ui.operation_state import OperationType
from core.ui.gui_commands._base import CommandContext, finish
from core.ui.gui_commands._rsch import RschCommandsMixin


class ReconciliationCommandsMixin(RschCommandsMixin):
    """Mixin providing the two reconciliation entry points.

    Inherits the RSCH source-resolution helpers because the auto
    reconciliation needs the same file-picker flow.
    """

    ctx: CommandContext

    def run_reconciliation_report(self) -> None:
        default_master_path = self.ctx.employee_vm.get_master_path(
            self.ctx.config_controller
        )
        filetypes = [("Excel files", "*.xlsx *.xlsm"), ("All files", "*.*")]
        master_dialog_kwargs: dict = {"filetypes": filetypes}
        if default_master_path:
            master_dialog_kwargs["initialdir"] = str(
                Path(default_master_path).parent
            )

        master_paths: dict = {}
        for label, title in (
            ("Общий", "Выберите общий мастер-отчет"),
            ("Борис", "Выберите отчет Бориса"),
            ("Иван", "Выберите отчет Ивана"),
        ):
            selected = filedialog.askopenfilename(
                title=title, **master_dialog_kwargs
            )
            if not selected:
                return
            master_paths[label] = Path(selected)
            master_dialog_kwargs["initialdir"] = str(Path(selected).parent)

        selected_date = self.ctx.sidebar.get_date()
        rsch_path = filedialog.askopenfilename(
            title="Выберите готовый РСЧ отчет",
            filetypes=filetypes,
            initialdir=self._suggest_rsch_output_dir([]),
        )
        if not rsch_path:
            return

        date_suffix = selected_date.strftime("%d.%m.%Y")
        output = filedialog.asksaveasfilename(
            title="Сохранить отчет расхождений",
            defaultextension=".xlsx",
            initialfile=f"Отчет_расхождений_{date_suffix}.xlsx",
            initialdir=str(Path(rsch_path).parent),
            filetypes=[("Excel files", "*.xlsx")],
        )
        if not output:
            return

        if not self.ctx.operation_state.begin(OperationType.RECONCILIATION):
            return
        self.ctx.app_controller.run_reconciliation_report(
            master_paths,
            Path(rsch_path),
            Path(output),
            selected_date,
            self._on_reconciliation_complete,
            cancel_event=self.ctx.operation_state.cancel_event,
        )
```

```
    def run_auto_reconciliation_day(self) -> None:
        source_paths = self._resolve_rsch_sources()
        if not source_paths:
            return

        selected_date = self.ctx.sidebar.get_date()
        date_suffix = selected_date.strftime("%d.%m.%Y")
        output = filedialog.asksaveasfilename(
            title="Сохранить автоматический отчет расхождений",
            defaultextension=".xlsx",
            initialfile=f"Отчет_расхождений_{date_suffix}.xlsx",
            initialdir=self._suggest_rsch_output_dir(source_paths),
            filetypes=[("Excel files", "*.xlsx")],
        )
        if not output:
            return

        self._save_rsch_preferences(source_paths, Path(output))
        if not self.ctx.operation_state.begin(OperationType.AUTO_RECONCILIATION):
            return
        self.ctx.app_controller.run_auto_reconciliation_day(
            source_paths,
            Path(output),
            selected_date,
            self._on_auto_reconciliation_complete,
            cancel_event=self.ctx.operation_state.cancel_event,
        )

    # --- completion handlers -------------------------------------------

    def _on_reconciliation_complete(
        self, ok: bool, msg: str, path: Optional[Path]
    ) -> None:
        def fin() -> None:
            finish(
                self.ctx,
                ok=ok,
                msg=msg,
                path=path,
                label="Сверка",
                ask_open_text=(
                    "Отчет расхождений собран. Открыть файл?" if ok else None
                ),
                record_result=self._record_dashboard_result,
            )
        self.ctx.post(fin)

    def _on_auto_reconciliation_complete(
        self, ok: bool, msg: str, path: Optional[Path]
    ) -> None:
        def fin() -> None:
            finish(
                self.ctx,
                ok=ok,
                msg=msg,
                path=path,
                label="Автосверка",
                ask_open_text=(
                    "Автоматический отчет расхождений собран. Открыть файл?"
                    if ok
                    else None
                ),
                record_result=self._record_dashboard_result,
            )
        self.ctx.post(fin)
```

APP\core\ui\gui_commands_rsch.py

242 lines • 9,092 bytes

```
"""RSCH (day / month) commands and their persistence helpers.

The RSCH build flow is the most stateful of the three: the user
selects source workbooks (optionally remembered from a previous run),
picks an output path, and the build runs in a background thread. All
the persistence helpers (saved sources, suggested dirs, safe file
names) live here too because nothing else needs them.
"""
from __future__ import annotations

from pathlib import Path
from tkinter import filedialog, messagebox
from typing import List, Optional

from core.ui.operation_state import OperationType
from core.ui.gui_commands._base import CommandContext, finish


class RschCommandsMixin:
    """Mixin providing RSCH commands and the related completion handler."""

    ctx: CommandContext

    # --- command entry points ---------------------------------------

    def run_rsch_report(self) -> None:
        source_paths = self._resolve_rsch_sources()
        if not source_paths:
            return

        selected_date = self.ctx.sidebar.get_date()
        date_suffix = selected_date.strftime("%d.%m.%Y")
        default_name = f"РСЧ_отчет_{date_suffix}.xlsx"
        output = filedialog.asksaveasfilename(
            title="Сохранить ШО отчет",
            defaultextension=".xlsx",
            initialfile=default_name,
            initialdir=self._suggest_rsch_output_dir(source_paths),
            filetypes=[("Excel files", "*.xlsx")],
        )
        if not output:
            return

        self._save_rsch_preferences(source_paths, Path(output))

        if not self.ctx.operation_state.begin(OperationType.RSCH_DAY):
            return
        self.ctx.app_controller.run_rsch_report(
            source_paths,
            Path(output),
            selected_date,
            self._on_rsch_complete,
            cancel_event=self.ctx.operation_state.cancel_event,
        )

    def run_rsch_report_month(self) -> None:
        start_date, end_date = self.ctx.sidebar.get_month_range()
        if start_date > end_date:
            messagebox.showerror(
                "Ошибка", "Дата начала должна быть раньше даты окончания"
            )
            return

        source_paths = self._resolve_rsch_sources()
        if not source_paths:
            return

        start_suffix = start_date.strftime("%d.%m.%Y")
        end_suffix = end_date.strftime("%d.%m.%Y")
        default_name = f"РСЧ_отчет_{start_suffix}-{end_suffix}.xlsx"
        output = filedialog.asksaveasfilename(
            title="Сохранить ШО отчет за период",
            defaultextension=".xlsx",
            initialfile=default_name,
            initialdir=self._suggest_rsch_output_dir(source_paths),
            filetypes=[("Excel files", "*.xlsx")],
        )
        if not output:
            return

        self._save_rsch_preferences(source_paths, Path(output))
        if not self.ctx.operation_state.begin(OperationType.RSCH_MONTH):
            return
```

```
        self.ctx.app_controller.run_rsch_report_period(
            source_paths,
            Path(output),
            start_date,
            end_date,
            self._on_rsch_complete,
            cancel_event=self.ctx.operation_state.cancel_event,
        )

    def _on_rsch_complete(self, ok: bool, msg: str, path: Optional[Path]) -> None:
        def fin() -> None:
            finish(
                self.ctx,
                ok=ok,
                msg=msg,
                path=path,
                label="РСЧ",
                ask_open_text=(
                    "РСЧ отчет собран. Открыть файл?" if ok else None
                ),
                record_result=self._record_dashboard_result,
            )
        self.ctx.post(fin)

    # --- source-file resolution (saved vs. fresh selection) ------------

    def _resolve_rsch_sources(self) -> List[Path]:
        """Apply the saved-or-pick dialog flow, returning the chosen paths.

        Returns an empty list when the user cancelled any of the
        dialogs; the caller treats that as 'abort the command'.
        """
        source_paths: List[Path] = []
        saved_sources = self._get_saved_rsch_sources()
        if saved_sources:
            existing_saved = [p for p in saved_sources if p.exists()]
            use_saved = messagebox.askyesnocancel(
                "РСЧ файлы",
                "Использовать сохраненный набор файлов РСЧ?",
            )
            if use_saved is None:
                return []
            if use_saved:
                if existing_saved:
                    source_paths = existing_saved
                else:
                    messagebox.showwarning(
                        "РСЧ файлы",
                        "Сохраненные пути не найдены. Выберите файлы заново.",
                    )

        if not source_paths:
            source_paths = self._select_rsch_sources(saved_sources)
        return source_paths

    def _select_rsch_sources(self, saved_sources: List[Path]) -> List[Path]:
        from core.ui.ui_dialogs import show_multi_file_select_dialog

        # Pre-populate the dialog with any paths that still exist on
        # disk so the user can re-open a session without losing work.
        seed = [str(p) for p in saved_sources if p.exists()]
        selected = show_multi_file_select_dialog(
            self.ctx.root,
            initial_paths=seed,
            title="Выберите ШР файлы РСЧ",
            filetypes=[
                ("Excel files", "*.xlsx *.xlsm *.xlsb"),
                ("All files", "*.*"),
            ],
        )
        return [Path(p) for p in selected]

    def _merge_unique_paths(
        self, current: List[Path], new_items: List[Path]
    ) -> List[Path]:
        merged = list(current)
        seen = {str(p) for p in current}
        for item in new_items:
            key = str(item)
            if key in seen:
                continue
            seen.add(key)
            merged.append(item)
```

```
    return merged

# --- config persistence ------------------------------------------

def _get_rsch_config(self) -> dict:
    cfg = getattr(self.ctx.config_controller, "config", None)
    if not isinstance(cfg, dict):
        cfg = {}
        try:
            self.ctx.config_controller.config = cfg
        except Exception:
            return {}

    rsch_cfg = cfg.get("rsch")
    if not isinstance(rsch_cfg, dict):
        rsch_cfg = {}
        cfg["rsch"] = rsch_cfg
    return rsch_cfg

def _get_saved_rsch_sources(self) -> List[Path]:
    rsch_cfg = self._get_rsch_config()
    raw_sources = rsch_cfg.get("recent_sources", [])
    if isinstance(raw_sources, str):
        raw_sources = [raw_sources]

    normalized: List[Path] = []
    seen: set = set()
    for raw in raw_sources:
        if not raw:
            continue
        path = Path(str(raw))
        key = str(path)
        if key in seen:
            continue
        seen.add(key)
        normalized.append(self._resolve_config_path(path))
    return normalized

def _resolve_config_path(self, path: Path) -> Path:
    if path.is_absolute():
        return path
    resolve_path = getattr(self.ctx.config_controller, "resolve_path", None)
    if callable(resolve_path):
        return Path(resolve_path(str(path)))
    return path

def _save_rsch_preferences(
    self, source_paths: List[Path], output_path: Path
) -> None:
    rsch_cfg = self._get_rsch_config()
    rsch_cfg["recent_sources"] = [str(p) for p in source_paths]
    rsch_cfg["last_output_dir"] = str(output_path.parent)
    rsch_cfg["mode"] = "combined"
    save_config = getattr(self.ctx.config_controller, "save_config", None)
    if callable(save_config):
        save_config()

def _apply_saved_rsch_mode(self) -> None:
    rsch_cfg = self._get_rsch_config()
    if rsch_cfg.get("mode") != "combined":
        rsch_cfg["mode"] = "combined"

# --- output naming -----------------------------------------------

def _suggest_rsch_output_dir(self, source_paths: List[Path]) -> str:
    rsch_cfg = self._get_rsch_config()
    last_output_dir = rsch_cfg.get("last_output_dir")
    if last_output_dir:
        parent = self._resolve_config_path(Path(str(last_output_dir)))
        if parent.exists():
            return str(parent)
    if source_paths:
        first_parent = source_paths[0].parent
        if first_parent.exists():
            return str(first_parent)
    return ""
```

APP\core\ui\gui_master.py

416 lines • 16,499 bytes

```
import logging
import tkinter as tk
from tkinter import ttk, messagebox
from pathlib import Path
from typing import Optional

from core.ui.ui_sidebar import Sidebar
from core.ui.ui_employee_table import EmployeeTable
from core.ui.ui_stats_panel import StatsPanel
from core.ui.ui_log_panel import LogPanel
from core.ui.ui_dashboard import DashboardTabs
from core.ui.theme import ThemeManager
from core.ui.geometry import (
    is_geometry_visible,
    load_geometry,
    save_geometry,
)
from core.ui.operation_state import OPERATION_LABELS, OperationState
from core.ui.operation_status_bar import OperationStatusBar
from core.ui.toast import ToastManager
from core.infra import ConfigController, setup_gui_logging, open_path
from core.app_controller import AppController

from core.view_models.employee_list_viewmodel import EmployeeListViewModel
from core.view_models.consolidation_viewmodel import ConsolidationViewModel
from core.ui.gui_commands import GuiCommands


def _fit_window_geometry(
    preferred_width: int,
    preferred_height: int,
    screen_width: int,
    screen_height: int,
) -> str:
    max_width = max(800, int(screen_width * 0.82))
    max_height = max(560, int(screen_height * 0.88))
    width = min(preferred_width, max_width)
    height = min(preferred_height, max_height)
    x = max(0, (screen_width - width) // 2)
    y = max(0, (screen_height - height) // 2)
    return f"{width}x{height}+{x}+{y}"


def _resolve_initial_geometry(
    preferred: tuple[int, int],
    screen_width: int,
    screen_height: int,
) -> str:
    """Pick the geometry to use at startup.

    Preference order:
    1. Persisted geometry (if it still fits the current display).
    2. Auto-fitted default (centred, capped to 82% of screen width).
    """
    saved = load_geometry()
    if saved and is_geometry_visible(saved, screen_width, screen_height):
        return saved
    return _fit_window_geometry(
        preferred[0], preferred[1], screen_width, screen_height
    )


class GuiMaster:
    def __init__(
        self,
        root: tk.Tk,
        config_controller: ConfigController,
        app_controller: AppController,
    ):
        self.root, self.config_controller, self.app_controller = (
            root,
            config_controller,
            app_controller,
        )
        self.root.title(" Master-Report v5.5 | Financial Consolidator")
        ThemeManager.init_scaling(self.root)
        ThemeManager.apply_styles()
        sw, sh = self.root.winfo_screenwidth(), self.root.winfo_screenheight()
        self.root.geometry(_resolve_initial_geometry((1100, 840), sw, sh))
        self.root.minsize(max(640, int(sw * 0.5)), max(420, int(sh * 0.5)))
        self.root.configure(bg=ThemeManager.COLORS["bg_main"])

        # Persist the geometry on close. The protocol is set by main.py
```

```
        # to call on_close; we intercept that to save the geometry.
        self._install_geometry_save_handler()

        self.employee_vm = EmployeeListViewModel()
        self.operation_state = OperationState()
        self.consolidation_vm = ConsolidationViewModel(
            app_controller, operation_state=self.operation_state
        )

        self.employee_vm.set_update_callback(self._on_employee_vm_update)
        self.consolidation_vm.set_callbacks(
            self._on_consolidation_progress, self._on_consolidation_complete
        )

        ThemeManager.apply_styles()
        self.setup_ui()
        # Show operation progress in the window title.
        # Two mechanisms working together:
        # 1. <<TitleUpdate>> virtual event: generated on the GUI thread
        #    from _on_consolidation_progress (which is itself called
        #    from the consolidation worker). The event_generate call is
        #    marshalled via root.after(0, …) so no Tk API is touched
        #    from a worker thread.
        # 2. Fallback poll every 500ms from GUI thread — catches the
        #    edge case where the virtual event is dropped.
        self._base_title = " Master-Report v5.5"

        def _refresh_title(_event=None) -> None:
            try:
                snap = self.operation_state.snapshot()
                if snap.current:
                    label = OPERATION_LABELS.get(snap.current, "")
                    self.root.title(
                        f"{self._base_title} — {label} {snap.progress:.0f}%"
                    )
                else:
                    self.root.title(self._base_title)
            except Exception:
                pass

        self.root.bind("<<TitleUpdate>>", _refresh_title)

        def _poll() -> None:
            _refresh_title()
            self.root.after(500, _poll)
        self.root.after(500, _poll)
        self.operation_state.add_listener(
            lambda snap: self.root.after(0, lambda: self.sidebar.apply(snap))
        )
        # Drive the StatsPanel operation progress bar from the same
        # snapshot stream. The OperationStatusBar reads this too, but
        # the StatsPanel bar lives in a different widget that the
        # listener must update explicitly.
        # Must be marshalled via root.after() — OperationState._notify
        # is called from background threads during consolidation.
        self.operation_state.add_listener(
            lambda snap: self.root.after(
                0, lambda: self._on_operation_snapshot(snap)
            )
        )
        setup_gui_logging(self.log_panel.get_widget())

        self.commands = GuiCommands(
            self.root,
            config_controller,
            app_controller,
            self.employee_vm,
            self.consolidation_vm,
            self.sidebar,
            self.employee_table,
            self.stats_panel,
            self.dashboard_tabs,
            self.operation_state,
            self.toasts,
        )

        self.root.after(100, self.commands.refresh_status)
        # Begin polling the OS theme every 5 s so that when the user
        # has picked "Системная" we follow OS dark/light changes without
        # an app restart. No-op when the user has chosen light/dark.
        self.start_theme_polling()

    def setup_ui(self):
```

```
        self.sidebar = Sidebar(self.root, self)
        self.sidebar.pack(side=tk.LEFT, fill=tk.Y)

        self.main_area = ttk.Frame(self.root, padding="15", style="Main.TFrame")
        self.main_area.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        self.stats_panel = StatsPanel(self.sidebar, self)
        self.stats_panel.pack(fill=tk.X, pady=(0, 10))

        self.log_panel = LogPanel(self.sidebar, self)
        self.log_panel.pack(fill=tk.BOTH, expand=True)

        # Status bar lives at the bottom of the main window (not the
        # sidebar) so the progress indicator is always visible
        # regardless of which tab is open or how tall the window is.
        self.status_bar = OperationStatusBar(
            self.root,
            self.operation_state,
            on_cancel=self._request_cancel,
        )
        self.status_bar.pack(side=tk.BOTTOM, fill=tk.X)

        self.notebook = ttk.Notebook(self.main_area)
        self.notebook.pack(fill=tk.BOTH, expand=True)

        self.dashboard_tabs = DashboardTabs(
            self.notebook,
            self,
            self.config_controller,
        )

        self.employee_table = EmployeeTable(self.dashboard_tabs.employees_tab, self)
        self.employee_table.pack(fill=tk.BOTH, expand=True)
        self.dashboard_tabs.refresh()

        self.toasts = ToastManager(self.root)

    def _on_employee_vm_update(self):
        self.root.after(0, self.commands._sync_employee_ui)

    def _request_cancel(self) -> None:
        if not self.operation_state.is_busy:
            return
        cancelled = self.operation_state.request_cancel()
        if cancelled:
            self.toasts.info("Отмена", "Операция будет остановлена на ближайшей контрольной точке")

    def _on_consolidation_progress(self, progress: float):
        # Called from the consolidation worker thread. OperationState is
        # thread-safe, but ``event_generate`` (and any Tk call) must run
        # on the GUI thread — calling it directly from the worker races
        # the Tcl event loop and corrupts Tk state under load (e.g.
        # several concurrent consolidations + cancel). Marshal it.
        self.operation_state.set_progress(progress)
        self.root.after(0, lambda: self.root.event_generate("<<TitleUpdate>>"))

    def _update_title_from_state(self, _event=None) -> None:
        """Refresh window title from OperationState — always runs on GUI thread."""
        snap = self.operation_state.snapshot()
        if snap.current:
            label = OPERATION_LABELS.get(snap.current, "")
            self.root.title(f"{self._base_title} — {label} {snap.progress:.0f}%")
        else:
            self.root.title(self._base_title)

    def _on_operation_snapshot(self, snap) -> None:
        """Forward the operation progress to the StatsPanel bar."""
        try:
            self.stats_panel.set_operation(snap.progress)
        except (AttributeError, tk.TclError):
            pass

    def _on_consolidation_complete(self, ok: bool, msg: str, path: Optional[Path]):
        def fin():
            self.operation_state.end()
            cancelled = "отменена пользователем" in (msg or "").lower()

            if ok:
                logging.info(f"Успешно: {msg}")
                self.dashboard_tabs.record_result("Master", ok, msg, path)
                if path and messagebox.askyesno(
                    "Успех", "Консолидация завершена. Открыть файл?"
                ):
```

```
                open_path(str(path))
            self.toasts.success("Master-отчет", msg, duration_ms=4000)
        elif cancelled:
            logging.info(f"Отменено: {msg}")
            self.dashboard_tabs.record_result("Master", False, msg, path)
            self.toasts.warning("Master-отчет: отменено", msg, duration_ms=4000)
        else:
            logging.error(f"Ошибка: {msg}")
            self.dashboard_tabs.record_result("Master", ok, msg, path)
            self.toasts.error("Master-отчет: ошибка", msg, duration_ms=6000)

    self.root.after(0, fin)

def refresh_status(self):
    self.commands.refresh_status()

def run_consolidation(self):
    self.commands.run_consolidation()

def run_consolidation_all_dates(self):
    self.commands.run_consolidation_all_dates()

def run_consolidation_month(self):
    self.commands.run_consolidation_month()

def run_rsch_report(self):
    self.commands.run_rsch_report()

def run_rsch_report_month(self):
    self.commands.run_rsch_report_month()

def run_reconciliation_report(self):
    self.commands.run_reconciliation_report()

def run_auto_reconciliation_day(self):
    self.commands.run_auto_reconciliation_day()

def add_employee_dialog(self):
    self.commands.add_employee_dialog()

def attach_file_dialog(self):
    self.commands.attach_file_dialog()

def open_master_folder(self):
    self.commands.open_master_folder()

def copy_missing_to_clipboard(self):
    self.commands.copy_missing_to_clipboard()

def filter_tree(self):
    self.commands.filter_tree()

def open_employee_folder(self):
    self.commands.open_employee_folder()

def open_employee_file(self):
    self.commands.open_employee_file()

def delete_employee(self):
    self.commands.delete_employee()

def _install_geometry_save_handler(self) -> None:
    """Wrap WM_DELETE_WINDOW so we can persist the geometry.

    ``main.py`` sets ``root.protocol("WM_DELETE_WINDOW", on_close)``
    *after* ``GuiMaster.__init__`` runs, which clobbers any protocol
    we install here. As a robust fallback we also bind to the root
    ``<Destroy>`` event — that fires when the root is torn down no
    matter which close path was taken, so the geometry is persisted
    even when ``main.py``'s handler is the one that ran.

    ``_persist_geometry`` is idempotent (guarded by a flag) so it is
    safe for both bindings to fire on the same shutdown.
    """
    self._geometry_saved = False
    original = self.root.protocol("WM_DELETE_WINDOW")

    def _save_then_close() -> None:
        self._persist_geometry()
        if original:
            original()
        else:
            self.root.destroy()
```

```
        self.root.protocol("WM_DELETE_WINDOW", _save_then_close)
        # Fallback for the case where main.py overwrites WM_DELETE_WINDOW
        # after __init__ returns. <Destroy> propagates up from any child
        # widget, so filter to root only.
        self.root.bind(
            "<Destroy>",
            lambda e: self._persist_geometry() if e.widget is self.root else None,
            add="+",
        )

    def _persist_geometry(self) -> None:
        """Save the current window geometry to disk — idempotent."""
        if getattr(self, "_geometry_saved", False):
            return
        try:
            save_geometry(self.root.geometry())
        except Exception as e:  # noqa: BLE001
            logging.debug("Geometry save failed: %s", e)
        finally:
            self._geometry_saved = True

    def save_geometry_and_close(self) -> None:
        """Persist the window geometry and then destroy the root."""
        self._persist_geometry()
        try:
            self.root.destroy()
        except tk.TclError:
            pass

    def start_theme_polling(self) -> None:
        """Begin polling the OS theme every 5 s.

        Only re-applies the theme when the user has selected "Системная"
        in the sidebar AND the resolved OS value has changed since the
        last poll. This avoids touching Tk widgets unnecessarily on every
        tick (which would also fight manual light/dark choices).
        """
        try:
            self._last_system_theme = ThemeManager.resolve_effective("system")
        except Exception:
            self._last_system_theme = "light"
        self._poll_system_theme()

    def _poll_system_theme(self) -> None:
        try:
            theme_var = getattr(self.sidebar, "theme_var", None)
            if theme_var is not None and theme_var.get() == "system":
                current = ThemeManager.resolve_effective("system")
                if current != self._last_system_theme:
                    self._last_system_theme = current
                    ThemeManager.set_theme(current)
                    self.root.configure(bg=ThemeManager.COLORS["bg_main"])
        except (tk.TclError, AttributeError):
            # Root or sidebar destroyed — let the polling loop die
            # quietly.
            return
        try:
            self.root.after(5000, self._poll_system_theme)
        except tk.TclError:
            pass

    def set_theme(self, name: str) -> None:
        """Switch the active palette and reconfigure ttk styles.

        ``name`` may be "light", "dark", or "system" (the last one
        resolves to whichever the OS reports — falling back to light
        on Windows 8). The main window's background is updated to
        match the new palette so the change is visible immediately.
        """
        resolved = ThemeManager.resolve_effective(name)
        if resolved not in ("light", "dark"):
            resolved = "light"
        ThemeManager.set_theme(resolved)
        self.root.configure(bg=ThemeManager.COLORS["bg_main"])
        if hasattr(self, "employee_table"):
            self.employee_table.refresh_theme()
        if hasattr(self, "dashboard_tabs"):
            self.dashboard_tabs.refresh_theme()
        self.toasts.info(
            "Тема",
            f"Применена тема: {resolved}",
            duration_ms=2000,
        )
```

APP\core\ui\operation_state.py

184 lines • 6,086 bytes

```
"""Global operation state machine.

Single source of truth for "is any long-running operation in flight?".
Widgets subscribe via ``add_listener``; the command layer calls
``begin()``/``end()``/``request_cancel()``.

This is the foundation that fixes two user-visible bugs:
- double-clicking two different action buttons causing state corruption
- not knowing what is currently running or how long it will take
"""
from __future__ import annotations

import logging
import threading
import time
from dataclasses import dataclass
from enum import Enum
from typing import Callable, List, Optional

logger = logging.getLogger(__name__)


class OperationType(str, Enum):
    NONE = ""
    REFRESH = "refresh"
    CONSOLIDATION_DAY = "consolidation_day"
    CONSOLIDATION_ALL = "consolidation_all"
    CONSOLIDATION_MONTH = "consolidation_month"
    RSCH_DAY = "rsch_day"
    RSCH_MONTH = "rsch_month"
    RECONCILIATION = "reconciliation"
    AUTO_RECONCILIATION = "auto_reconciliation"


OPERATION_LABELS: dict[OperationType, str] = {
    OperationType.NONE: "",
    OperationType.REFRESH: "Обновление статусов",
    OperationType.CONSOLIDATION_DAY: "Сборка за день",
    OperationType.CONSOLIDATION_ALL: "Сборка за все даты",
    OperationType.CONSOLIDATION_MONTH: "Сборка за период",
    OperationType.RSCH_DAY: "Сборка РСЧ",
    OperationType.RSCH_MONTH: "Сборка РСЧ за период",
    OperationType.RECONCILIATION: "Сверка отчётов",
    OperationType.AUTO_RECONCILIATION: "Автоматическая сверка",
}


@dataclass
class OperationSnapshot:
    current: OperationType
    cancelling: bool
    progress: float
    started_at: Optional[float]
    label: str


class OperationState:
    """Thread-safe singleton-like state machine for long-running operations.

    Listeners are notified after every state transition so widgets can
    re-render (button enable/disable, status bar update, toast).
    """

    def __init__(self) -> None:
        self._lock = threading.Lock()
        self._current: OperationType = OperationType.NONE
        self._cancelling: bool = False
        self._cancel_event: threading.Event = threading.Event()
        self._progress: float = 0.0
        self._started_at: Optional[float] = None
        self._listeners: List[Callable[[OperationSnapshot], None]] = []

    # --- read API ----------------------------------------------------

    def snapshot(self) -> OperationSnapshot:
        with self._lock:
            return OperationSnapshot(
                current=self._current,
                cancelling=self._cancelling,
                progress=self._progress,
                started_at=self._started_at,
                label=OPERATION_LABELS.get(self._current, ""),
            )
```

```
    @property
    def is_busy(self) -> bool:
        with self._lock:
            return self._current != OperationType.NONE

    @property
    def is_cancelling(self) -> bool:
        with self._lock:
            return self._cancelling

    @property
    def current(self) -> OperationType:
        with self._lock:
            return self._current

    @property
    def progress(self) -> float:
        with self._lock:
            return self._progress

    @property
    def started_at(self) -> Optional[float]:
        with self._lock:
            return self._started_at

    @property
    def cancel_event(self) -> threading.Event:
        return self._cancel_event

    # --- write API ---------------------------------------------------------

    def begin(self, op: OperationType) -> bool:
        """Try to start an operation.

        Returns True if started; False if another operation is already running
        (in which case the caller should silently abort the new request).
        """
        with self._lock:
            if self._current != OperationType.NONE:
                logger.warning(
                    "Cannot start %s: %s is already running",
                    op.value,
                    self._current.value,
                )
                return False
            self._current = op
            self._cancelling = False
            self._cancel_event.clear()
            self._progress = 0.0
            self._started_at = time.monotonic()
        self._notify()
        return True

    def end(self) -> None:
        with self._lock:
            if self._current == OperationType.NONE and not self._cancelling:
                return
            self._current = OperationType.NONE
            self._cancelling = False
            self._cancel_event.clear()
            self._progress = 0.0
            self._started_at = None
        self._notify()

    def request_cancel(self) -> bool:
        with self._lock:
            if self._current == OperationType.NONE:
                return False
            self._cancelling = True
            self._cancel_event.set()
        self._notify()
        return True

    def set_progress(self, value: float) -> None:
        with self._lock:
            if self._current == OperationType.NONE:
                return
            self._progress = max(0.0, min(100.0, value))
        self._notify()

    # --- listener API ------------------------------------------------------

    def add_listener(self, fn: Callable[[OperationSnapshot], None]) -> None:
```

```
        with self._lock:
            self._listeners.append(fn)

    def remove_listener(self, fn: Callable[[OperationSnapshot], None]) -> None:
        with self._lock:
            try:
                self._listeners.remove(fn)
            except ValueError:
                pass

    def _notify(self) -> None:
        snap = self.snapshot()
        for fn in list(self._listeners):
            try:
                fn(snap)
            except Exception:
                logger.exception("operation_state listener raised")
```

APP\core\ui\operation_status_bar.py

174 lines • 5,885 bytes

```
"""Status bar widget showing the active operation with progress, elapsed time, and cancel.

Addresses the user pain point: "не понятно сколько программа еще будет работать".
The bar shows:
  - operation name (Russian label from OperationState)
  - progress percentage
  - elapsed time
  - linear ETA estimate
  - Cancel button (visible only while busy)

Sits at the bottom of the sidebar, replacing the single-purpose StatsPanel progress.
"""
from __future__ import annotations

import time
import tkinter as tk
from tkinter import ttk
from typing import Callable, Optional

from core.ui.operation_state import (
    OPERATION_LABELS,
    OperationState,
    OperationSnapshot,
    OperationType,
)


def _format_duration(seconds: float) -> str:
    """Format seconds as M:SS (e.g. 0:42, 12:30, 1:05:30 for >1h)."""
    if seconds < 0:
        seconds = 0
    s = int(seconds)
    if s < 3600:
        return f"{s // 60}:{s % 60:02d}"
    return f"{s // 3600}:{(s % 3600) // 60:02d}:{s % 60:02d}"


class OperationStatusBar(ttk.Frame):
    """Bottom-of-sidebar status bar showing the running operation."""

    def __init__(
        self,
        parent,
        operation_state: OperationState,
        on_cancel: Callable[[], None],
    ) -> None:
        super().__init__(parent, style="StatusBar.TFrame", padding=(6, 4))
        self._operation_state = operation_state
        self._on_cancel = on_cancel

        self._label_var = tk.StringVar(value="")
        self._progress_var = tk.DoubleVar(value=0.0)
        self._elapsed_var = tk.StringVar(value="")
        self._eta_var = tk.StringVar(value="")

        self._title_label = ttk.Label(
            self,
            textvariable=self._label_var,
            style="StatusBar.TLabel",
            font=("Segoe UI", 9, "bold"),
            anchor=tk.W,
        )
        self._title_label.pack(fill=tk.X, anchor=tk.W)

        row = ttk.Frame(self, style="StatusBar.TFrame")
        row.pack(fill=tk.X, pady=(2, 0))

        self._progress_bar = ttk.Progressbar(
            row,
            orient=tk.HORIZONTAL,
            mode="determinate",
            maximum=100.0,
            variable=self._progress_var,
        )
        self._progress_bar.pack(side=tk.LEFT, fill=tk.X, expand=True)

        self._pct_label = ttk.Label(
            row,
            text="0%",
            style="StatusBar.TLabel",
            width=5,
            anchor=tk.E,
        )
```

```
        self._pct_label.pack(side=tk.LEFT, padx=(4, 0))

        timings = ttk.Frame(self, style="StatusBar.TFrame")
        timings.pack(fill=tk.X, pady=(2, 0))

        self._elapsed_label = ttk.Label(
            timings,
            textvariable=self._elapsed_var,
            style="StatusBar.TLabel",
            font=("Consolas", 9),
        )
        self._elapsed_label.pack(side=tk.LEFT)

        self._eta_label = ttk.Label(
            timings,
            textvariable=self._eta_var,
            style="StatusBar.TLabel",
            font=("Segoe UI", 8),
        )
        self._eta_label.pack(side=tk.RIGHT)

        self._cancel_btn = ttk.Button(
            self,
            text=" Отмена",
            style="Danger.TButton",
            command=self._on_cancel_clicked,
            width=12,
        )
        self._cancel_btn.pack(fill=tk.X, pady=(4, 0))

        self._operation_state.add_listener(self._on_snapshot)
        self._render(OperationType.NONE, 0.0, None, False)

    def _on_cancel_clicked(self) -> None:
        try:
            self._on_cancel()
        except Exception:
            pass

    def _on_snapshot(self, snap: OperationSnapshot) -> None:
        # ``self.after`` raises ``TclError`` once the widget is torn down.
        # If that happens, unsubscribe from OperationState so we stop
        # receiving snapshots (otherwise every future notify would
        # schedule another doomed ``after`` call and log a TclError).
        try:
            self.after(0, lambda: self._render(
                snap.current, snap.progress, snap.started_at, snap.cancelling
            ))
        except tk.TclError:
            self._operation_state.remove_listener(self._on_snapshot)

    def _render(
        self,
        current: OperationType,
        progress: float,
        started_at: Optional[float],
        cancelling: bool,
    ) -> None:
        busy = current is not OperationType.NONE
        if not busy:
            self._label_var.set("Готово к работе")
            self._progress_var.set(0.0)
            self._elapsed_var.set("")
            self._eta_var.set("")
            self._pct_label.config(text="")
            self._cancel_btn.config(state="disabled")
            return

        if cancelling:
            label = f" {OPERATION_LABELS.get(current, '')} (отмена...)"
        else:
            label = f" {OPERATION_LABELS.get(current, '')}"
        self._label_var.set(label)
        self._progress_var.set(progress)
        self._pct_label.config(text=f"{progress:.0f}%")

        if started_at is not None:
            elapsed = time.monotonic() - started_at
            self._elapsed_var.set(f" {_format_duration(elapsed)}")
            if progress > 1.0 and progress < 100.0:
                eta = elapsed / progress * (100.0 - progress)
                self._eta_var.set(f"~ {_format_duration(eta)} осталось")
            elif progress >= 100.0:
```

```
            self._eta_var.set("завершено")
        else:
            self._eta_var.set("")
    else:
        self._elapsed_var.set("")
        self._eta_var.set("")

    self._cancel_btn.config(state="disabled" if cancelling else "normal")
```

APP\core\ui\system_theme.py

164 lines • 5,387 bytes

```
"""Detect whether the OS is currently in dark mode.

Supports Windows 10/11 (registry query), macOS (osascript heuristic),
and Linux (gsettings heuristic for GNOME/Unity). On Windows 8 the
AppsUseLightTheme registry key does not exist, so the function returns
None and the caller is expected to fall back to a manual user choice
rather than guessing.

This module deliberately does not import any GUI toolkit; it is safe
to call from headless tests.
"""
from __future__ import annotations

import logging
import platform
import shutil
import subprocess
from typing import Optional

_logger = logging.getLogger(__name__)


def _windows_apps_use_light() -> Optional[bool]:
    """Read the Windows 10/11 AppsUseLightTheme registry value.

    Returns True if the system is in light mode, False for dark mode,
    None if the value is missing (typical of Windows 8/8.1) or the
    registry is inaccessible.
    """
    try:
        import winreg  # type: ignore[import-not-found]
    except ImportError:
        return None

    registry_path = r"Software\Microsoft\Windows\CurrentVersion\Themes\Personalize"
    value_name = "AppsUseLightTheme"

    for root in (winreg.HKEY_CURRENT_USER, winreg.HKEY_LOCAL_MACHINE):
        try:
            with winreg.OpenKey(root, registry_path) as key:
                value, _ = winreg.QueryValueEx(key, value_name)
                return bool(value)
        except FileNotFoundError:
            # Path or value missing — try the next root, or fall through.
            continue
        except OSError as e:
            _logger.debug("Registry query failed for %s: %s", root, e)
            return None
    return None


def _macos_appearance() -> Optional[bool]:
    """Return True for light, False for dark, None if unknown.

    Uses AppleScript which is universally available on macOS. Errors
    (no AppleScript, headless build agent) are swallowed and reported
    as unknown.
    """
    if not shutil.which("osascript"):
        return None
    try:
        result = subprocess.run(
            ["osascript", "-e", "tell application \"System Events\" to tell appearance preferences to get dark mode"],
            capture_output=True,
            text=True,
            timeout=2,
        )
        if result.returncode != 0:
            return None
        out = result.stdout.strip().lower()
        if "true" in out:
            return False
        if "false" in out:
            return True
        return None
    except (subprocess.TimeoutExpired, OSError) as e:
        _logger.debug("macOS appearance query failed: %s", e)
        return None


def _linux_gnome_dark() -> Optional[bool]:
    """Return True for light, False for dark, None if unknown.
```

```
    Only handles gsettings / GNOME-style desktop environments.
    """
    if not shutil.which("gsettings"):
        return None
    try:
        result = subprocess.run(
            ["gsettings", "get", "org.gnome.desktop.interface", "color-scheme"],
            capture_output=True,
            text=True,
            timeout=2,
        )
        if result.returncode != 0:
            return None
        out = result.stdout.strip().lower()
        if "dark" in out:
            return False
        if "default" in out or "light" in out or out == "''":
            return True
        return None
    except (subprocess.TimeoutExpired, OSError) as e:
        _logger.debug("gsettings query failed: %s", e)
        return None


def detect_system_theme() -> Optional[str]:
    """Return "light", "dark", or None when undetectable.

    None means the OS either does not publish a theme preference
    (Windows 8/8.1) or this code could not read it. Callers should
    treat None as "ask the user".
    """
    system = platform.system().lower()

    if system == "windows":
        is_light = _windows_apps_use_light()
        if is_light is None:
            _logger.info(
                "Windows theme preference not available "
                "(likely Windows 8/8.1). User must choose."
            )
            return None
        return "light" if is_light else "dark"

    if system == "darwin":
        is_light = _macos_appearance()
        if is_light is None:
            return None
        return "light" if is_light else "dark"

    if system == "linux":
        is_light = _linux_gnome_dark()
        if is_light is None:
            return None
        return "light" if is_light else "dark"

    return None


def is_windows_8_or_earlier() -> bool:
    """Return True when running on Windows 8/8.1 or earlier.

    Used to decide whether to surface the "auto-follow system" option
    in the UI: on Win 8 the registry lookup will always return None,
    so the user must pick manually.
    """
    if platform.system().lower() != "windows":
        return False
    # ``platform.windows_version`` was deprecated in 3.12 and removed
    # in 3.13. ``platform.release`` returns a string like ``"10"`` or
    # ``"8"`` that is good enough for the "≤Win 8" check.
    try:
        major = int(platform.release().split(".", 1)[0])
    except ValueError:
        return False
    return major <= 8  # Win 7, Win 8, Win 8.1


__all__ = [
    "detect_system_theme",
    "is_windows_8_or_earlier",
]
```

APP\core\ui\theme.py

649 lines • 22,906 bytes

```
"""Centralised colour palette and ttk style configuration.

Exposes two palettes (light + dark) and a runtime ``set_theme`` that
re-applies the ttk styles so widgets that are created later pick up
the active palette. Widgets already in the tree pick up the changes
on the next ``configure`` call, which the GuiMaster triggers after
the user switches themes.

Design tokens
-------------
- ``COLORS``    : active palette (dict of hex strings).
- ``FONTS``     : typography hierarchy (display / heading / body / ...).
- ``SPACING``   : 4-step spacing scale (xs/sm/md/lg/xl/xxl).
- ``animate_background`` : classmethod helper for smooth bg transitions.
"""
from __future__ import annotations

import tkinter as tk
from tkinter import ttk
from typing import ClassVar

# ---------------------------------------------------------------------------
# Palettes
# ---------------------------------------------------------------------------
# All pre-existing keys (primary, primary_active, success, success_active,
# danger, warning, info, info_active, bg_main, bg_sidebar, bg_panel, trough,
# text, text_inverse, accent, tooltip_bg, tooltip_fg) are preserved verbatim
# so existing call sites keep working. New refined keys are appended:
#   danger_active, text_muted, border_subtle, border_default,
#   success_subtle, danger_subtle, warning_subtle, info_subtle.
_LIGHT = {
    # --- accent / semantic ---
    "primary": "#2563eb",
    "primary_active": "#1d4ed8",
    "success": "#16a34a",
    "success_active": "#15803d",
    "danger": "#dc2626",
    "danger_active": "#b91c1c",
    "warning": "#d97706",
    "info": "#0891b2",
    "info_active": "#0e7490",
    "accent": "#2563eb",
    # --- backgrounds (warm off-white) ---
    "bg_main": "#f3f4f6",
    "bg_sidebar": "#ffffff",
    "bg_panel": "#e9ecef",
    "trough": "#cbd5e1",
    # --- text ---
    "text": "#334155",
    "text_muted": "#64748b",
    "text_inverse": "#ffffff",
    # --- borders ---
    "border_subtle": "#e2e8f0",
    "border_default": "#cbd5e1",
    # --- subtle semantic fills (badges / cards) ---
    "success_subtle": "#dcfce7",
    "danger_subtle": "#fee2e2",
    "warning_subtle": "#fef3c7",
    "info_subtle": "#e0f2fe",
    # --- misc ---
    "tooltip_bg": "#0f172a",
    "tooltip_fg": "#e2e8f0",
}

_DARK = {
    # --- accent / semantic ---
    "primary": "#60a5fa",
    "primary_active": "#93c5fd",
    "success": "#4ade80",
    "success_active": "#86efac",
    "danger": "#f87171",
    "danger_active": "#fca5a5",
    "warning": "#fbbf24",
    "info": "#22d3ee",
    "info_active": "#67e8f9",
    "accent": "#60a5fa",
    # --- backgrounds (rich charcoal) ---
    "bg_main": "#0f172a",
    "bg_sidebar": "#1e293b",
    "bg_panel": "#334155",
    "trough": "#334155",
    # --- text ---
    "text": "#e2e8f0",
```

```
    "text_muted": "#94a3b8",
    "text_inverse": "#0f172a",
    # --- borders ---
    "border_subtle": "#334155",
    "border_default": "#475569",
    # --- subtle semantic fills ---
    "success_subtle": "#052e16",
    "danger_subtle": "#450a0a",
    "warning_subtle": "#422006",
    "info_subtle": "#082f49",
    # --- misc ---
    "tooltip_bg": "#020617",
    "tooltip_fg": "#e2e8f0",
}


def _lerp_hex(a: str, b: str, t: float) -> str:
    """Linearly interpolate between two ``#rrggbb`` colours (0<=t<=1)."""
    ar, ag, ab = int(a[1:3], 16), int(a[3:5], 16), int(a[5:7], 16)
    br, bg, bb = int(b[1:3], 16), int(b[3:5], 16), int(b[5:7], 16)
    r = round(ar + (br - ar) * t)
    g = round(ag + (bg - ag) * t)
    bl = round(ab + (bb - ab) * t)
    return f"#{r:02x}{g:02x}{bl:02x}"


class ThemeManager:
    """Static palette + ttk style configuration."""

    _PALETTES: ClassVar[dict[str, dict[str, str]]] = {
        "light": _LIGHT,
        "dark": _DARK,
    }

    COLORS: ClassVar[dict[str, str]] = dict(_LIGHT)
    _current: ClassVar[str] = "light"
    _scale: ClassVar[float] = 1.0
    _scaled_fonts: ClassVar[dict[str, tuple]] = {}

    # Typography hierarchy. "Segoe UI Semibold" is not universally available
    # on Windows; ("Segoe UI", N, "bold") is, and visually close enough.
    _BASE_FONTS: ClassVar[dict[str, tuple]] = {
        "display": ("Segoe UI", 14, "bold"),
        "heading": ("Segoe UI", 12, "bold"),
        "body": ("Segoe UI", 10),
        "body_bold": ("Segoe UI", 10, "bold"),
        "caption": ("Segoe UI", 9),
        "mono": ("Consolas", 10),
        "button": ("Segoe UI", 10),
    }
    FONTS: ClassVar[dict[str, tuple]] = dict(_BASE_FONTS)

    # 4-step spacing scale (pixels). Other modules may reference
    # ThemeManager.SPACING["md"] instead of hardcoded numbers.
    _BASE_SPACING: ClassVar[dict[str, int]] = {
        "xs": 4,
        "sm": 8,
        "md": 12,
        "lg": 16,
        "xl": 24,
        "xxl": 32,
    }
    SPACING: ClassVar[dict[str, int]] = dict(_BASE_SPACING)

    # Widgets that use explicit ``background=ThemeManager.COLORS[...]``
    # (tk.Canvas, tk.Text, DateEntry, ...) do not pick up ttk style
    # changes automatically. Register them here so ``set_theme`` can
    # re-apply the current palette's colours to each. Entries are
    # ``(widget, bg_key, fg_key)`` tuples; destroyed widgets are pruned
    # lazily on the next ``set_theme`` call.
    _themed_widgets: ClassVar[list] = []

    @classmethod
    def register(cls, widget, *, bg_key: str = "bg_panel", fg_key: str = "text") -> None:
        """Track a non-ttk widget so its colours follow theme switches."""
        cls._themed_widgets.append((widget, bg_key, fg_key))

    @classmethod
    def init_scaling(cls, root: tk.Tk) -> None:
        """Calculate DPI scale factor and rebuild FONTS/SPACING accordingly.

        Must be called once after ``root = tk.Tk()`` so that Tk's
        ``winfo_fpixels`` reports the actual monitor DPI. On 100% DPI
```

```
        the scale is 1.0; on 150% it's ~1.5, etc.
        """
        try:
            dpi = root.winfo_fpixels("1i")
            cls._scale = max(1.0, round(dpi / 96.0 * 2) / 2)
        except Exception:
            cls._scale = 1.0
        cls._rebuild_fonts()
        cls._rebuild_spacing()

    @classmethod
    def _rebuild_fonts(cls) -> None:
        s = cls._scale
        cls._scaled_fonts = {}
        for key, val in cls._BASE_FONTS.items():
            font_name = val[0]
            font_size = max(8, int(val[1] * s))
            font_opts = val[2:]
            cls._scaled_fonts[key] = (font_name, font_size, *font_opts)
        cls.FONTS = cls._scaled_fonts

    @classmethod
    def _rebuild_spacing(cls) -> None:
        s = cls._scale
        cls.SPACING = {k: max(2, int(v * s)) for k, v in cls._BASE_SPACING.items()}

    @classmethod
    def current_theme(cls) -> str:
        return cls._current

    @classmethod
    def set_theme(cls, name: str) -> None:
        """Switch the palette AND re-apply ttk styles.

        Unknown theme names raise ``ValueError`` — the caller's UI
        should only offer choices that have been registered.
        """
        if name not in cls._PALETTES:
            raise ValueError(
                f"Unknown theme {name!r}. Known: {sorted(cls._PALETTES)}"
            )
        cls._current = name
        cls.COLORS = dict(cls._PALETTES[name])
        try:
            cls.apply_styles()
        except Exception:
            # If there is no Tk yet (e.g. unit test importing the module),
            # the styles are still in place and the next GUI boot will
            # call apply_styles() with the right palette active.
            pass
        # Re-apply explicit colours to non-ttk widgets (Canvas, Text,
        # DateEntry, ...). Destroyed widgets are pruned silently.
        for entry in list(cls._themed_widgets):
            try:
                widget, bg_key, fg_key = entry
                widget.configure(
                    background=cls.COLORS[bg_key], foreground=cls.COLORS[fg_key]
                )
            except Exception:
                try:
                    cls._themed_widgets.remove(entry)
                except ValueError:
                    pass

    @classmethod
    def palette(cls) -> dict[str, str]:
        """Return a fresh copy of the active palette."""
        return dict(cls._PALETTES[cls._current])

    @classmethod
    def resolve_effective(cls, preference: str) -> str:
        """Resolve "system" to the actual light/dark value.

        Used by the GUI when the user has "follow system" selected.
        On Windows 8 the registry key is absent, so we fall back to
        the light theme and let the user switch manually.
        """
        if preference != "system":
            return preference
        # Local import: avoid pulling in GUI/registry code at module
        # import time during tests.
        from core.ui.system_theme import detect_system_theme
```

```
        return detect_system_theme() or "light"

    # ------------------------------------------------------------------
    # Smooth background animation helper
    # ------------------------------------------------------------------
    @classmethod
    def animate_background(
        cls,
        widget,
        target: str,
        steps: int = 8,
        delay_ms: int = 16,
    ) -> None:
        """Interpolate ``widget`` background to ``target`` over ~``steps*delay_ms`` ms.

        Cancels any in-flight animation for the same widget before starting
        a new one (avoids flicker when Enter/Leave fire in quick succession).
        Intended for plain ``tk`` widgets (Frame, Label, ...). ttk widgets
        ignore ``background=`` for the most part — callers should pick
        appropriate targets. Best-effort: silently bails on any Tcl error.
        """
        try:
            current = widget.cget("background")
        except Exception:
            return
        # Cancel pending animation for this widget, if any.
        prev = getattr(widget, "_theme_anim_ids", None)
        if prev:
            for aid in prev:
                try:
                    widget.after_cancel(aid)
                except Exception:
                    pass
        ids: list[str] = []

        def _step(i: int) -> None:
            if i > steps:
                return
            t = i / steps
            try:
                widget.configure(background=_lerp_hex(current, target, t))
            except Exception:
                return

        for i in range(1, steps + 1):
            try:
                ids.append(widget.after(delay_ms * i, _step, i))
            except Exception:
                return
        try:
            widget._theme_anim_ids = ids  # type: ignore[attr-defined]
        except Exception:
            pass

    # ------------------------------------------------------------------
    # ttk style configuration
    # ------------------------------------------------------------------
    @classmethod
    def apply_styles(cls) -> None:
        c = cls.COLORS
        f = cls.FONTS
        s = cls._scale
        sp = cls.SPACING
        style = ttk.Style()
        try:
            style.theme_use("clam")
        except Exception:
            # Some custom themes may not exist on a system. The default
            # 'clam' is bundled with Tk; if even that fails we leave
            # whatever theme is active so the app still starts.
            pass

        # --- Frames ---------------------------------------------------
        style.configure("Main.TFrame", background=c["bg_main"])
        style.configure("Sidebar.TFrame", background=c["bg_sidebar"], relief="flat")

        # --- Labels ---------------------------------------------------
        style.configure(
            "TLabel",
            background=c["bg_main"],
            foreground=c["text"],
            font=f["body"],
        )
```

```
    style.configure(
        "Sidebar.TLabel",
        background=c["bg_sidebar"],
        foreground=c["text"],
        font=f["body"],
    )
    style.configure(
        "Header.TLabel",
        font=f["display"],
        foreground=c["primary"],
        background=c["bg_main"],
    )

    # --- Buttons (base + variants) ----------------------------------
    # Flat, breathable padding. Hover handled via style.map (instant,
    # reliable). Smooth animation is available via animate_background
    # for non-ttk widgets (toast frames, etc.).
    style.configure(
        "TButton",
        font=f["button"],
        padding=(sp["md"], sp["sm"]),
        relief="flat",
        borderwidth=0,
        background=c["bg_panel"],
        foreground=c["text"],
    )
    style.map(
        "TButton",
        background=[
            ("active", c["primary"]),
            ("disabled", c["bg_panel"]),
        ],
        foreground=[
            ("active", c["text_inverse"]),
            ("disabled", c["text_muted"]),
        ],
    )
    # Primary action (was: Action.TButton). Keep exact name.
    style.configure(
        "Action.TButton",
        font=f["button"],
        padding=(sp["md"], sp["sm"]),
        relief="flat",
        borderwidth=0,
        background=c["primary"],
        foreground=c["text_inverse"],
    )
    style.map(
        "Action.TButton",
        background=[("active", c["primary_active"]), ("disabled", c["bg_panel"])],
        foreground=[("disabled", c["text_muted"])],
    )
    # Success (positive actions).
    style.configure(
        "Success.TButton",
        font=f["button"],
        padding=(sp["md"], sp["sm"]),
        relief="flat",
        borderwidth=0,
        background=c["success"],
        foreground=c["text_inverse"],
    )
    style.map(
        "Success.TButton",
        background=[("active", c["success_active"]), ("disabled", c["bg_panel"])],
        foreground=[("disabled", c["text_muted"])],
    )
    # Audit (informational, was: info/blue).
    style.configure(
        "Audit.TButton",
        font=f["button"],
        padding=(sp["md"], sp["sm"]),
        relief="flat",
        borderwidth=0,
        background=c["info"],
        foreground=c["text_inverse"],
    )
    style.map(
        "Audit.TButton",
        background=[("active", c["info_active"]), ("disabled", c["bg_panel"])],
        foreground=[("disabled", c["text_muted"])],
    )
    # Danger (destructive actions like delete).
```

```
    style.configure(
        "Danger.TButton",
        font=f["button"],
        padding=(sp["md"], sp["sm"]),
        relief="flat",
        borderwidth=0,
        background=c["danger"],
        foreground=c["text_inverse"],
    )
    style.map(
        "Danger.TButton",
        background=[("active", c["danger_active"]), ("disabled", c["bg_panel"])],
        foreground=[("disabled", c["text_muted"])],
    )
    # Ghost (transparent secondary; border surfaces on hover).
    style.configure(
        "Ghost.TButton",
        font=f["button"],
        padding=(sp["md"], sp["sm"]),
        relief="flat",
        borderwidth=0,
        background=c["bg_main"],
        foreground=c["text"],
    )
    style.map(
        "Ghost.TButton",
        background=[
            ("active", c["bg_panel"]),
            ("disabled", c["bg_main"]),
        ],
        foreground=[
            ("active", c["primary"]),
            ("disabled", c["text_muted"]),
        ],
    )

    # --- LabelFrame / Card ------------------------------------------
    # Modern flat card: subtle border, muted label (not primary —
    # muted reads as a refined card title rather than a heading).
    style.configure(
        "TLabelframe",
        background=c["bg_sidebar"],
        relief="flat",
        borderwidth=1,
        bordercolor=c["border_subtle"],
    )
    style.configure(
        "TLabelframe.Label",
        background=c["bg_sidebar"],
        foreground=c["text_muted"],
        font=f["heading"],
    )
    # Card variant — same look, explicit so callers can opt in.
    style.configure(
        "Card.TLabelframe",
        background=c["bg_sidebar"],
        relief="flat",
        borderwidth=1,
        bordercolor=c["border_subtle"],
    )
    style.configure(
        "Card.TLabelframe.Label",
        background=c["bg_sidebar"],
        foreground=c["text_muted"],
        font=f["heading"],
    )

    # --- Treeview ---------------------------------------------------
    style.configure(
        "Treeview",
        font=f["body"],
        rowheight=int(32 * s),
        background=c["bg_sidebar"],
        fieldbackground=c["bg_sidebar"],
        foreground=c["text"],
        bordercolor=c["border_subtle"],
        borderwidth=0,
    )
    style.configure(
        "Treeview.Heading",
        font=f["body_bold"],
        background=c["bg_panel"],
        foreground=c["text_muted"],
```

```
        relief="flat",
        borderwidth=0,
        padding=(sp["sm"], int(6 * s)),
    )
    style.map(
        "Treeview",
        background=[("selected", c["primary"])],
        foreground=[("selected", c["text_inverse"])],
    )
    style.map(
        "Treeview.Heading",
        background=[("active", c["border_default"])],
    )

    # --- Entry / Combobox ---------------------------------------------
    style.configure(
        "TEntry",
        fieldbackground=c["bg_sidebar"],
        background=c["bg_sidebar"],
        foreground=c["text"],
        insertcolor=c["primary"],
        bordercolor=c["border_default"],
        lightcolor=c["border_default"],
        darkcolor=c["border_default"],
        selectbackground=c["primary"],
        selectforeground=c["text_inverse"],
        padding=(sp["xs"], int(4 * s)),
    )
    style.map(
        "TEntry",
        bordercolor=[("focus", c["primary"])],
        lightcolor=[("focus", c["primary"])],
        darkcolor=[("focus", c["primary"])],
    )
    style.configure(
        "TCombobox",
        fieldbackground=c["bg_sidebar"],
        background=c["bg_sidebar"],
        foreground=c["text"],
        arrowcolor=c["text_muted"],
        bordercolor=c["border_default"],
        lightcolor=c["border_default"],
        darkcolor=c["border_default"],
        selectbackground=c["primary"],
        selectforeground=c["text_inverse"],
        padding=(sp["xs"], int(4 * s)),
    )
    style.map(
        "TCombobox",
        fieldbackground=[("readonly", c["bg_sidebar"])],
        foreground=[("readonly", c["text"])],
        bordercolor=[("focus", c["primary"])],
        lightcolor=[("focus", c["primary"])],
        darkcolor=[("focus", c["primary"])],
        arrowcolor=[("active", c["primary"])],
    )

    # --- Scrollbars (thinner, subtler) --------------------------------
    for orient in ("Vertical", "Horizontal"):
        style.configure(
            f"{orient}.TScrollbar",
            background=c["border_default"],
            troughcolor=c["bg_main"],
            arrowcolor=c["text_muted"],
            bordercolor=c["bg_main"],
            relief="flat",
            width=int(10 * s) if orient == "Vertical" else int(10 * s),
        )
        style.map(
            f"{orient}.TScrollbar",
            background=[("active", c["text_muted"])],
            arrowcolor=[("active", c["text"])],
        )

    # --- Notebook / Tabs ----------------------------------------------
    style.configure(
        "TNotebook",
        background=c["bg_main"],
        borderwidth=0,
        tabmargins=(0, 4, 0, 0),
    )
    style.configure(
        "TNotebook.Tab",
```

```
            font=f["body"],
            background=c["bg_main"],
            foreground=c["text_muted"],
            padding=(sp["lg"], int(10 * s)),
            borderwidth=0,
        )
        style.map(
            "TNotebook.Tab",
            background=[
                ("selected", c["bg_sidebar"]),
                ("active", c["bg_panel"]),
            ],
            foreground=[
                ("selected", c["primary"]),
                ("active", c["text"]),
            ],
        )

        # --- Checkbutton / Radiobutton -----------------------------------
        style.configure(
            "TCheckbutton",
            background=c["bg_sidebar"],
            foreground=c["text"],
            font=f["body"],
        )
        style.map(
            "TCheckbutton",
            background=[("active", c["bg_sidebar"])],
            foreground=[("disabled", c["text_muted"])],
        )
        style.configure(
            "TRadiobutton",
            background=c["bg_sidebar"],
            foreground=c["text"],
            font=f["body"],
        )
        style.map(
            "TRadiobutton",
            background=[("active", c["bg_sidebar"])],
            foreground=[("disabled", c["text_muted"])],
        )

        # --- Progressbar (slimmer) ---------------------------------------
        style.configure(
            "TProgressbar",
            thickness=int(12 * s),
            troughcolor=c["bg_panel"],
            background=c["primary"],
            bordercolor=c["border_subtle"],
            lightcolor=c["primary"],
            darkcolor=c["primary"],
        )

        # --- Status bar --------------------------------------------------
        style.configure(
            "StatusBar.TFrame",
            background=c["bg_panel"],
            relief="flat",
        )
        style.configure(
            "StatusBar.TLabel",
            background=c["bg_panel"],
            foreground=c["text_muted"],
            font=f["caption"],
        )


__all__ = ["ThemeManager"]
```

APP\core\ui\toast.py

396 lines • 14,601 bytes

```
"""Non-modal toast notifications.

Replaces blocking ``messagebox.showinfo`` / ``showerror`` for non-critical
notifications. Stack of up to 3 toasts in the bottom-right corner of the
root window, each fading out after ``duration_ms``.

Each toast is a borderless ``tk.Toplevel`` so it can fade and slide
independently of the root window's alpha (Tkinter Frames do not support
per-widget alpha). On platforms where ``-alpha`` is unsupported (some
X11 builds), the fade is skipped and the toast appears at full opacity;
the slide-in still runs because it only depends on window position.

For notifications that genuinely require user choice (e.g. "open the
finished file?"), keep using ``messagebox.askyesno``.
"""
from __future__ import annotations

import logging
import tkinter as tk
from dataclasses import dataclass
from enum import Enum
from typing import Optional

logger = logging.getLogger(__name__)


class ToastLevel(str, Enum):
    INFO = "info"
    SUCCESS = "success"
    WARNING = "warning"
    ERROR = "error"


_LEVEL_ICONS = {
    ToastLevel.INFO: "",
    ToastLevel.SUCCESS: "",
    ToastLevel.WARNING: "",
    ToastLevel.ERROR: "",
}

# Level → (bg_key, fg_key) into ThemeManager.COLORS. Resolved at ``show``
# time so a theme switch picks up the new palette immediately.
_LEVEL_THEME_KEYS = {
    ToastLevel.INFO: ("info", "text_inverse"),
    ToastLevel.SUCCESS: ("success", "text_inverse"),
    ToastLevel.WARNING: ("warning", "text"),
    ToastLevel.ERROR: ("danger", "text_inverse"),
}

_FADE_IN_STEPS = 8       # ~150 ms total (8 × 18 ms)
_FADE_OUT_STEPS = 10     # ~180 ms total (10 × 18 ms)
_FADE_DELAY_MS = 18
_SLIDE_PX = 30           # subtle slide-in from the right edge


def _darken(hex_color: str, factor: float = 0.88) -> str:
    """Darken a ``#rrggbb`` color by ``factor`` (0..1). Best-effort."""
    try:
        h = hex_color.lstrip("#")
        r, g, b = (int(h[i : i + 2], 16) for i in (0, 2, 4))
        r = max(0, min(255, int(r * factor)))
        g = max(0, min(255, int(g * factor)))
        b = max(0, min(255, int(b * factor)))
        return f"#{r:02x}{g:02x}{b:02x}"
    except (ValueError, IndexError):
        return hex_color


@dataclass
class _Toast:
    level: ToastLevel
    title: str
    message: str
    toplevel: tk.Toplevel
    target_x: int = 0
    target_y: int = 0
    auto_after_id: Optional[str] = None      # auto-dismiss timer
    fade_after_id: Optional[str] = None      # active fade-step timer
    fading_out: bool = False
    fade_step: int = 0
    alpha_supported: bool = True
    pending_fade: bool = True                # False once _tick_fade has run
```

```
class ToastManager:
    """Stack of non-modal toast popups anchored to a Tk root."""

    def __init__(self, root: tk.Misc, max_stack: int = 3) -> None:
        self._root = root
        self._max_stack = max_stack
        self._stack: list[_Toast] = []
        # Cancel all pending animations when the app closes so we don't
        # leave orphaned ``after`` callbacks trying to touch a dead root.
        try:
            self._root.bind("<Destroy>", self._cleanup_all, add="+")
        except tk.TclError:
            pass

    # ------------------------------------------------------------------ public API

    def show(
        self,
        level: ToastLevel,
        title: str,
        message: str,
        duration_ms: int = 4000,
    ) -> None:
        """Show a toast. Older toasts above ``max_stack`` are dismissed early."""
        try:
            from core.ui.theme import ThemeManager

            colors = ThemeManager.COLORS
            bg_key, fg_key = _LEVEL_THEME_KEYS[level]
            bg = colors[bg_key]
            fg = colors[fg_key]
            icon = _LEVEL_ICONS[level]
            border = colors.get("border_subtle", colors.get("trough", "#ced4da"))
        except (KeyError, tk.TclError):
            return

        # Make room — dismiss oldest visible toasts beyond max_stack.
        visible = [t for t in self._stack if not t.fading_out]
        while len(visible) >= self._max_stack:
            self._dismiss(visible[0], early=True)
            visible = [t for t in self._stack if not t.fading_out]

        toplevel = tk.Toplevel(self._root)
        toplevel.wm_overrideredirect(True)
        try:
            toplevel.configure(bg=bg)
        except tk.TclError:
            pass

        # Initial alpha = 0 to prevent a flash before the fade-in starts.
        # On platforms that don't support per-window alpha (some X11 builds),
        # this raises TclError — we record that and skip further alpha animation.
        alpha_supported = True
        try:
            toplevel.wm_attributes("-alpha", 0.0)
        except tk.TclError:
            alpha_supported = False

        # Park the toplevel off-screen so the mapping that happens during
        # ``_repack``'s ``update_idletasks()`` is invisible on platforms
        # without alpha (otherwise the WM default position flashes briefly).
        try:
            toplevel.wm_geometry("+-10000+-10000")
        except tk.TclError:
            pass

        # Inner frame carries the 1px highlight border + content padding.
        inner = tk.Frame(
            toplevel,
            bg=bg,
            padx=16,
            pady=10,
            highlightthickness=1,
            highlightbackground=border,
            highlightcolor=border,
        )
        inner.pack(fill="both", expand=True)

        title_label = tk.Label(
            inner,
            text=f"{icon}  {title}",
            bg=bg,
            fg=fg,
```

```
            font=("Segoe UI", 10, "bold"),
            anchor="w",
        )
        title_label.pack(fill="x")

        if message:
            msg_label = tk.Label(
                inner,
                text=message,
                bg=bg,
                fg=fg,
                font=("Segoe UI", 9),
                anchor="w",
                wraplength=360,
                justify="left",
            )
            msg_label.pack(fill="x", pady=(2, 0))

        toast = _Toast(
            level=level,
            title=title,
            message=message,
            toplevel=toplevel,
            alpha_supported=alpha_supported,
        )

        # Close button (×), top-right corner. Clicking it dismisses early
        # via the same fade-out path as the auto-dismiss timer.
        close_bg_hover = _darken(bg, 0.88)
        close_label = tk.Label(
            toplevel,
            text="×",
            bg=bg,
            fg=fg,
            font=("Segoe UI", 11, "bold"),
            cursor="hand2",
        )
        close_label.place(relx=1.0, x=-8, y=4, anchor="ne")
        close_label.bind("<Button-1>", lambda e, t=toast: self._dismiss(t))
        close_label.bind(
            "<Enter>",
            lambda e, lbl=close_label, c=close_bg_hover: lbl.configure(bg=c),
        )
        close_label.bind(
            "<Leave>",
            lambda e, lbl=close_label, c=bg: lbl.configure(bg=c),
        )

        self._stack.append(toast)
        # Compute target positions for all toasts (also slides older ones up).
        self._repack()
        # Start the new toast offset to the right for the slide-in.
        self._set_position(toast, toast.target_x + _SLIDE_PX, toast.target_y)
        toast.fade_step = 0
        toast.fading_out = False
        self._tick_fade(toast)
        # Auto-dismiss after the requested duration.
        toast.auto_after_id = self._root.after(
            duration_ms, lambda t=toast: self._dismiss(t)
        )

    def info(self, title: str, message: str = "", **kwargs) -> None:
        self.show(ToastLevel.INFO, title, message, **kwargs)

    def success(self, title: str, message: str = "", **kwargs) -> None:
        self.show(ToastLevel.SUCCESS, title, message, **kwargs)

    def warning(self, title: str, message: str = "", **kwargs) -> None:
        self.show(ToastLevel.WARNING, title, message, **kwargs)

    def error(self, title: str, message: str = "", **kwargs) -> None:
        self.show(ToastLevel.ERROR, title, message, **kwargs)

    # ------------------------------------------------------------------ internals

    def _set_alpha(self, toast: _Toast, alpha: float) -> None:
        if not toast.alpha_supported:
            return
        try:
            toast.toplevel.wm_attributes("-alpha", alpha)
        except tk.TclError:
            # Platform doesn't support per-window alpha — disable further
            # attempts for this toast (it stays at full opacity).
```

```
            toast.alpha_supported = False

    def _set_position(self, toast: _Toast, x: int, y: int) -> None:
        try:
            toast.toplevel.wm_geometry(f"+{int(x)}+{int(y)}")
        except tk.TclError:
            pass

    def _tick_fade(self, toast: _Toast) -> None:
        """Advance one fade animation step. Reschedules itself via ``after``."""
        toast.pending_fade = False
        toast.fade_after_id = None
        if toast.fading_out:
            total = _FADE_OUT_STEPS
            step = toast.fade_step
            if step >= total:
                # Fade-out complete — destroy.
                self._destroy_toplevel(toast)
                return
            alpha = 1.0 - (step + 1) / total
            self._set_alpha(toast, alpha)
            toast.fade_step = step + 1
            toast.fade_after_id = self._root.after(
                _FADE_DELAY_MS, lambda: self._tick_fade(toast)
            )
        else:
            # Fade-in. Abort if the toast was dismissed out-of-band.
            if toast not in self._stack:
                self._destroy_toplevel(toast)
                return
            total = _FADE_IN_STEPS
            step = toast.fade_step
            if step >= total:
                # Fade-in complete — pin to final position & full opacity.
                self._set_alpha(toast, 1.0)
                self._set_position(toast, toast.target_x, toast.target_y)
                return
            alpha = (step + 1) / total
            x = toast.target_x + int(_SLIDE_PX * (1 - (step + 1) / total))
            self._set_alpha(toast, alpha)
            self._set_position(toast, x, toast.target_y)
            toast.fade_step = step + 1
            toast.fade_after_id = self._root.after(
                _FADE_DELAY_MS, lambda: self._tick_fade(toast)
            )

    def _dismiss(self, toast: _Toast, early: bool = False) -> None:
        if toast not in self._stack:
            return
        if toast.fading_out:
            if not early:
                return  # already dismissing — wait for the fade to complete
            # early=True: accelerate to instant destroy so the new toast
            # gets visual space immediately.
            self._destroy_toplevel(toast)
            self._repack()
            return

        # Cancel auto-dismiss timer.
        if toast.auto_after_id is not None:
            try:
                self._root.after_cancel(toast.auto_after_id)
            except tk.TclError:
                pass
            toast.auto_after_id = None
        # Cancel any pending fade-in step.
        if toast.fade_after_id is not None:
            try:
                self._root.after_cancel(toast.fade_after_id)
            except tk.TclError:
                pass
            toast.fade_after_id = None

        if early or not toast.alpha_supported:
            # Instant destroy — make room quickly or platform can't fade.
            self._destroy_toplevel(toast)
            self._repack()
            return

        # Fade out, then destroy. Keep in stack so _cleanup_all can find it
        # if the app exits mid-fade; _repack skips fading-out toasts.
        toast.fading_out = True
        toast.fade_step = 0
```

```
        self._repack()
        self._tick_fade(toast)

    def _destroy_toplevel(self, toast: _Toast) -> None:
        """Cancel timers, remove from stack, and destroy the underlying Toplevel."""
        if toast.fade_after_id is not None:
            try:
                self._root.after_cancel(toast.fade_after_id)
            except tk.TclError:
                pass
            toast.fade_after_id = None
        if toast.auto_after_id is not None:
            try:
                self._root.after_cancel(toast.auto_after_id)
            except tk.TclError:
                pass
            toast.auto_after_id = None
        if toast in self._stack:
            self._stack.remove(toast)
        try:
            toast.toplevel.destroy()
        except tk.TclError:
            pass

    def _repack(self) -> None:
        """Re-anchor all visible toasts vertically, newest at the bottom."""
        try:
            root_x = self._root.winfo_rootx()
            root_y = self._root.winfo_rooty()
            root_w = self._root.winfo_width()
            root_h = self._root.winfo_height()
        except tk.TclError:
            return
        margin = 20
        offset = margin
        for toast in reversed(self._stack):
            if toast.fading_out:
                continue
            try:
                toast.toplevel.update_idletasks()
                w = toast.toplevel.winfo_reqwidth() or 320
                h = toast.toplevel.winfo_reqheight() or 60
            except tk.TclError:
                w, h = 320, 60
            toast.target_x = root_x + root_w - w - margin
            toast.target_y = root_y + root_h - h - offset
            # If the toast is mid-fade-in (or about to start), the next
            # tick will reposition it using the updated target. Otherwise
            # jump to the new position instantly (YAGNI: no slide on repack).
            if toast.fade_after_id is None and not toast.pending_fade:
                self._set_position(toast, toast.target_x, toast.target_y)
            offset += h + 8

    def _cleanup_all(self, event: Optional[tk.Event] = None) -> None:
        """Cancel all pending animations and destroy toasts on app close."""
        if event is not None and getattr(event, "widget", None) is not self._root:
            return
        for toast in list(self._stack):
            self._destroy_toplevel(toast)
        self._stack.clear()


__all__ = ["ToastManager", "ToastLevel"]
```

APP\core\ui\tooltip.py

228 lines • 8,089 bytes

```
"""Lightweight tooltip widget for Tk/ttk.

A ``Tooltip`` is attached to any widget and shows a small Toplevel
next to the cursor on hover. The tooltip is non-modal, follows the
cursor, and hides itself as soon as the pointer leaves the bound
widget. No third-party dependency, no images.

The tooltip fades in over ~100 ms (alpha 0 → 1). On platforms that
do not support per-window alpha (some X11 builds), the fade is
skipped and the tooltip appears at full opacity instantly. Fade-out
is intentionally omitted — on mouse-leave an instant disappear feels
snappier than a slow fade.

Usage::

    btn = ttk.Button(parent, text="Run")
    Tooltip(btn, "Run the consolidation")

The widget is a thin wrapper — it does not own its data and can be
garbage-collected normally once the underlying widget is destroyed.
"""
from __future__ import annotations

import logging
import tkinter as tk
from typing import Callable, Optional, Union

_logger = logging.getLogger(__name__)

_DEFAULT_DELAY_MS = 400
_DEFAULT_OFFSET_PX = 12
_FADE_STEPS = 6       # ~100 ms total (6 × 16 ms)
_FADE_DELAY_MS = 16


class Tooltip:
    """Bind a tooltip to a Tk widget.

    Parameters
    ----------
    widget:
        The widget to attach the tooltip to. Must support the standard
        ``<Enter>``, ``<Leave>``, ``<Motion>``, and ``<Destroy>``
        events (all built-in Tk widgets do).
    text:
        Either a plain string or a callable that returns the text
        dynamically. The callable form is useful for status messages
        that depend on the current state of the application.
    delay_ms:
        Hover delay before the tooltip appears.
    """

    def __init__(
        self,
        widget: tk.Misc,
        text: Union[str, Callable[[], str]],
        delay_ms: int = _DEFAULT_DELAY_MS,
    ) -> None:
        self.widget = widget
        self.text = text
        self.delay_ms = delay_ms

        self._tip: Optional[tk.Toplevel] = None
        self._after_id: Optional[str] = None      # show-delay timer
        self._fade_after_id: Optional[str] = None  # active fade-step timer
        self._alpha_supported: bool = True
        self._last_x = 0
        self._last_y = 0

        widget.bind("<Enter>", self._on_enter, add="+")
        widget.bind("<Leave>", self._on_leave, add="+")
        widget.bind("<Motion>", self._on_motion, add="+")
        widget.bind("<ButtonPress>", self._on_leave, add="+")
        widget.bind("<Destroy>", self._on_destroy, add="+")

    def set_text(self, text: str) -> None:
        """Replace the tooltip text. Updates the visible window if any."""
        self.text = text
        if self._tip is not None and self._tip.winfo_exists():
            label = getattr(self._tip, "_label", None)
            if label is not None:
                label.configure(text=text)
```

```
    def _on_enter(self, event: tk.Event) -> None:
        self._last_x, self._last_y = event.x_root, event.y_root
        self._cancel_pending()
        self._after_id = self.widget.after(self.delay_ms, self._show)

    def _on_leave(self, _event: Optional[tk.Event] = None) -> None:
        self._cancel_pending()
        self._hide()

    def _on_motion(self, event: tk.Event) -> None:
        self._last_x, self._last_y = event.x_root, event.y_root
        if self._tip is not None and self._tip.winfo_exists():
            self._tip.geometry(
                f"+{self._last_x + _DEFAULT_OFFSET_PX}+{self._last_y + _DEFAULT_OFFSET_PX}"
            )

    def _on_destroy(self, _event: tk.Event) -> None:
        self._cancel_pending()
        self._hide()

    def _cancel_pending(self) -> None:
        if self._after_id is not None:
            try:
                self.widget.after_cancel(self._after_id)
            except tk.TclError:
                pass
            self._after_id = None

    def _cancel_fade(self) -> None:
        if self._fade_after_id is not None:
            try:
                self.widget.after_cancel(self._fade_after_id)
            except tk.TclError:
                pass
            self._fade_after_id = None

    def _show(self) -> None:
        self._after_id = None
        if self._tip is not None and self._tip.winfo_exists():
            return
        try:
            from core.ui.theme import ThemeManager

            colors = ThemeManager.COLORS
            bg = colors["tooltip_bg"]
            fg = colors["tooltip_fg"]
            border = colors.get("border_subtle", colors.get("trough", "#ced4da"))
        except (KeyError, tk.TclError):
            bg, fg, border = "#ffffe0", "#212529", "#ced4da"

        try:
            tip = tk.Toplevel(self.widget)
            tip.wm_overrideredirect(True)
            tip.wm_geometry(
                f"+{self._last_x + _DEFAULT_OFFSET_PX}+{self._last_y + _DEFAULT_OFFSET_PX}"
            )
            tip.configure(bg=bg)

            # tk.Frame (not ttk.Frame) so we can use highlightthickness
            # for a 1px border in ``border`` color. ttk.Frame does not
            # support highlightthickness reliably across themes.
            frame = tk.Frame(
                tip,
                bg=bg,
                padx=8,
                pady=6,
                highlightthickness=1,
                highlightbackground=border,
                highlightcolor=border,
            )
            frame.pack(fill="both", expand=True)

            label = tk.Label(
                frame,
                text=self._resolved_text(),
                bg=bg,
                fg=fg,
                font=("Segoe UI", 9),
                wraplength=280,
                justify="left",
            )
            label.pack()
            tip._label = label  # type: ignore[attr-defined]
```

```
            self._tip = tip
        except tk.TclError as e:
            # The widget tree may have been destroyed between the
            # after-schedule and the show call. Silently ignore —
            # the tooltip simply won't appear.
            _logger.debug("Tooltip show failed: %s", e)
            return

        # Fade-in: alpha 0 → 1 over _FADE_STEPS × _FADE_DELAY_MS.
        # Try alpha=0 first; if the platform rejects it, skip the fade.
        self._alpha_supported = True
        try:
            tip.wm_attributes("-alpha", 0.0)
        except tk.TclError:
            self._alpha_supported = False
            return  # tooltip already visible at full opacity — nothing to animate

        self._fade_after_id = self.widget.after(
            _FADE_DELAY_MS, lambda: self._tick_fade_in(1)
        )

    def _tick_fade_in(self, step: int) -> None:
        self._fade_after_id = None
        if self._tip is None or not self._tip.winfo_exists():
            return
        if step > _FADE_STEPS:
            # Finalize at full opacity.
            try:
                self._tip.wm_attributes("-alpha", 1.0)
            except tk.TclError:
                self._alpha_supported = False
            return
        try:
            self._tip.wm_attributes("-alpha", step / _FADE_STEPS)
        except tk.TclError:
            # Platform withdrew alpha support mid-fade — stop animating.
            self._alpha_supported = False
            return
        self._fade_after_id = self.widget.after(
            _FADE_DELAY_MS, lambda: self._tick_fade_in(step + 1)
        )

    def _resolved_text(self) -> str:
        if callable(self.text):
            try:
                return str(self.text())
            except Exception as e:  # noqa: BLE001
                _logger.debug("Tooltip text callable raised: %s", e)
                return ""
        return str(self.text)

    def _hide(self) -> None:
        self._cancel_fade()
        if self._tip is not None:
            try:
                self._tip.destroy()
            except tk.TclError:
                pass
            self._tip = None


__all__ = ["Tooltip"]
```

APP\core\ui\ui_aliases.py

247 lines • 11,019 bytes

```
"""UI tab for customer aliases — reconciliation name mappings.

Позволяет бухгалтерам вручную указывать соответствия между заказчиками
со стороны кураторов (master) и заказчиками со стороны РСЧ. Эти алиасы
используются сверкой: при точном совпадении суммы + исполнителя + имени
заказчика (или алиаса) строка получает статус "сверено".
"""

import tkinter as tk
from tkinter import ttk, messagebox
import logging

logger = logging.getLogger(__name__)


class AliasesTab:
    """Tab widget for managing customer aliases.

    Shows a sortable table of all aliases with add/edit/delete buttons.
    Double-click a row to edit. Changes take effect immediately on the
    next reconciliation run.
    """

    def __init__(self, parent: ttk.Frame, app_controller) -> None:
        self.parent = parent
        self.app = app_controller
        self._editing: dict | None = None
        self._setup_ui()
        self.refresh()

    def _setup_ui(self) -> None:
        # Header
        header = ttk.Frame(self.parent, style="Main.TFrame")
        header.pack(fill=tk.X, pady=(0, 8))
        ttk.Label(
            header,
            text=" Соответствия заказчиков для сверки",
            style="Header.TLabel",
        ).pack(side=tk.LEFT)
        ttk.Label(
            header,
            text="Бухгалтеры указывают, какой заказчик куратора = какой заказчик РСЧ",
            style="Muted.TLabel",
        ).pack(side=tk.LEFT, padx=(12, 0))

        # Toolbar
        toolbar = ttk.Frame(self.parent, style="Main.TFrame")
        toolbar.pack(fill=tk.X, pady=(0, 6))
        ttk.Button(toolbar, text=" Добавить", command=self._on_add).pack(side=tk.LEFT, padx=(0, 4))
        ttk.Button(toolbar, text=" Изменить", command=self._on_edit).pack(side=tk.LEFT, padx=4)
        ttk.Button(toolbar, text=" Удалить", command=self._on_delete).pack(side=tk.LEFT, padx=4)
        ttk.Button(toolbar, text=" Обновить", command=self.refresh).pack(side=tk.LEFT, padx=4)
        ttk.Label(
            toolbar,
            text="Двойной клик по строке — изменить",
            style="Muted.TLabel",
        ).pack(side=tk.RIGHT)

        # Treeview
        tree_frame = ttk.Frame(self.parent, style="Main.TFrame")
        tree_frame.pack(fill=tk.BOTH, expand=True)
        columns = ("id", "master", "rsch", "notes", "updated")
        self.tree = ttk.Treeview(
            tree_frame,
            columns=columns,
            show="headings",
            selectmode="browse",
        )
        self.tree.heading("id", text="ID")
        self.tree.heading("master", text="Заказчик (куратор)")
        self.tree.heading("rsch", text="Заказчик (РСЧ)")
        self.tree.heading("notes", text="Заметки")
        self.tree.heading("updated", text="Обновлено")
        self.tree.column("id", width=50, stretch=False)
        self.tree.column("master", width=300, stretch=True)
        self.tree.column("rsch", width=300, stretch=True)
        self.tree.column("notes", width=200, stretch=True)
        self.tree.column("updated", width=140, stretch=False)
        self.tree.grid(row=0, column=0, sticky="nsew")
        vsb = ttk.Scrollbar(tree_frame, orient="vertical", command=self.tree.yview)
        vsb.grid(row=0, column=1, sticky="ns")
        hsb = ttk.Scrollbar(tree_frame, orient="horizontal", command=self.tree.xview)
        hsb.grid(row=1, column=0, sticky="ew")
```

```
        self.tree.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
        tree_frame.grid_rowconfigure(0, weight=1)
        tree_frame.grid_columnconfigure(0, weight=1)
        self.tree.bind("<Double-1>", lambda e: self._on_edit())

        # Footer hint
        ttk.Label(
            self.parent,
            text=" Сверено = точное совпадение суммы + исполнителя + заказчика (или алиаса). "
                 "Строки с частичным совпадением попадают в «Проверить» — добавьте алиас, чтобы "
                 "в следующий раз они сверялись автоматически.",
            style="Muted.TLabel",
            wraplength=900,
            justify=tk.LEFT,
        ).pack(anchor=tk.W, pady=(8, 0))

    def refresh(self) -> None:
        """Reload aliases from DB into the treeview."""
        for item in self.tree.get_children():
            self.tree.delete(item)
        try:
            aliases = self.app.get_customer_aliases()
        except Exception as e:
            logger.error("Failed to load aliases: %s", e)
            messagebox.showerror("Ошибка", f"Не удалось загрузить алиасы: {e}", parent=self.parent)
            return
        for a in aliases:
            updated = (a.updated_at or "")[:19]
            self.tree.insert(
                "",
                tk.END,
                values=(a.id, a.master_customer, a.rsch_customer, a.notes, updated),
            )

    def _selected_id(self) -> int | None:
        sel = self.tree.selection()
        if not sel:
            return None
        values = self.tree.item(sel[0], "values")
        return int(values[0]) if values else None

    def _on_add(self) -> None:
        self._open_dialog(mode="add")

    def _on_edit(self) -> None:
        sel = self.tree.selection()
        if not sel:
            messagebox.showinfo("Изменение", "Выберите строку для изменения", parent=self.parent)
            return
        values = self.tree.item(sel[0], "values")
        self._open_dialog(
            mode="edit",
            alias_id=int(values[0]),
            master=str(values[1]),
            rsch=str(values[2]),
            notes=str(values[3]) if values[3] else "",
        )

    def _on_delete(self) -> None:
        alias_id = self._selected_id()
        if alias_id is None:
            messagebox.showinfo("Удаление", "Выберите строку для удаления", parent=self.parent)
            return
        if messagebox.askyesno(
            "Удаление алиаса",
            "Удалить выбранный алиас?",
            parent=self.parent,
        ):
            try:
                self.app.delete_customer_alias(alias_id)
                self.refresh()
            except Exception as e:
                messagebox.showerror("Ошибка", str(e), parent=self.parent)

    def _open_dialog(
        self,
        mode: str,
        alias_id: int | None = None,
        master: str = "",
        rsch: str = "",
        notes: str = "",
    ) -> None:
        dlg = tk.Toplevel(self.parent)
```

```
        dlg.title("Добавить алиас" if mode == "add" else "Изменить алиас")
        dlg.geometry("780x280")
        dlg.minsize(640, 240)
        dlg.transient(self.parent)
        dlg.grab_set()

        # Колонка полей ввода и строка заметок растягиваются вместе с окном.
        dlg.columnconfigure(1, weight=1)
        dlg.rowconfigure(2, weight=1)

        ttk.Label(dlg, text="Заказчик со стороны куратора:").grid(row=0, column=0, sticky="w", padx=8, pady=(12, 4))
        master_entry = ttk.Entry(dlg)
        master_entry.grid(row=0, column=1, sticky="ew", padx=8, pady=(12, 4))
        master_entry.insert(0, master)

        ttk.Label(dlg, text="Заказчик со стороны РСЧ:").grid(row=1, column=0, sticky="w", padx=8, pady=4)
        rsch_entry = ttk.Entry(dlg)
        rsch_entry.grid(row=1, column=1, sticky="ew", padx=8, pady=4)
        rsch_entry.insert(0, rsch)

        ttk.Label(dlg, text="Заметки:").grid(row=2, column=0, sticky="nw", padx=8, pady=4)
        notes_entry = tk.Text(dlg, height=3, wrap=tk.WORD)
        notes_entry.grid(row=2, column=1, sticky="nsew", padx=8, pady=4)
        notes_entry.insert("1.0", notes)

        btn_frame = ttk.Frame(dlg)
        btn_frame.grid(row=3, column=0, columnspan=2, pady=(12, 8))

        def save() -> None:
            m = master_entry.get().strip()
            r = rsch_entry.get().strip()
            n = notes_entry.get("1.0", tk.END).strip()
            if not m or not r:
                messagebox.showerror("Ошибка", "Заполните оба поля заказчиков", parent=dlg)
                return
            try:
                if mode == "add":
                    self.app.add_customer_alias(m, r, n)
                else:
                    self.app.update_customer_alias(alias_id, m, r, n)
                self.refresh()
                dlg.destroy()
            except ValueError as e:
                messagebox.showerror("Ошибка", str(e), parent=dlg)
            except Exception as e:
                messagebox.showerror("Ошибка", f"Не удалось сохранить: {e}", parent=dlg)

        ok_btn = ttk.Button(btn_frame, text="Сохранить", command=save)
        ok_btn.pack(side=tk.LEFT, padx=4)
        ttk.Button(btn_frame, text="Отмена", command=dlg.destroy).pack(side=tk.LEFT, padx=4)

        # Контекстное меню (правый клик): Вырезать/Копировать/Вставить/Выделить всё.
        def _send_event(virtual_event: str) -> None:
            widget = dlg.focus_get()
            if widget is not None:
                widget.event_generate(virtual_event)

        ctx = tk.Menu(dlg, tearoff=0)
        ctx.add_command(label="Вырезать", command=lambda: _send_event("<<Cut>>"))
        ctx.add_command(label="Копировать", command=lambda: _send_event("<<Copy>>"))
        ctx.add_command(label="Вставить", command=lambda: _send_event("<<Paste>>"))
        ctx.add_separator()
        ctx.add_command(label="Выделить всё", command=lambda: _send_event("<<SelectAll>>"))

        def _show_context_menu(event: tk.Event) -> None:
            ctx.tk_popup(event.x_root, event.y_root)

        for w in (master_entry, rsch_entry, notes_entry):
            w.bind("<Button-3>", _show_context_menu)

        # Горячие клавиши: Enter — сохранить, Escape — отмена.
        def _on_enter(_event: tk.Event) -> None:
            if "disabled" not in ok_btn.state():  # type: ignore[no-untyped-call]
                save()

        dlg.bind("<Return>", _on_enter)
        dlg.bind("<Escape>", lambda e: dlg.destroy())

        dlg.update_idletasks()
        dlg.geometry(f"+{self.parent.winfo_rootx() + 100}+{self.parent.winfo_rooty() + 100}")
        master_entry.focus_set()
```

APP\core\ui\ui_dashboard.py

751 lines • 32,925 bytes

```
import logging
import threading
import tkinter as tk
from datetime import datetime
from pathlib import Path
from tkinter import filedialog, messagebox, ttk
from typing import Callable, TypeVar

from core.infra import open_path
from core.ui.dashboard_state import (
    RuntimeResult,
    build_diagnostic_summary,
    collect_report_paths,
    load_runtime_results,
    read_recent_log_errors,
    read_recent_logs,
    run_light_diagnostics,
    safe_open_existing_path,
    save_runtime_results,
    status_text,
)
from core.ui.theme import ThemeManager

_T = TypeVar("_T")


def _c(key, fallback):
    """Safe theme-color lookup with fallback for keys POLISH-1 may add."""
    return ThemeManager.COLORS.get(key, fallback)


class DashboardTabs:
    def __init__(self, notebook: ttk.Notebook, controller, config_controller) -> None:
        self.notebook = notebook
        self.controller = controller
        self.config_controller = config_controller
        self.app_root = Path(getattr(config_controller, "config_path", "APP/config.yaml")).parent
        self.repo_root = self.app_root.parent
        self.runtime_state_path = self.app_root / "runtime_state.json"
        self._last_results: list[RuntimeResult] = load_runtime_results(
            self.runtime_state_path
        )

        self.home_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")
        self.employees_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")
        self.reports_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")
        self.diagnostics_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")
        self.logs_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")
        self.settings_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")
        self.aliases_tab = ttk.Frame(notebook, padding=10, style="Main.TFrame")

        notebook.add(self.employees_tab, text="Сотрудники")
        notebook.add(self.home_tab, text="Панель")
        notebook.add(self.reports_tab, text="Отчеты")
        notebook.add(self.diagnostics_tab, text="Диагностика")
        notebook.add(self.logs_tab, text="Логи")
        notebook.add(self.aliases_tab, text=" Алиасы сверки")
        notebook.add(self.settings_tab, text="Настройки")
        notebook.select(self.employees_tab)

        self._setup_home()
        self._setup_reports()
        self._setup_diagnostics()
        self._setup_logs()
        self._setup_settings()
        self._setup_aliases()

    def _setup_home(self) -> None:
        summary_frame = ttk.LabelFrame(self.home_tab, text="Состояние", padding=8)
        summary_frame.pack(fill=tk.X, pady=(0, 8))
        self.state_label = ttk.Label(summary_frame, text="", justify=tk.LEFT, wraplength=600)
        self.state_label.pack(fill=tk.X)

        quick_frame = ttk.LabelFrame(self.home_tab, text="Быстрые действия", padding=8)
        quick_frame.pack(fill=tk.X, pady=(0, 8))
        ttk.Button(
            quick_frame,
            text="Открыть последний Master",
            command=lambda: self._open_report_by_label("Master_Report.xlsx"),
        ).pack(side=tk.LEFT, padx=(0, 8))
        ttk.Button(
            quick_frame,
            text="Открыть последнюю сверку",
```

```
            command=lambda: self._open_report_by_label("Последняя сверка"),
        ).pack(side=tk.LEFT, padx=(0, 8))
        ttk.Button(
            quick_frame,
            text="Открыть backup",
            command=self._open_backup_folder,
        ).pack(side=tk.LEFT, padx=(0, 8))
        ttk.Button(
            quick_frame,
            text="Открыть логи",
            command=lambda: self._open_path(self.repo_root),
        ).pack(side=tk.LEFT)

        last_frame = ttk.LabelFrame(self.home_tab, text="Последние результаты", padding=8)
        last_frame.pack(fill=tk.BOTH, expand=True)
        self.results_tree = ttk.Treeview(
            last_frame,
            columns=("type", "status", "time", "path", "message"),
            show="headings",
        )
        for column, text, width in (
            ("type", "Тип", 120),
            ("status", "Статус", 90),
            ("time", "Время", 140),
            ("path", "Путь", 260),
            ("message", "Сообщение", 360),
        ):
            self.results_tree.heading(column, text=text)
            self.results_tree.column(column, width=width, anchor=tk.W, stretch=(column == "message"))
        # Empty-state placeholder: when no results have been recorded
        # yet, show a muted hint instead of a blank tree.
        self.results_tree.tag_configure(
            "empty", foreground=_c("text_muted", _c("text", "#6c757d"))
        )
        self.results_tree.pack(fill=tk.BOTH, expand=True)
        self.results_tree.bind("<Double-1>", lambda _event: self._open_selected_result())
        # Initial empty-state hint so the tree isn't a blank grid on
        # first launch — replaced by real rows once ``_refresh_home``
        # completes (or re-inserted if there are no results yet).
        self.results_tree.insert(
            "",
            tk.END,
            values=("Нет данных", "", "", "", "Запустите сборку отчета"),
            tags=("empty",),
        )

    def _setup_reports(self) -> None:
        action_frame = ttk.Frame(self.reports_tab, style="Main.TFrame")
        action_frame.pack(fill=tk.X, pady=(0, 8))
        ttk.Button(action_frame, text="Обновить", command=self.refresh).pack(side=tk.LEFT)
        ttk.Button(action_frame, text="Открыть", command=self._open_selected_report).pack(
            side=tk.LEFT, padx=(6, 0)
        )
        ttk.Button(
            action_frame,
            text="Открыть папку",
            command=self._open_selected_report_folder,
        ).pack(side=tk.LEFT, padx=(6, 0))
        ttk.Button(action_frame, text="Backup", command=self._open_backup_folder).pack(
            side=tk.LEFT, padx=(6, 0)
        )

        self.reports_tree = ttk.Treeview(
            self.reports_tab,
            columns=("name", "status", "mtime", "size", "path"),
            show="headings",
        )
        for column, text, width in (
            ("name", "Файл", 160),
            ("status", "Статус", 90),
            ("mtime", "Изменен", 130),
            ("size", "Размер", 80),
            ("path", "Путь", 520),
        ):
            self.reports_tree.heading(column, text=text)
            self.reports_tree.column(column, width=width, anchor=tk.W, stretch=(column == "path"))
        self.reports_tree.pack(fill=tk.BOTH, expand=True)
        self.reports_tree.bind("<Double-1>", lambda _event: self._open_selected_report())

    def _setup_diagnostics(self) -> None:
        action_frame = ttk.Frame(self.diagnostics_tab, style="Main.TFrame")
        action_frame.pack(fill=tk.X, pady=(0, 8))
        ttk.Button(
```

```
            action_frame,
            text="Проверить систему",
            command=self.run_diagnostics,
            style="Action.TButton",
        ).pack(side=tk.LEFT)
        ttk.Button(
            action_frame,
            text="Скопировать диагностику",
            command=self.copy_diagnostics,
        ).pack(side=tk.LEFT, padx=(6, 0))

        self.diagnostics_tree = ttk.Treeview(
            self.diagnostics_tab,
            columns=("check", "status", "detail", "path"),
            show="headings",
        )
        for column, text, width in (
            ("check", "Проверка", 170),
            ("status", "Статус", 90),
            ("detail", "Детали", 260),
            ("path", "Путь", 520),
        ):
            self.diagnostics_tree.heading(column, text=text)
            self.diagnostics_tree.column(column, width=width, anchor=tk.W, stretch=(column in ("detail", "path")))
        self.diagnostics_tree.pack(fill=tk.BOTH, expand=True)

    def _setup_logs(self) -> None:
        action_frame = ttk.Frame(self.logs_tab, style="Main.TFrame")
        action_frame.pack(fill=tk.X, pady=(0, 8))

        ttk.Button(
            action_frame,
            text="Обновить",
            command=self._refresh_logs,
        ).pack(side=tk.LEFT)

        ttk.Button(
            action_frame,
            text="Открыть лог работы (Python)",
            command=lambda: self._open_path(self.app_root / "startup_log.txt"),
        ).pack(side=tk.LEFT, padx=(6, 0))

        ttk.Button(
            action_frame,
            text="Открыть лог запуска (Батник)",
            command=lambda: self._open_path(self.repo_root / "startup_log.txt"),
        ).pack(side=tk.LEFT, padx=(6, 0))

        ttk.Button(
            action_frame,
            text="Открыть gui_error_log.txt",
            command=lambda: self._open_path(self.repo_root / "gui_error_log.txt"),
        ).pack(side=tk.LEFT, padx=(6, 0))

        ttk.Button(
            action_frame,
            text="Открыть папку логов",
            command=lambda: self._open_path(self.repo_root / "logs"),
        ).pack(side=tk.LEFT, padx=(6, 0))

        ttk.Label(self.logs_tab, text="Последние события лога:").pack(anchor=tk.W)

        self.log_errors = tk.Text(
            self.logs_tab,
            state="disabled",
            wrap=tk.WORD,
        )

        scroll_y = ttk.Scrollbar(self.logs_tab, orient=tk.VERTICAL, command=self.log_errors.yview)
        scroll_x = ttk.Scrollbar(self.logs_tab, orient=tk.HORIZONTAL, command=self.log_errors.xview)

        self.log_errors.configure(yscrollcommand=scroll_y.set, xscrollcommand=scroll_x.set)

        scroll_y.pack(side=tk.RIGHT, fill=tk.Y)
        scroll_x.pack(side=tk.BOTTOM, fill=tk.X)
        self.log_errors.pack(fill=tk.BOTH, expand=True)
        ThemeManager.register(self.log_errors, bg_key="bg_panel", fg_key="text")

    def _setup_settings(self) -> None:
        self.settings_mode = "visual"  # "visual" или "yaml"

        # Переменные для формы настроек
        self.master_file_var = tk.StringVar()
```

```
self.archive_dir_var = tk.StringVar()
self.drive_letters_var = tk.StringVar()
self.excel_keep_var = tk.IntVar(value=10)

# Общий action_frame
action_frame = ttk.Frame(self.settings_tab, style="Main.TFrame")
action_frame.pack(fill=tk.X, pady=(0, 8))

self.save_btn = ttk.Button(
    action_frame,
    text="Сохранить",
    command=self._save_settings,
)
self.save_btn.pack(side=tk.LEFT)

self.reset_btn = ttk.Button(
    action_frame,
    text="Сбросить",
    command=self._refresh_settings,
)
self.reset_btn.pack(side=tk.LEFT, padx=(6, 0))

self.mode_btn = ttk.Button(
    action_frame,
    text="Режим YAML-кода",
    command=self._toggle_settings_mode,
)
self.mode_btn.pack(side=tk.LEFT, padx=(6, 0))

self.external_btn = ttk.Button(
    action_frame,
    text="Открыть в Блокноте",
    command=lambda: self._open_path(Path(self.config_controller.config_path)),
)
self.external_btn.pack(side=tk.LEFT, padx=(6, 0))

# Контейнер для двух режимов
self.settings_container = ttk.Frame(self.settings_tab, style="Main.TFrame")
self.settings_container.pack(fill=tk.BOTH, expand=True)

# 1. Фрейм визуального режима
self.settings_visual_frame = ttk.Frame(self.settings_container, style="Main.TFrame")

# Поле Master-отчета
master_frame = ttk.LabelFrame(self.settings_visual_frame, text="Основной сводный отчет (Master_Report.xlsx)", padding=8)
master_frame.pack(fill=tk.X, pady=(0, 8))
ttk.Entry(master_frame, textvariable=self.master_file_var).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 8))
ttk.Button(master_frame, text="Обзор...", command=self._browse_master_file).pack(side=tk.RIGHT)

# Поле папки бэкапов
backup_frame = ttk.LabelFrame(self.settings_visual_frame, text="Папка для резервных копий (archive_dir)", padding=8)
backup_frame.pack(fill=tk.X, pady=(0, 8))
ttk.Entry(backup_frame, textvariable=self.archive_dir_var).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 8))
ttk.Button(backup_frame, text="Обзор...", command=self._browse_archive_dir).pack(side=tk.RIGHT)

# Список папок-источников (кураторов)
sources_frame = ttk.LabelFrame(self.settings_visual_frame, text="Папки с отчетами кураторов (source_roots)", padding=8)
sources_frame.pack(fill=tk.BOTH, expand=True, pady=(0, 8))

list_container = ttk.Frame(sources_frame)
list_container.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 8))

self.sources_listbox = tk.Listbox(list_container)
ThemeManager.register(self.sources_listbox, bg_key="bg_panel", fg_key="text")

scroll_y_lb = ttk.Scrollbar(list_container, orient=tk.VERTICAL, command=self.sources_listbox.yview)
self.sources_listbox.configure(yscrollcommand=scroll_y_lb.set)

self.sources_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scroll_y_lb.pack(side=tk.RIGHT, fill=tk.Y)

btn_container = ttk.Frame(sources_frame)
btn_container.pack(side=tk.RIGHT, fill=tk.Y)
ttk.Button(btn_container, text="Добавить...", command=self._add_source_root).pack(fill=tk.X, pady=(0, 4))
ttk.Button(btn_container, text="Удалить", command=self._remove_source_root).pack(fill=tk.X)

# Дополнительные настройки (сетевые диски и число бэкапов)
misc_frame = ttk.LabelFrame(self.settings_visual_frame, text="Дополнительно", padding=8)
misc_frame.pack(fill=tk.X)
misc_frame.grid_columnconfigure(1, weight=1)

# Сетевые диски
ttk.Label(misc_frame, text="Сетевые диски (буквы через запятую):").grid(row=0, column=0, sticky=tk.W, pady=4, padx=(0, 8))
```

```
        ttk.Entry(misc_frame, textvariable=self.drive_letters_var).grid(row=0, column=1, sticky="ew", pady=4)

        # Хранение бэкапов
        ttk.Label(misc_frame, text="Количество хранимых бэкапов Excel:").grid(row=1, column=0, sticky=tk.W, pady=4, padx=(0, 8))
        ttk.Entry(misc_frame, textvariable=self.excel_keep_var).grid(row=1, column=1, sticky="ew", pady=4)

        # 2. Фрейм YAML-режима
        self.settings_yaml_frame = ttk.Frame(self.settings_container, style="Main.TFrame")

        self.settings_text = tk.Text(
            self.settings_yaml_frame,
            undo=True,
            wrap=tk.NONE,
        )
        scroll_y = ttk.Scrollbar(self.settings_yaml_frame, orient=tk.VERTICAL, command=self.settings_text.yview)
        scroll_x = ttk.Scrollbar(self.settings_yaml_frame, orient=tk.HORIZONTAL, command=self.settings_text.xview)
        self.settings_text.configure(yscrollcommand=scroll_y.set, xscrollcommand=scroll_x.set)

        scroll_y.pack(side=tk.RIGHT, fill=tk.Y)
        scroll_x.pack(side=tk.BOTTOM, fill=tk.X)
        self.settings_text.pack(fill=tk.BOTH, expand=True)
        ThemeManager.register(self.settings_text, bg_key="bg_panel", fg_key="text")

        self.settings_visual_frame.pack(fill=tk.BOTH, expand=True)

    def _setup_aliases(self) -> None:
        from core.ui.ui_aliases import AliasesTab
        app_ctl = getattr(self.controller, "app_controller", self.controller)
        self._aliases_tab = AliasesTab(self.aliases_tab, app_ctl)

    def refresh(self) -> None:
        self._refresh_reports()
        self._refresh_home()
        self._refresh_logs()
        self._refresh_settings()

    def refresh_theme(self) -> None:
        """Re-apply theme-dependent tag colours after a palette switch."""
        self.results_tree.tag_configure(
            "empty", foreground=_c("text_muted", _c("text", "#6c757d"))
        )

    def run_diagnostics(self) -> None:
        for item in self.diagnostics_tree.get_children():
            self.diagnostics_tree.delete(item)
        for item in run_light_diagnostics(self.config_controller, self.app_root):
            self.diagnostics_tree.insert(
                "",
                tk.END,
                values=(
                    item.label,
                    status_text(item.status),
                    item.detail,
                    str(item.path) if item.path else "",
                ),
            )
        self._refresh_home()

    def copy_diagnostics(self) -> None:
        diagnostics = run_light_diagnostics(
            self.config_controller,
            self.app_root,
            include_network=False,
        )
        log_errors = self._recent_log_errors()
        summary = build_diagnostic_summary(
            diagnostics,
            self._last_results,
            log_errors,
        )
        self.notebook.clipboard_clear()
        self.notebook.clipboard_append(summary)
        messagebox.showinfo("Диагностика", "Диагностика скопирована в буфер обмена")

    def record_result(
        self,
        label: str,
        ok: bool,
        message: str,
        path: Path | None = None,
        rows: int | None = None,
    ) -> None:
        self._last_results.insert(
```

```
            0,
            RuntimeResult(label, ok, message, path, rows, datetime.now()),
        )
        self._last_results = self._last_results[:10]
        save_runtime_results(self.runtime_state_path, self._last_results)
        self.refresh()

    def _run_in_background(
        self, work: Callable[[], _T], on_done: Callable[[_T], None]
    ) -> None:
        """Run ``work`` on a daemon thread, then call ``on_done`` on the
        GUI thread via ``root.after(0, ...)``.

        Used to keep ``path.exists()`` / network-drive probes off the
        Tk event loop so consolidations do not freeze the UI.
        """

        def runner() -> None:
            try:
                result = work()
            except Exception:
                logging.exception("Background dashboard task failed")
                result = None

            def safe_done() -> None:
                try:
                    on_done(result)
                except tk.TclError:
                    pass

            try:
                self.notebook.after(0, safe_done)
            except tk.TclError:
                pass

        threading.Thread(target=runner, daemon=True).start()

    def _refresh_reports(self) -> None:
        for item in self.reports_tree.get_children():
            self.reports_tree.delete(item)
        self.reports_tree.insert("", tk.END, values=("проверка...", "", "", "", ""))

        def work():
            return collect_report_paths(self.config_controller, self.app_root)

        def done(payload):
            if payload is None:
                return
            for item in self.reports_tree.get_children():
                self.reports_tree.delete(item)
            for item in payload:
                status = "найден" if item.exists else "не найден"
                self.reports_tree.insert(
                    "",
                    tk.END,
                    values=(
                        item.label,
                        f" {status}" if item.exists else f" {status}",
                        item.mtime,
                        item.size,
                        str(item.path) if item.path else "",
                    ),
                )

        self._run_in_background(work, done)

    def _refresh_home(self) -> None:
        self.state_label.config(text="проверка...")

        def work():
            diagnostics = run_light_diagnostics(self.config_controller, self.app_root)
            reports = collect_report_paths(self.config_controller, self.app_root)
            return diagnostics, reports

        def done(payload):
            if payload is None:
                return
            diagnostics, reports = payload
            errors = sum(1 for item in diagnostics if item.status == "error")
            warnings = sum(1 for item in diagnostics if item.status == "warning")
            existing_reports = sum(1 for item in reports if item.exists)
            self.state_label.config(
                text=(
```

```
                f"Готовность: {'можно работать' if errors == 0 else 'есть ошибки'}\n"
                f"Диагностика: ошибок {errors}, предупреждений {warnings}\n"
                f"Найдено результатов: {existing_reports}/{len(reports)}"
            )
        )

        for item in self.results_tree.get_children():
            self.results_tree.delete(item)
        if not self._last_results:
            # Empty-state hint so the tree isn't a blank grid on
            # first launch (before any consolidation has run).
            self.results_tree.insert(
                "",
                tk.END,
                values=(
                    "Нет данных",
                    "",
                    "",
                    "",
                    "Запустите сборку отчета",
                ),
                tags=("empty",),
            )
            return
        for result in self._last_results:
            time_text = result.created_at.strftime("%d.%m.%Y %H:%M") if result.created_at else "-"
            message = result.message
            if result.rows is not None:
                message = f"{message} | строк: {result.rows}"
            self.results_tree.insert(
                "",
                tk.END,
                values=(
                    result.label,
                    " успех" if result.ok else " ошибка",
                    time_text,
                    str(result.path) if result.path else "",
                    message,
                ),
            )

    self._run_in_background(work, done)

def _refresh_logs(self) -> None:
    logs = read_recent_logs(
        [self.app_root / "startup_log.txt", self.repo_root / "gui_error_log.txt"],
        limit=150
    )
    self.log_errors.configure(state="normal")
    self.log_errors.delete("1.0", tk.END)
    if logs:
        self.log_errors.insert(tk.END, "\n".join(logs))
        self.log_errors.see(tk.END)
    else:
        self.log_errors.insert(tk.END, "Событий в логах не найдено.")
    self.log_errors.configure(state="disabled")

def _refresh_settings(self) -> None:
    # 1. Загрузка в YAML-редактор
    self.settings_text.configure(state="normal")
    self.settings_text.delete("1.0", tk.END)
    config_path = Path(self.config_controller.config_path)
    if config_path.exists():
        try:
            text = config_path.read_text(encoding="utf-8")
            self.settings_text.insert(tk.END, text)
        except Exception as e:
            self.settings_text.insert(tk.END, f"# Ошибка чтения файла настроек: {e}")
    else:
        self.settings_text.insert(tk.END, "# Файл config.yaml не найден.")

    # 2. Загрузка в визуальную форму
    cfg = self.config_controller.config
    paths_cfg = cfg.get("paths", {})

    self.master_file_var.set(paths_cfg.get("master_file", ""))
    self.archive_dir_var.set(paths_cfg.get("archive_dir", ""))

    # Загрузка списка источников
    self.sources_listbox.delete(0, tk.END)
    source_roots = paths_cfg.get("source_roots", [])
    if isinstance(source_roots, list):
        for root in source_roots:
```

```
            self.sources_listbox.insert(tk.END, str(root))

        # Сетевые диски
        network_cfg = cfg.get("network", {})
        drive_letters = network_cfg.get("drive_letters", [])
        if isinstance(drive_letters, list):
            self.drive_letters_var.set(", ".join(str(d) for d in drive_letters))
        else:
            self.drive_letters_var.set(str(drive_letters))

        # Число бэкапов
        backup_cfg = cfg.get("backup", {})
        self.excel_keep_var.set(backup_cfg.get("excel_keep", 10))

    def _save_settings(self) -> None:
        config_path = Path(self.config_controller.config_path)
        from ruamel.yaml import YAML
        yaml = YAML()

        if self.settings_mode == "yaml":
            # Сохранение в режиме YAML-кода
            text = self.settings_text.get("1.0", tk.END)
            try:
                parsed = yaml.load(text)
                if not isinstance(parsed, dict):
                    raise ValueError("Корневой элемент конфигурации должен быть объектом (словарем)")
            except Exception as e:
                messagebox.showerror("Ошибка сохранения", f"Некорректный синтаксис YAML:\n{e}")
                return

            try:
                config_path.write_text(text, encoding="utf-8")
                self.config_controller.config = parsed
                messagebox.showinfo("Успех", "Настройки успешно сохранены!")
            except Exception as e:
                messagebox.showerror("Ошибка сохранения", f"Не удалось записать файл config.yaml:\n{e}")
        else:
            # Сохранение из визуальной формы
            cfg = self.config_controller.config

            if "paths" not in cfg:
                cfg["paths"] = {}
            if "network" not in cfg:
                cfg["network"] = {}
            if "backup" not in cfg:
                cfg["backup"] = {}

            cfg["paths"]["master_file"] = self.master_file_var.get()
            cfg["paths"]["archive_dir"] = self.archive_dir_var.get()

            roots = list(self.sources_listbox.get(0, tk.END))
            cfg["paths"]["source_roots"] = roots

            drives_str = self.drive_letters_var.get()
            drives = [d.strip().upper() for d in drives_str.split(",") if d.strip()]
            cfg["network"]["drive_letters"] = drives

            try:
                keep = int(self.excel_keep_var.get())
                cfg["backup"]["excel_keep"] = keep
            except (ValueError, tk.TclError):
                messagebox.showerror("Ошибка сохранения", "Количество бэкапов должно быть целым числом!")
                return

            try:
                self.config_controller.save_config()
                self._refresh_settings()
                messagebox.showinfo("Успех", "Настройки успешно сохранены!")
            except Exception as e:
                messagebox.showerror("Ошибка сохранения", f"Не удалось сохранить настройки:\n{e}")

    def _toggle_settings_mode(self) -> None:
        if self.settings_mode == "visual":
            self.settings_mode = "yaml"
            self.mode_btn.configure(text="Визуальный режим")
            self.settings_visual_frame.pack_forget()
            self._refresh_settings()
            self.settings_yaml_frame.pack(fill=tk.BOTH, expand=True)
        else:
            self.settings_mode = "visual"
            self.mode_btn.configure(text="Режим YAML-кода")
            self.settings_yaml_frame.pack_forget()
            self._refresh_settings()
```

```
        self.settings_visual_frame.pack(fill=tk.BOTH, expand=True)

    def _browse_master_file(self) -> None:
        path = filedialog.askopenfilename(
            title="Выберите основной сводный отчет",
            filetypes=[("Excel Files", "*.xlsx;*.xlsb"), ("All Files", "*.*")]
        )
        if path:
            try:
                rel_path = Path(path).relative_to(self.repo_root)
                self.master_file_var.set(str(Path("..") / rel_path))
            except ValueError:
                self.master_file_var.set(path)

    def _browse_archive_dir(self) -> None:
        path = filedialog.askdirectory(title="Выберите папку для резервных копий")
        if path:
            try:
                rel_path = Path(path).relative_to(self.repo_root)
                self.archive_dir_var.set(str(Path("..") / rel_path))
            except ValueError:
                self.archive_dir_var.set(path)

    def _add_source_root(self) -> None:
        path = filedialog.askdirectory(title="Выберите папку с отчетами кураторов")
        if path:
            self.sources_listbox.insert(tk.END, path)

    def _remove_source_root(self) -> None:
        selected = self.sources_listbox.curselection()
        if selected:
            self.sources_listbox.delete(selected[0])

    def _selected_report_path(self) -> Path | None:
        selected = self.reports_tree.selection()
        if not selected:
            return None
        values = self.reports_tree.item(selected[0])["values"]
        if len(values) < 5 or not values[4]:
            return None
        return Path(str(values[4]))

    def _selected_result_path(self) -> Path | None:
        selected = self.results_tree.selection()
        if not selected:
            return None
        values = self.results_tree.item(selected[0])["values"]
        if len(values) < 4 or not values[3]:
            return None
        return Path(str(values[3]))

    def _open_selected_report(self) -> None:
        self._open_path(self._selected_report_path())

    def _open_selected_report_folder(self) -> None:
        path = self._selected_report_path()
        if path and path.is_file():
            path = path.parent
        self._open_path(path)

    def _open_selected_result(self) -> None:
        self._open_path(self._selected_result_path())

    def _open_backup_folder(self) -> None:
        for item in collect_report_paths(self.config_controller, self.app_root):
            if item.label == "Backup":
                self._open_path(item.path)
                return
        self._show_missing("Backup-папка не настроена")

    def _open_report_by_label(self, label: str) -> None:
        for item in collect_report_paths(self.config_controller, self.app_root):
            if item.label == label:
                self._open_path(item.path)
                return
        self._show_missing(f"Путь не найден: {label}")

    def _recent_log_errors(self) -> list[str]:
        return read_recent_log_errors(
            [self.repo_root / "startup_log.txt", self.repo_root / "gui_error_log.txt"]
        )

    def _open_path(self, path: Path | None) -> None:
```

```
        safe_open_existing_path(path, open_path, self._show_missing)

    def _show_missing(self, message: str) -> None:
        messagebox.showwarning("Путь не найден", message)
```

APP\core\ui\ui_dialogs.py

273 lines • 10,467 bytes

```
import os
import tkinter as tk
from tkinter import ttk, filedialog
from pathlib import Path
from typing import Callable, List, Optional


def show_multi_file_select_dialog(
    parent,
    initial_paths: Optional[List[str]] = None,
    title: str = "Выберите файлы",
    filetypes: Optional[List[tuple]] = None,
) -> List[str]:
    """Modal dialog for selecting multiple files with a live preview.

    Replaces the previous *askopenfilenames → askyesno → loop* pattern:
    the user adds files via the file picker one or more times, sees
    a live list of name/size/mtime, can remove individual entries,
    and confirms with a single click.  The dialog is empty until at
    least one file has been added; cancelling returns the original
    list (so the user can re-open and keep working with it).
    """
    if filetypes is None:
        filetypes = [("Excel files", "*.xlsx *.xlsm *.xlsb"), ("All files", "*.*")]

    dialog = tk.Toplevel(parent)
    dialog.title(title)
    dialog.geometry("800x500")
    dialog.minsize(480, 280)
    dialog.transient(parent)
    dialog.grab_set()

    _title_label = ttk.Label(
        dialog, text=f"{title} — выбрано: 0", padding=(10, 10, 10, 0)
    )
    _title_label.pack(anchor=tk.W)

    list_frame = ttk.Frame(dialog, padding=(10, 4))
    list_frame.pack(fill=tk.BOTH, expand=True)
    tree = ttk.Treeview(
        list_frame,
        columns=("path", "size", "mtime"),
        show="headings",
        height=12,
        selectmode="extended",
    )
    tree.heading("path", text="Путь / файл")
    tree.heading("size", text="Размер")
    tree.heading("mtime", text="Изменён")
    tree.column("path", width=360, anchor=tk.W)
    tree.column("size", width=80, anchor=tk.E)
    tree.column("mtime", width=130, anchor=tk.W)
    tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    scroll = ttk.Scrollbar(list_frame, orient=tk.VERTICAL, command=tree.yview)
    scroll.pack(side=tk.RIGHT, fill=tk.Y)
    tree.configure(yscrollcommand=scroll.set)

    def _format_size(n: int) -> str:
        for unit in ("Б", "КБ", "МБ", "ГБ"):
            if n < 1024:
                return f"{n:.0f} {unit}" if unit == "Б" else f"{n:.1f} {unit}"
            n /= 1024
        return f"{n:.1f} ТБ"

    def _refresh(paths: List[str]) -> None:
        for row in tree.get_children():
            tree.delete(row)
        for p in paths:
            pp = Path(p)
            display_path = str(pp.parent)
            try:
                st = os.stat(p)
                size = _format_size(st.st_size)
                mtime = (
                    Path(p)
                    .stat()
                    .st_mtime
                )
                from datetime import datetime
                mtime_str = datetime.fromtimestamp(mtime).strftime(
                    "%d.%m.%Y %H:%M"
                )
                value_path = f"{display_path}\\{pp.name}"
```

```
                tree.insert("", tk.END, iid=p, values=(value_path, size, mtime_str))
            except OSError:
                tree.insert(
                    "", tk.END, iid=p,
                    values=(f"{display_path}\\{pp.name} (нет доступа)", "-", "-"),
                )
        _title_label.configure(text=f"{title} — выбрано: {len(paths)}")

    selected: List[str] = list(initial_paths or [])
    _refresh(selected)

    def _add() -> None:
        initial_dir = None
        if selected:
            initial_dir = str(Path(selected[0]).parent)
        new = filedialog.askopenfilenames(
            title="Добавить файлы",
            filetypes=filetypes,
            initialdir=initial_dir,
        )
        if not new:
            return
        seen = set(selected)
        for p in new:
            if p not in seen:
                selected.append(p)
                seen.add(p)
        _refresh(selected)

    def _remove() -> None:
        for iid in tree.selection():
            if iid in selected:
                selected.remove(iid)
        _refresh(selected)

    def _ok() -> None:
        dialog.destroy()

    def _cancel() -> None:
        # Cancelling reverts to the *original* list — the user can
        # always re-open the dialog and start over without losing
        # what they had staged.
        selected[:] = list(initial_paths or [])
        dialog.destroy()

    btn_frame = ttk.Frame(dialog, padding=10)
    btn_frame.pack(side=tk.BOTTOM, fill=tk.X)
    ttk.Button(btn_frame, text=" Добавить ещё", command=_add).pack(side=tk.LEFT)
    ttk.Button(btn_frame, text=" Удалить", command=_remove).pack(side=tk.LEFT, padx=(5, 0))
    ttk.Button(btn_frame, text="Готово", command=_ok).pack(side=tk.RIGHT)
    ttk.Button(btn_frame, text="Отмена", command=_cancel).pack(side=tk.RIGHT, padx=5)

    dialog.wait_window()
    return list(selected)


def show_attach_file_dialog(
    parent,
    employees: List[str],
    on_attach: Callable[[str, List[str], bool], None],
    preselect: Optional[str] = None,
):
    dialog = tk.Toplevel(parent)
    dialog.title("Привязать файлы")
    dialog.geometry("450x400")
    dialog.transient(parent)
    dialog.grab_set()

    ttk.Label(dialog, text="Выберите сотрудника:", padding=10).pack()
    employee_var = tk.StringVar()
    employee_combo = ttk.Combobox(
        dialog,
        textvariable=employee_var,
        values=employees,
        # Pre-typed characters narrow the dropdown — Tk's built-in
        # ``postcommand`` filters via the *starts-with* heuristic, which
        # is good enough for ≤100 employees.
    )
    employee_combo.pack(fill=tk.X, padx=10, pady=5)
    if preselect and preselect in employees:
        # Pre-select the row that was right-clicked on, so the user
        # does not have to re-pick from the dropdown.
        employee_var.set(preselect)
```

```
    employee_combo.focus_set()

    selected_files_frame = ttk.LabelFrame(dialog, text="Выбранные файлы", padding=6)
    selected_files_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
    files_listbox = tk.Listbox(selected_files_frame, height=6)
    files_listbox.pack(fill=tk.BOTH, expand=True)

    append_var = tk.BooleanVar(value=True)
    ttk.Checkbutton(dialog, text="Добавить к существующим", variable=append_var).pack(pady=5)

    def pick_files() -> None:
        paths = filedialog.askopenfilenames(title="Файлы для привязки")
        for p in paths:
            files_listbox.insert(tk.END, p)

    def pick() -> None:
        emp_name = employee_var.get().strip()
        if not emp_name:
            return
        files = list(files_listbox.get(0, tk.END))
        if not files:
            # No files staged yet — open the picker once before bailing.
            paths = filedialog.askopenfilenames(
                title=f"Файлы для {emp_name}"
            )
            files = list(paths)
        if not files:
            return
        on_attach(emp_name, files, append_var.get())
        dialog.destroy()

    btn_frame = ttk.Frame(dialog, padding=10)
    btn_frame.pack(fill=tk.X)
    ttk.Button(btn_frame, text="Выбрать файлы", command=pick_files).pack(side=tk.LEFT)
    ttk.Button(btn_frame, text="Привязать", command=pick).pack(side=tk.RIGHT)
    ttk.Button(btn_frame, text="Отмена", command=dialog.destroy).pack(side=tk.RIGHT, padx=5)


def ask_employee_name(existing_names: Optional[List[str]] = None) -> str:
    """Open a custom Add-Employee dialog with realtime validation.

    The dialog is modal, gives the user live feedback (red/green
    border + tooltip) on empty/whitespace/duplicate input, and only
    enables the OK button when the name is valid.  The result is
    returned as a stripped string, or ``""`` if the user cancelled.
    """
    existing = {n.strip().lower() for n in (existing_names or []) if n and n.strip()}

    dialog = tk.Toplevel()
    dialog.title("Добавить сотрудника")
    dialog.geometry("380x200")
    dialog.transient(dialog.master)
    dialog.grab_set()
    dialog.resizable(False, False)

    ttk.Label(dialog, text="Имя сотрудника:", padding=(10, 10, 10, 0)).pack(anchor=tk.W)

    name_var = tk.StringVar()
    entry = ttk.Entry(dialog, textvariable=name_var, width=40)
    entry.pack(fill=tk.X, padx=10, pady=4)
    entry.focus_set()

    status_var = tk.StringVar()
    status_label = ttk.Label(dialog, textvariable=status_var, padding=(10, 0))
    status_label.pack(fill=tk.X)

    btn_frame = ttk.Frame(dialog, padding=10)
    btn_frame.pack(fill=tk.X)
    cancel_btn = ttk.Button(btn_frame, text="Отмена", command=dialog.destroy)
    cancel_btn.pack(side=tk.RIGHT, padx=(5, 0))
    ok_btn = ttk.Button(btn_frame, text="Добавить", command=dialog.destroy)
    ok_btn.pack(side=tk.RIGHT)
    ok_btn.state(["disabled"])

    result_holder: List[str] = [""]

    def _validate(_event=None) -> None:
        raw = name_var.get()
        name = raw.strip()
        if not raw:
            status_var.set("")
            ok_btn.state(["disabled"])
            return
```

```
        if not name:
            status_var.set("Имя не может состоять только из пробелов")
            ok_btn.state(["disabled"])
            return
        if name.lower() in existing:
            status_var.set(f"Сотрудник «{name}» уже существует")
            ok_btn.state(["disabled"])
            return
        status_var.set(f"Готово к добавлению: «{name}»")
        ok_btn.state(["!disabled"])

    def _submit() -> None:
        # Only invoked when the OK button is enabled, so the input is
        # already valid by the time we get here.
        result_holder[0] = name_var.get().strip()
        dialog.destroy()

    ok_btn.configure(command=_submit)
    name_var.trace_add("write", lambda *_args: _validate())
    entry.bind("<Return>", lambda _e: _submit() if "disabled" not in ok_btn.state() else None)
    entry.bind("<Escape>", lambda _e: dialog.destroy())

    dialog.wait_window()
    return result_holder[0]
```

APP\core\ui\ui_employee_table.py

181 lines • 7,077 bytes

```
import tkinter as tk
from tkinter import ttk

from core.ui.debounce import Debouncer
from core.ui.theme import ThemeManager


def _c(key, fallback):
    """Safe theme-color lookup with fallback for keys POLISH-1 may add."""
    return ThemeManager.COLORS.get(key, fallback)


class EmployeeTable(ttk.Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, style='Main.TFrame')
        self.controller = controller
        self._search_debouncer = Debouncer(
            self, self.controller.filter_tree, delay_ms=300
        )
        # Row index for alternating stripes (reset on clear_tree); and
        # the current hovered item id for hover-row highlight.
        self._row_index = 0
        self._hover_row = None
        self._setup_ui()

    def _setup_ui(self):
        # Header with actions
        header_frame = ttk.Frame(self, style='Main.TFrame')
        header_frame.pack(fill=tk.X, pady=(0, 10))

        ttk.Label(
            header_frame,
            text=" СТАТУС ГОТОВНОСТИ ОТЧЕТОВ",
            style='Header.TLabel'
        ).pack(side=tk.LEFT)

        btn_frame = ttk.Frame(header_frame, style='Main.TFrame')
        btn_frame.pack(side=tk.RIGHT)

        ttk.Button(
            btn_frame,
            text=" Папка Master",
            command=self.controller.open_master_folder
        ).pack(side=tk.LEFT, padx=5)

        ttk.Button(
            btn_frame,
            text=" Копировать должников",
            command=self.controller.copy_missing_to_clipboard
        ).pack(side=tk.LEFT)

        # Search
        search_frame = ttk.Frame(self, style='Main.TFrame')
        search_frame.pack(fill=tk.X, pady=(0, 5))

        self.search_var = tk.StringVar()
        self.search_var.trace_add("write", lambda *args: self._search_debouncer())

        ttk.Entry(
            search_frame,
            textvariable=self.search_var
        ).pack(side=tk.LEFT, fill=tk.X, expand=True)

        # Treeview
        self.tree_frame = ttk.Frame(self)
        self.tree_frame.pack(fill=tk.BOTH, expand=True)

        self.tree = ttk.Treeview(
            self.tree_frame,
            columns=("Employee", "Status", "Path"),
            show='headings',
            selectmode='browse'
        )
        self.tree.heading("Employee", text="Сотрудник")
        self.tree.heading("Status", text="Статус")
        self.tree.heading("Path", text="Файл отчета")

        self.tree.column("Employee", width=220, stretch=False)
        self.tree.column("Status", width=150, stretch=False)
        self.tree.column("Path", width=400, stretch=True)

        # Alternating row stripes + hover highlight (POLISH-3).
        # Status tags ("ready"/"missing"/"partial") get a coloured
```

```
        # foreground so the user can scan readiness at a glance.
        self._configure_tags()
        self.tree.bind("<Motion>", self._on_tree_motion, add="+")
        self.tree.bind("<Leave>", self._on_tree_leave, add="+")

        self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        # Scrollbar
        scrollbar = ttk.Scrollbar(self.tree_frame, orient=tk.VERTICAL, command=self.tree.yview)
        self.tree.configure(yscrollcommand=scrollbar.set)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Context Menu
        self.menu = tk.Menu(self.tree, tearoff=0)
        self.menu.add_command(label=" Открыть папку", command=self.controller.open_employee_folder)
        self.menu.add_command(label=" Открыть файл", command=self.controller.open_employee_file)
        self.menu.add_separator()
        self.menu.add_command(label=" Удалить сотрудника", command=self.controller.delete_employee)

        self.tree.bind("<Button-3>", self._show_context_menu)
        self.tree.bind("<Double-1>", lambda e: self.controller.open_employee_file())
        self.bind("<Destroy>", self._on_destroy, add="+")

    def _configure_tags(self) -> None:
        """Apply current theme colours to Treeview row tags.

        Called at init and again on every ``refresh_theme()`` so the
        stripes/status colours follow light↔dark switches.
        """
        self.tree.tag_configure("odd", background=_c("bg_panel", "#f3f4f6"))
        self.tree.tag_configure("even", background=_c("bg_sidebar", "#ffffff"))
        self.tree.tag_configure(
            "hover", background=_c("border_subtle", _c("trough", "#e5e7eb"))
        )
        self.tree.tag_configure("ready", foreground=_c("success", "#28a745"))
        self.tree.tag_configure("missing", foreground=_c("danger", "#dc3545"))

    def refresh_theme(self) -> None:
        """Re-apply theme-dependent tag colours after a palette switch."""
        self._configure_tags()

    def _on_destroy(self, _event):
        self._search_debouncer.cancel()

    def _show_context_menu(self, event):
        item = self.tree.identify_row(event.y)
        if item:
            self.tree.selection_set(item)
            self.menu.post(event.x_root, event.y_root)

    def get_search_query(self):
        return self.search_var.get().lower()

    def clear_tree(self):
        self._row_index = 0
        self._hover_row = None
        for item in self.tree.get_children():
            self.tree.delete(item)

    def insert_row(self, values, tag):
        # Alternate stripe tag (odd/even) is merged with the caller's
        # status tag so both the row background AND the status text
        # colour apply — tags are additive in ttk.Treeview.
        stripe = "odd" if self._row_index % 2 else "even"
        self._row_index += 1
        tags = [stripe]
        if tag:
            tags.append(tag)
        self.tree.insert('', tk.END, values=values, tags=tuple(tags))

    def _on_tree_motion(self, event):
        """Highlight the row under the cursor for visual scanning."""
        row = self.tree.identify_row(event.y)
        if row == self._hover_row:
            return
        if self._hover_row:
            self._strip_hover_tag(self._hover_row)
        if row:
            tags = [t for t in self.tree.item(row, "tags") if t != "hover"]
            tags.append("hover")
            self.tree.item(row, tags=tags)
        self._hover_row = row or None
```

```
def _on_tree_leave(self, _event):
    if self._hover_row:
        self._strip_hover_tag(self._hover_row)
        self._hover_row = None

def _strip_hover_tag(self, item):
    existing = [t for t in self.tree.item(item, "tags") if t != "hover"]
    self.tree.item(item, tags=existing)

def get_selected_employee(self):
    sel = self.tree.selection()
    if not sel:
        return None
    return self.tree.item(sel[0])['values'][0]
```

APP\core\ui\ui_log_panel.py

58 lines • 2,107 bytes

```
import tkinter as tk
from tkinter import ttk, scrolledtext
from core.ui.theme import ThemeManager

class LogPanel(ttk.Frame):
    def __init__(self, parent, controller):
        super().__init__(parent, style='Sidebar.TFrame')
        self.controller = controller
        self._setup_ui()

    def _setup_ui(self):
        header_frame = ttk.Frame(self, style='Sidebar.TFrame')
        header_frame.pack(fill=tk.X, pady=(10, 2))

        ttk.Label(
            header_frame,
            text=" Логи:",
            font=ThemeManager.FONTS['caption'],
            background=ThemeManager.COLORS['bg_sidebar']
        ).pack(side=tk.LEFT)

        ttk.Button(
            header_frame,
            text=" Очистить",
            command=self.clear_logs
        ).pack(side=tk.RIGHT)

        self.log_area = scrolledtext.ScrolledText(
            self,
            state='disabled',
            font=ThemeManager.FONTS['mono'],
            borderwidth=1,
            relief='solid',
            bg=ThemeManager.COLORS["bg_panel"],
            fg=ThemeManager.COLORS["text"],
            insertbackground=ThemeManager.COLORS["text"],
            selectbackground=ThemeManager.COLORS["primary"],
            selectforeground=ThemeManager.COLORS["text_inverse"],
        )
        # ScrolledText uses explicit colours that do not follow ttk
        # styles — register so dark-mode switches re-colour the bg/fg.
        ThemeManager.register(self.log_area, bg_key="bg_panel", fg_key="text")
        self.log_area.pack(fill=tk.BOTH, expand=True, padx=8, pady=6)

    def clear_logs(self):
        self.log_area.configure(state='normal')
        self.log_area.delete('1.0', tk.END)
        self.log_area.configure(state='disabled')

    def append_log(self, message):
        self.log_area.configure(state='normal')
        self.log_area.insert(tk.END, message + '\n')
        self.log_area.configure(state='disabled')
        self.log_area.yview(tk.END)

    def get_widget(self):
        return self.log_area
```

APP\core\ui\ui_sidebar.py

457 lines • 18,721 bytes

```
import sys
import tkinter as tk
from tkinter import ttk
from tkcalendar import DateEntry
from datetime import datetime, timedelta
from core.ui.theme import ThemeManager
from core.ui.debounce import Debouncer


def _last_day_of_month(month_start: datetime) -> datetime:
    if month_start.month == 12:
        return month_start.replace(day=31)
    return (month_start.replace(day=28) + timedelta(days=4)).replace(
        day=1
    ) - timedelta(days=1)


def _previous_month_start(today: datetime) -> datetime:
    if today.month == 1:
        return today.replace(year=today.year - 1, month=12, day=1)
    return today.replace(month=today.month - 1, day=1)


def get_default_period_range(today: datetime | None = None) -> tuple[datetime, datetime]:
    today = today or datetime.now()
    if today.day == 1:
        start = _previous_month_start(today)
        return start, _last_day_of_month(start)

    start = today.replace(day=1)
    return start, today


class Sidebar(ttk.Frame):
    def __init__(self, parent, controller):
        screen_w = parent.winfo_screenwidth()
        sidebar_w = max(260, min(360, int(screen_w * 0.22)))
        super().__init__(parent, width=sidebar_w, padding="10", style="Sidebar.TFrame")
        self.controller = controller
        self.pack_propagate(False)
        # Debounce date changes so clicking through dates in the picker
        # does not fire a full refresh per click.
        self._date_debouncer = Debouncer(
            self, self.controller.refresh_status, delay_ms=250
        )
        self.canvas = tk.Canvas(
            self,
            borderwidth=0,
            highlightthickness=0,
            background=ThemeManager.COLORS["bg_sidebar"],
        )
        # Canvas uses explicit colours (not a ttk style), so register
        # it to be re-themed on ThemeManager.set_theme().
        ThemeManager.register(self.canvas, bg_key="bg_sidebar")
        self.scrollbar = ttk.Scrollbar(
            self, orient=tk.VERTICAL, command=self.canvas.yview
        )
        self.content = ttk.Frame(self.canvas, style="Sidebar.TFrame")
        self._content_window = self.canvas.create_window(
            (0, 0), window=self.content, anchor=tk.NW
        )
        self.canvas.configure(yscrollcommand=self.scrollbar.set)
        self.canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
        self.content.bind("<Configure>", self._on_content_configure)
        self.canvas.bind("<Configure>", self._on_canvas_configure)
        self.canvas.bind("<Enter>", self._bind_mousewheel)
        self.canvas.bind("<Leave>", self._unbind_mousewheel)
        self._setup_ui()

    def _on_content_configure(self, _event):
        self.canvas.configure(scrollregion=self.canvas.bbox("all"))

    def _on_canvas_configure(self, event):
        self.canvas.itemconfigure(self._content_window, width=event.width)

    def _bind_mousewheel(self, _event):
        # Bind on the canvas only — the previous implementation used
        # ``bind_all``, which routed the wheel to every widget while the
        # cursor was over the sidebar (a focus-eating overreach).
        self.canvas.bind("<MouseWheel>", self._on_mousewheel)
        # X11 / Linux use discrete Button-4 (up) and Button-5 (down) for
        # the wheel — ``<MouseWheel>`` does not fire on those platforms.
```

```
        self.canvas.bind("<Button-4>", self._on_button4)
        self.canvas.bind("<Button-5>", self._on_button5)

    def _unbind_mousewheel(self, _event):
        self.canvas.unbind("<MouseWheel>")
        self.canvas.unbind("<Button-4>")
        self.canvas.unbind("<Button-5>")

    def _on_mousewheel(self, event):
        # macOS reports delta as ±1 (small integers); Windows uses
        # multiples of 120. ``int(delta / 120)`` truncates to zero on
        # macOS, so use the platform-specific scaling.
        if sys.platform == "darwin":
            steps = -event.delta
        else:
            steps = -int(event.delta / 120)
        self.canvas.yview_scroll(steps, "units")

    def _on_button4(self, _event):
        self.canvas.yview_scroll(-1, "units")
        return "break"

    def _on_button5(self, _event):
        self.canvas.yview_scroll(1, "units")
        return "break"

    def _setup_ui(self):
        ttk.Label(
            self.content,
            text=" ПАНЕЛЬ УПРАВЛЕНИЯ",
            style="Header.TLabel",
            background=ThemeManager.COLORS["bg_sidebar"],
        ).pack(pady=(0, 10), anchor=tk.W)

        # Date Selection
        date_frame = ttk.LabelFrame(self.content, text=" ВЫБОР ДАТЫ", padding=6)
        date_frame.pack(fill=tk.X, pady=(4, 8))

        self.date_entry = DateEntry(
            date_frame,
            width=12,
            background=ThemeManager.COLORS["primary"],
            foreground="white",
            borderwidth=2,
            date_pattern="dd.mm.yyyy",
            locale="ru_RU",
            headersbackground=ThemeManager.COLORS["primary"],
            selectbackground=ThemeManager.COLORS["accent"]
            if "accent" in ThemeManager.COLORS
            else "#007bff",
        )
        self.date_entry.pack(pady=(0, 4), fill=tk.X)
        self.date_entry.bind(
            "<<DateEntrySelected>>", lambda e: self._date_debouncer()
        )
        self.bind("<Destroy>", self._on_destroy, add="+")

        self.refresh_btn = ttk.Button(
            date_frame,
            text=" Обновить статус",
            command=self.controller.refresh_status,
        )
        self.refresh_btn.pack(fill=tk.X, pady=(0, 4))

        # Employee Management
        emp_frame = ttk.LabelFrame(self.content, text=" СОТРУДНИКИ", padding=6)
        emp_frame.pack(fill=tk.X, pady=(0, 8))

        self.add_employee_btn = ttk.Button(
            emp_frame, text=" Добавить", command=self.controller.add_employee_dialog
        )
        self.add_employee_btn.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 5))

        self.attach_btn = ttk.Button(
            emp_frame, text=" Привязать", command=self.controller.attach_file_dialog
        )
        self.attach_btn.pack(side=tk.LEFT, fill=tk.X, expand=True)

        # Consolidation Actions
        action_frame = ttk.LabelFrame(self.content, text=" СБОРКА", padding=6)
        action_frame.pack(fill=tk.X, pady=(0, 8))

        ttk.Label(action_frame, text="Кураторский отчет:").pack(anchor=tk.W)
```

```
self.curator_var = tk.StringVar(value="Общий отчет")
self.curator_combo = ttk.Combobox(
    action_frame,
    textvariable=self.curator_var,
    state="readonly",
    values=["Общий отчет", "Куратор Борис", "Куратор Иван"],
)
self.curator_combo.pack(fill=tk.X, pady=(0, 4))

self.consolidate_btn = ttk.Button(
    action_frame,
    text=" СОБРАТЬ ДЕНЬ",
    style="Action.TButton",
    command=self.controller.run_consolidation,
)
self.consolidate_btn.pack(fill=tk.X, pady=(0, 4))

self.consolidate_all_btn = ttk.Button(
    action_frame,
    text=" СОБРАТЬ ВСЕ ДАТЫ",
    command=self.controller.run_consolidation_all_dates,
)
self.consolidate_all_btn.pack(fill=tk.X, pady=(0, 4))

# Monthly Range Section
month_frame = ttk.LabelFrame(self.content, text=" СБОРКА ЗА ПЕРИОД", padding=6)
month_frame.pack(fill=tk.X, pady=(0, 8))

ttk.Label(month_frame, text="С:").pack(anchor=tk.W)
self.month_start_entry = DateEntry(
    month_frame,
    width=12,
    background=ThemeManager.COLORS["primary"],
    foreground="white",
    borderwidth=2,
    date_pattern="dd.mm.yyyy",
    locale="ru_RU",
    headersbackground=ThemeManager.COLORS["primary"],
)
self.month_start_entry.pack(pady=(0, 4), fill=tk.X)
# По умолчанию — текущий месяц, а 1-го числа — прошлый полный месяц.
today = datetime.now()
default_start, default_end = get_default_period_range(today)
self.month_start_entry.set_date(default_start)

ttk.Label(month_frame, text="По:").pack(anchor=tk.W)
self.month_end_entry = DateEntry(
    month_frame,
    width=12,
    background=ThemeManager.COLORS["primary"],
    foreground="white",
    borderwidth=2,
    date_pattern="dd.mm.yyyy",
    locale="ru_RU",
    headersbackground=ThemeManager.COLORS["primary"],
)
self.month_end_entry.pack(pady=(0, 4), fill=tk.X)
self.month_end_entry.set_date(default_end)
# Realtime validation: when the user changes either date, refresh
# the visual state of the affected DateEntry (red border when
# the end is earlier than the start) and update the tooltip.
self.month_start_entry.bind(
    "<<DateEntrySelected>>", lambda e: self._validate_period_range()
)
self.month_end_entry.bind(
    "<<DateEntrySelected>>", lambda e: self._validate_period_range()
)
self._range_warning_tooltip = None
self._range_invalid = False

# Quick month preset buttons
preset_frame = ttk.Frame(month_frame)
preset_frame.pack(fill=tk.X, pady=(0, 4))
ttk.Button(
    preset_frame, text="Этот месяц", command=self._set_current_month
).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 3))
ttk.Button(
    preset_frame, text="Пред. месяц", command=self._set_previous_month
).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(3, 0))

self.consolidate_month_btn = ttk.Button(
    month_frame,
    text=" СОБРАТЬ ПЕРИОД",
```

```
            style="Action.TButton",
            command=self.controller.run_consolidation_month,
        )
        self.consolidate_month_btn.pack(fill=tk.X, pady=(0, 4))

        self.rsch_month_btn = ttk.Button(
            month_frame,
            text=" СОБРАТЬ РСЧ ЗА ПЕРИОД",
            command=self.controller.run_rsch_report_month,
        )
        self.rsch_month_btn.pack(fill=tk.X)

        self.rsch_btn = ttk.Button(
            action_frame,
            text=" СОБРАТЬ РСЧ",
            command=self.controller.run_rsch_report,
        )
        self.rsch_btn.pack(fill=tk.X, pady=(0, 4))

        self.reconciliation_btn = ttk.Button(
            action_frame,
            text=" СВЕРИТЬ С РСЧ",
            command=self.controller.run_reconciliation_report,
        )
        self.reconciliation_btn.pack(fill=tk.X, pady=(0, 4))

        self.auto_reconciliation_btn = ttk.Button(
            action_frame,
            text=" АВТОСВЕРКА",
            style="Action.TButton",
            command=self.controller.run_auto_reconciliation_day,
        )
        self.auto_reconciliation_btn.pack(fill=tk.X, pady=(0, 4))

        # Theme switcher: 3 options (light/dark/system). The "system"
        # option is auto-disabled on Windows 8 where the registry key
        # we read is not present — there's nothing to follow.
        theme_frame = ttk.LabelFrame(self.content, text=" ТЕМА", padding=6)
        theme_frame.pack(fill=tk.X, pady=(0, 8))
        self.theme_var = tk.StringVar(value="light")
        from core.ui.system_theme import is_windows_8_or_earlier
        win8 = is_windows_8_or_earlier()
        for value, label in (
            ("light", " Светлая"),
            ("dark", " Тёмная"),
        ):
            ttk.Radiobutton(
                theme_frame,
                text=label,
                variable=self.theme_var,
                value=value,
                command=self._on_theme_changed,
            ).pack(anchor=tk.W)
        # "Системная" only useful on Win 10/11, macOS, modern Linux.
        rb_system = ttk.Radiobutton(
            theme_frame,
            text=" Системная" + (" (нет данных)" if win8 else ""),
            variable=self.theme_var,
            value="system",
            command=self._on_theme_changed,
        )
        rb_system.pack(anchor=tk.W)
        if win8:
            rb_system.state(["disabled"])
        from core.ui.tooltip import Tooltip
        Tooltip(rb_system, "Следовать теме ОС. На Windows 8/8.1 недоступно — ОС не публикует тему.")

    def _on_theme_changed(self):
        """Forward the chosen theme to the master controller."""
        value = self.theme_var.get()
        setter = getattr(self.controller, "set_theme", None)
        if callable(setter):
            setter(value)

    def _on_destroy(self, _event):
        # Cancel any pending debounced calls so a fast destroy during
        # the user's date-picking does not fire a stale refresh.
        self._date_debouncer.cancel()

    def _set_current_month(self):
        today = datetime.now()
        self.month_start_entry.set_date(today.replace(day=1))
        self.month_end_entry.set_date(_last_day_of_month(today))
```

```
def _set_previous_month(self):
    today = datetime.now()
    prev_month = _previous_month_start(today)
    self.month_start_entry.set_date(prev_month)
    self.month_end_entry.set_date(_last_day_of_month(prev_month))
    self._validate_period_range()

def _validate_period_range(self) -> None:
    """Highlight the period DateEntries when end < start.

    The validation is purely visual — the actual guard still lives
    in the build commands, but giving the user immediate feedback
    when the picker shows an inverted range prevents them from
    clicking "СОБРАТЬ ПЕРИОД" only to be told about the error.
    """
    try:
        start = self.month_start_entry.get_date()
        end = self.month_end_entry.get_date()
    except (tk.TclError, ValueError):
        return
    invalid = start > end
    # Tk's ``configure`` does not change border colour on ttk
    # widgets, so we use the surrounding LabelFrame text colour as
    # a soft signal: red when invalid, normal otherwise.
    if invalid == self._range_invalid:
        # No change, but the preset buttons may have just been
        # clicked; nothing to do beyond the early-exit guard.
        return
    self._range_invalid = invalid
    if invalid:
        self.month_start_entry.configure(foreground="#dc3545")
        self.month_end_entry.configure(foreground="#dc3545")
        from core.ui.tooltip import Tooltip
        if self._range_warning_tooltip is None:
            self._range_warning_tooltip = Tooltip(
                self.month_end_entry,
                "Дата окончания раньше даты начала. Поменяйте одну из дат.",
            )
        else:
            self._range_warning_tooltip.set_text(
                "Дата окончания раньше даты начала. Поменяйте одну из дат."
            )
    else:
        self.month_start_entry.configure(foreground="white")
        self.month_end_entry.configure(foreground="white")
        from core.ui.tooltip import Tooltip
        if self._range_warning_tooltip is not None:
            self._range_warning_tooltip.set_text("")

def get_date(self):
    return self.date_entry.get_date()

def get_curator(self):
    value = self.curator_var.get()
    if value == "Куратор Борис":
        return "boris"
    if value == "Куратор Иван":
        return "ivan"
    return None

def get_month_range(self):
    """Возвращает (start_date, end_date) для месячного отчёта."""
    start_date = self.month_start_entry.get_date()
    end_date = self.month_end_entry.get_date()
    today = datetime.now().date()
    if end_date > today:
        end_date = today
    return start_date, end_date

def set_refresh_state(self, state):
    self.refresh_btn.config(state=state)

def set_consolidate_state(self, state):
    self.consolidate_btn.config(state=state)

def set_consolidate_all_state(self, state):
    self.consolidate_all_btn.config(state=state)

def set_consolidate_month_state(self, state):
    self.consolidate_month_btn.config(state=state)

def set_rsch_state(self, state):
    self.rsch_btn.config(state=state)
```

```
        if hasattr(self, "rsch_month_btn"):
            self.rsch_month_btn.config(state=state)

    def set_reconciliation_state(self, state):
        self.reconciliation_btn.config(state=state)
        self.auto_reconciliation_btn.config(state=state)

    # --- global state ---------------------------------------------------

    _OPERATION_BUTTONS_ATTRS = (
        "refresh_btn",
        "add_employee_btn",
        "attach_btn",
        "consolidate_btn",
        "consolidate_all_btn",
        "consolidate_month_btn",
        "rsch_btn",
        "rsch_month_btn",
        "reconciliation_btn",
        "auto_reconciliation_btn",
    )

    def apply(self, snap) -> None:
        """Apply global operation state to all action widgets.

        When any operation is running, every action button is disabled.
        This fixes the original bug where only the clicked button was disabled
        and the user could fire a second operation that conflicted with the first.
        """
        busy = snap.current.value != ""
        for attr in self._OPERATION_BUTTONS_ATTRS:
            btn = getattr(self, attr, None)
            if btn is None:
                continue
            try:
                if busy:
                    btn.state(["disabled"])
                else:
                    btn.state(["!disabled"])
            except tk.TclError:
                pass
```

APP\core\ui\ui_stats_panel.py

90 lines • 3,616 bytes

```
import tkinter as tk
from tkinter import ttk


class StatsPanel(ttk.LabelFrame):
    """Two progress bars: readiness (top, small) and operation (big, main).

    The two are intentionally separated so the user can see at a glance:

    * ``readiness`` — fraction of employees whose reports are ready for
      the currently selected date. Driven by
      :meth:`EmployeeListViewModel.get_progress_percent` whenever the
      employee table is re-synced.
    * ``operation`` — fraction of the long-running build/RSC/
      reconciliation task that is currently running. Driven by
      :class:`OperationState` snapshots.
    """

    def __init__(self, parent, controller):
        super().__init__(parent, text=" СТАТИСТИКА ДНЯ", padding=10)
        self.controller = controller
        self._setup_ui()

    def _setup_ui(self):
        self.stats_label = ttk.Label(
            self,
            text="Сотрудников: 0\nОтчетов готово: 0\nОшибок: 0",
            justify=tk.LEFT,
            style="Sidebar.TLabel",
            wraplength=280,
        )
        self.stats_label.pack(fill=tk.X)

        # Readiness (small, on top). Tracks how many of today's reports
        # are present on disk — a "how much work is left" indicator.
        self._readiness_caption = ttk.Label(
            self,
            text="Готовность сотрудников:",
            style="Sidebar.TLabel",
        )
        self._readiness_caption.pack(fill=tk.X, pady=(10, 0))
        self.readiness_var = tk.DoubleVar()
        self.readiness_bar = ttk.Progressbar(
            self,
            variable=self.readiness_var,
            maximum=100,
        )
        self.readiness_bar.pack(fill=tk.X)

        # Operation (big, main). Tracks the running consolidation/RSC/
        # reconciliation operation, identical to what
        # OperationStatusBar shows but as a graphical bar.
        self._operation_caption = ttk.Label(
            self,
            text="Текущая операция:",
            style="Sidebar.TLabel",
        )
        self._operation_caption.pack(fill=tk.X, pady=(8, 0))
        self.operation_var = tk.DoubleVar()
        self.operation_bar = ttk.Progressbar(
            self,
            variable=self.operation_var,
            maximum=100,
        )
        self.operation_bar.pack(fill=tk.X, pady=(2, 0))

    def update_stats(self, ready_count, total_count, total_income=None, total_expense=None, error_count=0):
        text = f"Готово: {ready_count}/{total_count}"
        if total_income is not None and total_expense is not None:
            text += f"\nПриход: {total_income:,.2f}\nРасход: {total_expense:,.2f}"
        text += f"\nОшибки: {error_count}"
        self.stats_label.config(text=text)

    def set_readiness(self, value: float) -> None:
        """Set the small readiness bar to ``value`` (0-100)."""
        self.readiness_var.set(max(0.0, min(100.0, value)))

    def set_operation(self, value: float) -> None:
        """Set the big operation bar to ``value`` (0-100)."""
        self.operation_var.set(max(0.0, min(100.0, value)))

    def set_progress(self, value):
        """Back-compat shim. Treats the value as the operation progress
```

```
        because that was the original single-bar use; the readiness
        bar is updated explicitly via :meth:`set_readiness`.
        """
        self.set_operation(value)

    def get_progress(self):
        return self.operation_var.get()
```

APP\core\view_models__init__.py

1 lines • 83 bytes

```
"""ViewModels — слой презентации для MVVM паттерна."""
```

APP\core\view_models\consolidation_viewmodel.py

119 lines • 4,073 bytes

```
"""Consolidation ViewModel — состояние и логика консолидации."""

import threading
from typing import Callable
from datetime import datetime
from pathlib import Path
import logging

logger = logging.getLogger(__name__)


class ConsolidationViewModel:
    def __init__(self, app_controller=None, operation_state=None) -> None:
        self._app_controller = app_controller
        self._operation_state = operation_state
        self._lock = threading.Lock()
        self._is_running = False
        self._progress = 0.0
        self._on_progress: Callable[[float], None] | None = None
        self._on_complete: Callable[[bool, str, Path | None], None] | None = None

    def set_callbacks(
        self,
        on_progress: Callable[[float], None],
        on_complete: Callable[[bool, str, Path | None], None],
    ) -> None:
        self._on_progress = on_progress
        self._on_complete = on_complete

    def _execute(self, action_fn) -> None:
        with self._lock:
            if self._is_running:
                return
            self._is_running = True
            self._progress = 0.0

        try:
            def on_progress(p: float) -> None:
                with self._lock:
                    self._progress = p
                if self._on_progress:
                    self._on_progress(p)

            def on_complete(ok: bool, msg: str, path: Path | None) -> None:
                with self._lock:
                    self._is_running = False
                    self._progress = 100.0 if ok else self._progress
                if self._on_complete:
                    self._on_complete(ok, msg, path)

            action_fn(on_progress, on_complete)
        except Exception:
            with self._lock:
                self._is_running = False
            if self._operation_state is not None:
                try:
                    self._operation_state.end()
                except Exception:
                    logger.exception("operation_state.end() failed in VM _execute")
            raise

    def can_run_single_date(self, missing_count: int) -> bool:
        return missing_count == 0

    def run_consolidation(
        self, app_controller=None, selected_date: datetime | None = None, curator=None,
    ) -> None:
        ctl = app_controller or self._app_controller
        cancel_event = (
            self._operation_state.cancel_event if self._operation_state else None
        )
        self._execute(
            lambda p, c: ctl.run_consolidation(
                selected_date, p, c, curator=curator, cancel_event=cancel_event,
            )
        )

    def run_consolidation_all_dates(
        self, app_controller=None, curator=None,
    ) -> None:
        ctl = app_controller or self._app_controller
        cancel_event = (
            self._operation_state.cancel_event if self._operation_state else None
```

```
        )
        self._execute(
            lambda p, c: ctl.run_consolidation_all_dates(
                p, c, curator=curator, cancel_event=cancel_event,
            )
        )

    def run_consolidation_month(
        self,
        app_controller=None,
        start_date: datetime | None = None,
        end_date: datetime | None = None,
        curator=None,
    ) -> None:
        ctl = app_controller or self._app_controller
        cancel_event = (
            self._operation_state.cancel_event if self._operation_state else None
        )
        self._execute(
            lambda p, c: ctl.run_consolidation_month(
                start_date, end_date, p, c, curator=curator, cancel_event=cancel_event,
            )
        )

    @property
    def is_running(self) -> bool:
        with self._lock:
            return self._is_running

    @property
    def progress(self) -> float:
        with self._lock:
            return self._progress

    def get_button_state(self) -> str:
        return "disabled" if self._is_running else "normal"
```

APP\core\view_models\employee_list_viewmodel.py

224 lines • 8,267 bytes

```
"""Employee List ViewModel — состояние и логика списка сотрудников."""

import threading
from typing import List, Dict, Any, Callable, Optional
from datetime import datetime
from pathlib import Path
import logging
from dataclasses import dataclass

logger = logging.getLogger(__name__)


@dataclass
class EmployeeStatus:
    """Статус одного сотрудника."""

    name: str
    status_text: str  # " ГОТОВО", " НЕТ ФАЙЛА", etc
    path_text: str  # "file.xlsx" или "[3 файлов]"
    tag: str  # "ready", "missing", "partial"

    def to_tuple(self) -> tuple:
        return (self.name, self.status_text, self.path_text, self.tag)


class EmployeeListViewModel:
    """ViewModel для списка сотрудников.

    Отделяет логику:
    - Фильтрация по поисковому запросу
    - Сортировка
    - Подсчёт статистики
    - Определение должников
    """

    def __init__(self) -> None:
        self._lock = threading.Lock()
        # Separate lock for config saves. save_config() is invoked from
        # worker-thread completion callbacks (refresh_status → on_complete)
        # and writes config.yaml to disk; without serialization this races
        # with GUI-thread writes (add_employee, attach_files) and can
        # corrupt the file. Kept distinct from _lock so a slow disk write
        # does not block UI reads of viewmodel state.
        self._save_lock = threading.Lock()
        self._all_employees: List[EmployeeStatus] = []
        self._employees_map: Dict[str, List[Dict[str, Any]]] = {}
        self._search_query: str = ""
        self._rename_detected: bool = False
        self._last_error: Optional[str] = None
        self._on_update: Optional[Callable] = None

    def set_update_callback(self, callback: Callable) -> None:
        """Установить callback при обновлении данных."""
        self._on_update = callback

    # === Commands (действия из UI) ===

    def refresh(
        self,
        app_controller,
        selected_date: datetime,
        on_progress: Callable[[float], None],
        curator=None,
        on_error: Optional[Callable[[str], None]] = None,
        on_complete: Optional[Callable[[], None]] = None,
    ) -> None:
        """Запустить обновление статуса сотрудников."""

        def on_complete_inner(
            data: List[tuple], employees_map: Dict, rename_detected: bool
        ) -> None:
            with self._lock:
                self._all_employees = [
                    EmployeeStatus(name=n, status_text=s, path_text=p, tag=t)
                    for n, s, p, t in data
                ]
                self._employees_map = employees_map
                self._rename_detected = rename_detected
                self._last_error = None

            if rename_detected:
                with self._save_lock:
                    app_controller.config_controller.save_config()
```

```
            if self._on_update:
                self._on_update()
            if callable(on_complete):
                on_complete()

        def handle_error(message: str) -> None:
            with self._lock:
                self._last_error = message
            if callable(on_error):
                on_error(message)

        app_controller.refresh_status(
            selected_date,
            on_progress,
            on_complete_inner,
            curator=curator,
            on_error=handle_error,
        )

    def add_employee(self, config_controller, name: str) -> None:
        """Добавить нового сотрудника."""
        config_controller.add_employee(name)

    def delete_employee(self, config_controller, name: str) -> None:
        """Удалить сотрудника."""
        config_controller.delete_employee(name)

    def attach_files(
        self,
        config_controller,
        name: str,
        paths: List[str],
        append: bool = True,
    ) -> None:
        """Прикрепить файлы к сотруднику."""
        config_controller.attach_files(name, paths, append=append)

    def get_master_path(self, config_controller) -> Optional[Path]:
        """Получить путь к мастер-файлу."""
        p = config_controller.resolve_path(
            config_controller.get_paths().get("master_file", "Master_Report.xlsx")
        )
        return p

    # === Queries (состояние для UI) ===

    @property
    def search_query(self) -> str:
        return self._search_query

    @search_query.setter
    def search_query(self, value: str) -> None:
        self._search_query = value.lower()

    def get_all_names(self) -> List[str]:
        """Return the names of every known employee.

        Used by the add-employee dialog to detect duplicate names
        before the user clicks "Добавить" and by the attach dialog to
        populate the employee combobox.
        """
        with self._lock:
            return [e.name for e in self._all_employees]

    def get_filtered_employees(self) -> List[EmployeeStatus]:
        """Получить отфильтрованный список по поисковому запросу."""
        with self._lock:
            employees = list(self._all_employees)
        if not self._search_query:
            return employees

        return [
            emp
            for emp in employees
            if any(
                self._search_query in x.lower()
                for x in (emp.name, emp.status_text, emp.path_text)
            )
        ]

    def get_ready_count(self) -> int:
        """Количество готовых сотрудников."""
        with self._lock:
```

```
        return sum(1 for e in self._all_employees if " ГОТОВО" in e.status_text)

    def get_total_count(self) -> int:
        """Общее количество."""
        with self._lock:
            return len(self._all_employees)

    def get_missing_names(self) -> List[str]:
        """Имена должников."""
        with self._lock:
            return [e.name for e in self._all_employees if " ГОТОВО" not in e.status_text]

    @property
    def rename_detected(self) -> bool:
        with self._lock:
            return self._rename_detected

    @property
    def employees_map(self) -> Dict[str, List[Dict[str, Any]]]:
        """Return a shallow copy of the employees map for thread safety."""
        with self._lock:
            return self._employees_map.copy()

    def get_selected_employee_path(self, selected_name: Optional[str]) -> Optional[str]:
        """Получить путь выбранного сотрудника."""
        if not selected_name:
            return None
        with self._lock:
            metas = self._employees_map.get(selected_name, [])
        if not metas:
            return None
        meta = metas[0]
        return meta["path"] if isinstance(meta, dict) else meta

    def get_selected_employee_paths(self, selected_name: Optional[str]) -> List[str]:
        """Получить все пути выбранного сотрудника."""
        if not selected_name:
            return []
        with self._lock:
            metas = list(self._employees_map.get(selected_name, []))
        paths: List[str] = []
        for meta in metas:
            path = meta["path"] if isinstance(meta, dict) else meta
            if path:
                paths.append(path)
        return paths

    def get_progress_percent(self) -> float:
        """Процент готовности (для прогресс-бара)."""
        if not self._all_employees:
            return 0.0
        return (self.get_ready_count() / self.get_total_count()) * 100

    @property
    def last_error(self) -> Optional[str]:
        with self._lock:
            return self._last_error
```

APP\headless_run.py

93 lines • 3,246 bytes

```
import os
import sys
import signal
import logging
import threading
import argparse
from datetime import datetime
from pathlib import Path

app_root = Path(__file__).parent.absolute()
if str(app_root) not in sys.path:
    sys.path.insert(0, str(app_root))

from core.bootstrap import Bootstrap  # noqa: E402


def main():
    parser = argparse.ArgumentParser(description="Run headless consolidation.")
    parser.add_argument("--date", type=str, help="Date to consolidate (DD.MM.YYYY)")
    args = parser.parse_args()

    logging.basicConfig(
        level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s"
    )
    logger = logging.getLogger("Headless")
    app_ctl = None
    db = None
    timeout = int(os.environ.get("HEADLESS_RUN_TIMEOUT", "600"))
    cancel_event = threading.Event()

    def _signal_handler(signum, _frame):
        # SIGTERM/SIGINT: signal the worker to cancel at its next
        # checkpoint, then exit. The worker thread is daemon=True so it
        # will be reaped when the process dies, but setting the event
        # lets it flush partial state cleanly.
        logger.warning("Received signal %s, cancelling consolidation...", signum)
        cancel_event.set()
        sys.exit(1)

    signal.signal(signal.SIGTERM, _signal_handler)
    signal.signal(signal.SIGINT, _signal_handler)

    try:
        try:
            _, app_ctl, db = Bootstrap.setup(
                app_root, str(app_root / "config.yaml")
            )
        except RuntimeError as e:
            # Bootstrap signals missing critical dependency (python-calamine)
            # by raising RuntimeError. Print to stderr for shell users / CI.
            print(f"Bootstrap failed: {e}", file=sys.stderr)
            logger.critical(f"Bootstrap failed: {e}")
            sys.exit(1)
        done = threading.Event()

        def on_comp(ok, msg, path):
            if ok:
                logger.info(f"DONE: {msg}")
            else:
                logger.error(msg)
            done.set()

        if args.date:
            target_date = datetime.strptime(args.date, "%d.%m.%Y")
            logger.info(f"Triggering run_consolidation for {args.date}...")
            app_ctl.run_consolidation(
                target_date, lambda p: None, on_comp, cancel_event=cancel_event
            )
        else:
            logger.info("Triggering run_consolidation_all_dates...")
            app_ctl.run_consolidation_all_dates(
                lambda p: None, on_comp, cancel_event=cancel_event
            )

        finished = done.wait(timeout=timeout) if timeout > 0 else done.wait()
        if not finished:
            logger.error("Timed out waiting for consolidation to complete")
            # Signal the worker to cancel at its next checkpoint before
            # exiting; otherwise the background thread keeps running.
            cancel_event.set()
            sys.exit(1)
    except Exception as e:
        logger.critical(f"Failed: {e}")
```

```
        sys.exit(1)
    finally:
        if app_ctl is not None:
            app_ctl.shutdown()
        elif db is not None:
            db.close()


if __name__ == "__main__":
    main()
```

APP\main.py

111 lines • 3,494 bytes

```
import sys
import tkinter as tk
import logging
import logging.handlers
import ctypes
from pathlib import Path
from tkinter import messagebox

app_root = Path(__file__).parent.absolute()
if str(app_root) not in sys.path:
    sys.path.insert(0, str(app_root))

from core.bootstrap import Bootstrap  # noqa: E402


def preflight_gui_imports():
    from core.ui.gui_master import GuiMaster

    return GuiMaster


def _enable_dpi_awareness() -> None:
    """Enable per-monitor DPI awareness so the UI is crisp on HiDPI displays."""
    try:
        ctypes.windll.shcore.SetProcessDpiAwareness(2)
    except Exception:
        try:
            ctypes.windll.user32.SetProcessDPIAware()
        except Exception:
            pass


def main():
    log_file = app_root / "startup_log.txt"
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        handlers=[
            logging.handlers.RotatingFileHandler(
                log_file,
                encoding="utf-8",
                maxBytes=10 * 1024 * 1024,  # 10 MB
                backupCount=5,
            ),
            logging.StreamHandler(),
        ],
    )
    logger = logging.getLogger("Main")
    logger.info(f"--- STARTING: {app_root} ---")
    _enable_dpi_awareness()
    app_ctl = None
    root = None
    try:
        GuiMaster = preflight_gui_imports()
        try:
            cfg_ctl, app_ctl, db = Bootstrap.setup(
                app_root, str(app_root / "config.yaml")
            )
        except RuntimeError as e:
            # Bootstrap signals missing critical dependency (or other
            # hard config failures) by raising RuntimeError. Surface a
            # friendly dialog *before* Tk root exists; messagebox will
            # spin up a default root for us.
            logger.critical(f"Bootstrap failed: {e}")
            try:
                messagebox.showerror("Ошибка запуска", str(e))
            except Exception:
                pass
            sys.exit(1)
        root = tk.Tk()
        if Bootstrap._integrity_warning:
            # Surfaced by _check_db_integrity_on_startup during bootstrap.
            # Tk root is now available so we can show a real messagebox.
            try:
                messagebox.showwarning(
                    "Предупреждение: целостность БД",
                    Bootstrap._integrity_warning,
                )
            except Exception:
                logger.warning(
                    "Could not show integrity warning dialog: %s",
                    Bootstrap._integrity_warning,
                )
```

```
        GuiMaster(root, cfg_ctl, app_ctl)

        def on_close():
            logger.info("Shutdown...")
            try:
                if app_ctl:
                    app_ctl.shutdown()
            except Exception as e:
                logger.error(f"Shutdown error: {e}")
            finally:
                if root:
                    root.destroy()
                sys.exit(0)

        root.protocol("WM_DELETE_WINDOW", on_close)
        root.mainloop()
    except Exception as e:
        logger.critical(f"Failed: {e}", exc_info=True)
        if app_ctl:
            try:
                app_ctl.shutdown()
            except Exception:
                pass
        sys.exit(1)


if __name__ == "__main__":
    main()
```

requirements.txt

7 lines • 130 bytes

```
openpyxl==3.1.5
ruamel.yaml==0.18.6
tkcalendar==1.6.1
pyxlsb==1.0.10
PyYAML==6.0.1
pydantic==2.10.6
python-calamine==0.6.2
```

requirements-dev.txt

10 lines • 216 bytes

```
-r requirements.txt
pytest==9.0.2
pytest-mock==3.15.1
pytest-cov==7.0.0
hypothesis==6.151.10
ruff==0.14.6
mypy==1.18.2
pandas-stubs==2.3.2.250926
types-openpyxl==3.1.5.20260408
types-psutil==7.2.2.20260408
```

pyproject.toml

22 lines • 512 bytes

```
# Minimal dev config for the runtime-only main branch.
# Full pytest config and tests-specific ignores live in the experiments branch.

[tool.ruff]
target-version = "py312"
line-length = 100
exclude = [
    ".git",
    ".pytest_cache",
    ".ruff_cache",
    "tmp_*",
    "output",
    "APP/output",
]

[tool.ruff.lint]
select = ["E", "F"]
ignore = ["E402", "E501", "E701"]

[tool.ruff.lint.per-file-ignores]
"APP/core/domain/__init__.py" = ["F401"]
"APP/core/infra/__init__.py" = ["F401"]
```

ЗАПУСК.bat

262 lines • 7,799 bytes

```
@echo off
setlocal EnableExtensions
chcp 65001 >nul

set "PIP_NO_KEYRING=1"
set "PIP_DISABLE_PIP_VERSION_CHECK=1"
set "PIP_NO_INPUT=1"

set "ARG1=%~1"
set "ARG2=%~2"

title Master Report Launcher v5.5

set "APPDIR=%~dp0APP"
set "MAINFILE=%APPDIR%\main.py"
set "LOGFILE=%~dp0startup_log.txt"
set "GUIERRLOG=%~dp0gui_error_log.txt"
set "TARGET_PY=3.12"
set "VENV_DIR=%~dp0.venv"
set "FAIL_MESSAGE="

set "PYTHON_CMD="
set "PYTHONW_CMD="
set "PYTHON_IS_VENV="

:: Clean up inherited environment variables if they contain empty quotes or invalid placeholders
if "%PYTHON_CMD%"=="""" set "PYTHON_CMD="
if "%PYTHON_CMD%"=="""""" set "PYTHON_CMD="
if "%PYTHONW_CMD%"=="""" set "PYTHONW_CMD="
if "%PYTHONW_CMD%"=="""""" set "PYTHONW_CMD="

break > "%LOGFILE%"
break > "%GUIERRLOG%"

call :log "============================================================"
call :log "MASTER REPORT CONSOLIDATOR v5.5"
call :log "============================================================"
call :log "Start time: %date% %time%"

echo ============================================================
echo MASTER REPORT CONSOLIDATOR v5.5
echo ============================================================
echo.
echo Checking environment...

if not exist "%MAINFILE%" goto missing_main

if not exist "%~dp0requirements.txt" goto missing_requirements

cd /d "%APPDIR%"
if errorlevel 1 goto appdir_unavailable

call :step "Step 1/5. Finding Python"
call :refresh_paths
call :find_python

if not defined PYTHON_CMD goto python_not_found
if "%PYTHON_CMD%"=="""" goto python_not_found
if "%PYTHON_CMD%"=="""""" goto python_not_found
goto python_ok

:python_not_found
call :log "Python not found. Install it from python.org"
set "FAIL_MESSAGE=Python not found. Please install Python 3.10 or newer and make sure it is in PATH."
goto fail

:python_ok

call :step "Step 2/5. Checking pip"
"%PYTHON_CMD%" -m pip --version >> "%LOGFILE%" 2>&1
if not errorlevel 1 goto pip_ok
call :log "pip missing. Repairing..."
"%PYTHON_CMD%" -m ensurepip --upgrade >> "%LOGFILE%" 2>&1
if errorlevel 1 goto pip_repair_failed

:pip_ok

call :step "Step 3/5. Checking packaging tools"
"%PYTHON_CMD%" -m pip --version >> "%LOGFILE%" 2>&1
if errorlevel 1 goto tool_upgrade_failed

call :step "Step 4/5. Checking libraries"
"%PYTHON_CMD%" -c "import tkinter, openpyxl, yaml, ruamel.yaml, tkcalendar, pyxlsb, pydantic, python_calamine" >> "%LOGFILE%" 2>&1
```

```
if errorlevel 1 goto install_requirements
goto after_requirements

:install_requirements
call :log "Installing missing libraries..."
if defined PYTHON_IS_VENV (
    "%PYTHON_CMD%" -m pip install -r "%~dp0requirements.txt" >> "%LOGFILE%" 2>&1
) else (
    call :log "WARNING: local .venv is unavailable; installing into user site-packages."
    "%PYTHON_CMD%" -m pip install --user -r "%~dp0requirements.txt" >> "%LOGFILE%" 2>&1
)
if errorlevel 1 goto requirements_install_failed

:after_requirements

call :step "Step 4.5/5. GUI import preflight"
"%PYTHON_CMD%" -c "import main; main.preflight_gui_imports()" >> "%GUIERRLOG%" 2>&1
if not errorlevel 1 goto gui_ok
call :log "GUI preflight failed. See gui_error_log.txt"
goto gui_preflight_failed

:gui_ok

call :step "Step 5/5. Launching application"
if not defined PYTHONW_CMD goto start_python
call :log "Launching via pythonw (no console)"
start "" "%PYTHONW_CMD%" "%MAINFILE%"
goto launch_done

:start_python
call :log "Launching via python (with console)"
start "" "%PYTHON_CMD%" "%MAINFILE%"

:launch_done

echo.
echo ------------------------------------------------------
echo DONE. The application is starting...
echo This window will close automatically in 3 seconds.
echo ------------------------------------------------------
timeout /t 3 >nul
exit /b 0


:step
echo %~1
call :log "%~1"
exit /b 0

:log
>> "%LOGFILE%" echo [%date% %time%] %~1
exit /b 0

:fail
echo.
echo [ERROR] %FAIL_MESSAGE%
call :log "[ERROR] %FAIL_MESSAGE%"
if not "%ARG1%"=="/nopause" if not "%ARG2%"=="/nopause" pause
exit /b 1

:missing_main
set "FAIL_MESSAGE=main.py was not found inside APP."
goto fail

:missing_requirements
set "FAIL_MESSAGE=requirements.txt was not found next to the launcher. Copy the whole project, not just APP."
goto fail

:appdir_unavailable
set "FAIL_MESSAGE=Could not switch into the APP directory."
goto fail

:pip_repair_failed
if exist "%LOGFILE%" type "%LOGFILE%"
set "FAIL_MESSAGE=Could not repair pip."
goto fail

:tool_upgrade_failed
if exist "%LOGFILE%" type "%LOGFILE%"
set "FAIL_MESSAGE=Could not update pip/setuptools/wheel."
goto fail

:requirements_install_failed
```

```
if exist "%LOGFILE%" type "%LOGFILE%"
set "FAIL_MESSAGE=Could not install dependencies from requirements.txt."
goto fail

:gui_preflight_failed
if exist "%GUIERRLOG%" type "%GUIERRLOG%"
set "FAIL_MESSAGE=GUI startup failed."
goto fail

:refresh_paths
set "PATH=%PATH%;%USERPROFILE%\AppData\Local\Microsoft\WindowsApps;%LocalAppData%\Programs\Python\Launcher"
exit /b 0

:find_python
rem 1. Prefer an isolated local virtual environment next to the launcher.
if exist "%VENV_DIR%\Scripts\python.exe" (
    call :check_venv_valid
    if not errorlevel 1 (
        set "PYTHON_CMD=%VENV_DIR%\Scripts\python.exe"
        set "PYTHON_IS_VENV=1"
        if exist "%VENV_DIR%\Scripts\pythonw.exe" set "PYTHONW_CMD=%VENV_DIR%\Scripts\pythonw.exe"
        goto python_launcher_done
    )
    call :log "WARNING: Local .venv is broken (likely copied from another PC). Re-creating..."
    call :cleanup_broken_venv
)

rem 2. Try to create the local virtual environment using different Python handles.
rem Try py -3.12 first
py -%TARGET_PY% --version >nul 2>&1
if not errorlevel 1 (
    call :log "Creating local virtual environment using py -%TARGET_PY%: %VENV_DIR%"
    py -%TARGET_PY% -m venv "%VENV_DIR%" >> "%LOGFILE%" 2>&1
    goto check_created_venv
)

rem Try py launcher generally
py --version >nul 2>&1
if not errorlevel 1 (
    call :log "Creating local virtual environment using py: %VENV_DIR%"
    py -m venv "%VENV_DIR%" >> "%LOGFILE%" 2>&1
    goto check_created_venv
)

rem Try python in PATH
python --version >nul 2>&1
if not errorlevel 1 (
    call :log "Creating local virtual environment using python: %VENV_DIR%"
    python -m venv "%VENV_DIR%" >> "%LOGFILE%" 2>&1
    goto check_created_venv
)

:check_created_venv
if exist "%VENV_DIR%\Scripts\python.exe" (
    call :check_venv_valid
    if not errorlevel 1 (
        set "PYTHON_CMD=%VENV_DIR%\Scripts\python.exe"
        set "PYTHON_IS_VENV=1"
        if exist "%VENV_DIR%\Scripts\pythonw.exe" set "PYTHONW_CMD=%VENV_DIR%\Scripts\pythonw.exe"
        goto python_launcher_done
    )
)
call :log "WARNING: could not create local .venv; falling back to PATH Python."

rem 3. Fallback to regular python in PATH
for /f "delims=" %%I in ('where python 2^>nul') do (
    if not defined PYTHON_CMD set "PYTHON_CMD=%%~fI"
)
for /f "delims=" %%I in ('where pythonw 2^>nul') do (
    if not defined PYTHONW_CMD set "PYTHONW_CMD=%%~fI"
)

rem 4. Last fallback to py launcher directly
if defined PYTHON_CMD goto python_launcher_done
py --version >nul 2>&1
if not errorlevel 1 (
    set "PYTHON_CMD=py"
    set "PYTHONW_CMD=pyw"
)

:python_launcher_done
exit /b 0
```

```
:cleanup_broken_venv
set "TEMP_VENV=%VENV_DIR%_broken_%RANDOM%"
move "%VENV_DIR%" "%TEMP_VENV%" >nul 2>&1
if exist "%VENV_DIR%" rd /s /q "%VENV_DIR%" >nul 2>&1
if exist "%TEMP_VENV%" rd /s /q "%TEMP_VENV%" >nul 2>&1
exit /b 0

:check_venv_valid
for %%I in ("%VENV_DIR%\Scripts\python.exe") do (
    if %%~zI LSS 10000 exit /b 1
)
"%VENV_DIR%\Scripts\python.exe" --version >nul 2>&1
exit /b %errorlevel%
```

ОБНОВИТЬ_И_ЗАПУСТИТЬ.bat

28 lines • 594 bytes

```
@echo off
setlocal EnableExtensions
chcp 65001 >nul

title Master Report Updater

set "ROOT=%~dp0"
set "SCRIPT=%ROOT%tools\update_from_github.ps1"
set "POWERSHELL_CMD=powershell"

if not exist "%SCRIPT%" (
    echo [ERROR] Updater not found: %SCRIPT%
    pause
    exit /b 1
)

where pwsh >nul 2>&1
if not errorlevel 1 set "POWERSHELL_CMD=pwsh"

"%POWERSHELL_CMD%" -NoProfile -ExecutionPolicy Bypass -File "%SCRIPT%" -Launch
if errorlevel 1 (
    echo.
    echo [ERROR] Update failed. See details above and in update_log.txt.
    pause
    exit /b 1
)

exit /b 0
```

tools\cleanup.ps1

49 lines • 1,355 bytes

```
<#
.SYNOPSIS
  Cleans transient files left by launcher bat scripts and stale pycache.

.DESCRIPTION
  Removes known junk files from the project root and stale __pycache__
  directories under APP/. Safe to run at any time — only deletes
  regenerable/transient artifacts.

.USAGE
  .\tools\cleanup.ps1              # clean root junk
  .\tools\cleanup.ps1 -Full        # also prune __pycache__ and .pyc
#>
[CmdletBinding()]
param(
    [switch]$Full
)

$ErrorActionPreference = 'SilentlyContinue'

$junkFiles = @(
    'console_err.txt',
    'console_out.txt',
    'err.txt',
    'out.txt',
    'last_bat_err.txt',
    'last_bat_out.txt',
    'update_log.txt'
)

$root = Split-Path -Parent $PSScriptRoot

foreach ($f in $junkFiles) {
    $p = Join-Path $root $f
    if (Test-Path $p) {
        Remove-Item -LiteralPath $p -Force
        Write-Host "Removed: $f"
    }
}

if ($Full) {
    Get-ChildItem -Path (Join-Path $root 'APP') -Recurse -Directory -Filter '__pycache__' |
        ForEach-Object { Remove-Item -LiteralPath $_.FullName -Recurse -Force; Write-Host "Removed: $($_.FullName)" }
    Get-ChildItem -Path $root -Filter '*.pyc' -Recurse |
        ForEach-Object { Remove-Item -LiteralPath $_.FullName -Force }
    Write-Host "Pruned __pycache__ and .pyc files."
}

Write-Host "Cleanup complete."
```

tools\diff_workspaces.py

42 lines • 1,165 bytes

```
import sys
import os
import difflib

ws1 = r"e:\Dlia Dashi\Version5\APP"
ws2 = r"C:\Users\vanya\Desktop\workspace-25b6a0ab-0a35-4c0a-aa7c-abe7168cc6fa\upload\extracted\APP"

files = [
    r"core/infra/staging_repo.py",
    r"core/infra/ingestion_reader.py",
    r"core/domain/logic_service.py",
    r"core/domain/data_loss_log.py"
]

for rel_path in files:
    p1 = os.path.join(ws1, rel_path.replace("/", "\\"))
    p2 = os.path.join(ws2, rel_path.replace("/", "\\"))

    if not os.path.exists(p1):
        print(f"File missing in ws1: {p1}")
        continue
    if not os.path.exists(p2):
        print(f"File missing in ws2: {p2}")
        continue

    with open(p1, "r", encoding="utf-8") as f1, open(p2, "r", encoding="utf-8") as f2:
        lines1 = f1.readlines()
        lines2 = f2.readlines()

    diff = list(difflib.unified_diff(
        lines1, lines2,
        fromfile=f"ws1/{rel_path}",
        tofile=f"ws2/{rel_path}",
        n=2
    ))

    if diff:
        print(f"=== DIFF FOR {rel_path} ===")
        sys.stdout.writelines(diff)
        print("\n" + "="*50 + "\n")
    else:
        print(f"No diff for {rel_path}")
```

tools\find_large.ps1

5 lines • 330 bytes

```
Get-ChildItem -Path REPORTS, РСЧ, "рсч всех" -Recurse -File -ErrorAction SilentlyContinue |
    Where-Object { $_.Length -gt 2MB -and $_.Extension -ne ".bak" } |
    Select-Object FullName, @{Name='SizeMB';Expression={[math]::Round($_.Length / 1MB, 2)}} |
    Sort-Object SizeMB -Descending |
    Format-Table -AutoSize
```

tools\merge_to_pdf.py

158 lines • 5,093 bytes

```
#!/usr/bin/env python3
"""Merge all project source files into a single PDF document.

Includes:
- All .py files under APP/
- Root config files (requirements.txt, pyproject.toml)
- Batch launchers (.bat)
- tools/ scripts (.py, .ps1)

Usage:
    python tools/merge_to_pdf.py          # -> project_source_code.pdf
    python tools/merge_to_pdf.py -o out.pdf
"""
from __future__ import annotations

import argparse
import subprocess
import sys
from pathlib import Path

from fpdf import FPDF

REPO_ROOT = Path(__file__).resolve().parents[1]

# --- File groups and ordering ---

APP_PY: list[Path] = sorted(
    p for p in Path(REPO_ROOT, "APP").rglob("*.py")
)

ROOT_CONFIGS: list[Path] = [
    Path(REPO_ROOT, "requirements.txt"),
    Path(REPO_ROOT, "requirements-dev.txt"),
    Path(REPO_ROOT, "pyproject.toml"),
]

ROOT_BATS: list[Path] = sorted(Path(REPO_ROOT).glob("*.bat"))

TOOLS_SCRIPTS: list[Path] = sorted(
    p for p in Path(REPO_ROOT, "tools").glob("*")
    if p.suffix in (".py", ".ps1")
)

# Combine into final order
ALL_FILES: list[Path] = APP_PY + ROOT_CONFIGS + ROOT_BATS + TOOLS_SCRIPTS
ALL_FILES = [p for p in ALL_FILES if p.is_file()]


class CodePDF(FPDF):
    """PDF builder with syntax-friendly monospace layout."""

    FONT_SIZE = 7
    LINE_HEIGHT = 3.2
    LEFT_MARGIN = 10
    RIGHT_MARGIN = 10

    def header(self) -> None:
        self.set_font(self._font_family, "B", 7)
        self.set_text_color(120, 120, 120)
        rel = self._current_rel_path()
        self.cell(0, 4, f"Master-Report — {rel}", align="R", new_x="LMARGIN", new_y="NEXT")
        self.set_draw_color(200, 200, 200)
        self.line(self.l_margin, self.get_y(), self.w - self.r_margin, self.get_y())
        self.ln(2)

    def footer(self) -> None:
        self.set_y(-12)
        self.set_font(self._font_family, "", 6)
        self.set_text_color(150, 150, 150)
        self.cell(0, 8, f"{self.page_no()}/{379}", align="C")

    def _current_rel_path(self) -> str:
        return getattr(self, "_rel_path", "")

    def add_file(self, path: Path) -> None:
        rel = path.relative_to(REPO_ROOT)
        self._rel_path = str(rel)

        # --- Title page / separator ---
        self.add_page()
        self.ln(15)
        self.set_font(self._font_family, "B", 14)
        self.set_text_color(30, 30, 30)
```

```
        self.cell(0, 8, str(rel), new_x="LMARGIN", new_y="NEXT", align="C")

        self.set_font(self._font_family, "", 8)
        self.set_text_color(100, 100, 100)
        lines = len(path.read_text(encoding="utf-8", errors="replace").splitlines())
        self.cell(0, 5, f"{lines} lines  •  {path.stat().st_size:,} bytes", new_x="LMARGIN", new_y="NEXT", align="C")

        self.set_draw_color(180, 180, 180)
        self.line(self.l_margin + 30, self.get_y() + 2, self.w - self.r_margin - 30, self.get_y() + 2)

        # --- Code content ---
        self.add_page()
        self.set_font(self._font_family, "", self.FONT_SIZE)
        self.set_text_color(20, 20, 20)

        text = path.read_text(encoding="utf-8", errors="replace")
        for raw_line in text.splitlines():
            display = raw_line.replace("\t", "    ")
            if display.strip() == "":
                self.ln(self.LINE_HEIGHT)
            else:
                self.cell(0, self.LINE_HEIGHT, display, new_x="LMARGIN", new_y="NEXT")

    def _setup_fonts(self) -> None:
        """Register a Unicode-safe monospace font.

        Prefers Consola (Windows system font), falls back to Courier.
        """
        import os

        windir = os.environ.get("WINDIR", r"C:\Windows")
        fonts_dir = os.path.join(windir, "Fonts")
        regular = os.path.join(fonts_dir, "consola.ttf")
        bold = os.path.join(fonts_dir, "consolab.ttf")

        if os.path.isfile(regular):
            self.add_font("Code", "", regular)
            if os.path.isfile(bold):
                self.add_font("Code", "B", bold)
            self._font_family = "Code"
        else:
            self._font_family = "Courier"

    def finish(self, output: Path) -> None:
        self.alias_nb_pages()
        self.output(str(output))


def main() -> None:
    parser = argparse.ArgumentParser(description="Merge project source into one PDF")
    parser.add_argument("-o", "--output", type=Path, default=Path(REPO_ROOT, "project_source_code.pdf"))
    args = parser.parse_args()

    pdf = CodePDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.set_left_margin(pdf.LEFT_MARGIN)
    pdf.set_right_margin(pdf.RIGHT_MARGIN)
    pdf._setup_fonts()

    print(f"Merging {len(ALL_FILES)} files into {args.output.name} ...")
    for i, f in enumerate(ALL_FILES, 1):
        rel = f.relative_to(REPO_ROOT)
        print(f"  [{i:3d}/{len(ALL_FILES)}] {rel}")
        try:
            pdf.add_file(f)
        except Exception as exc:
            print(f"  WARNING: {rel}: {exc}", file=sys.stderr)

    pdf.finish(args.output)
    size_mb = args.output.stat().st_size / (1024 * 1024)
    print(f"\nDone: {args.output} ({size_mb:.1f} MB, {pdf.page_no()} pages)")


if __name__ == "__main__":
    main()
```

tools\merge_to_txt.py

65 lines • 2,271 bytes

```
#!/usr/bin/env python3
"""Merge all project source files into a single text file.

Includes:
- All .py files under APP/
- Root config files (requirements.txt, pyproject.toml)
- Batch launchers (.bat)
- tools/ scripts (.py, .ps1)
"""
from __future__ import annotations

import sys
from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parents[1]

# --- File groups and ordering ---
APP_PY = sorted(p for p in Path(REPO_ROOT, "APP").rglob("*.py"))
ROOT_CONFIGS = [
    Path(REPO_ROOT, "requirements.txt"),
    Path(REPO_ROOT, "requirements-dev.txt"),
    Path(REPO_ROOT, "pyproject.toml"),
]
ROOT_BATS = sorted(Path(REPO_ROOT).glob("*.bat"))
TOOLS_SCRIPTS = sorted(
    p for p in Path(REPO_ROOT, "tools").glob("*")
    if p.suffix in (".py", ".ps1")
)

# Combine into final order
ALL_FILES = APP_PY + ROOT_CONFIGS + ROOT_BATS + TOOLS_SCRIPTS
ALL_FILES = [p for p in ALL_FILES if p.is_file()]

# Exclude itself and target output files to avoid self-inclusion or duplicate runs
EXCLUDED_NAMES = {"merge_to_txt.py", "project_source_code.txt", "project_code.txt"}
ALL_FILES = [p for p in ALL_FILES if p.name not in EXCLUDED_NAMES]

def main() -> None:
    output_path = REPO_ROOT / "project_source_code.txt"
    print(f"Merging {len(ALL_FILES)} files into {output_path.name} ...")

    try:
        with open(output_path, "w", encoding="utf-8") as f_out:
            for i, f in enumerate(ALL_FILES, 1):
                rel = f.relative_to(REPO_ROOT)
                print(f"  [{i:3d}/{len(ALL_FILES)}] {rel}")

                try:
                    content = f.read_text(encoding="utf-8", errors="replace")
                    f_out.write("=" * 80 + "\n")
                    f_out.write(f"FILE: {rel}\n")
                    f_out.write("=" * 80 + "\n\n")
                    f_out.write(content)
                    f_out.write("\n\n")
                except Exception as exc:
                    print(f"  WARNING: {rel}: {exc}", file=sys.stderr)

        size_kb = output_path.stat().st_size / 1024
        print(f"\nDone: {output_path} ({size_kb:.1f} KB)")
    except Exception as e:
        print(f"ERROR: Failed to write output file: {e}", file=sys.stderr)
        sys.exit(1)

if __name__ == "__main__":
    main()
```

tools\precommit_check.py

103 lines • 3,522 bytes

```
#!/usr/bin/env python3
"""Pre-commit quality gate for Master-Report.

Cross-platform replacement for the old bash hook. Invoked by
.git/hooks/pre-commit (a thin sh wrapper). Does three things:

1. ruff --select F401 --fix  : auto-remove unused imports, re-staged by caller
2. ruff full check           : blocks on lint errors
3. mypy baseline mode        : blocks only NEW errors (line numbers
                               normalised, so pure line-shifts don't count)

The baseline file is .mypy_baseline.txt at the repo root. If it is
missing the first run captures it and allows the commit.
"""
from __future__ import annotations

import subprocess
import sys
from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parents[1]
APP_DIR = REPO_ROOT / "APP"
TEST_DIR = REPO_ROOT / "tests"
BASELINE = REPO_ROOT / ".mypy_baseline.txt"


def _python() -> str:
    """Prefer the project venv python; fall back to the invoking python."""
    venv_py = REPO_ROOT / ".venv" / "Scripts" / "python.exe"
    if venv_py.exists():
        return str(venv_py)
    return sys.executable


def _run(cmd: list[str]) -> subprocess.CompletedProcess[str]:
    return subprocess.run(cmd, capture_output=True, text=True, encoding="utf-8")


def _normalise(text: str) -> set[str]:
    """Strip line numbers so a pure shift from editing nearby code is
    not counted as a new error."""
    import re

    out = set()
    for line in text.splitlines():
        line = line.strip()
        if not line:
            continue
        line = re.sub(r":\d+:", ":N:", line)
        out.add(line)
    return out


def main() -> int:
    py = _python()

    print("=== [1/3] ruff: auto-fix unused imports ===")
    _run([py, "-m", "ruff", "check", "--select", "F401", "--fix",
          str(APP_DIR), str(TEST_DIR)])
    # The caller re-stages; we don't touch git here to stay testable.

    print("=== [2/3] ruff: full check ===")
    ruff = _run([py, "-m", "ruff", "check", str(APP_DIR), str(TEST_DIR)])
    print(ruff.stdout, end="")
    print(ruff.stderr, end="", file=sys.stderr)
    if ruff.returncode != 0:
        print("[pre-commit] ruff failed.")
        return ruff.returncode

    print("=== [3/3] mypy: baseline mode ===")
    mypy = _run([py, "-m", "mypy", str(APP_DIR),
                 "--ignore-missing-imports", "--strict", "--no-error-summary"])
    current_lines = _normalise(mypy.stdout + mypy.stderr)

    if not BASELINE.exists():
        BASELINE.write_text(
            mypy.stdout + mypy.stderr, encoding="utf-8"
        )
        print(f"Baseline created: {BASELINE} ({len(current_lines)} issues)")
        print("No errors blocked — baseline captured.")
        print("\n=== All pre-commit checks passed ===")
        return 0
```

```
    baseline_lines = _normalise(BASELINE.read_text(encoding="utf-8"))
    new = sorted(current_lines - baseline_lines)
    if new:
        print("=== BLOCKED: new mypy errors (line numbers normalised) ===")
        for line in new:
            print(line)
        print(
            "\nFix the above, or refresh the baseline if intentional:\n"
            f"  {py} -m mypy APP --strict --ignore-missing-imports "
            "--no-error-summary > .mypy_baseline.txt"
        )
        return 1

    print(f"mypy: no new errors (baseline: {len(baseline_lines)} lines)")
    print("\n=== All pre-commit checks passed ===")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

tools\update_from_github.ps1

503 lines • 19,306 bytes

```
param(
    [string]$RepoOwner = "sunil123456897",
    [string]$RepoName = "Exel_otchet",
    [string]$Branch = "main",
    # Pin a specific commit SHA for integrity verification. When set,
    # the updater ABORTS if the fetched commit does not match the pin —
    # this stops a compromised repository (or MITM with a valid TLS
    # cert) from pushing arbitrary code to user PCs. Empty = accept
    # branch HEAD (LESS SECURE — only use during development).
    # Bumping the pin requires manual review of the diff between the
    # old and new SHA on github.com/<owner>/<repo>/compare/<old>...<new>.
    [string]$PinnedCommitSha = "",
    [switch]$Launch
)

$ErrorActionPreference = "Stop"

$env:PIP_NO_KEYRING = "1"
$env:PIP_DISABLE_PIP_VERSION_CHECK = "1"
$env:PIP_NO_INPUT = "1"

try {
    [Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManager]::SecurityProtocol -bor [Net.SecurityProtocolType]::Tls12
} catch {
    Write-Warning "Could not force TLS 1.2 for GitHub downloads: $($_.Exception.Message)"
}

$ScriptDir = Split-Path -Parent $MyInvocation.MyCommand.Path
$Root = Split-Path -Parent $ScriptDir
$LogFile = Join-Path $Root "update_log.txt"
$StateDir = Join-Path $Root ".update_state"
$BackupRoot = Join-Path $Root ".update_backups"
$LocalSettings = @("APP\config.yaml", "APP\employees.yaml")
$LocalData = @(
    "APP\cache.db",
    "APP\cache.db-wal",
    "APP\cache.db-shm",
    "APP\runtime_state.json"
)
$ProtectedUpdatePaths = $LocalSettings + $LocalData + @(
    "APP\output",
    "REPORTS",
    "РСЧ",
    "рсч всех",
    "Master_Report.xlsx"
)
$UpdateItems = @(
    "APP\core",
    "APP\main.py",
    "APP\headless_run.py",
    "APP\db_schema.sql",
    "APP\config.example.yaml",
    "APP\migrations",
    "requirements.txt",
    "ЗАПУСК.bat",
    "ОБНОВИТЬ_И_ЗАПУСТИТЬ.bat",
    "tools\update_from_github.ps1"
)

function Get-GitHubToken {
    $tokenFile = Join-Path $Root ".github_token"
    if (Test-Path -LiteralPath $tokenFile) {
        $token = (Get-Content -LiteralPath $tokenFile -Raw).Trim()
        if ($token) { return $token }
    }
    $envToken = $env:GITHUB_TOKEN
    if ($envToken) { return $envToken }
    return $null
}

function Get-GitHubHeaders {
    param([string]$Token)
    $headers = @{"User-Agent" = "Master-Report-Updater"}
    if ($Token) {
        $headers["Authorization"] = "token $Token"
    }
    return $headers
}

function Write-Log {
    param([string]$Message)
    $line = "[{0}] {1}" -f (Get-Date -Format "yyyy-MM-dd HH:mm:ss"), $Message
    Write-Host $Message
```

```
    Add-Content -Path $LogFile -Value $line -Encoding UTF8
}

function Ensure-Directory {
    param([string]$Path)
    if (-not (Test-Path -LiteralPath $Path)) {
        New-Item -ItemType Directory -Path $Path | Out-Null
    }
}

function Normalize-RelativePath {
    param([string]$Path)
    return (($Path -replace "/", "\").TrimEnd("\")).ToLowerInvariant()
}

function Test-IsSameOrChildPath {
    param(
        [string]$Candidate,
        [string]$Parent
    )

    $candidateNorm = Normalize-RelativePath $Candidate
    $parentNorm = Normalize-RelativePath $Parent
    return $candidateNorm -eq $parentNorm -or $candidateNorm.StartsWith($parentNorm + "\")
}

function Test-IsProtectedByUpdateItem {
    param(
        [string]$ProtectedPath,
        [string]$UpdateItem
    )

    return (Test-IsSameOrChildPath $ProtectedPath $UpdateItem) -or (Test-IsSameOrChildPath $UpdateItem $ProtectedPath)
}

function Assert-UpdatePolicy {
    foreach ($item in $UpdateItems) {
        $normalized = Normalize-RelativePath $item
        if ($normalized -in @(".", "app", "reports", "рсч", "рсч всех")) {
            throw "Небезопасный update allowlist item: $item"
        }

        foreach ($protectedPath in $ProtectedUpdatePaths) {
            if (Test-IsProtectedByUpdateItem $protectedPath $item) {
                throw "Update item '$item' пересекается с локальными данными '$protectedPath'"
            }
        }
    }
}

function Backup-LocalFiles {
    param([string[]]$RelativePaths)

    $stamp = Get-Date -Format "yyyyMMdd_HHmmss"
    $backupDir = Join-Path $BackupRoot $stamp
    Ensure-Directory $backupDir

    foreach ($relativePath in $RelativePaths) {
        $source = Join-Path $Root $relativePath
        if (Test-Path -LiteralPath $source) {
            $target = Join-Path $backupDir $relativePath
            Ensure-Directory (Split-Path -Parent $target)
            Copy-Item -LiteralPath $source -Destination $target -Force
        }
    }

    return $backupDir
}

function Restore-LocalFiles {
    param(
        [string]$BackupDir,
        [string[]]$RelativePaths
    )

    foreach ($relativePath in $RelativePaths) {
        $source = Join-Path $BackupDir $relativePath
        if (Test-Path -LiteralPath $source) {
            $target = Join-Path $Root $relativePath
            Ensure-Directory (Split-Path -Parent $target)
            Copy-Item -LiteralPath $source -Destination $target -Force
        }
    }
```

```
}

function Get-GitCommand {
    $cmd = Get-Command git -ErrorAction SilentlyContinue
    if ($null -eq $cmd) {
        return $null
    }
    return $cmd.Source
}

function Get-RemoteSha {
    param([string]$Token)
    $apiUrl = "https://api.github.com/repos/$RepoOwner/$RepoName/commits/$Branch"
    try {
        $headers = Get-GitHubHeaders $Token
        $response = Invoke-RestMethod -Uri $apiUrl -Headers $headers -TimeoutSec 30
        return [string]$response.sha
    } catch {
        Write-Log "Не удалось проверить последнюю версию через GitHub API: $($_.Exception.Message)"
        return $null
    }
}

function Get-InstalledSha {
    param([string]$FallbackSha)

    $lastShaFile = Join-Path $StateDir "last_sha.txt"
    if (Test-Path -LiteralPath $lastShaFile) {
        $storedSha = (Get-Content -LiteralPath $lastShaFile -Raw).Trim()
        if ($storedSha) {
            return $storedSha
        }
    }

    return $FallbackSha
}

function Assert-PinnedCommit {
    param([string]$ResolvedSha)

    # Integrity gate: refuse to apply an update whose commit SHA does not
    # match the operator-pinned value. Without this check, a compromised
    # GitHub repo (or a TLS-MITM with a valid cert) could push arbitrary
    # code to every user PC running ОБНОВИТЬ_И_ЗАПУСТИТЬ.bat.
    if (-not $PinnedCommitSha) {
        Write-Warning "PinnedCommitSha не задан — обновление из ветки '$Branch' без проверки целостности. Задайте -PinnedCommitSha для защиты от ко
        return
    }
    if ([string]::IsNullOrWhiteSpace($ResolvedSha)) {
        throw "Не удалось определить SHA загруженного коммита. Обновление отменено (нет целостности)."
    }
    $resolvedNorm = $ResolvedSha.Trim().ToLowerInvariant()
    $pinnedNorm = $PinnedCommitSha.Trim().ToLowerInvariant()
    if ($resolvedNorm -ne $pinnedNorm) {
        throw "Целостность обновления НЕ подтверждена: полученный commit '$resolvedNorm' не совпадает с закреплённым '$pinnedNorm'. Обновление ОТМЕ
    }
}

function Copy-UpdateItems {
    param([string]$SourceRoot)

    foreach ($item in $UpdateItems) {
        $source = Join-Path $SourceRoot $item
        if (-not (Test-Path -LiteralPath $source)) {
            Write-Log "Пропускаю отсутствующий файл обновления: $item"
            continue
        }

        $target = Join-Path $Root $item
        if (Test-Path -LiteralPath $source -PathType Container) {
            Ensure-Directory $target
            Get-ChildItem -LiteralPath $source -Force | Copy-Item -Destination $target -Recurse -Force
        } else {
            Ensure-Directory (Split-Path -Parent $target)
            Copy-Item -LiteralPath $source -Destination $target -Force
        }
    }
}

function Expand-GitArchive {
    param(
        [string]$Git,
        [string]$Treeish,
```

```
        [string]$Destination
    )

    Ensure-Directory $Destination
    $archivePath = Join-Path $Destination "update.tar"
    $normalizedItems = $UpdateItems | ForEach-Object { $_ -replace "\\", "/" }

    & $Git archive --format=tar --output=$archivePath $Treeish -- $normalizedItems
    if ($LASTEXITCODE -ne 0) {
        throw "git archive завершился с кодом $LASTEXITCODE"
    }

    tar -xf $archivePath -C $Destination
    if ($LASTEXITCODE -ne 0) {
        throw "tar завершился с кодом $LASTEXITCODE"
    }
}

function Update-WithGit {
    param([string]$Git, [string]$Token)

    if (-not (Test-Path -LiteralPath (Join-Path $Root ".git"))) {
        return $false
    }

    Push-Location $Root
    try {
        $remoteUrl = & $Git remote get-url origin 2>$null
        if (-not $remoteUrl) {
            if ($Token) {
                & $Git remote add origin "https://$Token@github.com/$RepoOwner/$RepoName.git" | Out-Null
            } else {
                & $Git remote add origin "https://github.com/$RepoOwner/$RepoName.git" | Out-Null
            }
        } elseif ($Token -and $remoteUrl -notmatch "@github\.com") {
            & $Git remote set-url origin "https://$Token@github.com/$RepoOwner/$RepoName.git" | Out-Null
        }

        Write-Log "Проверяю обновления через Git..."
        & $Git fetch origin $Branch --quiet
        if ($LASTEXITCODE -ne 0) {
            throw "git fetch завершился с кодом $LASTEXITCODE"
        }

        $headSha = (& $Git rev-parse HEAD).Trim()
        $localSha = Get-InstalledSha $headSha
        $remoteSha = (& $Git rev-parse "origin/$Branch").Trim()
        Assert-PinnedCommit $remoteSha
        if ($localSha -eq $remoteSha) {
            Write-Log "Новых версий нет. Текущая версия: $localSha"
            return $true
        }

        $backupDir = Backup-LocalFiles ($LocalSettings + $LocalData)
        Write-Log "Найдена новая версия: $remoteSha. Backup локальных настроек/БД: $backupDir"

        $tempDir = Join-Path $env:TEMP ("master_report_git_update_" + [guid]::NewGuid().ToString("N"))
        try {
            Expand-GitArchive $Git "origin/$Branch" $tempDir
            Copy-UpdateItems $tempDir
        } finally {
            if (Test-Path -LiteralPath $tempDir) {
                Remove-Item -LiteralPath $tempDir -Recurse -Force
            }
        }

        Restore-LocalFiles $backupDir ($LocalSettings + $LocalData)
        Ensure-Directory $StateDir
        Set-Content -LiteralPath (Join-Path $StateDir "last_sha.txt") -Value $remoteSha -Encoding UTF8
        Write-Log "Минимальное обновление через Git выполнено. Локальные настройки/БД возвращены."
        return $true
    } catch {
        Write-Log "WARNING: Git update failed: $($_.Exception.Message). Falling back to snapshot update..."
        return $false
    } finally {
        Pop-Location
    }
}

function Update-WithGitSnapshot {
    param([string]$Git)

    $tempRepo = Join-Path $env:TEMP ("master_report_git_snapshot_repo_" + [guid]::NewGuid().ToString("N"))
```

```
    $tempArchive = Join-Path $env:TEMP ("master_report_git_snapshot_update_" + [guid]::NewGuid().ToString("N"))
    $pushed = $false

    try {
        Ensure-Directory $tempRepo
        Ensure-Directory $tempArchive
        Push-Location $tempRepo
        $pushed = $true

        try {
            & $Git init --quiet
            if ($LASTEXITCODE -ne 0) {
                throw "git init завершился с кодом $LASTEXITCODE"
            }

            & $Git remote add origin "https://github.com/$RepoOwner/$RepoName.git"
            if ($LASTEXITCODE -ne 0) {
                throw "git remote add завершился с кодом $LASTEXITCODE"
            }

            Write-Log "Папка без .git. Скачиваю минимальный snapshot через Git..."
            & $Git fetch --depth=1 origin $Branch --quiet
            if ($LASTEXITCODE -ne 0) {
                throw "git fetch snapshot завершился с кодом $LASTEXITCODE"
            }

            $remoteSha = (& $Git rev-parse FETCH_HEAD).Trim()
            Assert-PinnedCommit $remoteSha
            $localSha = Get-InstalledSha ""
            if ($localSha -eq $remoteSha) {
                Write-Log "Новых версий нет. Текущая версия: $localSha"
                return $true
            }

            $backupDir = Backup-LocalFiles ($LocalSettings + $LocalData)
            Write-Log "Найдена новая версия: $remoteSha. Backup локальных настроек/БД: $backupDir"

            Expand-GitArchive $Git "FETCH_HEAD" $tempArchive
            Copy-UpdateItems $tempArchive
            Restore-LocalFiles $backupDir ($LocalSettings + $LocalData)
            Ensure-Directory $StateDir
            Set-Content -LiteralPath (Join-Path $StateDir "last_sha.txt") -Value $remoteSha -Encoding UTF8
            Write-Log "Минимальное обновление через Git snapshot выполнено. Локальные настройки/БД возвращены."
            return $true
        } catch {
            Write-Log "WARNING: Git snapshot update failed: $($_.Exception.Message). Falling back to zip update..."
            return $false
        }
    } finally {
        if ($pushed) {
            Pop-Location
        }
        if (Test-Path -LiteralPath $tempArchive) {
            Remove-Item -LiteralPath $tempArchive -Recurse -Force
        }
        if (Test-Path -LiteralPath $tempRepo) {
            Remove-Item -LiteralPath $tempRepo -Recurse -Force
        }
    }
}

function Update-WithZip {
    param([string]$Token)
    $remoteSha = Get-RemoteSha $Token
    Assert-PinnedCommit $remoteSha
    $lastShaFile = Join-Path $StateDir "last_sha.txt"
    if ($remoteSha -and (Test-Path -LiteralPath $lastShaFile)) {
        $currentSha = (Get-Content -LiteralPath $lastShaFile -Raw).Trim()
        if ($currentSha -eq $remoteSha) {
            Write-Log "Новых версий нет. Текущая версия: $currentSha"
            return
        }
    }

    $backupDir = Backup-LocalFiles ($LocalSettings + $LocalData)
    $tempDir = Join-Path $env:TEMP ("master_report_update_" + [guid]::NewGuid().ToString("N"))
    $zipPath = Join-Path $tempDir "source.zip"
    Ensure-Directory $tempDir

    try {
        if ($Token) {
            $zipUrl = "https://api.github.com/repos/$RepoOwner/$RepoName/zipball/$Branch"
            $headers = Get-GitHubHeaders $Token
```

```
            $headers["Accept"] = "application/vnd.github+json"
        } else {
            $zipUrl = "https://github.com/$RepoOwner/$RepoName/archive/refs/heads/$Branch.zip"
            $headers = Get-GitHubHeaders $Token
        }
        Write-Log "Git не найден или папка без .git. Скачиваю zip из GitHub..."
        Invoke-WebRequest -Uri $zipUrl -OutFile $zipPath -UseBasicParsing -TimeoutSec 120 -Headers $headers
        Expand-Archive -LiteralPath $zipPath -DestinationPath $tempDir -Force
        $sourceRoot = Get-ChildItem -LiteralPath $tempDir -Directory | Where-Object { $_.Name -like "$RepoName-*" -or $_.Name -like "$RepoOwner-$Re
        if ($null -eq $sourceRoot) {
            throw "Не найдена распакованная папка проекта."
        }

        Copy-UpdateItems $sourceRoot.FullName

        Restore-LocalFiles $backupDir ($LocalSettings + $LocalData)
        Ensure-Directory $StateDir
        if ($remoteSha) {
            Set-Content -LiteralPath $lastShaFile -Value $remoteSha -Encoding UTF8
        }
        Write-Log "Минимальное обновление через zip выполнено. Локальные настройки/БД возвращены."
    } finally {
        if (Test-Path -LiteralPath $tempDir) {
            Remove-Item -LiteralPath $tempDir -Recurse -Force
        }
    }
}

function Update-Dependencies {
    $venvPython = Join-Path $Root ".venv\Scripts\python.exe"
    $requirements = Join-Path $Root "requirements.txt"
    if ((Test-Path -LiteralPath $venvPython) -and (Test-Path -LiteralPath $requirements)) {
        $size = (Get-Item -LiteralPath $venvPython).Length
        $valid = $false
        if ($size -gt 10000) {
            $null = & $venvPython --version 2>$null
            if ($LASTEXITCODE -eq 0) {
                $valid = $true
            }
        }
        if ($valid) {
            Write-Log "Обновляю зависимости в локальном .venv..."
            & $venvPython -m pip install -r $requirements | Tee-Object -FilePath $LogFile -Append
            if ($LASTEXITCODE -ne 0) {
                throw "pip install завершился с кодом $LASTEXITCODE"
            }
        } else {
            Write-Warning "Локальное виртуальное окружение .venv неисправно. Пропускаю обновление зависимостей в апдейтере. ЗАПУСК.bat автоматическ
        }
    }
}

Set-Content -Path $LogFile -Value "" -Encoding UTF8
Write-Log "Master Report updater started."
Write-Log "Project folder: $Root"
Assert-UpdatePolicy

$gitHubToken = Get-GitHubToken
if ($gitHubToken) {
    Write-Log "GitHub токен найден. Используется авторизованный доступ."
} else {
    Write-Log "GitHub токен не найден. Попытка анонимного доступа..."
}

$git = Get-GitCommand
$updatedWithGit = $false
if ($git) {
    $updatedWithGit = Update-WithGit $git
    if (-not $updatedWithGit) {
        $updatedWithGit = Update-WithGitSnapshot $git
    }
}
if (-not $updatedWithGit) {
    Update-WithZip $gitHubToken
}

Update-Dependencies

if ($Launch) {
    $launcher = Join-Path $Root "ЗАПУСК.bat"
    if (-not (Test-Path -LiteralPath $launcher)) {
        throw "Не найден ЗАПУСК.bat"
    }
```

```
    Write-Log "Запускаю приложение..."
    Start-Process -FilePath $launcher -ArgumentList "/nopause" -WorkingDirectory $Root
}

Write-Log "Updater finished successfully."
```